Ryutaro Ichise (Ed.)

Natural Language Processing and Information Systems

LNCS 15837

30th International Conference on Applications of Natural Language to Information Systems, NLDB 2025
Kanazawa, Japan, July 4–6, 2025
Proceedings, Part II

2 Part II

Springer

Lecture Notes in Computer Science **15837**

Founding Editors

Gerhard Goos
Juris Hartmanis

Editorial Board Members

Elisa Bertino, *Purdue University, West Lafayette, IN, USA*
Wen Gao, *Peking University, Beijing, China*
Bernhard Steffen, *TU Dortmund University, Dortmund, Germany*
Moti Yung, *Columbia University, New York, NY, USA*

The series Lecture Notes in Computer Science (LNCS), including its subseries Lecture Notes in Artificial Intelligence (LNAI) and Lecture Notes in Bioinformatics (LNBI), has established itself as a medium for the publication of new developments in computer science and information technology research, teaching, and education.

LNCS enjoys close cooperation with the computer science R & D community, the series counts many renowned academics among its volume editors and paper authors, and collaborates with prestigious societies. Its mission is to serve this international community by providing an invaluable service, mainly focused on the publication of conference and workshop proceedings and postproceedings. LNCS commenced publication in 1973.

Ryutaro Ichise
Editor

Natural Language Processing and Information Systems

30th International Conference on Applications
of Natural Language to Information Systems, NLDB 2025
Kanazawa, Japan, July 4–6, 2025
Proceedings, Part II

Springer

Editor
Ryutaro Ichise
Institute of Science Tokyo
Tokyo, Japan

ISSN 0302-9743 ISSN 1611-3349 (electronic)
Lecture Notes in Computer Science
ISBN 978-3-031-97143-3 ISBN 978-3-031-97144-0 (eBook)
https://doi.org/10.1007/978-3-031-97144-0

© The Editor(s) (if applicable) and The Author(s), under exclusive license
to Springer Nature Switzerland AG 2026

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

Preface

We are delighted to present the proceedings of the 30th International Conference on Natural Language and Information Systems (NLDB 2025), held in Kanazawa, Japan, from July 4 to July 6, 2025.

As a well-established forum at the intersection of natural language processing and information systems, NLDB has brought together researchers, practitioners, and industry professionals for three decades to share advances, explore challenges, and foster interdisciplinary collaboration. The 30th edition continued this tradition, providing a venue for both foundational research and innovative applications across a range of domains.

This year, NLDB 2025 received 120 submissions. Following a rigorous single-blind peer-review process in which submissions received on average 2.1 reviews each, 33 full papers, 19 short papers, and 2 demo papers were accepted. Among the short papers, 4 were selected for the Industry Track, reflecting the conference's commitment to bridging academic research and real-world applications. Due to the exceptionally high quality of submissions this year, the selection process was particularly competitive. Papers with an average review score of approximately 1.0 or higher were generally accepted as long papers, while those scoring around 0.5 or higher were considered for short paper presentation.

We sincerely thank all authors for their contributions and the reviewers and Program Committee members for their thoughtful evaluations and dedication to maintaining the high standard of the conference. We also acknowledge the invited speakers, whose keynotes enriched the program by highlighting emerging trends and open challenges in the field.

Finally, we extend our deepest appreciation to the Organizing Committee, Local Chairs, volunteers, and sponsors whose support made NLDB 2025 possible. We hope that these proceedings will serve as a valuable resource and inspiration for future research in natural language and information systems.

May 2025 Ryutaro Ichise

Organization

General Chairs

Akiko Aizawa	National Institute of Informatics, Japan
Farid Meziane	University of Derby, UK
Kiyoaki Shirai	Japan Advanced Institute of Science and Technology, Japan
Vijayan Sugumaran	Oakland University, USA

Program Committee Chair

Ryutaro Ichise	Institute of Science Tokyo, Japan

Local Chair

Natthawut Kertkeidkachorn	Japan Advanced Institute of Science and Technology, Japan

Web and Publicity Chairs

Rungsiman Nararatwong	OUTSCALE, Dassault Systèmes, France
Patipon Wiangnak	Japan Advanced Institute of Science and Technology, Japan

Local Committee

Miura Takuto	Japan Advanced Institute of Science and Technology, Japan
Oda Kohei	Japan Advanced Institute of Science and Technology, Japan

Program Committee

Asad Abdi	University of Derby, UK
Jacky Akoka	CEDRIC-CNAM, France
Yasir Arfat	University of Derby, UK
Davide Audrito	University of Bologna, Italy
Mithun Balakrishna	Lymba Corporation, USA
Pier Felice Balestrucci	Università degli Studi di Torino, Italy
Somnath Banerjee	University of Tartu, Estonia
Valerio Basile	University of Turin, Italy
Patrizio Bellan	Fondazione Bruno Kessler, Italy
Imene Bensalem	ESCF de Constantine and Constantine 2 University, Algeria
Alessandro Bondielli	Università di Pisa, Italy
Ana-Maria Bucur	University of Bucharest, Romania
Davide Buscaldi	Université Sorbonne Paris Nord & École Polytechnique, France
Elena Cabrio	Université Côte d'Azur, CNRS, Inria, I3S, France
Lucia Cascone	University of Salerno, Italy
Raja Chiky	3iL ingénieurs, Xlim, France
Isabelle Comyn-Wattiau	ESSEC Business School, France
Danilo Croce	University of Rome Tor Vergata, Italy
Greta Damo	Université Côte d'Azur, France
Luigi Di Caro	University of Turin, Italy
Mauro Dragoni	Fondazione Bruno Kessler, Italy
Komal Florio	University of Turin, Italy
Vladimir Fomichov	Moscow Aviation Institute (National Research University), Russia
Flavius Frasincar	Erasmus University Rotterdam, Netherlands
Simona Frenda	Heriot-Watt University, UK
Valeriya Goloviznina	Vyatka State University, Russia
Fouzi Harrag	Ferhat Abbas University, Algeria
Helmut Horacek	Saarland University, Germany
Ryutaro Ichise	Institute of Science Tokyo, Japan
Epaminondas Kapetanios	University of Hertfordshire, UK
Zoubida Kedad	University of Versailles Saint-Quentin-en-Yvelines, France
Natthawut Kertkeidkachorn	Japan Advanced Institute of Science and Technology, Japan
Christian Kop	University of Klagenfurt, Austria
Evgeny Kotelnikov	European University at St. Petersburg, Russia
Stefano Locci	Università degli Studi di Torino, Italy

Luigi Lomasto	Università degli studi di Salerno, Italy
Cédric Lopez	Emvista, France
Natalia Loukachevitch	Moscow State University, Russia
Vladimiro Lovera Rulfi	Alma Mater Studiorum - Università Di Bologna, Italy
Paloma Martínez Fernández	Universidad Carlos III de Madrid, Spain
Farid Meziane	University of Derby, UK
Alessio Miaschi	Institute for Computational Linguistics "A. Zampolli" (CNR-ILC), Italy
Luisa Mich	University of Trento, Italy
Nada Mimouni	Conservatoire National des Arts et Métiers Paris, France
Jelena Mitrović	University of Passau, Germany
Soto Montalvo	Universidad Rey Juan Carlos, Spain
Manuel Montes-y-Gómez	Instituto Nacional de Astrofísica, Óptica y Electrónica, Mexico
Dror Mughaz	Lev Academic Center, Israel
Benedetta Muscato	University of Pisa, Italy
Rafael Muñoz	University of Alicante, Spain
Roberto Nai	University of Torino, Italy
Rungsiman Nararatwong	National Institute of Advanced Industrial Science and Technology, Japan
Thanapon Noraset	Mahidol University, Thailand
Manjula Perkinian	VIT.chennai, India
Davide Picca	University of Lausanne, Switzerland
Stephan Reiff-Marganiec	University of Derby, UK
Mathieu Roche	Cirad, TETIS, France
Muhammad Yasir Shabir	University of Turin, Italy
Kiyoaki Shirai	Japan Advanced Institute of Science and Technology, Japan
Lucia Siciliani	University of Bari Aldo Moro, Italy
Krung Sinapiromsaran	Chulalongkorn University, Thailand
Irene Siragusa	Università degli Studi di Palermo, Italy
Giancarlo Sperlì	University of Naples Federico II, Italy
Vijayan Sugumaran	Oakland University, USA
Emilio Sulis	University of Turin, Italy
Maguelonne Teisseire	Irstea - UMR Tetis, France
Krishnaprasad Thirunarayan	Wright State University, USA
Federico Torrielli	University of Torino, Italy
Areerat Trongratsameethong	Chiang Mai University, Thailand
Dan Tufis	Institutul de Cercetări pentru Inteligență Artificială, Academia Română
L. Alfonso Ureña-López	University of Jaén, Spain

Tiberio Uricchio	University of Pisa, Italy
Sunil Vadera	University of Salford, UK
Peerapon Vateekul	Chulalongkorn University, Thailand
Gennaro Vessio	University of Bari, Italy
Sabrina Villata	Università degli Studi di Torino, Italy
Esau Villatoro-Tello	Idiap, Switzerland
Dittaya Wanvarie	Chulalongkorn University, Thailand
Konlakorn Wongpatikaseree	Mahidol University, Thailand
Ziwei Xu	Institute of Science Tokyo, Japan
Sumeth Yuenyong	Mahidol University, Thailand

Additional Reviewers

Rathachai Chawuthai	King Mongkut's Institute of Technology Ladkrabang, Thailand
Greta Damo	Université Côte d'Azur, France
Francesca Grasso	University of Turin, Italy
Rajalida Lipikorn	Chulalongkorn University, Thailand
Yun Liu	Kyoto Institute of Technology, Japan
Lankeshwara Munasinghe	Robert Gordon University, UK
Minh-Phuong Nguyen	Japan Advanced Institute of Science and Technology, Japan
Naruemon Pratanwanich	Chulalongkorn University, Thailand
Mohammad Golam Sohrab	National Institute of Advanced Industrial Science and Technology, Japan
Kamonphop Srisopha	Chiang Mai University, Thailand
Ekaterina Sviridova	Université Côte d'Azur, CNRS, Inria, I3S, France
Xiaoou Wang	Inria, CNRS, Université Côte d'Azur, France
Ratsameetip Wita	Chiang Mai University, Thailand
Nicola Zeni	University of Trento, University of Bergamo, Italy

Contents – Part II

Short Papers (Main Track)

Exploration of Rationale-Extraction Methods for Closed-Domain Question Answering with a New Sentence-Level Rationale Dataset 3
Lize Pirenne, Samy Mokeddem, Damien Ernst, and Gilles Louppe

Hybrid-SET: Few-Shot Example Selection Combining Sentence Similarity and Set Coverage - A Case Study on Material Science Domain 14
Chencheng Zhu, Tomoki Taniguchi, Tomoko Ohkuma, and Kazutaka Shimada

Efficiently Summarizing Norwegian Legal Texts 25
Tu My Doan, David Baumgartner, and Benjamin Kille

FairNM: Fairness in Name Matching 36
Yuan Liu and Flavius Frasincar

Is a Similar Task Useful for Few-Shot Selection? Aspect Term Extraction Using LLM 47
Koki Imazato and Kazutaka Shimada

IRIS: Rapid Curation Framework for Iterative Improvement of Noisy Named Entity Annotations 58
Ken Yano, Makoto Miwa, and Sophia Ananiadou

Fusion-in-LLM: A Parallel Approach to Improve Retrieval-Augmented Generation for Quiz-Based Question Answering 70
Yuuki Tachioka

Unveiling Emotional Signature in Conspiracy Topics on Social Media Through Vector Analysis 82
Luigi Lomasto, Nicola Lettieri, Delfina Malandrino, Valerio Mosca, and Rocco Zaccagnino

LLM-Supervised Multilingual Skill Extraction and Classification from Job Ads 94
Jakob Mørup Wang and Zhiru Sun

Cross-Domain Named Entity Recognition: A Resource-Efficient Transfer Learning Approach 105
Gianmaria Balducci, Elisabetta Fersini, and Enza Messina

Explainable AI for NLP: Enhancing Transparency in Sentiment Analysis and Named Entity Recognition 116
Feras Elkharrat, Mohamed Ghoniem, Ahmed Sherif, and Caroline Sabty

Utilization of SLMs as Generator in RAG 128
Kouya Abe and Hiroyuki Shinnou

Repurposing Annotation Guidelines to Instruct LLM Annotators: A Case Study 140
Kon Woo Kim, Rezarta Islamaj, Jin-Dong Kim, Florian Boudin, and Akiko Aizawa

Pretraining Data Exposure in Large Language Models: A Survey of Membership Inference, Data Contamination, and Security Implications 152
Ziyi Tong, Feifei Sun, and Le Minh Nguyen

EcoRAG: A Multi-hop Economic QA Benchmark for Retrieval Augmented Generation Using Knowledge Graphs 163
Hanieh Khorashadizadeh, Sanju Tiwari, Farah Benamara, Nandana Mihindukulasooriya, Jinghua Groppe, Soror Sahri, Morteza Ezzabady, Frédéric Ieng, and Sven Groppe

Short Papers (Industry Track)

Should Embedding-Based News Recommendation be Revisited? A Focus on the Differences Between News Publishers and Aggregators 177
Takumi Tamura, Yoichiro Ito, Masaki Aota, Kenta Yamada, and Shotaro Ishihara

A Simple Ensemble Strategy for LLM Inference: Towards More Stable Text Classification 189
Junichiro Niimi

Testing Procedure Generation Based on Testing Requirements for Automotive Components with LLM Fine-Tuning 200
Man Yiu Chow and Mitsuhiro Kitani

OpenThaiGPT 1.5: A Thai-Centric Open Source Large Language Model 210
Sumeth Yuenyong, Kobkrit Viriyayudhakorn, Apivadee Piyatumrong, and Jillaphat Jaroenkantasima

Demo Papers

ESG-Consultant: Developing of an ESG Compliance Consulting Tool for Companies Using RAG 223
Angel Ontiveros, Irina Nikishina, Moritz Gomm, Christopher Schmitt, and Chris Biemann

CaLuX: A Catalyst and Lubricant Properties Extraction System for Domain Experts 230
Catalina Riano, Fumian Chen, and Hui Fang

Author Index 237

Contents – Part I

Full Papers

Multi-scale Convolutional Fusion with Contrastive Feature Alignment for Imbalanced Data Classification 3
Keito Inoshita, Takato Ueno, and Xiaokang Zhou

Memory Efficient LM Compression Using Fisher Information from Low-Rank Representations 19
Daniil Moskovskiy, Sergey Pletenev, Sergey Zagoruyko, and Alexander Panchenko

Accelerating End-to-End PDF to Markdown Conversion Through Assisted Generation 34
Changxu Duan

Comparison of Pipelines, Seq2seq Models, and LLMs for Rare Disease Information Extraction 49
Shashank Gupta, Xuguang Ai, Yuhang Jiang, and Ramakanth Kavuluru

A Multiple Graphical Convolution Networks Approach for Aspect-Based Sentiment Classification 64
Diana Kazakova, Flavius Frasincar, and Jasmijn Klinkhamer

How Important is Domain-Specific Language Model Pretraining and Instruction Finetuning for Biomedical Relation Extraction? 80
Aviv Brokman and Ramakanth Kavuluru

ShortPathQA: A Dataset for Controllable Fusion of Large Language Models with Knowledge Graphs 95
Mikhail Salnikov, Andrey Sakhovskiy, Irina Nikishina, Aida Usmanova, Angelie Kraft, Cedric Möller, Debayan Banerjee, Junbo Huang, Longquan Jiang, Rana Abdullah, Xi Yan, Elena Tutubalina, Ricardo Usbeck, and Alexander Panchenko

Explaining Bias in Internal Representations of Large Language Models via Concept Activation Vectors 111
Jasper Kyle Catapang

Duplicate Bug Report Retrieval for New Bug Reports 126
Miao Hu, Zhiwei Lin, and Adele H. Marshall

Context is the Key for LLM-Based Text Segmentation 142
Amit Maraj and Miguel Vargas Martin

MultiProSE: A Multi-label Arabic Dataset for Propaganda, Sentiment, and Emotion Detection 156
Lubna Al-Henaki, Hend Al-Khalifa, Abdulmalik Al-Salman, Hajar Alqubayshi, Hind Al-Twailay, Gheeda Alghamdi, and Hawra Aljasim

CLIP-Driven Deep Hashing for Cross-Modal Retrieval 173
Zhichao Han, Azreen Azman, Mas Rina Mustaffa, and Fatimah Khalid

WIP: Iterative Post-training Pruning with Weighted Importance Estimation for Large Language Models 186
Dinh-Truong Do, Kiyoaki Shirai, and Le-Minh Nguyen

A Comparative Study on the Development of a Thai Legal QA Framework Using Large Language Models and Mixed Legal Datasets 201
Supachoke Hanwiboonwat, Chaichana Thavornthaveekul, Prachya Boonkwan, Apivadee Piyatumrong, and Peerapon Vateekul

Leveraging Generative AI for Chronic Kidney Disease Prediction: A Comparative Study of Open-Source LLMs 216
B. Abhin, Hemanth Kumar M, Sowmya Kamath S, and Vijayan Sugumaran

LADDER: Latent Attention and Decomposition for Deep Enhanced Retrieval in Medical Question-Answering Systems 229
Sidhaarth Murali, Sowmya Kamath S, and Vijayan Sugumaran

Test It Before You Trust It: Applying Software Testing for Trustworthy In-Context Learning 243
Teeradaj Racharak, Chaiyong Ragkhitwetsagul, Chommakorn Sontesadisai, and Thanwadee Sunetnanta

CAFE: Context-Aware Applicability-Weighted Fairness Evaluation 259
Fumian Chen and Hui Fang

A Survey of LoRA Algorithm Variations for Language Models 275
Mengyao Zhu and Phuc Huu Nguyen

ViMRHP: A Vietnamese Benchmark Dataset for Multimodal Review Helpfulness Prediction via Human-AI Collaborative Annotation 291
Truc Mai-Thanh Nguyen, Dat Minh Nguyen, Son T. Luu, and Kiet Van Nguyen

Towards an Ontological Approach to Browser Fingerprinting Detection and Privacy Risk Assessment 306
Christopher D. McDermott, Lankeshwara Munasinghe, and Mathew Nicho

Dual-Style Transcription of Historical Manuscripts Based on Multimodal Small Language Models with Switchable Adapters 321
Sergio Torres Aguilar

Semantic Reasoning for Device Identification in Dynamic Internet of Things Environments with Large Language Models 335
Zhou Gui, Layla Kuty, Daniel Schraudner, and Andreas Harth

Instruction Tuning TextFlow Semi-automatic RFCs Generation 350
Jie Bian and Michael Welzl

News Timeline Summarization: Recent Methods 365
Vandana Yadav, Jon Atle Gulla, Özlem Özgöbek, and Lemei Zhang

QA2HALL: A Framework for Generating Non-trivial Hallucination Detection Datasets from KGQA Datasets 380
Kosuke Nakamura, Rie Hasegawa, Kotaro Otomura, Ryutaro Ichise, and Jumpei Hato

Causal Relation-Aware Data Augmentation for Legal Textual Entailment 396
Huy Chu, Hoang Chu, Phuong Nguyen, and Minh Nguyen

Improve Smart Contract Vulnerability Explanation with Synthetic Data and Chain-of-Thought Prompting 411
Ngoc Minh Nguyen, Naoya Inoue, and Le Minh Nguyen

The Benefits of Query-Based KGQA Systems for Complex and Temporal Questions in LLM Era 426
Artem Alekseev, Mikhail Chaichuk, Miron Butko, Alexander Panchenko, Elena Tutubalina, and Oleg Somov

TopicVD: A Topic-Based Dataset of Video-Guided Multimodal Machine Translation for Documentaries 442
Jinze Lv, Jian Chen, Zi Long, Xianghua Fu, and Yin Chen

Multimodal Event Detection: Current Approaches and Defining the New Playground Through LLMs and VLMs 457
Abhishek Dey, Aabha Bothera, Samhita Sarikonda, Rishav Aryan, Sanjay Kumar Podishetty, Akshay Havalgi, Saurabh Srivastava, and Gaurav Singh

ReSHQ: Re-ranking Based on the Similarity Between Hypothetical Query and Query 472
Keito Fukuoka and Hisashi Miyamori

Multi-hop Question Answering in SlideVQA Based on Beam Retrieval 484
Mizuki Yamano and Hisashi Miyamori

Author Index 501

Short Papers (Main Track)

Exploration of Rationale-Extraction Methods for Closed-Domain Question Answering with a New Sentence-Level Rationale Dataset

Lize Pirenne(✉), Samy Mokeddem, Damien Ernst, and Gilles Louppe

University of Liège, Liège, Belgium
{lize.pirenne,samy.mokeddem}@uliege.be

Abstract. In this paper, we address the problem of Rationale Extraction (RE) from Natural Language Processing: given a context (C), a related question (Q) and its answer (A), the task is to find the best sentence-level rationale (R^*). This rationale is loosely defined as being the subset of sentences of the context C such that producing A would require at least R^*. We have constructed a dataset where each entry is composed of the four terms (C, Q, A, R^*) to explore different methods in the particular case where the answer is one or multiple full sentences. The methods studied are based on TF-IDF scores, embedding similarity, classifiers and attention and have been evaluated using a sentence overlap metric akin to the Intersection over Union (IoU). Results show that the best scores were achieved by the classifier-based approach with the nuance of a better scaling with the attention-based method as the size of the context increases, which is a challenge for all other methods. We also show that generating A significantly decreases the performance of the attention-based method, but training the model to generate A can improve the results, linking the ability to generate with the accomplishment of the task.

1 Introduction

Reliable Question and Answer (QA) systems are as useful as they are challenging to implement. Even in the Closed-Domain Question Answering (CQA) task, where the answer is restricted by the information explicitly provided within the context, hallucinations can be interleaved in or substitute the answer sought; the smallest set of sentences necessary to produce a correct answer, called the rationale, can be different from what was actually used to produce the answer.

Extracting the rationale of an answer A from a given context C and a question Q offers significant benefits for CQA systems [29]. Indeed, they enhance explainability: by identifying the rationale behind an answer, users can gain insights into the decision-making process of the underlying model of the system and assess its reliability. This is particularly valuable in domains demanding high levels of trust and transparency. Furthermore, finding the rationale can potentially improve the quality of generated responses. For example, research suggests that leveraging the rationale during prompt

L. Pirenne and S. Mokeddem—Equal contribution.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 3–13, 2026.
https://doi.org/10.1007/978-3-031-97144-0_1

engineering can lead to better generation outcomes [16]. Others implicitly compare their generation against the rationale to sample away from hallucinations [6].

The objectives of our research is three-folds: we want to find the most appropriate method to perform rationale extraction in CQA, use rationales to guide models to find rationales better, and explore the limitations these methods face when context grows in length. The importance of the last point can be motivated by previous studies on large language models (LLMs) that have shown repeated weaknesses with increasing context size [28]. We provide the following contributions:

- We bring additional annotations to a dataset such that it becomes tailored for sentence-level rationale extraction in closed-domain question answering with full-sentence answers.
- We reveal increased rationale finding capabilities of the attention-based method when the underlying model is trained to generate the similar answer, and further improvement when the model is trained with reinforcement learning (RL) using the rationale in the reward.
- We study the effect of increasing the number of sentences in the context on performance and compare selection characteristics of the methods such as whether they use a threshold or a ranking approach.

1.1 Related Work

Explainability. The majority of the methods explored here (all except the attention-based methods) can also be classified as attribution-based explanations using surrogate models. Surrogate model-involving methods are also known as post hoc explanation methods, as discussed in AMPLIFY [16]. Our work also aligns with the category of local explanation models defined in Zhao [34].

Rationale Extraction in Natural Language Processing. Similarly to Moradi [22], we use an attention-based method to extract the rationale, although their use is for machine translation as a regularisation parameter. Moreover, as in Glockner [13] some of the methods proposed in our work aim to extract the k most relevant sentences from the context based on a relevance measurement while others use a more traditional threshold. Lamm [17] calls this rationale extraction task explanation prediction.

GopherCite [21] produces the rationale in line by adding special tokens and learning to produce exact quotes between them. RAFT [33] uses a special sequence to indicate the start and end of the rationale while LongCite [32] improve this technique by adding sentence numbers in the context to refer to. These methods rely on the model to be truthful and complete, which is not the case in our work.

Explanation Regularisation. Explanation Regularisation (ER) [15] explores how rationale can be used to provide supplementary training objectives for models. This can involve techniques such as introducing loss penalties that encourage the model to focus on informative parts of the context [25] or enforcing attention sparsity to prevent the model from becoming overwhelmed with excessive information [22]. Frameworks like UNIREX [4] demonstrate how these methods that leverage rationales can be

integrated into a larger system for improved CQA performance. Similarly, in our reinforcement learning attention-based method, we have regularised the reward with our rationale metric.

Datasets for Rationale Extraction. For complete sentence answers and rationales, and to the best of our knowledge, we have found five datasets. QED [17] that has the strong assumption of there being only one sentence for the rationale which is rarely the case in our own examples. There are also QuoRef [9] and QuAC [5] which are more focused on solving co-references. Hotpot-QA [31] is good for our task but is quite challenging for smaller models due to the objective of using multi-hop reasoning and LooGLE [18] defines the rationales but the tracing it back in the text is not straightforward due to formatting inconsistencies. Not being able to find a dataset that fits our needs, we have created our own.

2 Problem Statement

Given the triplet question-context-answer (Q, C, A), we are interested in finding a method that uses this triplet to produce a good approximation R of the best rationale $R^* \subset C$ to explain A. More formally, let $\mathcal{M}$ the set of all methods taking (Q, C, A) as input and outputting a subset of sentences R in the context C ($R \subset C$). The objective is to find the method $M \in \mathcal{M}$ that provides a good approximation $R = M(Q, C, A)$ of R^*.

To identify a high-performing method $M \in \mathcal{M}$, we have at our disposal a training set $TS = \{(Q_i, C_i, A_i, R_i^*)\}_{i=1}^N$ where each sample is composed of the (i) question, (ii) context, (iii) answer and (iv) rationale. The quality of the approximated rationale R will be assessed using the Intersection-over-Union (IoU) score defined by

$$IoU(R,\ R^*) = \frac{|R \cap R^*|}{|R \cup R^*|}, \tag{1}$$

where the operator $|\ |$ in Eq. 1 gives the number of character in all sentences in the set it operates on, $\cup$ outputs the set of sentences that appear in at least one operand, and $\cap$ computes the set of sentences appearing in both operands. This metric is also known as the Jaccard coefficient/index/similarity.[1]

This choice of metric was motivated by its use in DeYoung [11] but we differ in that we work on characters rather than tokens and our R and R^* always correspond to complete sentences. We prefer working on sentences because we believe they are more interpretable for end-users and are easier to annotate.

3 Methods

In this section, we will explain the four different methods, named Embedding Similarity, TF-IDF, LLM classifier and LLM attention, that will be later used in the experiments.

[1] We note that this IoU is equivalent to $\frac{1}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}} - 1}$ if both precision and recall are also defined per character.

3.1 TF-IDF

The first class of methods tested uses Term Frequency-Inverse Document Frequency (TF-IDF) [26]. Each column of the TF-IDF matrix corresponds to a term (word) in the vocabulary and each row is a document. The value represents the TF-IDF score of the corresponding term in the document. This score, derived from the Term Frequency (TF) and Inverse Document Frequency (IDF) values, highlights terms that are prevalent within a document but rare across the corpus, thereby underlining their significance within that document.

We have defined two methods using two different cutoff functions: the first chooses the top k_{emb} sentences with the highest scores and the other picks all sentences above a certain threshold t_{emb}. They both use the **Score** described in Eq. 2, where TF-IDF() is a function that takes a sentence as input and returns a vector $v \in \mathbb{R}^{d_{voc}}$, with d_{voc} being the size of the vocabulary.

$$\textbf{Score}_{\text{NG}}(A,\, C) = \text{cos_sim}(\text{TF-IDF}(A), \text{TF-IDF}(s)) \tag{2}$$

3.2 Embedding Similarity

The second class of methods tested uses sentence-embeddings based on LLMs pre-trained for Semantic Textual Similarity, which aims to determine the degree of similarity between two pieces of text.

To generate the embedding of a sentence using an LLM, the most commonly employed approaches are to either average the final hidden vectors (before the classification layer of a classical causal Transformer LLM) of the tokens in the sentence, or simply pool the final hidden vector of the special first token (the [CLS] token). We have chosen to use the latter.

Similarly to the first method, we analysed both a cutoff and a ranking approach. Both operate on the **Score** computed in Eq. 3 using the cosine similarity between the vectors $v \in \mathbb{R}^{d_{emb}}$, with d_{emb} the size of the embedding, returned by Emb() that takes a sentence as input.

$$\textbf{Score}_{\text{Embed}}(A,\ s \in C, C) = \text{cos_sim}(\text{Emb}(A), \text{Emb}(s)) \tag{3}$$

3.3 LLM Classifier

The third class of method that will be used in our experiments is inspired by Sun [30] and Chae [3]. It involves fine-tuning a pre-trained LLM for binary text classification with the rationale as a class. The input of the classifier consists in the concatenation of the sentence to be classified (s) surrounded by its neighbouring sentences ($N(s)$), the answer (A), and the question (Q). This method can be formalised as:

$$\textbf{LLM}_{\text{Classifier}}(Q,\, C,\, A) = \left\{ s \in C \;\middle|\; \text{classify}((N(s),\, A,\, Q))^p > 0.5 \right\} \tag{4}$$

where p denotes the positive label of the soft-maxed output of classify$((s,\ A,\ Q))$, representing the LLM classifier. We only use the first left and right neighbouring sentences of s to avoid the model being overwhelmed by the context but this assumption was not tested.

3.4 LLM Attention

This last method is based upon the attention mechanism present in most LLMs. The attention relates two parts of the input together with a numerical value, akin to a correlation matrix. Incidentally, it is of the form $N \times N$, where N is the number of parts in the input. These parts are called tokens. The attention mechanism is replicated multiple times in a single layer, all with different weights (multi-head attention). This means that for a given model with L layers, H heads and for each token produced, there are $L \times H$ attention results to consider, each attending to different parts of the input and enabling it to understand different linguistic features [7].

We start from the internal values of attention $a(i,j)$ per token of the answer $i \in T(A)$ and of the context $j \in T(C)$, where $T()$ is the tokenizer. We average these values over the tokens of the answer to have only one per token of the context by following the equation: $\mathtt{A}(j) = \frac{1}{|T(A)|} \sum_{i \in T(A)} a(i,j)$. We get the following criterion:

$$\mathbf{LLM}_{Att}(A,Q,C) = \underset{s \text{ in } C}{\text{Top-K}} \left(\left\{ s \;\middle|\; \left(\frac{1}{|T(s)|} \sum_{j \in T(s)} \mathtt{A}(j) \right) > t_{att} \right\} \right). \tag{5}$$

We note that the LLM can itself generate an answer A' that replaces A using only Q and C. Thus producing the equation $\mathbf{LLM}_{Att}(Q,C) = \mathbf{LLM}_{Att}(A',Q,C)$.

4 Experiments

4.1 Dataset

Data Source and Filtering. We chose the closed-QA part of the `databricks-dolly-15k` [8] as our base CQA dataset. We filtered[2] the triplets (Q,C,A) in the dataset by excluding those where the answer A did not respond to the question Q strictly using the context C or when the answer was wrong. When little change was required to avoid discards (e.g., deleting a sentence, adding a word,...), we tried applying those instead. This filtered dataset contains 1595 triplets.

Dataset Utilisation. The Rationale Databricks Dolly CQA (RDD) dataset has been shuffled using the same random seed for all experiments to ensure consistency, and then divided into three sets: the training, validation and evaluation set. They represent respectively 80%, 10% and 10% of the original dataset. The training set has been used to train the methods, the validation set to fine-tune the parameters, and lastly the evaluation to compare their performance.

[2] Using Doccano [23].

We decided to split the data points into four categories based on the number of sentences in the context. To do so, we created four intervals $[1;3]$, $[4;6]$, $[7;10]$ and $[11;\inf]$ of different sizes to keep the number of samples in each one comparable. In particular, the last category covers all triplets above ten sentences that have less than 2048 tokens for Q and C.

4.2 Training

All methods explored were able to run on our two Nvidia 2080tis and will be further explained in this section. To achieve this, for the methods necessitating training, we have used Low Rank Adaptation (LoRA) [14] together with quantisation (called QLoRA [10]).

TF-IDF. Since this method does not centre on a model, we did not have to make any specific choice other than the hyper-parameters, for which we have made the same sweep as for the embedding method.

Embedding. For the experiments based on the embeddings, we utilised two pre-trained LLMs: `Sentence-Bert` [24] and `SFR-Embedding-Mistral` [20]. To choose the appropriate hyper-parameters, we have swept over $k \in [1 \dots 5]$ and $t \in [0.1 \dots 0.9]$ (90 steps).

LLM Classifier. We have finetuned the different pre-trained models[3] over 20 epochs using the standard cross-entropy loss. The selection of the best models and checkpoint is based on accuracy; our IoU metric is only used during evaluation. The hyper-parameter selection is primarily based on empirical results, the final parameters used for all fine-tuned models are the following: Learning rate=5e-5, LoRa rank=4, alpha LoRa=4, LoRa dropout=0.1.

LLM Attention. There are three variations of the attention-based method. The first (suffixed *Base* in tables and graphs) is the base pre-trained model (`Gemma-2B`). The second (*FT*) is fine-tuned[4] for one epoch on the base CQA dataset by performing a standard causal language modelling training with Q, C and A always present concurrently. The third (*RL*) is an RL-tuned[5] version of the second, where the reward is the average of the IoU score and the METEOR metric [2] with flat penalty for not including the *EOS* token.

Due to hardware constraints, we have limited the sizes of the examples to $|(Q, C)| < 2048$ and $|A| < 542$ such that the total number of tokens respected $|(Q, C, A)| < 2600$. For the RL training, these values are respectively 450, 50 and 500. The prompts have been formatted by adding "### Question:", "### Context:",

[3] `Gemma-2B` [12], `DistilBERT` [27] and `RoBERTa-Base` [19].
[4] via Huggingface:Trainer.
[5] via HuggingFace:TRL:PPOTrainer.

"### Answer:" and "### End" separated by double line breaks in order to provide a clear description of the task.

The attention computation of Eq. 5 requires the *head*, *layer*, *threshold* and k to be set. To do so, we have swept over all eight heads, 18 layers for $k \in [0 \dots 4]$ (0 indicates no restriction) and $t \in [0.006, \dots, 0.001, 0.0005, 0.0003, \dots, 0.0]$ to compute the average score on the validation set and took the best combination of parameters. The range for t was motivated by an analysis of reoccurring values.

4.3 Results

Best Method. As can be seen in Table 1, summarising the performance of all methods over the entire test set, the best method on the dataset is the Gemma classifier, which is the largest of the models tested for that category. However, when more sentences are present in the context, the attention method seems to scale better.

Table 1. Summary table of experiment results, including the number of parameters for each method (size) and the average IoU score obtained on the evaluation set, presented with a 95% confidence interval assuming a student-t distribution.

Model	Size	IoU
Sentence-Bert-large (k_{emb} = 1)	109M	0.61 ± 0.05
Sentence-Bert-large (t_{emb} = 0.68)	109M	0.54 ± 0.06
SFR-Embedding-Mistral (k_{emb} = 1)	7.11B	0.65 ± 0.05
SFR-Embedding-Mistral (t_{emb} = 0.72)	7.11B	0.59 ± 0.06
TF-IDF (k_{tf-idf} = 1)	/	0.64 ± 0.05
TF-IDF (t_{tf-idf} = 0.25)	/	0.66 ± 0.05
Classifier DistilBERT	67M	0.64 ± 0.06
Classifier RoBERTa	125M	0.75 ± 0.05
Classifier Gemma	2.51B	$\mathbf{0.79 \pm 0.04}$
Gemma Base (L = 5, H = 4, k = 0, t = 0.002)	2.51B	0.74 ± 0.05
Gemma FT (L = 8, H = 6, k = 0, t = 0.002)	2.51B	0.75 ± 0.05
Gemma RL (L = 5, H = 4, k = 0, t = 0.002)	2.51B	0.76 ± 0.05
+ Generation of Answer (Gen)		
Gemma Base (L = 14, H = 7, k_{att} = 2, t_{att} = 0.000)	2.51B	0.53 ± 0.05
Gemma FT (L = 8, H = 6, k_{att} = 0, t_{att} = 0.002)	2.51B	0.60 ± 0.06
Gemma RL (L = 8, H = 6, k_{att} = 0, t_{att} = 0.002)	2.51B	0.61 ± 0.06

Influence of Hyper-Parameters. The sweep of hyper-parameters has shown that embedding methods using a ranking approach perform best with small k values (i.e., 1). This is likely due to the skew of the dataset for smaller numbers of sentences. The small values of k being preferred may also be the reason why TF-IDF achieves better

scores than the embedding methods. This is not the case for the attention method, for which the best score is often achieved with $k = 0$ (i.e. no limit).

Influence of Training. The results of Table 1 demonstrate that the attention method performs better after training the LLM model when generating the answer. The impact of the training is less noticeable when the golden answer is provided, meaning modern LLMs can have good results as post-hoc checkers. RL training does not significantly improve the results on the evaluation or test set despite showing a 5% absolute increase at training time. This can indicate that the SFT training was sufficient and going beyond would only over-fit.

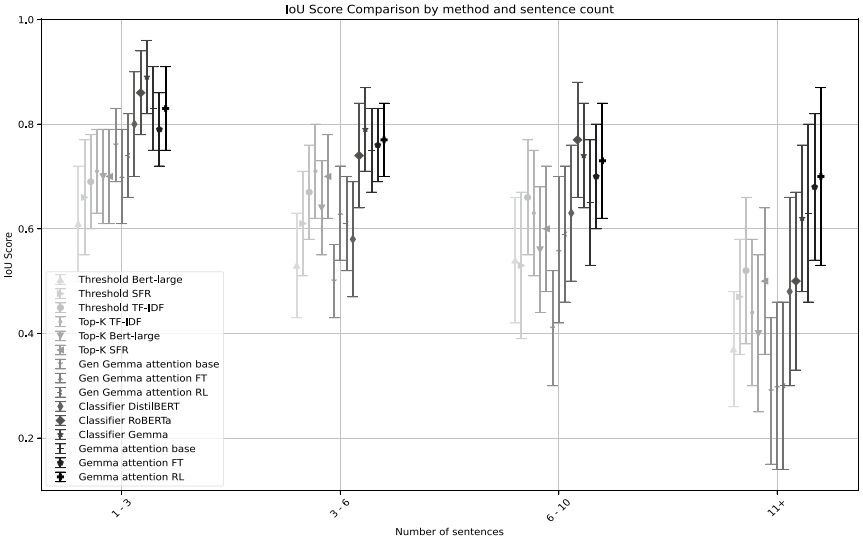


Fig. 1. IoU scores with 95% confidence error bars (student-t).

Influence of the Number of Sentences in the Context. As shown in Fig. 1, for all methods, as the number of sentences in the context increases, the performance of the models decrease. This supports the results of [1] who reported (converted from Precision and Recall) over three datasets of increasing size IoUs of 0.89, 0.66 and 0.59. For comparison, the projected, context size weighted, scores of our RL attention would be 0.80, 0.71 and 0.70.

5 Conclusion and Limitations

In this paper, we have underlined the difficulty in reaching satisfying scores as the scale of the context grows. Nonetheless, we have found that classifier models could

achieve an IoU of **79%** and the attention with reinforcement learning followed closely with **76%**, which is on a par with previous work for smaller sizes of contexts (a few sentences) but is projected to scale slightly better as the number of sentences grows.

We have constructed the CQA Rationale Databricks Dolly dataset[6] for the express purpose of improving sentence-level rationale extraction in closed-domain question answering where the rationale to the answer appears as full sentences. One thing to note is that it has been annotated by us and thus may not have been reviewed impartially and/or in sufficient depth.

This research calls for several future works. First, in the produced dataset there have been numerous instances of questions that contained sub-questions while taking care of differentiating the sub-answers and corresponding sub-rationales. This is of particular interest because no other dataset reviewed seemed to differentiate this case.

Second, it may be interesting to see other approaches compete, such as gradient-based methods, to see how the scores could be improved. The methods studied do not cover all existing ones and should not be regarded as an exhaustive comparison but rather an educated guess of potentially well-performing methods.

Finally, it might be interesting to address the issue of finding both A and R similarly to our generation case. To do this, reward regularisation, context filtering or compression might be the most straightforward of approaches as has been shown empirically in this and related works.

Acknowledgments. Lize Pirenne gratefully acknowledges the financial support of the Walloon Region for Grant No. 2010235 - ARIAC by DW4AI.

Disclosure of Interests. The authors declare no conflict of interest.

References

1. Atanasova, P., Simonsen, J.G., Lioma, C., Augenstein, I.: Diagnostics-guided explanation generation. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, no. 10, pp. 10445–10453 (2022). https://doi.org/10.1609/aaai.v36i10.21287
2. Banerjee, S., Lavie, A.: METEOR: an automatic metric for MT evaluation with improved correlation with human judgments. In: Goldstein, J., Lavie, A., Lin, C.Y., Voss, C.R. (eds.) Proceedings of the Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization@ACL 2005, Ann Arbor, Michigan, USA, 29 June 2005, pp. 65–72. Association for Computational Linguistics (2005)
3. Chae, Y., Davidson, T.: Large language models for text classification: from zero-shot learning to fine-tuning. Open Science Foundation (2023)
4. Chan, A., et al.: UNIREX: a unified learning framework for language model rationale extraction. In: Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., Sabato, S. (eds.) Proceedings of the 39th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 162, pp. 2867–2889. PMLR (2022)
5. Choi, E., et al.: QuAC: question answering in context. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 2174–2184. Association

[6] Available on Huggingface hf:Inversta/rationale-databricks-dolly-cqa. Code available on github:Inversta/rationale_extraction.

for Computational Linguistics, Brussels, Belgium (2018). https://doi.org/10.18653/v1/D18-1241
6. Chuang, Y.S., Qiu, L., Hsieh, C.Y., Krishna, R., Kim, Y., Glass, J.: Lookback lens: detecting and mitigating contextual hallucinations in large language models using only attention maps. arXiv preprint arXiv:2407.07071 (2024)
7. Clark, K., Khandelwal, U., Levy, O., Manning, C.D.: What does BERT look at? An analysis of BERT's attention. arXiv preprint arXiv:1906.04341 (2019)
8. Conover, M., et al.: Free dolly: introducing the world's first truly open instruction-tuned LLM (2023). https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm
9. Dasigi, P., Liu, N.F., Marasović, A., Smith, N.A., Gardner, M.: Quoref: a reading comprehension dataset with questions requiring coreferential reasoning. arXiv preprint arXiv:1908.05803 (2019)
10. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: Qlora: efficient finetuning of quantized LLMs. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) Advances in Neural Information Processing Systems, vol. 36, pp. 10088–10115. Curran Associates, Inc. (2023)
11. DeYoung, J., et al.: Eraser: a benchmark to evaluate rationalized NLP models. arXiv preprint arXiv:1911.03429 (2019)
12. Gemma, T.: Gemma: Open Models Based on Gemini Research and Technology (2024)
13. Glockner, M., Habernal, I., Gurevych, I.: Why do you think that? Exploring faithful sentence-level rationales without supervision. In: Cohn, T., He, Y., Liu, Y. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1080–1095. Association for Computational Linguistics, Online (2020). https://doi.org/10.18653/v1/2020.findings-emnlp.97
14. Hu, E.J., et al.: LoRA: low-rank adaptation of large language models. In: The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, 25–29 April 2022. OpenReview.net (2022)
15. Joshi, B., et al.: ER-test: evaluating explanation regularization methods for language models. In: Goldberg, Y., Kozareva, Z., Zhang, Y. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2022, Abu Dhabi, United Arab Emirates, 7–11 December 2022, pp. 3315–3336. Association for Computational Linguistics (2022). https://doi.org/10.18653/V1/2022.FINDINGS-EMNLP.242
16. Krishna, S., Ma, J., Slack, D., Ghandeharioun, A., Singh, S., Lakkaraju, H.: Post hoc explanations of language models can improve language models. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, 10–16 December 2023 (2023)
17. Lamm, M., et al.: QED: a framework and dataset for explanations in question answering. Trans. Assoc. Comput. Linguist. **9**, 790–806 (2021). https://doi.org/10.1162/tacl_a_00398
18. Li, J., Wang, M., Zheng, Z., Zhang, M.: Loogle: can long-context language models understand long contexts? arXiv preprint arXiv:2311.04939 (2023)
19. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., et al.: Roberta: a robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692 (2019)
20. Meng, R., Liu, Y., Joty, S.R., Xiong, C., Zhou, Y., Yavuz, S.: Sfrembedding-mistral: enhance text retrieval with transfer learning. Salesforce AI Res. Blog **3**, 6 (2024)
21. Menick, J., et al.: Teaching language models to support answers with verified quotes. arXiv preprint arXiv:2203.11147 (2022)
22. Moradi, P., Kambhatla, N., Sarkar, A.: Measuring and improving faithfulness of attention in neural machine translation. In: Merlo, P., Tiedemann, J., Tsarfaty, R. (eds.) Proceedings

of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume, pp. 2791–2802. Association for Computational Linguistics, Online (2021). https://doi.org/10.18653/v1/2021.eacl-main.243
23. Nakayama, H., Kubo, T., Kamura, J., Taniguchi, Y., Liang, X.: doccano: text annotation tool for human (2018). https://github.com/doccano/doccano. Accessed 02 Mar 2024
24. Reimers, N., Gurevych, I.: Sentence-BERT: sentence embeddings using Siamese BERT-networks. In: Conference on Empirical Methods in Natural Language Processing (2019). https://doi.org/10.18653/v1/D19-1410
25. Ross, A.S., Hughes, M.C., Doshi-Velez, F.: Right for the right reasons: training differentiable models by constraining their explanations. In: Sierra, C. (ed.) Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, 19–25 August 2017, pp. 2662–2670. ijcai.org (2017). https://doi.org/10.24963/IJCAI.2017/371
26. Salton, G., Buckley, C.: Term-weighting approaches in automatic text retrieval. Inf. Process. Manag. **24**(5), 513–523 (1988). https://doi.org/10.1016/0306-4573(88)90021-0
27. Sanh, V., Debut, L., Chaumond, J., Wolf, T.: Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108 (2019)
28. Shi, F., et al.: Large language models can be easily distracted by irrelevant context. In: Proceedings of the 40th International Conference on Machine Learning, pp. 31210–31227. PMLR (2023)
29. Sun, J., Swayamdipta, S., May, J., Ma, X.: Investigating the benefits of free-form rationales. In: Goldberg, Y., Kozareva, Z., Zhang, Y. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2022, Abu Dhabi, United Arab Emirates, 7–11 December 2022, pp. 5867–5882. Association for Computational Linguistics (2022). https://doi.org/10.18653/V1/2022.FINDINGS-EMNLP.432
30. Sun, X., et al.: Text classification via large language models. In: Bouamor, H., Pino, J., Bali, K. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, 6–10 December 2023, pp. 8990–9005. Association for Computational Linguistics (2023). https://doi.org/10.18653/V1/2023.FINDINGS-EMNLP.603
31. Yang, Z., et al.: HotpotQA: a dataset for diverse, explainable multi-hop question answering. In: Conference on Empirical Methods in Natural Language Processing (EMNLP) (2018)
32. Zhang, J., et al.: Longcite: enabling LLMs to generate fine-grained citations in long-context QA. arXiv preprint arXiv:2409.02897 (2024)
33. Zhang, T., et al.: Raft: adapting language model to domain specific rag. arXiv preprint arXiv: 2403.10131 (2024)
34. Zhao, H., et al.: Explainability for large language models: a survey. ACM Trans. Intell. Syst. Technol. **15**(2) (2024). https://doi.org/10.1145/3639372

Hybrid-SET: Few-Shot Example Selection Combining Sentence Similarity and Set Coverage - A Case Study on Material Science Domain

Chencheng Zhu[1,2(✉)], Tomoki Taniguchi[2], Tomoko Ohkuma[2], and Kazutaka Shimada[1]

[1] Kyushu Institute of Technology, Fukuoka, Japan
shimada@ai.kyutech.ac.jp
[2] Asahi Kasei Corporation, Tokyo, Japan
{zhu.cd,taniguchi.tcr,okuma.td}@om.asahi-kasei.co.jp

Abstract. Named entity recognition (NER) in the domain of materials science and chemistry presents significant challenges, including limited availability of annotated training data and the necessity for domain-specific expertise during the annotation process. Large language models (LLMs) demonstrate the capability to perform various tasks with minimal labeled examples, a technique known as in-context learning (ICL). However, the ICL performance is highly sensitive to the given examples, highlighting the importance of an effective selection strategy. This paper introduces Hybrid-SET, a novel selection approach that combines example selection with sentence representation similarity and set-level coverage. Experimental results indicate that Hybrid-SET surpasses both conventional supervised methods and existing selection methods. Notably, the performance exhibits a significant improvement in recognizing domain-specific entities.

Keywords: Large language model · Named entity recognition · Material science

1 Introduction

Over the past several decades, there has been significant exponential growth in the volume of published materials science literature. However, historical knowledge in materials science is contained in unstructured text. Valuable data remain buried deep in extensive publications, posing difficulties for researchers in retrieving relevant information efficiently [11].

Recent advancements in pre-trained language models (PLMs), applicable to both general and scientific domains, have illustrated significant abilities in scientific NER [2,5,19]. Notwithstanding these developments, the field continues to encounter a diverse array of challenges. Generally, a large amount of high-quality labeled data is required as the training data to achieve high performance.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 14–24, 2026.
https://doi.org/10.1007/978-3-031-97144-0_2

However, the availability of publicly accessible corpora is constrained, resulting in a problem of data scarcity. Furthermore, annotating the domain-specific documents requires specialized knowledge, thereby increasing the costs of creating labeled data. Although several open-source corpora do exist [8,12], the annotation guidelines for each specific corpus vary even within the same subdomain. This variability presents a challenge to their general applicability as machine learning datasets.

The most recent cycle of NLP innovations with the emergence of large language models (LLMs), such as GPT-3 [3] and Llama 3 [7], among others, highlights their noteworthy performance across various NLP tasks [14]. Some research indicated that LLMs are capable of automatic annotation with accuracy approaching human levels [6,9]. Unlike the conventional supervised-based methods, LLMs can adapt to new tasks and domains through limited demonstrations (few-shot examples) without updating their parameters. This paradigm, termed in-context learning (ICL) and initially introduced by Brown et al. [3], has attracted attention due to its demonstrated potential to circumvent extensive data labeling requirements.

Meanwhile, many prior studies argue that ICL is sensitive to the provided few-shot examples. Noticeable fluctuations in performance have been reported when varied randomly chosen demonstrations are incorporated into the prompt [16,17,23]. Therefore, the selection strategy for few-shot examples plays an important role in fully harnessing the potential of LLMs. The prevalent methodology involves retrieving examples with high relevance to the test example from a pool of labeled few-shot candidates. These candidates are independently evaluated using predefined relevance metrics, and the top-ranked ones will be chosen. Liu et al. [17] used the semantic similarity of sentences as a measure of relevance and retrieved similar examples from the training data using kNN. However, potential challenges can be considered when adapting this approach to domain-specific information extraction tasks. (1) Since the semantic representation of a sentence assigns equal importance to all tokens, it is unsuitable for NER, which prioritizes potential entities and semantically related tokens [21]. (2) Given that each demonstration is selected independently, this method overlooks the utility of few-shot examples as a cohesive set [15]. In other words, the relationships among the selected examples are not taken into account. Consequently, there exists a potential issue of selecting examples that convey similar information, leading to inadequate coverage [10].

In this paper, we introduce a SET-BSR proposed by Gupta et al. [10], to address these two issues. SET-BSR leverages BERTScore [22] to evaluate relevance through contextual token-level similarity, rather than sentence-level similarity. Additionally, this method utilizes a greedy algorithm to optimize the coverage across the entire set of few-shot examples for the test examples.

Since SET-BSR considers both token-level similarity and coverage, we believe it can be well-suited for NER tasks. On the other hand, few-shot selection based on sentence representation similarity also plays an effective role, offering insights from a sentence-level perspective. Therefore, in this research, we propose **Hybrid-SET**, which integrates two selection approaches. Figure 1 illustrates

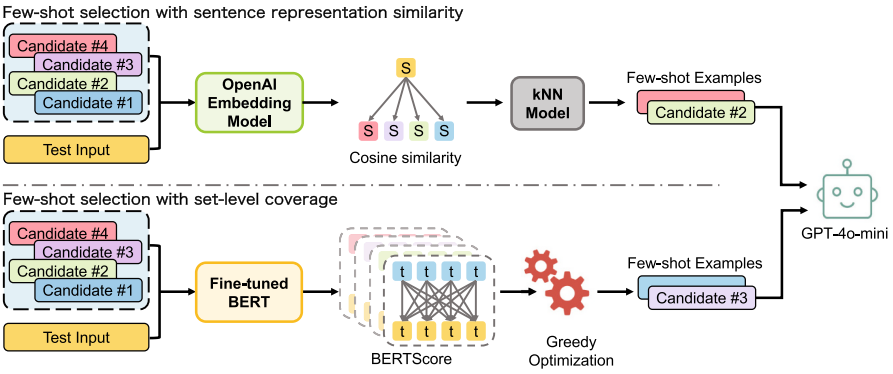


Fig. 1. The overview of Hybrid-SET.

the overview of Hybird-SET. This method considers both **sentence-level** and **token-level** relevance between few-shot candidates and test data. In the experiment, we evaluate the effectiveness of the proposed method using a material science corpus. The experimental results demonstrate that Hybrid-SET outperforms the existing few-shot selection methods.

2 Related Work

2.1 In-Context Learning

A novel paradigm, in-context learning (ICL), has emerged as a substantial advancement in LLMs. Unlike traditional fine-tuned base models that depend on task-specific supervised datasets and require parameter updates, ICL empowers LLMs to execute multiple tasks using only a limited set of "input-output" pairs (few-shot examples) for reference [3]. This method eliminates the necessity for retraining the model. Nevertheless, previous studies [16,17,23] suggested that ICL's performance is highly sensitive to the given few-shot examples. As a result, various techniques have been developed for selecting the optimal few-shot examples. A prevalent approach entails retrieving cases most relevant to the test instances from a pool of few-shot candidate examples. Numerous studies employed semantic similarity of a sentence as a measure of relevance [1,17]. Gupta et al. [10] introduced an approach using BERTScore [22] to gauge relevance when selecting examples. They advanced this concept by proposing SET-BSR, an extension of BERTScore, to evaluate the overall coverage of the few-shot example set. Evaluations across various tasks demonstrated that SET-BSR is particularly effective for complex reasoning tasks.

In-Context Learning for Named Entity Recognition. Several studies have indicated that LLMs are not ideally designed for NER tasks [20,21]. Unlike tasks such as classification and question-answering, which align well with LLMs' pre-training objectives focused on generating natural language, NER requires more

complex span extraction and detailed type classification [4]. The challenge is particularly notable in scientific domains. Research conducted by Gutiérrez et al. [13] demonstrates that GPT-3 performs significantly worse than a smaller PLM, even with advanced techniques like contextual calibration and few-shot example retrieval applied.

Several methods specifically tailored to Named Entity Recognition (NER) tasks have been proposed in prior work. Adiga et al. [1] introduced informative metrics featuring three distinct sentence- and word-level metrics. Wu et al. [21] proposed a three-stage framework integrating both ontological and entity-aware contextual information. However, both studies focused on general-domain NER tasks. Zhu et al. [24] proposed a hybrid approach that combines representativeness sampling and retrieval-based method to mitigate the potential misleading caused by similar demonstrations. However, the representative examples selected can vary significantly depending on the performance of the selection model, leading to instability in this approach.

3 Proposed Method: Hybrid-SET

As shown in Fig. 1, Hybrid-SET consists of few-shot selection with sentence representation similarity and few-shot selection with set-level coverage. The following will elaborate on these two parts.

3.1 Few-Shot Selection with Sentence Representation Similarity

For the few-shot selection with Sentence Representation Similarity (SRS), we utilize the OpenAI embedding model to convert both few-shot candidates and test data into embeddings. Subsequently, for each test input x_{test}, the nearest k neighbors, d_1, d_2, ..., d_k, are retrieved from the labeled data pool ($D_{lab} = \{d_i\}_{i=1}^{N_i}$) based on their distances in the embedding space. Given a pre-defined similarity measure sim such as cosine similarity, the neighbors are ranked in order such that $sim(d_i, x_{test}) \leq sim(d_j, x_{test})$ when $i < j$.

3.2 Few-Shot Selection with Set-Level Coverage

Relevance Metric. Following the prior research [10], we utilize BERTScore-Recall (BSR) [22] to compute the relevance between labeled few-shot candidates and test inputs. BERTScore is an evaluation metric designed to measure the similarity between text generated by language models (e.g., summaries and translations) and reference text. In the context of few-shot selection, BERTScore serves as a relevance metric to identify the most effective examples. Given a candidate example $ca = \{c_1, c_2, ...c_n\}$, $ca \in D_{lab}$, and a test example $x_{test} = \{x_1, x_2, ...x_m\}$, where c_i and x_i represent token contextual embeddings, BSR is computed as defined in Eq. 1.

$$BSR(x_{test}, ca) = \sum_{x_i \in x_{test}} \frac{1}{|x_{test}|} \cdot \max_{c_j \in ca} Cosine(x_i, c_j) \tag{1}$$

Here, *Cosine* denotes the calculation of cosine similarity.

In our research, a fine-tuned BERT is employed to generate the token embeddings for both candidate examples and test inputs.

Set-Level Coverage (SET-BSR). Coverage indicates the extent to which each token in the test example is comprehensively covered by the selected few-shot examples. The coverage of a single example can be computed by Eq. 1.

Here, we consider the challenges that arise when independently scoring candidates. In low-resource scenarios, no single candidate is sufficiently representative of the diverse patterns present in test inputs. Moreover, the complementary relationships between selected examples are not taken into account. As a result, the selected few-shot sets often include multiple redundant demonstrations with similar structures that contribute no additional information. Therefore, an extended Equation is designed to compute the coverage of a few-shot candidate set.

Given a few-shot candidate set, C, the Eq. 1 can be extended to compute the set coverage as follows:

$$setC(x_{test}, C) = \sum_{x_i \in x_{test}} \frac{1}{|x_{test}|} \cdot \max_{ca \in C} \max_{c_j \in ca} \text{Cosine}(x_i, c_j) \tag{2}$$

Additionally, Greedy Algorithm 1 is employed to optimize the set coverage.

Algorithm 1. Greedy Optimization of Set Coverage

Require: Few-shot candidate pool $D = \{d_i\}_{i=1}^{N_i}$, test input x_{test}, the number of few-shot example k, coverage scoring function $setC$

$F \leftarrow \emptyset$ ▷ Selected Demonstrations
$F_{curr} \leftarrow \emptyset$ ▷ Current Set Cover
curr_cov $\leftarrow -\infty$
while $|F| < k$ **do**
  f^*, next_cov $\leftarrow \underset{f \in D \setminus F}{\text{argmax}}$ setC$(x_{test}, F_{curr} \cup f)$
  if next_cov $>$ curr_cov **then** ▷ Pick f^*
    curr_cov $\leftarrow$ next_cov
    $F \leftarrow F \cup f^*$
    $F_{curr} \leftarrow F_{curr} \cup f^*$
  else ▷ Start new cover
    $F_{curr} \leftarrow \emptyset$, curr_cov $\leftarrow -\infty$
  end if
end while
return F

4 Experiment

4.1 Dataset

In this study, we utilize the Material Science Procedural Text Corpus (MSPT) [18] to evaluate the performance of automatic annotation. The MSPT contains

230 synthesis procedures annotated by three domain experts. A total of 21 entity types, including materials and measurement units, are labeled. This study focuses on the three most important entity types: `Material (MAT)`, `Operation (OPE)`, and `Property (PRO)`[1] for evaluation.

4.2 Baseline Methods

Fine-Tuned BERT: In order to draw a comparison between LLM and traditional supervised learning approaches, a NER model was trained using BERT within a low-resource framework. These fine-tuned BERT models are also utilized in coverage-based selection to obtain the token embeddings for both test examples and few-shot candidates.

Zero-Shot: Within the zero-shot setup, we remove the few-shot examples in the prompt.

Random Sampling: k few-shot examples are selected randomly from a pool of labeled data.

4.3 Experimental Setup

To better reflect real-world scenarios, we assume a low-resource setting where training data is limited. We randomly extracted 10% of the MSPT training set (170 to 200 sentences) as the labeled data pool. This labeled data pool is employed for fine-tuning BERT models. The few-shot examples are chosen from the same data pool. To address potential data variability, all experiments are conducted using three distinct data pools, and the mean results are reported.

All BERT models were fine-tuned using the HuggingFace Transformers library[2]. The hyperparameters used are the max length of 350, the batch size of 32, the learning rate of 2e-5, and 20 epochs of training. The EarlyStopping mechanism was applied to prevent overfitting. For the validation data, we utilized LLM to generate pseudo labels for approximately 200 sentences randomly extracted from the original validation set.

In this research, GPT was employed as the LLM to assess its in-context learning performance on material science annotation. The model employed is gpt-4o-mini[3] The temperature was set to 0. The OpenAI embedding model we used in few-shot selection with sentence representation similarity (SRS) is text-embedding-3-small[4].

[1] In MSPT, the distinctions between the `Material-Descriptor` and `Property-Misc` are often too nuanced to effectively distinguish without considering the entire article. Given that LLM provides responses at the sentence level, we merge the `Material-Descriptor` and `Property-Misc` into the `Property` label.

[2] https://huggingface.co/google-bert/bert-base-uncased.

[3] https://platform.openai.com/docs/models/gpt-4o-mini.

[4] https://platform.openai.com/docs/guides/embeddings.

Table 1. Overall results for baseline methods and various few-shot example selection methods ($k = 8$). The best results are given in **bold**.

Entity (Support)		Baseline Methods			Few-shot Selection Methods			
		BERT	Zero-shot	Random	SRS	BSR	SET-BSR	Hybrid-SET
MAT	(338)	0.507	0.508	0.641	0.657	0.657	**0.663**	0.660
OPE	(242)	0.719	0.619	0.765	0.788	0.793	**0.802**	0.801
PRO	(105)	0.071	0.155	0.275	0.264	0.269	0.284	**0.319**
Overall	(685)	0.547	0.495	0.623	0.633	0.637	0.645	**0.649**

We evaluated our method using $k = 8$ few-shot examples. In the case of Hybrid-SET, we allocate 2 examples to SRS, and 6 examples to SET-BSR[5]. If there was an overlap between the examples selected by SET-BSR and those selected by SRS, we retained the examples chosen by SRS and replaced the overlapping examples with alternative candidates ranked by SET-BSR. We utilize entity-level F1 scores for individual entity types and micro F1 scores for overall evaluation.

5 Results and Discussions

5.1 Main Results

Table 1 shows the experimental results for baseline methods and various few-shot selection methods. Notably, most methods employing GPT surpassed the fine-tuned BERT baseline, except in the zero-shot context. This suggests that GPT-4o-mini may be a more suitable choice when only a limited amount of annotated data is available. Furthermore, all few-shot selection methods achieved better performance than random sampling, highlighting the significance of selecting informative examples. For the overall performance, our proposed method achieved the best micro F1 score.

Drilling into the entity-level results, there are significant differences in extraction performance across entity types. For instance, `Operation` entities peaked at F1 score of 0.802, while `Property` entities only reached 0.319. This disparity may be attributed to the entity frequency within the corpus and the inherent complexity of entity types. Entities such as `Operation`, typically represented as verbs, are more easily identifiable by GPT. In contrast, entities like `Material`, and `Property`, require more domain knowledge or contextual understanding, making their recognition more challenging. While most methods struggled with the `Property` entity, Hybrid-SET demonstrated superior efficacy in extracting this domain-specific entity.

[5] We conducted experiments utilizing varying ratios of few-shot samples obtained through the two selection methods. The results indicate that a ratio of 1:3 yields the optimal performance.

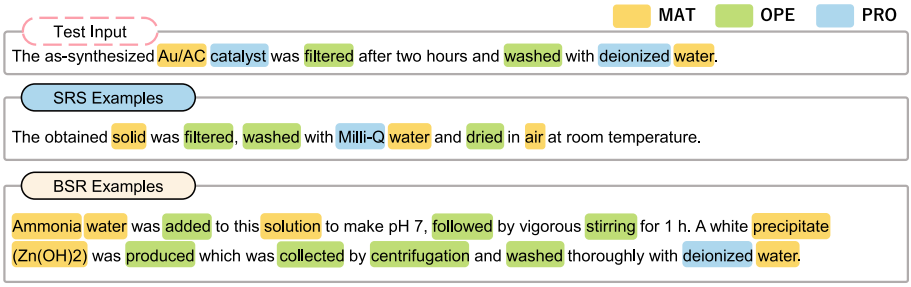


Fig. 2. The few-shot examples selected by different relevance metrics.

5.2 Comparison of Sentence-Level and Token-Level Relevance Metrics

As shown in Table 1, BSR outperformed SRS. This superior performance can be attributed to the fact that BSR utilize token contextual embeddings, which align closely with the nature of NER tasks, where capturing entities and their co-related tokens is crucial.

Figure 2 compares few-shot examples selected by SRS and BSR. Examples selected by SRS tend to have sentence structures that are syntactically similar to the test examples (e.g., "[*noun*] was filtered" and "washed with [*noun*]"), as SRS primarily operates at the sentence level. On the other hand, BSR focus more on token sequence alignment. For instance, the sequence "washed with deionized water" is exactly matched with the test input. Moreover, BSR leverages token contextual embeddings to retrieve semantically aligned examples successfully (e.g., aligning "(Zn(OH)2)" with "Au/AC" in the test input). This highlights the advantage of using BSR in capturing contextual equivalence.

5.3 The Effectiveness of Set-Level Coverage

As shown in Table 1, set-level selection methods outperformed methods that select few-shot examples independently. This finding emphasizes the significance of considering the complementary relationships among the selected examples.

To better understand why set-level selection is so effective, we compare the demonstrations selected by SET-BSR with those selected by SRS and BSR in Fig. 3. The examples chosen by the independent scoring methods (SRS and BSR) often exhibit redundancy, as they tend to include examples conveying similar information. For instance, the two examples selected by SRS both repeat the phrase "[*noun*] was dried". Similarly, examples selected by BSR, matched using token embeddings, also show overlap, such as the repeated inclusion of "washed (...) with deionized water". Conversely, SET-BSR accounts for the complementarity of selected few-shot examples, thus reducing the likelihood of choosing examples with recurring information. For instance, the second example selected by SET-BSR includes the phrase "The as-synthesised", which is absent from the

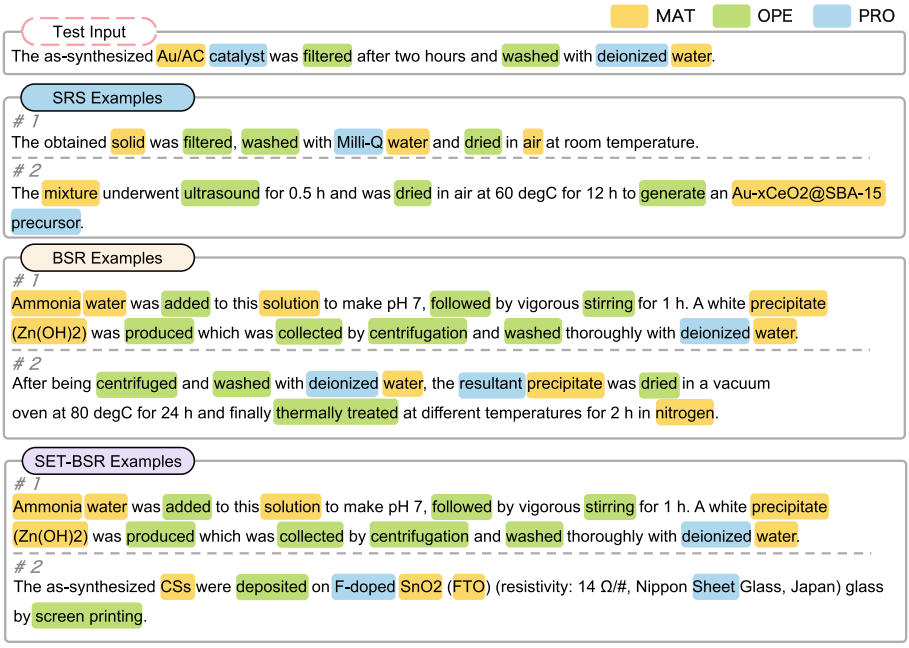


Fig. 3. A comparison of the few-shot examples selected by various relevance metrics.

first example but present in the test input. Consequently, SET-BSR avoids selecting redundant examples while maintaining high coverage with the test input.

The proposed method, Hybrid-SET, integrates SRS and SET-BSR in a complementary way. It considers both sentence-level and token-level similarities while optimizing the informativeness of the few-shot set. We believe these factors enable Hybrid-SET to effectively address challenging cases.

6 Conclusion

In this paper, we proposed a novel few-shot example selection method, Hybrid-SET, for material science NER using GPT. It considers both sentence-level semantic similarity and set-level coverage. The experimental results shows that Hybrid-SET outperformed other existing selection methods, and the improvements were more pronounced for domain-specific entities.

Despite the advancements, several limitations must be considered. First, this study focuses exclusively on the materials science domain and evaluates few-shot selection methods using a single dataset. Further validation on general-domain datasets and other tasks is necessary to establish broader applicability. Second, all experiments were conducted using GPT models. Comparing our approach with high-performance open-source large language models, such as Llama-3, would provide valuable insights. Additionally, the number of few-shot examples

included in the prompt was limited to 8. Given the observed trend of performance improvement with an increasing number of examples, future research should explore the potential for further enhancements by expanding the number of few-shot examples.

References

1. Adiga, R., Subramanian, L., Chandrasekaran, V.: Designing informative metrics for few-shot example selection. In: Findings of the Association for Computational Linguistics: ACL 2024, pp. 10127–10135. Association for Computational Linguistics (2024)
2. Alsentzer, E., et al.: Publicly available clinical BERT embeddings. In: Proceedings of the 2nd Clinical Natural Language Processing Workshop, pp. 72–78 (2019)
3. Brown, T., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901 (2020)
4. Chen, J., et al.: Learning in-context learning for named entity recognition. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 13661–13675 (2023)
5. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pp. 4171–4186 (2019)
6. Ding, B., et al.: Is GPT-3 a good data annotator? In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 11173–11195 (2023)
7. Dubey, A., et al.: The llama 3 herd of models (2024)
8. Friedrich, A., et al.: The SOFC-exp corpus and neural approaches to information extraction in the materials science domain. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 1255–1268 (2020)
9. Gilardi, F., Alizadeh, M., Kubli, M.: Chatgpt outperforms crowd workers for text-annotation tasks. Proc. Natl. Acad. Sci. **120**(30), e2305016120 (2023)
10. Gupta, S., Gardner, M., Singh, S.: Coverage-based example selection for in-context learning. In: Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 13924–13950. Association for Computational Linguistics, Singapore (2023)
11. Hong, Z., Ward, L., Chard, K., Blaiszik, B., Foster, I.: Challenges and advances in information extraction from scientific literature: a review. JOM **73**(11), 3383–3400 (2021)
12. Huang, S., Cole, J.M.: A database of battery materials auto-generated using chemdataextractor. Sci. Data **7**(1), 2052–4463 (2020)
13. Jimenez Gutierrez, B., et al.: Thinking about GPT-3 in-context learning for biomedical IE? Think again. In: Findings of the Association for Computational Linguistics: EMNLP 2022, pp. 4497–4512 (2022)
14. Laskar, M.T.R., Bari, M.S., Rahman, M., Bhuiyan, M.A.H., Joty, S., Huang, J.: A systematic study and comprehensive evaluation of ChatGPT on benchmark datasets. In: Findings of the Association for Computational Linguistics: ACL 2023, pp. 431–469 (2023)

15. Levy, I., Bogin, B., Berant, J.: Diverse demonstrations improve in-context compositional generalization. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1401–1422. Association for Computational Linguistics (2023)
16. Liu, H., Wang, Y.: Towards informative few-shot prompt with maximum information gain for in-context learning. In: Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 15825–15838. Singapore (2023)
17. Liu, J., Shen, D., Zhang, Y., Dolan, B., Carin, L., Chen, W.: What makes good in-context examples for GPT-3? In: Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures, pp. 100–114. Dublin, Ireland and Online (2022)
18. Mysore, S., et al.: The materials science procedural text corpus: annotating materials synthesis procedures with shallow semantic structures. In: Proceedings of the 13th Linguistic Annotation Workshop, pp. 56–64 (2019)
19. Trewartha, A., et al.: Quantifying the advantage of domain-specific pre-training on named entity recognition tasks in materials science. Patterns **3**(4), 100488 (2022)
20. Wan, Z., et al.: GPT-RE: in-context learning for relation extraction using large language models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 3534–3547 (2023)
21. Wu, C., Ke, W., Wang, P., Luo, Z., Li, G., Chen, W.: ConsistNER: towards instructive NER demonstrations for LLMs with the consistency of ontology and context. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, no. 17, pp. 19234–19242 (2024)
22. Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., Artzi, Y.: Bertscore: evaluating text generation with BERT. In: 8th International Conference on Learning Representations, ICLR 2020 (2020)
23. Zhang, Y., Feng, S., Tan, C.: Active example selection for in-context learning. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pp. 9134–9148. Abu Dhabi, United Arab Emirates (2022)
24. Zhu, C., Shimada, K., Taniguchi, T., Ohkuma, T.: Staykate: hybrid in-context example selection combining representativeness sampling and retrieval-based approach – a case study on science domains. CoRR abs/2412.20043, pp. 1–11 (2024). https://arxiv.org/abs/2412.20043

Efficiently Summarizing Norwegian Legal Texts

Tu My Doan, David Baumgartner, and Benjamin Kille(✉)

Norwegian University of Science and Technology, Trondheim, Norway
{tu.m.doan,david.baumgartner,benjamin.u.kille}@ntnu.no

Abstract. We present a study on summarizing the Norwegian legal texts. The methodology combines different models with pruning strategies to obtain summaries faster. Evaluation of more than 22,000 documents finds that pruning helps accelerate the process while sacrificing some accuracy. Future research has to show whether these insights generalize to other languages, domains, and tasks.

Keywords: Text summarization · Norwegian legal texts · LLMs

1 Introduction

Automatic summarization takes long documents and produces concise summaries. Extractive methods compile summaries composed from existing sentences. Abstractive summarization creates a new summary that does not necessarily correspond exactly to any of the existing sentences. This study explores abstractive summarization methods for Norwegian legal texts (Our code and example generated summaries are available at: https://github.com/doantumy/Efficiently-Summarizing-Norwegian-Legal-Texts).

Large Language Models (LLMs) are powerful tools for various natural language processing tasks. Pre-training exposes LLMs to a variety of texts that let them exploit statistical patterns. Abstractive summarization methods aim to imitate human summarization. A preliminary study showed that LLMs can summarize Norwegian legal texts, but the process takes a long time. We estimate the likelihood that sentences inform the summaries in an effort to remove less informative parts, thus accelerating the process. In addition, we explore whether considering the structure of a document can help to improve the quality of abstractive summaries. State of the art methods—such as SUMM^N—discard the structure of documents. We ask the following research questions:

RQ$_1$ How can we identify passages that are of little value to a summary?
RQ$_2$ What is the trade-off between summarization speed and summary quality when parts of a long document are eliminated?
RQ$_3$ How can we maintain and leverage the document structure when producing abstractive summaries?

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 25–35, 2026.
https://doi.org/10.1007/978-3-031-97144-0_3

The remainder of this document is structured as follows: Sect. 2 illustrates the gap between this study and previous work. Section 3 presents the data set. Section 4 describes pruning and summarizing methods. Section 5 outlines the evaluation protocol. Section 6 presents the observations from the experiments. Section 7 reflects on the findings and emphasizes limitations. Section 8 summarizes the findings and provides directions for future research.

2 Related Work

In this section, we discuss related work on abstractive summarization for long documents and legal texts. We refer readers to two survey papers [4,5] for a more comprehensive review.

Long Document Summarization: Multi-stage approaches handle lengthy documents. Zhang et al. [13] introduced SUMM^N, which iteratively splits, summarizes, and refines texts into coarse and fine-grained summaries. Lim et al. [7] improved this method by aligning target-focused segments and incorporating contrastive learning. Bleiweiss [1] proposed a two-step approach using LexRank and SBERT for extractive selection followed by BART-based abstractive summarization. OpenAI [12] employed recursive decomposition and human feedback to summarize fiction novels, fine-tuning GPT-3 on smaller sections. Pang et al. [9] introduced a hierarchical model that combines bottom-up local self-attention with top-down global context correction for better efficiency.

Legal Document Summarization: Legal texts pose unique challenges due to their complexity and length. We find fewer studies for legal documents. Feijo et al. [3] introduced LegalSumm, which segments Brazilian Supreme Court rulings into chunks and applies textual entailment with BERT to reduce hallucinations. Dan et al. [2] proposed a legal judgment summarization system for Chinese texts, structuring documents into case information, description, and judgment. They utilize an extractive model (BSLT) and an abstractive model (LPGN) based on Lawformer to generate coherent summaries.

3 Data

We have access to around 61,000 court decisions from[1]. The texts' character distribution shows many short texts and a noticeably high standard deviation indicating the presence of very long texts. Our focus is on long document summarization which is why we have decided to remove documents with fewer than 10,000 characters. After the reduction, we are left with 22,644 documents. We

[1] Lovdata (https://lovdata.no/) is a foundation established in 1981 that operates legal information systems in Norway on a non-profit basis. Lovdata has pre-processed the texts in order to remove personal information.

have split the documents into three sets for the experiments: the training set has 15,850 documents, the validation set has 2,264 documents, and the test set includes 4,530 documents. The documents have two separate parts needed for summarization. A relatively long text describes the court's reasoning. This text is called "premiss" in Norwegian. A relatively short text presents a summary of the reasoning. This text is referred to as "sammendrag". Figure 1 shows the distribution of characters over the collection of documents. The initial text contains on average 13,928.24 characters with a standard deviation of 15,229.44 characters. The summaries are on average 460.32 characters long with a standard deviation of 264.07.

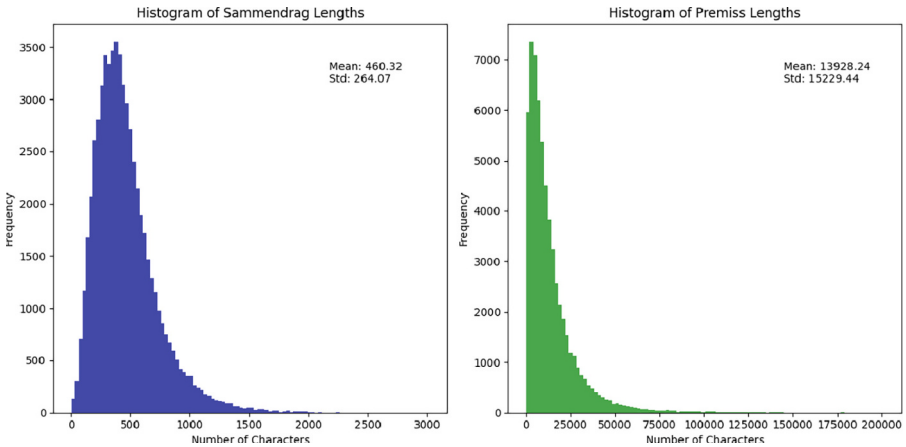


Fig. 1. The histograms show the distribution of characters over the sets of summaries (left) and initial texts (right). Note, the Norwegian term for summary is "sammendrag" and the data provider calls the initial texts "premiss".

4 Methodology

Suppose we want to summarize a document D of length L, but our abstractive summarizer is limited to length K with $K \ll L$. SUMM^N [13] addresses the problem by fitting as much into one chunk as possible and combining the summaries of the chunks with a separate model. We explore ways to keep related information together while sacrificing some efficiency. Documents from Lovdata are structured in paragraphs. Thus, we leverage this structure to develop different methods for grouping paragraphs together or splitting them for better coherence. Therefore, we introduce two other methods for generating summaries focusing on data segmentation: *paragraph level* and *paragraph similarity*. The former either splits long paragraphs or combines short ones to form a new paragraph determined by the context length of the model. The latter focuses on grouping similar paragraphs together using ROUGE-1 (R-1) or BERT-score metric.

Text Pruning

Summaries omit some parts of documents. These parts present information that is not necessary to present the core ideas. To improve the summary quality, and reduce training time, we performed text pruning before the segmentation process. This technique removes irrelevant sentences from the document:

1. Calculate weighted[2] similarity between every sentence in document text and every sentence in document summary (see Algorithm 1).
2. Identify pruning threshold by calculating mean and standard deviation of minimum similarity scores in the training set (as computed in Algorithm 1).
3. Create a dataset of all sentences in the training set with their corresponding scores as label.
4. Train a regressor to predict similarity scores of documents (training, validation, test set). We use a neural network model with nb-bert-large [6].
5. Prune training, validation, test data using threshold or proportion criteria.

Algorithm 1. Calculate Summary-Text Similarity Scores

$S = \{s_1, s_2, s_3, \ldots, s_n\}$ ▷ summary sentences
$T = \{t_1, t_2, \ldots, t_m\}$ ▷ text sentences
$L = \{\}$
Step 1: Iterate each summary-text pair
for each $s_i \in S$ **do**
    for each $t_j \in T$ **do**
        Calculate weighted similarity between s_i and t_j
    end for
    Build $L = \{$highest similarity score sentences in T until $|T| = |S|\}$
end for
Return: min L

5 Experiments

For all experiments, we use the T5 architecture [10]. As our texts are in Norwegian, we adopt the nor-t5-large model [11] using 512 tokens as the context window. The experiments feature four models combined with different pruning settings.

5.1 Models

We perform experiments for four different methods:

[2] The term *weighted* means to give more weight to rare tokens and less weight to frequent ones. This is based on the TF-IDF idea.

- **Classic fine-tune**: this corresponds to the fine-tuning without segmentation. We truncate text that is outside the model's context length. Note, that the elimination will sacrifice some important information for the summary.
- **SUMM$^{\mathbf{N}}$**: this is a two-stage approach summarization method without structured data.
- **Paragraph level**: We modify the segmentation process of SUMM^N by grouping/splitting paragraphs to fit model context length.
- **Paragraph similarity**: We group similar sentences in a paragraph together to fit into model context length using similarity scores (R-1 or BERT-score).

5.2 Data Pruning

Global Threshold Data Pruning. After having determined the mean (0.64) and standard deviation (0.07), we define three thresholds: 0.57, 0.64, and 0.71. These are used to prune data before the segmentation. In other words, we remove all sentences that score below the threshold. The same threshold is applied to all documents.

Local Threshold Data Pruning. The global threshold might result in some documents with few sentences. As an alternative, we introduce local thresholds that consider the available similarity scores.

Proportion Data Pruning. After calculating similarity scores for all sentences, we set to keep top percentage of data. We define three proportional values: 30%, 50%, and 70%. In other words, we discard a proportion of least similar sentences: 70%, 50%, and 30%.

Shuffle Pruning Data. We experiment with two settings: keeping the sentence order as they appear in the original document or shuffle them. This tells us whether the order affects the performance. We expect the shuffled documents to result in worse summaries.

6 Results

Table 1 shows results for different pruning and threshold strategies using different ROUGE metrics [8]. Evaluations are conducted for three main summarization approaches: Classic Fine-tune, SUMM^N, and Paragraph Level[3]. For non-shuffling and shuffling text pruning, we use global, proportional $p = \{0.3, 0.5, 0.7\}$, and local threshold setting $t = \{0.57, 0.64, 0.71, \overline{\text{sim}}\}$.

In proportion settings (shuffle and non-shuffle), the performance across all pruning proportions shows a decline as p increases, with the best results observed

[3] Due to limited space, we put examples of generated summaries in the GitHub README.

at $p = 0.3$. For example, classic fine-tune achieves an R-1 of 36.45, while SUMM^N registers 34.54 in non-shuffling setting. This implies that only a small part of the text is relevant to summary generation.

Weighted similarity scores approach shows that global threshold pruning performs slightly better than local threshold pruning in preserving summary quality for shuffle and non-shuffle settings. For instance, at $t = 0.57$, R-1 score for classic fine-tune reaches 36.91, while SUMM^N achieves 35.82 in non-shuffling setting.

Across all settings, non-shuffling text pruning generally outperforms shuffling pruning, particularly in paragraph similarity metrics. For example, under non-shuffling global threshold pruning, SUMM^N achieves an R-1 of 35.82, compared to 35.42 for shuffling.

The highest ROUGE-1 score (36.91) is achieved by classic fine-tune with global threshold pruning ($t = 0.57$) with SUMM^N being the second best across all methods. Paragraph-level performance consistently improves summary coherence, particularly in R-L metrics, demonstrating the benefit of leveraging structural context in summarization tasks. Paragraph similarity method achieves comparable, marginally worse results.

Shuffling text has reduced performance slightly in most cases except for paragraph similarity method. For example, an increase of 0.26 and 0.74 in $t = 0.57$ in ROUGE-1 score for both similarity metrics (R-1 and BERT-score respectively).

These results demonstrate that a combination of global threshold pruning and non-shuffling text processing leads to superior performance in both ROUGE and BERT-scores. Furthermore, the SUMM^N method proves effective in maintaining summary quality with lower proportions of text pruning and optimal threshold settings (e.g., $t = 0.57$). This suggests that incorporating paragraph-level strategies enhances both semantic accuracy and structural conformity in summarization.

Table 1 shows the empirical means of the considered methods and conditions. We compare the performance and runtime for pairs of methods that segment long documents—SUMM^N, paragraph-level, and paragraph-similarity—across conditions. Figure 2 shows runtime comparison among Paragraph-level, SUMM^N, and Paragraph similarity method. Each subfigure compares the normalized runtime of two methods. The diagonal line indicates identical runtime. The side on which the points land takes longer than the compared method. Paragraph-level method despite being slightly lower in performance compared to SUMM^N takes significantly shorter training time. Paragraph-similarity R-1 is the slowest method.

Figure 3 compares the performance of pairs of methods with respect to R-1, R-2, and R-L. Each row compares exactly two methods. The diagonal line indicate identical performance. Points landing on one side of the diagonal show better performance. For instance, the second row illustrates that SUMM^N and Paragraph Level perform comparably. In contrast, Paragraph Level outperforms paragraph Rouge and BERT (rows 4–5), with consistent R-1, R-2, and R-L scores.

Table 1. Summary of Experimental Results.

Settings	Classic Fine-tune			SUMM^N			Paragraph Level			Paragraph Similarity					
										ROUGE-1			BERT-score		
	R-1	R-2	R-L	R-1	R-2	R-L	R-1	R-2	R-L	R-1	R-2	R-L	R-1	R-2	R-L
No Text Pruning															
$p = 1.0$	34.92	14.22	23.89	28.53	07.28	17.52	27.88	06.89	17.16	26.87	06.17	16.58	27.55	06.72	16.95
Non-shuffling Text Pruning															
Proportion Pruning															
$p = 0.3$	36.45	15.23	24.43	34.54	13.61	22.98	35.01	13.78	23.15	30.04	09.28	19.30	31.27	10.45	20.30
$p = 0.5$	35.88	14.88	24.19	31.70	10.00	19.89	31.85	10.68	20.52	27.93	06.97	17.31	28.69	07.60	17.93
$p = 0.7$	35.28	14.39	23.90	29.64	08.20	18.41	29.88	08.86	18.80	27.57	06.75	16.99	28.25	07.11	17.33
Weighted Similarity Scores Global Threshold Pruning															
$t = 0.57$	36.91	16.04	24.97	35.82	15.32	24.41	35.66	15.26	24.30	34.95	14.65	23.76	34.50	13.94	22.00
$t = 0.64$	34.24	14.14	23.10	33.55	13.61	22.89	33.42	13.33	22.59	32.98	13.02	21.78	32.70	12.67	21.45
$t = 0.71$	35.27	14.83	24.00	33.61	13.86	22.96	33.94	13.72	22.62	33.75	13.60	22.64	33.61	13.51	22.19
Weighted Similarity Scores Local Threshold Pruning															
$t = \overline{\text{sim}}$	35.91	14.83	24.16	32.02	10.73	20.50	33.56	12.07	21.80	28.65	07.69	17.85	28.69	07.64	25.97
Shuffling Text Pruning															
Proportion Pruning															
$p = 0.3$	36.26	15.13	24.14	35.25	13.73	23.35	35.41	14.18	23.63	30.71	09.45	19.42	32.79	11.46	21.25
$p = 0.5$	35.29	14.17	23.52	33.07	11.15	21.04	32.83	10.87	20.73	28.52	07.23	17.56	29.47	08.15	18.42
$p = 0.7$	34.66	13.72	23.10	31.29	09.43	19.53	32.24	10.29	20.43	28.23	07.20	17.38	28.96	07.64	17.82
Weighted Similarity Scores Global Threshold Pruning															
$t = 0.57$	36.74	15.86	24.75	35.42	14.94	23.84	35.42	15.15	24.17	35.21	14.76	23.81	35.24	14.78	23.71
$t = 0.64$	34.33	14.11	23.05	31.99	12.52	20.75	31.98	11.70	20.01	33.06	12.53	21.04	33.60	13.52	22.85
$t = 0.71$	35.12	14.75	23.95	33.79	13.69	22.89	33.69	13.63	22.55	33.87	13.86	22.97	33.99	13.94	23.00
Weighted Similarity Scores Local Threshold Pruning															
$t = \overline{\text{sim}}$	35.62	14.53	23.57	33.45	11.99	21.69	33.75	11.97	21.74	29.13	07.76	18.08	30.05	08.47	18.71

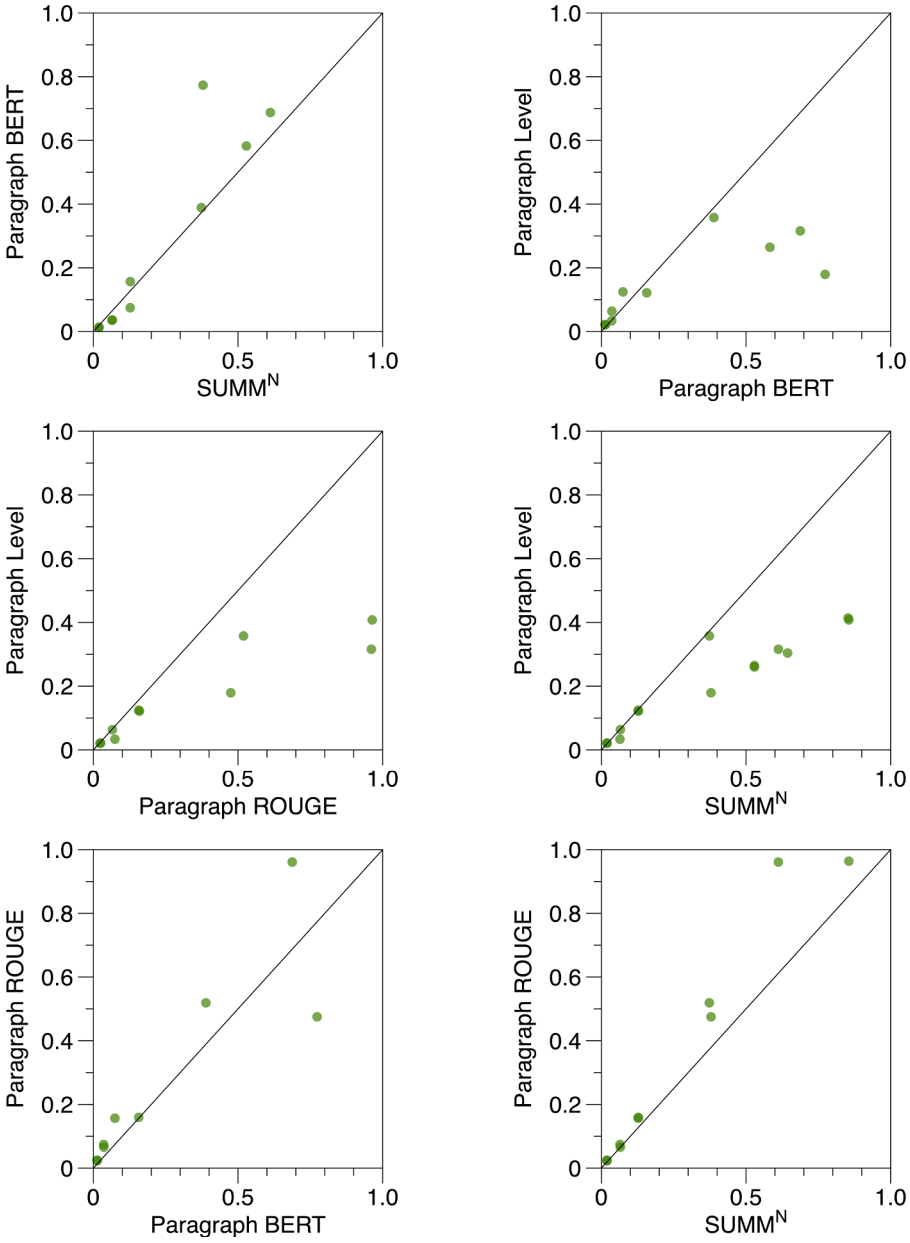


Fig. 2. Each subplot compares the normalized runtime of two summarization methods. The line indicates equal runtime. The side, on which the points are shown, takes longer than the other method.

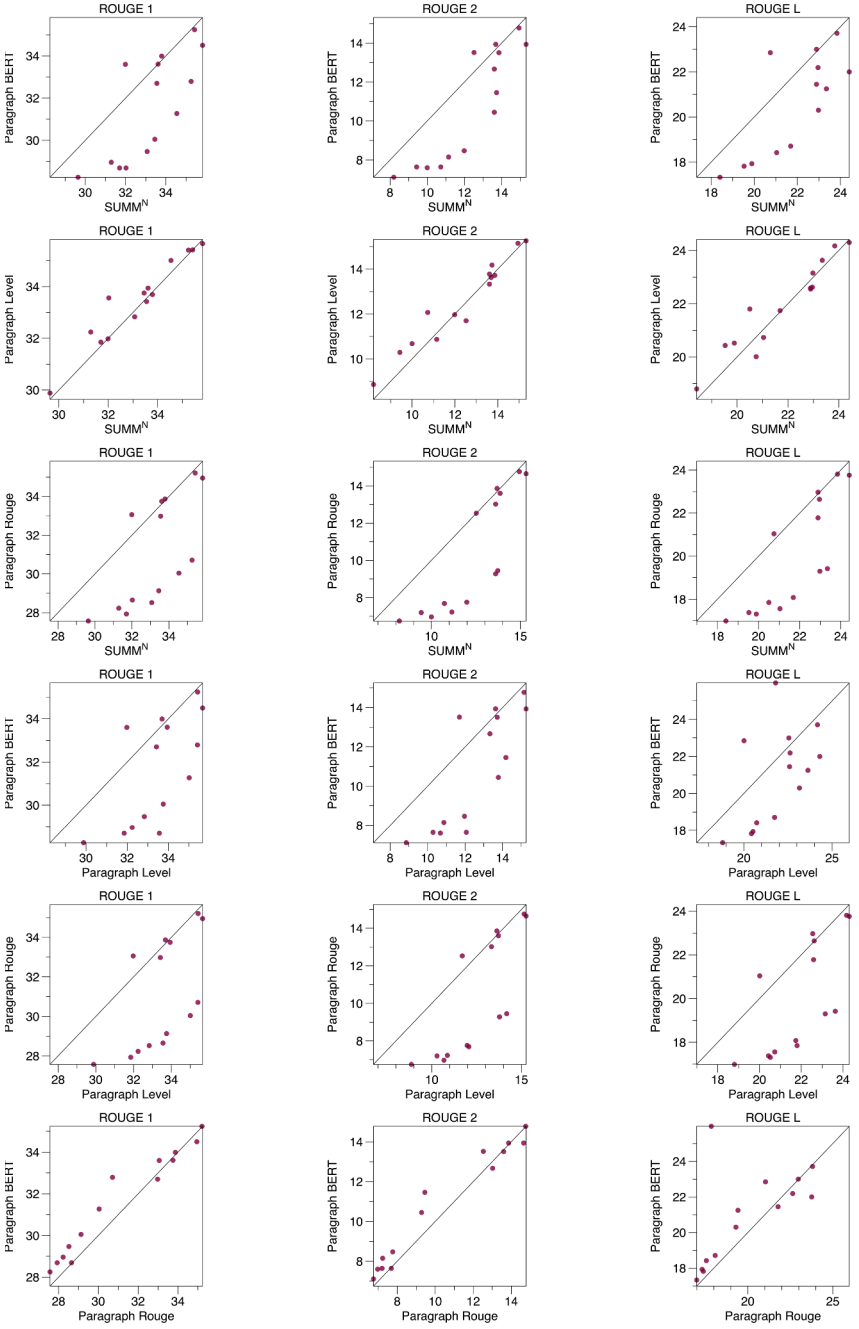


Fig. 3. Comparison of different summarization methods in terms of *Rouge-1* (left column), *Rouge-2* (middle column), and *Rouge-L* (right column).

7 Discussions

Summarizing long documents represents a challenge even with powerful language models. Systems need to balance the available computational resources with quality requirements. Our experiments highlight the performance of four models in terms of Rouge scores. The scores vary between 28.69 to 36.91 (R-1), 6.75 to 16.04 (R-2), and 17.31 to 25.97 (R-L) for non-shuffled, pruned texts. Shuffling yields comparable scores, pointing to models' ability to generate abstractive summaries without requiring ordered input.

The analysis compared the summary quality and runtime for combinations of methods. Results indicate that paragraph-level summarization yields results more quickly than traditional methods while maintaining a comparable, somewhat lower performance.

Our experiments found that pruning texts sped up the process. In fact, without pruning, the results took so long that they could hardly be obtained. Determining which parts of a text give little or no information to a summary represents a challenging task. Machine learning constitutes a promising way to tackle the problem. Given sufficiently many examples, a sophisticated classifier should be able to identify whether sentences convey the information present in the summary. On the other hand, it remains unclear which collection of document, summary pairs are needed or representative.

The evaluation protocol embodies another limitation. Human feedback would allow us to more accurately reflect on the value of generated abstractive summaries. An experiment could present users with summaries comprised of generated and human-written examples. Unfortunately, we lack the resources to conduct such a study. On the one hand, we would need legal experts as the texts convey jurisprudence which the general public could not summarize. On the other hand, the limited availability of expert annotators restricts our focus to a small sample size. Thus, we cannot obtain results that generalize to arbitrary legal texts.

8 Conclusion and Future Work

This study investigates methods to summarize long legal texts in Norwegian. We examined whether pruning texts would allow us to provide summarization faster while maintaining the quality. We compared four summarization methods with respect to Rouge scores. We found that pruning somewhat reduces the performance while also speeding up the process. Still, there are some open questions to be addressed by future research. It remains unclear what method to identifying less informative parts of texts links well with summarization methods. A detailed analysis of the retained proportion of documents could reveal what strategy works best in given circumstances. Second, further research needs to assess whether humans appreciate the abstractive summaries. Checking whether they can discern generated summaries from human-written summaries could be a first step. Finally, future research should check how well these method generalize to other domains.

Acknowledgments. This work is done as part of AI4Interviews Project. Data is provided by LovData.no under agreements between NTNU and LovData.

References

1. Bleiweiss, A.: Two-step text summarization for long-form biographical narrative genre. In: Strube, M., Braud, C., Hardmeier, C., Li, J.J., Loaiciga, S., Zeldes, A. (eds.) Proceedings of the 4th Workshop on Computational Approaches to Discourse. ACL (2023). https://doi.org/10.18653/v1/2023.codi-1.20
2. Dan, J., Hu, W., Wang, Y.: Enhancing legal judgment summarization with integrated semantic and structural information. Artif. Intell. Law (2023)
3. Feijo, D., Moreira, V.P.: Improving abstractive summarization of legal rulings through textual entailment. Artif. Intell. Law **31**(1), 91–113 (2023)
4. Kanapala, A., Pal, S., Pamula, R.: Text summarization from legal documents: a survey. Artif. Intell. Rev. **51**(3), 371–402 (2017). https://doi.org/10.1007/s10462-017-9566-2
5. Koh, H.Y., Ju, J., Liu, M., Pan, S.: An empirical survey on long document summarization: datasets, models, and metrics. ACM Comput. Surv. **55**(8) (2022). https://doi.org/10.1145/3545176
6. Kummervold, P.E., De la Rosa, J., Wetjen, F., Brygfjeld, S.A.: Operationalizing a national digital library: the case for a Norwegian transformer model. In: NoDaLiDa. Linköping University Electronic Press, Sweden (2021). https://aclanthology.org/2021.nodalida-main.3
7. Lim, J., Song, H.J.: Improving multi-stage long document summarization with enhanced coarse summarizer. In: Dong, Y., Xiao, W., Wang, L., Liu, F., Carenini, G. (eds.) Proceedings of the 4th New Frontiers in Summarization Workshop. ACL (2023). https://doi.org/10.18653/v1/2023.newsum-1.13
8. Lin, C.Y., Och, F.: Looking for a few good metrics: ROUGE and its evaluation. In: NTCIR Workshop, pp. 1–8 (2004)
9. Pang, B., Nijkamp, E., Kryscinski, W., Savarese, S., Zhou, Y., Xiong, C.: Long document summarization with top-down and bottom-up inference. In: Findings of the Association for Computational Linguistics: EACL 2023. ACL (2023). https://doi.org/10.18653/v1/2023.findings-eacl.94
10. Raffel, C., et al.: Exploring the limits of transfer learning with a unified text-to-text transformer. J. Mach. Learn. Res. **21**(140) (2020)
11. Samuel, D., et al.: NorBench – a benchmark for Norwegian language models. In: NoDaLiDa. University of Tartu Library (2023). https://aclanthology.org/2023.nodalida-1.61
12. Wu, J., et al.: Recursively Summarizing Books with Human Feedback. CoRR arxiv:2109.10862 (2021)
13. Zhang, Y., et al.: SUMM^N: a multi-stage summarization framework for long input dialogues and documents. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics, vol. 1: Long Papers. ACL (2022). https://doi.org/10.18653/v1/2022.acl-long.112

FairNM: Fairness in Name Matching

Yuan Liu and Flavius Frasincar(✉)

Erasmus University Rotterdam, Burgemeester Oudlaan 50,
3062 PA Rotterdam, The Netherlands
yuanliuenzo@gmail.com, frasincar@ese.eur.nl

Abstract. Ensuring fairness in automated systems is critical in today's data-driven landscape. Prior work has exposed algorithmic bias in popular name matching systems, particularly affecting names from certain racial backgrounds, but the root causes and effective solutions remain underexplored. This paper introduces FairNM, a novel system that reduces bias through token-based similarity scoring, a Siamese Neural Network-based Short Name Module, and Name Weighting. Using a novel test bench and fairness metric, we show that FairNM achieves accurate fuzzy name matching while significantly improving fairness. Its performance is especially valuable in Web-based contexts requiring high recall and fairness, such as fraud detection, where mismatches can have serious impact.

Keywords: Fairness · Entity matching · Fuzzy name matching · NLP

1 Introduction

With the rise of machine learning, data-driven algorithms are increasingly used in high-stakes decisions, such as loans [14] and hiring [15]. A key method in these systems is Entity Matching (EM), which identifies records referring to the same real-world entity. Within EM, fuzzy name matching aims to match name variations caused by formatting differences, typos, or misspellings.

However, names often encode sensitive characteristics like race, ethnicity, or religion [12], making name-based systems vulnerable to bias. While many studies compare name matching algorithms [3,4,7,16], fairness has received little attention. One exception [10] found that popular algorithms perform worse for names of Asian origin, which was attributed to differences in name length. But, this work did not explore other linguistic factors or propose mitigation strategies.

This paper addresses those gaps by introducing FairNM, a novel system that improves fairness in fuzzy name matching without sacrificing accuracy. Unlike prior studies that classify individuals by race, we group names by linguistic origin, recognizing that language-based differences (e.g., Chinese vs. Japanese names) are more meaningful than broad racial categories. We define fairness as equal treatment across these linguistic groups.

Due to the lack of existing name datasets with language labels, we created our own, using methods from [1,17]. The dataset and code are available at: https://github.com/yuanliueur/FairNM.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 36–46, 2026.
https://doi.org/10.1007/978-3-031-97144-0_4

The rest of the paper continues as follows. Section 2 discusses data acquisition and cross-lingual name characteristics. Section 3 details the experimental setup and fairness metric. Section 4 presents the results and Sect. 5 the conclusion.

2 Data

This section describes the data used in the paper, including the construction of the name database and a descriptive analysis. Identifying bias in name characteristics across linguistic origins offers insight into potential algorithmic bias.

2.1 Name Database Generation

Due to the lack of available name databases with linguistic information, we generated our own data, inspired by previous work in ethnicity classification [1,17]. Using a custom Web scraping tool, we collected data from Wikipedia.

Starting from a root category for a specific country on Wikipedia[1], we employed a Breadth-First Search (BFS) algorithm to retrieve all subcategories up to a depth of four. We then applied filtering criteria to these subcategories, as outlined below with examples:

1. Contains *people*: German people by occupation;
2. Contains *s of*: Members of the German Football Bond;
3. Ends with *s*: German footballers.

Non-person entries were filtered out. From each country's subtree, we extracted names at the leaf level and excluded entries unrelated to the target language. A schematic of the Chinese example is shown in Fig. 1.

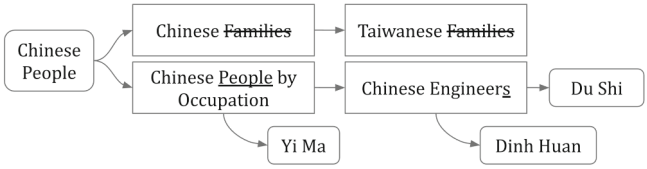


Fig. 1. Web scraping process for Chinese names, with filtered-out entries indicated.

This process was applied to various root categories, resulting in 260,150 names across 13 language groups, as detailed in Table 1.

Most names follow the Western format (first name followed by last name), but Chinese and Korean names often reverse this order. To standardize formatting, we swapped name order for these groups, except where a Vowel-Consonant-Vowel pattern or known compound surname suggested a different structure. For instance, 'Jin Baofang' is reordered to 'Baofang Jin', while 'Xiaolan Bao' remains unchanged, as the multiple vowels in 'Xiaolan' suggest a first name.

[1] The root page for 'Chinese': https://wikipedia.org/wiki/Category:Chinese_people.

Table 1. Obtained Wikipedia name database categorized per language with examples.

Language	# Names	Example Name	Root Category
BRI	57,385	Sarah Wigglesworth	British
FRA	36,683	Hervé Berville	French
GER	29,286	Magdalena Neuner	German
IND	26,146	Sukhbir Sinha	Indian
ITA	21,769	Marcello Farabegoli	Italian
RUS	17,801	Alexander Abrosimov	Russian
JAP	16,612	Toshizō Nishio	Japanese
SPA	14,462	Begoña Sánchez	Spanish
POR	13,632	Candido Rondon	Portuguese, Brazilian
CHI	13,378	Baofang Jin	Chinese
ARAB	7,036	Mohamed Al-Fayed	Egyptian, Iraq, Saudi Arabian
KOR	5,960	Seung-hyun Choi	South Korean
Total	260,150		

2.2 Data Descriptive Analysis

Two key patterns were identified across language groups in the name database: differences in name length and name uniqueness. As shown in Fig. 2, most languages exhibit name lengths between 13 to 18 characters, while Chinese and Korean names clearly peak at shorter lengths. Name uniqueness also varies substantially: Korean and Chinese surnames are far less diverse, consistent with reports that only 288 Korean surnames exist [5] and that the top five Chinese surnames are shared by 433 million people [18]. In contrast, these languages show increased variability in the first name.

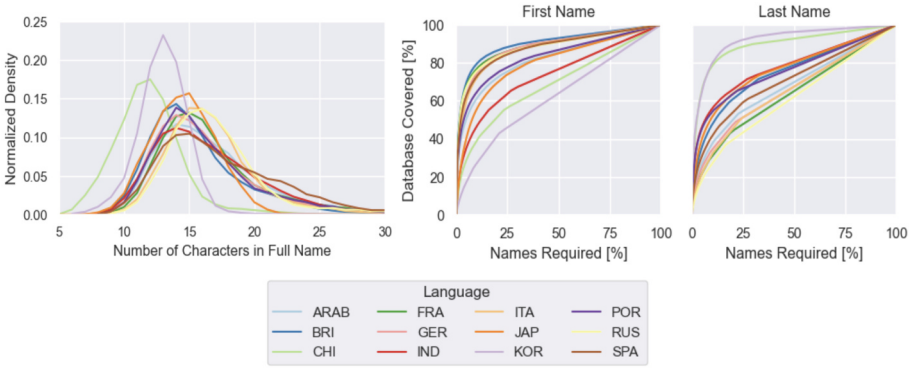


Fig. 2. Cross-lingual comparison of (left) full name length and (right) percentage of unique names required to cover database.

3 Methodology

This section describes the experimental setup and the components of the FairNM algorithm designed to enhance fairness and accuracy.

3.1 Test Bench

We simulate a fuzzy name matching task in which a query name must be linked to the correct entity within a screening list, where names are artificially modified. The test database contains 36,000 names (3,000 samples per language group). By running the test bench with screening lists of different linguistic origins, we can assess performance discrepancies across language groups. As shown in Fig. 3, the test bench consists of two steps: a High Recall Filter, which efficiently eliminates clear non-matches, and a Similarity Calculation step, where similarity scores are computed for the remaining candidate pairs using both the proposed FairNM algorithm and the normalized Levenshtein distance[2] [11], identified as the best-performing benchmark in previous works [9,10].

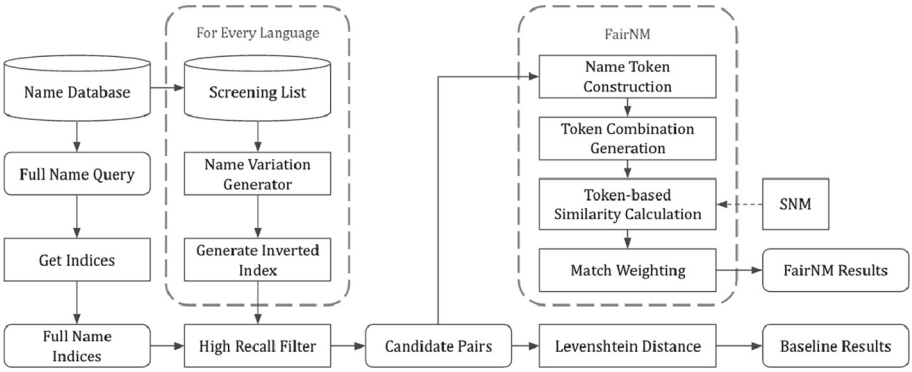


Fig. 3. Schematic overview of the test bench.

Name fuzziness is introduced through the Name Variation Generator by applying the variations proposed in [10], extended with swapped name tokens, of which examples are provided in Table 2[3].

To pre-filter candidates, we employ an inverted-index n-gram filter (using padded trigrams). Query and screening names are indexed, and only candidates with an overlap coefficient greater than 0.5 are retained for similarity scoring. This approach ensures full recall while substantially reducing computational cost.

[2] We use the RapidFuzz Python package: https://pypi.org/project/rapidfuzz/.
[3] A fat-finger error swaps a character with its neighboring key on a QWERTY keyboard.

Table 2. Example of output of name variation generator.

Name Variation	Output
Full Name	Juan de Salamanca
Swapped Names	de Salamanca Juan
Random Deletion	Jun Cerezo de Salamanca
Fat-finger Error	Juan Cerezl de Salamanca

3.2 FairNM

FairNM is a novel name matching solution that improves fairness by combining three key enhancements, as previously summarized in Fig. 3.

Token-Based Similarity Calculation. We adapt a variation of the token-based approach in [17] to relieve the challenges of handling swapped and merged tokens. Given two names, we construct all name token combinations, including merged forms (e.g., matching 'JuanCerezo' with both 'Juan' and 'Cerezo'). Each pair is scored using normalized Levenshtein similarity, and an optimal one-to-one alignment is computed based on the highest scores. Merged tokens are kept only if their score exceeds all corresponding individual token matches. The resulting score is the average of the best-aligned token similarities. A full pseudocode of the alignment process is provided in Algorithm 1.

To illustrate the working principle, let us consider the matching name pair ('Ahmed Mohammed Ahmad', 'Ahmad Achmed') of which the optimal token-based normalized Levenshtein similarities are shown below.

	Ahmed	Mohammed	Ahmad
Ahmad	×	×	1.00
Achmed	0.83	×	×

The resulting similarity score is computed by averaging the matched scores, yielding $(1.0 + 0.83)/2 = 0.92$. In contrast, the conventional normalized Levenshtein similarity would have yielded a score of 0.45, whilst the similarity between the names can be considered significantly higher.

Short Name Module. Short Name Module (SNM) enhances accuracy in cases where both name tokens are shorter than four characters, as shorter tokens are more susceptible to distortion from typographical errors. For these instances, we employ a Siamese Neural Network (SNN) specifically trained on short-token similarity (see Appendix A for training details). This module is particularly effective in reducing fairness disparities for linguistic groups, such as Chinese and Korean, where shorter names are more prevalent.

Algorithm 1. optimalAlignment

Input: allSim, a dictionary which keys are the location of the assessed token in name1 (e.g., (0, 1) if merged first and second name, etc.) and the values are lists containing tuples that are sorted on simScore from high to low. The tuples are formatted as: (token1, token2, location token1, location token2, simScore).

- findNext(token) : Finds next best available match for a token in name1 and adds it to bestMatch dictionary

```
bestMatch ← {}
matchedNames ← {}
for all key, value ∈ allSim do
    token1, token2, loc1, loc2, simScore = first entry of value
    if token2 ∉ matchedNames.keys then
        bestMatch[key] = (token1, token2, loc1, loc2, simScore)
        matchedNames[token2] = key
    else                                   ▷ If we already found a match for this token2
        if simScore > bestMatch[key].get(simScore) then
            findNext(bestMatch[key].get(token1))
            bestMatch[key] = (token1, token2, loc1, loc2, simScore)
        else
            findNext(token1)
for all key ∈ bestMatch do                 ▷ Analyze merged tokens
    if len(key) > 1 then
        if sim merged tokens > sim separate tokens then
            remove separate tokens from bestMatch
        else
            remove merged tokens from bestMatch
return bestMatch
```

Match Weighting. Match Weighting aims to increase emphasize on rare names while reducing focus on common ones. To derive weights, we create an up-sampled dataset of 50,000 full names per language code. Each name is tokenized, and the frequency of each distinct token (n_t) is counted in this dataset. Hereafter, the weight (w_t) per name token is calculated using 'inverse document frequency', $w_t = \log(N/n_t)$, where N is the total number of distinct name tokens.

During similarity scoring, each matched token pair receives a combined weight based on the sum of the individual token weights. These combined weights are normalized by their total sum (w_{tot}), and the final weighted similarity score is computed as:

$$simScore = \sum_{m=1}^{M} \frac{w_{m1} + w_{m2}}{w_{tot}} \times sim_m$$

where M denotes the number of matched token pairs, w_{m1} and w_{m2} are the weights of the tokens from each name, and sim_m is the corresponding similarity

score. To illustrate, consider the following example, with token weights shown in parentheses:

	Ho-young (74.8%)	Lee (25.2%)
Hyun-Jun (72.0%)	0.36	×
Lee (28.0%)	×	1.0

This results in a weighted score of

$$\frac{0.720 + 0.748}{2} \times 0.36 + \frac{0.280 + 0.252}{2} \times 1.0 = 0.53.$$

Without weighting, the perfect match in the most common Korean surname 'Lee' would dominate the score (yielding 0.68). However, applying match weighting appropriately reduces the influence of common tokens, leading to a more balanced and fair similarity assessment.

A summarizing overview of the proposed FairNM Similarity Calculation is presented in Algorithm 2.

Algorithm 2. FairNM Similarity Calculation

Input: A name pair containing two full names, where name1 is the name with the least tokens.

- combGen(name1, name2) performs all required steps up until Token Combination Generation from Fig. 3;
- SNM(token1, token2) returns SNM token similarity;
- normLev(token1, token2) returns normalized Levenshtein token similarity;
- sortMatch(dict) returns a dictionary where the found matches per token of name1 are sorted from high to low based on their simScore;

```
allComb = combGen(name1, name2)
allSim ← {}
for all combination ∈ allComb do
    token1, loc1 = combination[0]
    token2, loc2 = combination[1]
    if (|token1| < 4 and |token2| < 3) or (|token1| < 3 and |token2| < 4) then
        sim = SNM(token1, token2)
    else
        sim = normLev(token1, token2)
    add (token1, token2, loc1, loc2, sim) to allSim
sortMatch(allSim)
bestMatch = optimalAlignment(allSim)
finalScore = matchWeight(bestMatch)
return finalScore
```

3.3 Evaluation Measures

In this section, we discuss the metrics used to assess and compare the performance and fairness of the matching algorithms. While performance evaluation in EM is well-studied, fairness measures specific to name matching are relatively scarce in literature.

Performance is assessed via F_1 scores under two conditions: a recall-restricted environment (recall > 0.98), relevant to high-stakes applications such as law enforcement or healthcare, and an unrestricted environment to reflect general contexts.

Most fairness metrics in machine learning assume binary classification, where each instance is assigned a positive or negative label [13]. In contrast, name matching or EM algorithms may return one, multiple, or no matches, making fairness assessment less straightforward.

Equalized Odds [8], a common fairness criterion, requires equal True Positive Rates (TPR) and False Positive Rates (FPR) across groups. However, in name matching, the abundance of true negatives makes satisfying the FPR condition trivial. As a result, Equalized Odds reduces to requiring equal recall (TPR), which is insufficient on its own, since many applications also demand equal precision.

To address this, we propose a stricter fairness measure combining Equalized Odds and Predictive Parity [6], requiring parity in both recall and precision across groups. It is defined as:

$$1 - \frac{\Delta p + \Delta r}{2} = 1 - \frac{1}{2}\left(\left(\max_{\forall x \in A} p_x - \min_{\forall x \in A} p_x\right) + \left(\max_{\forall x \in A} r_x - \min_{\forall x \in A} r_x\right)\right) \quad (1)$$

Here, Δp and Δr represent the largest differences in precision and recall between any two subpopulations $x \in A$, where A is the set of all groups. A fairness score near 1 indicates similar model behavior across groups, while larger disparities yield lower scores.

For example, if an algorithm yields similar precision and recall for Western, African, and Asian name groups, the fairness score approaches 1. Significant performance gaps between groups would reduce it accordingly.

4 Results

Figure 4 shows that all three FairNM components improve the $F1$ score, while Match Weighting and SNM enhance fairness. Each module contributes individually, and their combination results in both improved performance and fairness, especially in recall-restricted environments.

This is remarkable, as improving fairness often comes at the expense of other performance metrics. For example, fairness-aware decisions in a hiring decision system, such as adjusting criteria based on race, can reduce bias but may also lead to the rejection of qualified applicants who would have been approved under a purely performance-driven approach.

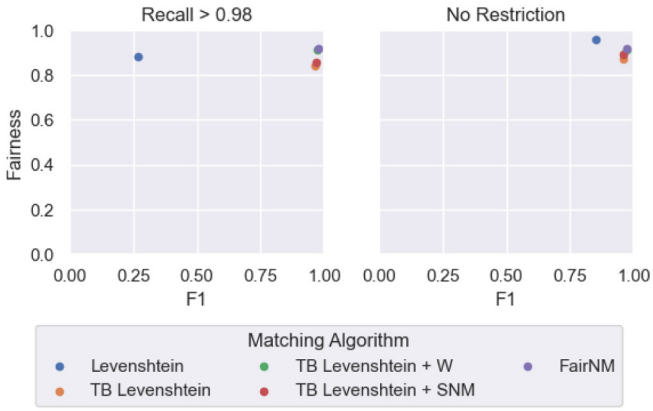


Fig. 4. Impact of FairNM modules (TB = Token-Based, W = Weighted). Dots represent results for the threshold with maximum harmonic mean between Fairness and $F1$.

In the recall-restricted setup, the token-based Levenshtein increases $F1$ but initially reduces fairness. Match Weighting and SNM mitigate this decline, with weighting offering the largest improvement. When combined, the modules show to complement each other, indicating that they tackle different fairness issues.

In the unrestricted setup, fairness drops slightly with token-based Levenshtein, but this is largely recovered (<0.04 loss) when SNM and weighting are applied. Overall, FairNM shows to be most effective under recall constraints, which are relevant to high-stakes scenarios where fairness is often critical.

5 Conclusion

This study provides valuable insights into cross-lingual differences in name characteristics and their impact on name matching performance by categorizing person names based on their lingual origin rather than race or ethnicity. The data descriptive analysis reveals that names of Korean and Chinese origin tend to be shorter on average and display less variation in family names but more variation in first names.

A novel test bench architecture is proposed to simulate real-world scenarios of fuzzy name matching tasks, specifically screening operations. This test bench facilitates comprehensive testing of name matching across a wide spectrum of real-life name variations. Additionally, a novel fairness measure is introduced to analyze the fairness of results in name matching.

Drawing from insights gained in the data analysis and existing research, the FairNM algorithm is introduced as a novel solution. FairNM adopts a token-based approach designed to effectively manage challenges posed by missing or swapped name tokens. Further enhancements, such as the inclusion of SNM

and Match Weighting, are integrated to enhance fairness in the name matching process.

Benchmarking against the normalized Levenshtein distance, the FairNM algorithm and its individual components demonstrate significant improvements in both performance and fairness, particularly in recall-constrained environments. This is particularly relevant as these constrained scenarios typically encompass high-stakes situations where fairness holds paramount importance.

While this work contributes to the understanding of fairness in the context of name matching, further research is deemed valuable to enhance this understanding. Specifically, conducting a reassessment using a non-artificial name database would help validate the proposed method in a real-life setting.

A SNN Training Setup

The setup of the utilized SNNs is inspired by a previously proposed algorithm called Deezymatch [9]. SNN consists of two identical artificial neural networks (i.e., they share their weights and biases) each capable of learning the hidden representation of a certain input name. Since the work of [2], which focused on matching location names, strongly resembles our goal, we adopt their best performing hyperparameters for this research.

To assess the performance of SNN, it is necessary to obtain representative training and test sets, where the training set can be subdivided in a training and validation part. The test set is constructed by extracting 3,000 names from each evaluated language group, specifically for the purpose of executing the previously presented test bench. Importantly, this test set is not used during the training process. To train the SNN, a large dataset consisting of name pairs categorized as either matching or non-matching is required. This dataset is obtained as follows:

1. Sample 50,000 full names, with replacement, from each language code out of the training set. We opt for a balanced training dataset in terms of language codes instead of a dataset that represents the original data distribution to reduce the chance of inherent bias in the training phase.
2. For half of the sampled names, pair each name with one of the four name variations: full name, swapped names, fat-finger error, or random deletion. This pairing results in the creation of the 'matching' training samples.
3. For the remaining half of the sampled names, 'mismatching' training samples are generated. Half of the samples are linked to a randomly selected mismatching name, whereas the other half is linked to a mismatching name with a padded tri-gram overlap coefficient of at least 0.5. This allows the model to better differentiate between non-matching names that exhibit slight similarities and those that are distinctly dissimilar.

Following the above steps, we obtain a balanced dataset consisting of 300,000 matching pairs and 300,000 non-matching pairs. This dataset can be further divided into training (90%) and validation (10%) subsets to facilitate the training of the algorithm. Note that for the short name module, solely names shorter than 4 characters are sampled.

References

1. Ambekar, A., Ward, C.B., Mohammed, J., Male, S., Skiena, S.: Name-ethnicity classification from open sources. In: Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, SIGKDD 2009, pp. 49–58. ACM (2009)
2. Ardanuy, M.C., Hosseini, K., McDonough, K., Krause, A., van Strien, D., Nanni, F.: A deep learning approach to geographical candidate selection through toponym matching. In: Proceedings of the 28th International Conference on Advances in Geographic Information Systems, SIGSPATIAL 2020, pp. 385–388. ACM (2020)
3. Christen, P.: Data Matching - Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection. Data-Centric Systems and Applications, 1st edn. Springer, Cham (2012)
4. Cohen, W., Ravikumar, P., Fienberg, S.: A comparison of string metrics for matching names and records. In: Proceedings of the 2003 International Conference on Information Integration on the Web, IIWEB 2003, pp. 73–78. ACM (2003)
5. Cruz, K.: A guide to Korean family names (2021)
6. Dwork, C., Hardt, M., Pitassi, T., Reingold, O., Zemel, R.S.: Fairness through awareness. In: Proceedings of the 3rd Conference on Innovations in Theoretical Computer Science, ITCS 2012, pp. 214–226. ACM (2012)
7. Elmagarmid, A.K., Ipeirotis, P.G., Verykios, V.S.: Duplicate record detection: a survey. IEEE Trans. Knowl. Data Eng. **19**(1), 1–16 (2007)
8. Hardt, M., Price, E., Srebro, N.: Equality of opportunity in supervised learning. In: Proceedings of the 30th International Conference on Neural Information Processing Systems, NIPS 2016, pp. 3315–3323. Curran Associates Inc. (2016)
9. Hosseini, K., Nanni, F., Ardanuy, M.C.: Deezymatch: a flexible deep learning approach to fuzzy string matching. In: Proceedings of the 25th Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, pp. 62–69. ACL (2020)
10. Karakasidis, A., Pitoura, E.: Identifying bias in name matching tasks. In: Proceedings of the 22nd International Conference on Extending Database Technology, EDBT 2019, pp. 626–629. OpenProceedings.org (2019)
11. Levenshtein, V.I.: Binary codes capable of correcting deletions, insertions, and reversals. Soviet Physics-Doklad **10**, 707–710 (1965)
12. Mateos, P.: A review of name-based ethnicity classification methods and their potential in population studies. Popul. Space Place **13**(4), 243–263 (2007)
13. Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., Galstyan, A.: A survey on bias and fairness in machine learning. ACM Comput. Surv. **54**(6), 115:1–115:35 (2014)
14. Mukerjee, A., Biswas, R., Deb, K., Mathur, A.P.: Multi-objective evolutionary algorithms for the risk-return trade-off in bank loan management. Int. Trans. Oper. Res. **9**(5), 583–597 (2002)
15. O'Keeffe, A., McCarthy, M.J.: The Routledge Handbook of Corpus Linguistics, 1st. edn. Routledge (2010)
16. Snae, C.: A comparison and analysis of name matching algorithms. Int. J. Comput. Inf. Eng. **4**(1), 107–112 (2007)
17. Treeratpituk, P., Giles, C.L.: Name-ethnicity classification from open sources. In: Proceedings of the 26th AAAI Conference on Artificial Intelligence, AAAI 2012, vol. 1, pp. 1141–1147. AAAI Press (2012)
18. Yeung, J.: Why 1.2 billion people share the same 100 surnames in China (2021)

Is a Similar Task Useful for Few-Shot Selection? Aspect Term Extraction Using LLM

Koki Imazato[1(✉)] and Kazutaka Shimada[2]

[1] Department of Creative Informatics, Kyushu Institute of Technology, Fukuoka, Japan
imazato.kouki927@mail.kyutech.jp

[2] Department of Artificial Intelligence, Kyushu Institute of Technology, Fukuoka, Japan
shimada@ai.kyutech.ac.jp

Abstract. In-Context Learning (ICL) has emerged as a powerful method for Large Language Models (LLMs) to perform tasks by utilizing prompts without additional training. Few-shot learning, a subset of ICL, enhances model performance by providing a small number of task-specific examples. However, the effectiveness of few-shot learning depends heavily on the selection of examples. Therefore, it is crucial to choose representative few-shot instances. In this study, we propose a novel few-shot selection method that leverages similar tasks instead of directly using the target task data. The target task in this paper is aspect term extraction on sentiment analysis. We introduce two selection strategies as similar tasks: one based on named entity recognition (NER) utilizing Predictive Entropy (PE) and another based on positive-negative classification using Information Gain (IG). In the experiment, the IG-based selection significantly outperformed baseline approaches: random selection and confidence-based selection. Our findings highlight the importance of choosing few-shot instances with small uncertainty. This suggests that informative and clearly structured instances contribute to better performance in LLM-based few-shot learning. The proposed method provides an effective way to optimize few-shot selection without relying on target task data.

Keywords: Few-shot selection · Large language model · Aspect Term Extraction

1 Introduction

In recent years, Large Language Models (LLMs), such as GPT, have become capable of performing tasks using only instructions, known as prompts. As a result, these models no longer require large amounts of data. The method of performing tasks through prompts is called In-Context Learning (ICL). Among

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 47–57, 2026.
https://doi.org/10.1007/978-3-031-97144-0_5

ICL methods, the approach that incorporates input-output examples into the prompt is known as few-shot learning. When few-shot examples are provided to an LLM, the model learns task patterns and solutions [1]. As a result, the model can achieve higher accuracy. However, few-shot learning has been reported to have a problem where task performance varies significantly depending on the few-shot examples provided [8]. Since LLMs are sensitive to the content of few-shot examples, selecting the few-shot is an important issue in ICL.

To solve this problem, this paper proposes an appropriate few-shot selection method. We refer to the task performed by an LLM as a target task. In this paper, the target task is aspect term extraction [11,12] on sentiment analysis. It is ideal to use data from the target task to select appropriate examples. However, in practice, there are often cases where sufficient data is not available. Therefore, we select appropriate few-shot instances without the dataset of the target task. We utilize similar tasks to select few-shot instances instead of directly relying on the target task. We refer to the similar tasks used for few-shot selection as sub-tasks.

In this paper, we introduce two types of sub-tasks for the few-shot selection: named entity recognition-based and positive-negative classification-based approaches. Aspect term extraction aims to extract specific elements or items from text that are the target of evaluation. For example, in product reviews, aspect term extraction automatically extracts which elements users are evaluating, such as "quality", "design", and "price". The following are an example sentence and its aspect term.

- Sentence: The comfort of this chair is excellent.
- Aspect term: The comfort of this chair.

Since the aspect term extraction is to extract strings that match specific conditions, it is similar to named entity recognition (NER). Therefore, we apply general NER, such as extraction of personal names and location, to a sub-task. On the other hand, aspect term extraction is a task within sentiment analysis. In other words, another task in sentiment analysis is similar to aspect term extraction. One of the most famous sentiment analysis is positive-negative (PN) classification. Therefore, we apply PN classification to a sub-task.

Figure 1 shows the outline of our method. It consists of two phases: few-shot selection based on a sub-task and few-shot learning with LLM for the target task. Our contribution is to perform appropriate few-shot selection by using only the knowledge from similar tasks (the blue dashed line in Fig. 1), without directly using the knowledge from the target task[1]. As mentioned above, we introduce NER-based and PN-based approaches into the few-shot selection. In the few-shot selection, the model trained on the sub-task data selects appropriate few-shot instances from the few-shot candidates. In other words, a measure for selection is necessary. For the NER-based approach, we apply Predictive Entropy (PE) by Zhu et al. [13]. PE is a measure that quantifies the uncertainty in the predictions

[1] Note that the few-shot learning with an LLM (the red dashed line in Fig. 1) is conducted through standard procedures, with no special adjustments in this paper.

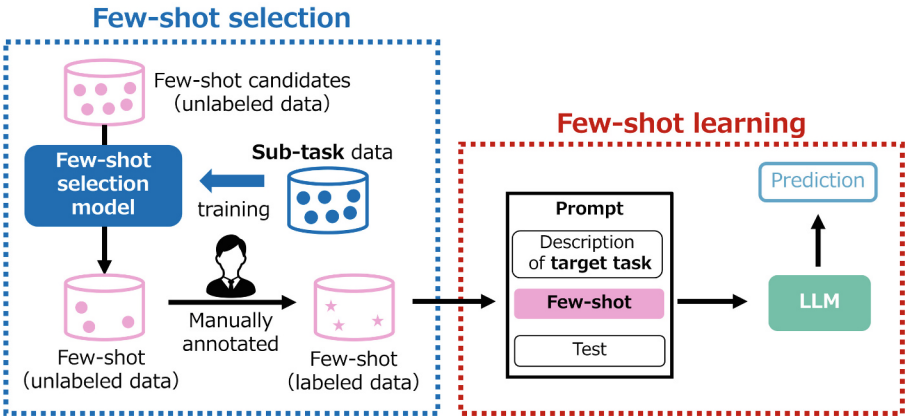


Fig. 1. The outline of our method in this paper. (Color figure online)

of a sequence labeling model. It is used to evaluate the representativeness of given data. For the PN-based approach, we apply Information Gain (IG) by Liu et al. [7]. IG is a measure that quantifies the uncertainty in the predictions of a text classification model. It is used to evaluate the value of information in given data. In the experiment, we compare the performance of two approaches. The experimental result shows the effectiveness of the IG-based approach. It demonstrates that the proposed method can select useful instances by leveraging knowledge from similar tasks, even when there is limited data available for the target task.

2 Proposed Method

2.1 Basic Idea

The proposed method does not directly use the target task for the few-shot selection model. Instead, it utilizes a sub-task to select the few-shot examples. The condition for a sub-task is that it is similar to the target task and has sufficient data for training. We employ two sub-tasks for few-shot selection.

One is named entity recognition (NER). NER is a task that extracts named entities in a sentence and assigns specific labels to them. An example of NER is shown below:

- Sentence: Yesterday, I went to Hakata, Yoshizuka, Chihaya, and Kashii, but Hakata Station was very crowded.
- Named Entities: [`Location`] Hakata Yoshizuka Chihaya Kashi

NER extracts entities from a sentence, while aspect term extraction extracts aspects. It is similar to aspect term extraction because they are to identify strings with specific roles from a text. Additionally, there are a lot of available datasets for many domains in NER.

The other is Positive-Negative (PN) classification. PN classification is a task that determines whether a text is positive or negative opinions. An example of PN classification is shown below:

- Sentence: This movie is interesting. I love it.
- Sentiment: Positive

PN classification is similar to aspect term extraction because they are both within sentiment analysis. Since PN classification is a representative task in sentiment analysis and a well-researched area, a lot of datasets are publicly available.

2.2 Measure for Selection

In this paper, we introduce NER-based few-shot selection and PN-based few-shot selection. Here, we need a measure for each selection approach.

Predicted Entropy (PE) for NER: PE is a measure that quantifies the uncertainty inherent in the sequence labeling model's predictions. It is used to evaluate the representativeness of the instance. Zhu et al. [13] used PE to measure the uncertainty of the instance and selected the few-shot instances for the sequence labeling task. The formula for calculating PE is shown in Eq. 1.

$$PE(x_i) = -\sum_{j=1}^{C} p(y_i = c_j \mid x_i) \log p(y_i = c_j \mid x_i) \tag{1}$$

PE is calculated for each token, and the PE of the instance x is defined as the average value $H(x)$.

$$H(x) = \frac{1}{N_x} \sum_{i=1}^{N_x} PE(x_i) \tag{2}$$

where N_x is the number of tokens in the instance x. If $H(x)$ is too low, the instance may be very simple, while if it is too high, the instance may be noisy. Zhu et al. demonstrated that when $H(x)$ is close to the average μ_H of the entire dataset, the instance is considered representative.

$$R_{score} = |H(x) - (\mu_H + \lambda * \sigma_H)| \tag{3}$$

where σ_H is the standard deviation, and λ takes the value of either 0 or 1. By selecting the instance with the lowest R_{score}, it becomes possible to acquire representative few-shot instances.

The proposed few-shot selection method using PE on NER in this study is referred to as Few-shot$_{PE}$. Few-shot$_{PE}$ calculates PE values from a general NER model and selects few-shot instances. The overview of Few-shot$_{PE}$ is shown in Fig. 2. First, a dataset on NER tasks is used to fine-tune the pre-trained model. Next, the fine-tuned model is used to perform NER on each few-shot candidate

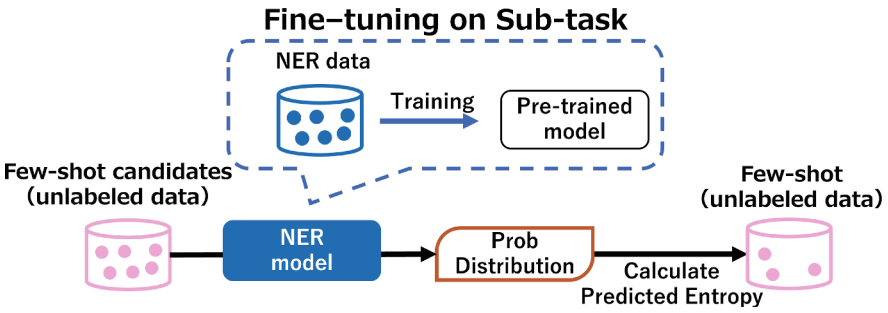


Fig. 2. The outline of Few-shot$_{PE}$.

(the unlabeled data). The model outputs the predicted probabilities for each label for each token. Using these predicted probabilities, the PE values ($H(x)$) for each candidate are calculated. After sorting all the few-shot candidates by the R_{score} values, the top n instances are selected and provided to the LLM.

Information Gain (IG) for PN: IG is a measure that quantifies how much the uncertainty of predictions decreases by observing an instance. It is used to measure the informational value of the instance. Liu et al. [7] used IG to measure the informational ability of the instance and selected the few-shot instances for the text classification task. The formula for calculating IG is shown in Eq. 4.

$$IG\left(Y|x_i\right) = H\left(Y\right) - H\left(Y|x_i\right) \tag{4}$$

where $H(Y)$ is the entropy of the LLM's predictions. $H(Y|x_i)$ represents the entropy of the LLM's predictions when the few-shot candidate instance x_i is input into the LLM. Entropy quantifies the uncertainty of information, that is, the difficulty of making predictions. $H(Y)$ is constant because it does not depend on the few-shot candidate instance x_i. Therefore, the value of IG is maximized when the conditional entropy $H(Y|x_i)$ is minimized. By selecting the instance x_i that maximizes IG, it becomes possible to acquire few-shot instances that contain information of reducing prediction uncertainty.

Here we discuss the difference between our approach and Liu's approach (the previous research with IG for few-shot selection). Calculating IG requires the prediction probabilities from the model. However, in LLMs like GPT-4, it is often not possible to directly obtain the probability distribution of predictions from the model. Furthermore, the previous research has focused solely on text classification tasks. In text classification tasks, a label is assigned to a single instance (the entire text), which allows for a clear definition of IG. However, in sequence labeling tasks like aspect term extraction, the labeling target often consists of multiple elements, such as compound nouns. As a result, the assignment of aspect labels becomes partial and variable, and it is difficult to clearly define IG for a single instance. Therefore, we introduce PN classification as the sub-task for aspect term extraction (our target task in this paper). Since PN classification

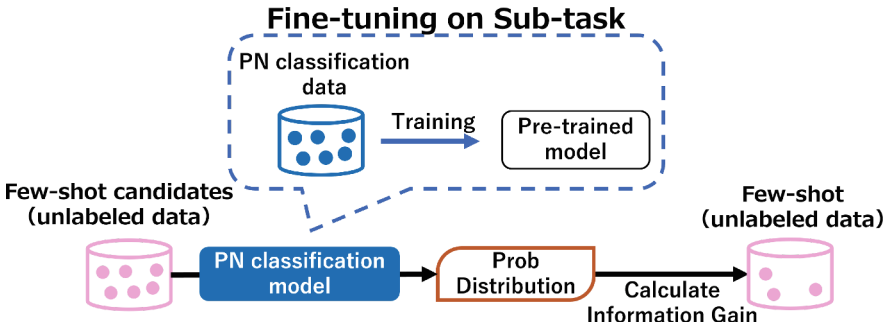


Fig. 3. The outline of Few-shot$_{IG}$.

is a text classification task, we can easily calculate IG values. The overview of Few-shot$_{IG}$ is shown in Fig. 3. First, a dataset on PN tasks is used to fine-tune the pre-trained model. Next, the fine-tuned model is used to perform PN classification on each few-shot candidate (the unlabeled data). The model outputs the predicted probabilities about the positive and negative classes of each instance by using the softmax function. Using these predicted probabilities, the IG values for each candidate are calculated. After sorting all the few-shot candidates by their IG values, the top n instances are selected and provided to the LLM.

The proposed few-shot selection method using IG on PN classification in this study is referred to as Few-shot$_{IG}$. Few-shot$_{IG}$ calculates Information Gain from the PN classification model and selects few-shot instances.

3 Experimental Setup

In this section, we describe the result of the aspect term extraction task with few-shot selection based on the sub-task. We compare two types of the number of few-shot instances: 2-shot and 8-shot.

3.1 Dataset and Model

Target Task: Aspect Term Extraction. We use the dataset developed by Kurihara et al. [6] for the aspect term extraction. This dataset consists of tweets containing opinions and impressions, with aspect terms manually annotated. In this study, we focus on the 4,762 tweets in the dataset that contain aspect terms. Each tweet contains one aspect term. The experiments are conducted using 5-fold cross-validation. In other words, we use the $\frac{1}{5}$ of the dataset for testing and the $\frac{4}{5}$ for few-shot candidates in Fig. 1[2].

[2] In our method, we need a manual annotation process for selected instances because the candidates are essentially unlabeled. However, in the experiment, since we used $\frac{4}{5}$ of the target task dataset as the few-shot candidates, we used the label in the dataset instead of the manual annotation process. Note that we did not use the label information for few-shot selection processes.

The LLM in Fig. 1 for aspect term extraction is gpt-4o-mini[3] (GPT). As mentioned in Sect. 1, the few-shot learning with an LLM has no special adjustments in this experiment because our contribution is few-shot selection with sub-tasks. The temperature value was set to 0.

Sub-task: Few-Shot$_{PE}$: We use a Japanese named entity recognition dataset[4] published by Stockmark as the training data. This dataset consists of sentences extracted from Wikipedia, with named entities manually annotated. Each instance is composed of one sentence, and the total number of data is 5,343. The number of entity labels[5] is 8.

For the NER model, we use BERT[6] [2]. The learning rate, number of epochs, and batch size are 1e−5, 15, and 16 respectively.

Sub-task: Few-Shot$_{IG}$: We also use the ACP Corpus [5] developed by Kaji and Kitsuregawa for the PN classification model. In this experiment, 10,000 instances randomly selected from the corpus are used as the training data for the few-shot selection model.

We also use BERT for the PN classification model. The learning rate, number of epochs, and batch size are 5e−5, 8, and 16, respectively.

3.2 Baselines

We prepared two baseline methods: Few-shot$_{RND}$ and Few-shot$_{LCS}$. The details are as follows:

- **Few-shot$_{RND}$** is a method that randomly selects instances from the few-shot candidates and provides them as few-shot to GPT.
- **Few-shot$_{LCS}$** is a method proposed by Imazato and Shimada. [4]. Few-shot instances are selected on the basis of the concept of Least Confidence Score (LCS) that is used in active learning [9]. In the same way as Few-shot$_{IG}$, the model is also fine-tuned for PN classification. It provides instances with low confidence in the prediction (i.e., instances that are judged difficult to predict by the model) as few-shot to GPT.

3.3 Evaluation Criteria

Aspect term extraction is evaluated based on exact match and partial match. The exact match is counted when both the starting and ending points of the GPT's predicted range correspond to the gold standard. The partial match is counted by the number of matching characters between the predicted string and the gold standard string. Below are examples of the exact match and partial match.

[3] https://platform.openai.com/docs/models.

[4] https://github.com/stockmarkteam/ner-wikipedia-dataset.

[5] Names about Personal, Corporate, Political organization, Other organization, Location, Facility, Product, and Event. Note that there is no relation from our target task, namely aspect term extraction.

[6] https://huggingface.co/tohoku-nlp/bert-base-japanese-v3.

Table 1. Extraction accuracy of aspect term

	Exact match			Partial match		
	Pre	Rec	F1	Pre	Rec	F1
2-shot$_{RND}$	0.397	0.392	0.395	0.475	0.517	0.492
2-shot$_{LCS}$	0.433	0.417	0.425	0.515	0.437	0.472
2-shot$_{PE}$	0.468	0.455	0.461	0.557	0.480	0.513
2-shot$_{IG}$	**0.506**	**0.500**	**0.503**	**0.560**	**0.596**	**0.577**
8-shot$_{RND}$	0.484	0.477	0.481	**0.561**	0.520	0.539
8-shot$_{LCS}$	0.428	0.420	0.424	0.520	0.437	0.474
8-shot$_{PE}$	0.482	0.476	0.479	**0.561**	0.502	0.528
8-shot$_{IG}$	**0.516**	**0.511**	**0.513**	0.559	**0.576**	**0.567**

– Sentence: Hakata ramen is the best.
– Aspect term$_{gold}$: Hakata ramen
– Aspect term$_{prediction}$: ramen is

In this example, since the prediction does not match the gold standard exactly, it is not counted as an exact match. In the case of partial match, since 5 characters (ramen) of the prediction are contained within the 12 characters (Hakata ramen)[7] of the gold standard, the Recall is $\frac{5}{12}$. Precision is $\frac{5}{8}$ because 5 of the 8 predicted characters (ramen is) are included in the gold standard.

4 Results and Discussion

The experimental results of aspect term extraction using GPT are shown in Table 1. The number at the beginning of each method represents the number of few-shot instances, namely 2-shot of 8-shot. The Few-show$_{PE}$ has a parameter λ (Eq. 3). In the experiment, we followed the $\lambda = 0$ and $\lambda = 1$ settings proposed by Zhu et al. [13]. As a result, the model with $\lambda = 0$ obtained higher accuracy. Therefore, the scores in the table are based on $\lambda = 0$. The highest values for each criterion in 2-shot and 8-shot are shown in bold.

The experimental result shows that Few-shot$_{IG}$ achieves the highest scores for most criteria in 2-shot and 8-shot. In general, a larger number of few-shot instances tends to improve the accuracy. However, for the partial match, Few-shot$_{IG}$ obtained the highest accuracy with 2-shot, as compared with 8-shot. These results show the effectiveness of Few-shot$_{IG}$. On the basis of the definition, Few-shot$_{IG}$ selects easy instances for PN classification. In other words, instances that are easy to classify in PN classification are well-suited as few-shot instances for GPT and the target task. The following is an instance from Few-shot$_{IG}$:

– Few-shot$_{IG}$: **東急東横線渋谷駅のドラクエ発車BGMは、どんなに疲れていても元気になれる。** (**The Dragon Quest departure BGM at Tokyu**

[7] Spaces are counted as characters..

Toyoko Line Shibuya Station can lift my spirits, no matter how tired I am.)

The words highlighted in bold are the aspect terms in the sentence[8]. Sentences that are easy for PN classification tend to contain explicit aspect terms. As a result, they are expected to serve as effective instances when GPT performs aspect term extraction. Gonen et al. [3] reported that prompts with lower uncertainty for the model enable it to perform tasks more effectively. The results in this experiment are consistent with this finding.

Next, we compare LCS and IG, both of which use the PN classification as the sub-task. The scores of Few-shot$_{LSC}$, one baseline in this experiment, are lower than those of Few-shot$_{IG}$. The following is an instance from Few-shot$_{LSC}$:

– Few-shot$_{LCS}$: なかなか会場に入れない隼達がたくさん。そのくらい集まってます!すげぇ。 #**隼駅まつり** URL (There are many Hayabusa that can't get into the venue easily. It's that crowded! Amazing. #**Hayabusa Station Festival** URL)

The principle of LCS is the opposite of that of IG. LCS selects instances with high uncertainty, i.e., the model has low confidence in the predictions. This indicates that it selects instances with higher difficulty for the task. Unlike in the case of IG, Few-shot$_{LCS}$ selected instances that are not efficient when GPT performed the target task, aspect term extraction. As a result, the accuracy decreased. Peng et al. [10] proposed a method that considers the similarity to test data while resampling instances with low uncertainty. The results from Peng's study and our experiment demonstrate that it is essential to select instances with low uncertainty during the few-shot selection process.

We proposed not only Few-shot$_{IG}$ but also Few-shot$_{PE}$ based on NER. Few-shot$_{PE}$ obtained good results in 2-shot selection, as compared with two baselines (Few-shot$_{RND}$ and Few-shot$_{LCS}$. From the definition, Few-shot$_{PE}$ has the characteristic of selecting representative instances within the dataset. This characteristic is effective from a diversity perspective when the number of selected instances is small. However, this advantage decreases if the number of selected instances increases. As a result, Few-shot$_{PE}$ did not outperform Few-shot$_{RND}$ in the 8-shot situation. As another characteristic, Few-shot$_{PE}$ tends to select shorter sentences compared to Few-shot$_{IG}$. The average number of characters in the instances selected by PE was 6.42, while it was 11.1 for IG. One characteristic of PE is to avoid instances that are either extremely difficult or extremely easy for NER. As a result, it may have tended to select sentences with simple structures and little information. The experimental results suggest that it is more important to "select instances that are rich in information with low uncertainty" rather than "select representative instances in the dataset," in few-shot selection.

[8] In the actual procedure, the bolded words correspond to the parts that are manually annotated during the few-shot selection process in Fig. 1.

5 Conclusion

This paper proposed a method for selecting effective Few-shot instances for Large Language Models (LLMs). The key point of our method is that it does not use information from the target task for selection. For the selection, we introduced a sub-task that is similar to the target task. In this paper, the target task is aspect term extraction on sentiment analysis, and we attempted to select appropriate instances by using NER or PN as the sub-task. NER is similar in the extraction processing, and PN is similar in sentiment analysis. For NER, we used Predicted Entropy (PE) as the selection criterion, and for PN, we used Information Gain (IG). In the experiment, we prepared two baseline methods with random selection and LCS. The experimental results showed that Few-shot$_{IG}$ achieved the best accuracy. In cases with fewer shots, namely 2-shot, Few-shot$_{PE}$ was also effective. The results suggest the effectiveness of selecting instances with high information and low uncertainty.

This paper focuses on aspect term extraction as the target task. However, the important point of the proposed method is "selecting similar tasks" for the target task. Therefore, our method can be applied to other tasks easily. In addition, although GPT was employed as the LLM in this study, we should validate the proposed method across multiple LLMs to ensure its generality. Applying our method to various tasks and validating its effectiveness across different LLMs is important future work.

Acknowledgements. This work was partly supported by JSPS KAKENHI Grant Number 23K11368.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Brown, T., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901. Curran Associates, Inc. (2020). https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf
2. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pp. 4171–4186. Association for Computational Linguistics (2019). https://doi.org/10.18653/v1/N19-1423, https://aclanthology.org/N19-1423
3. Gonen, H., Iyer, S., Blevins, T., Smith, N., Zettlemoyer, L.: Demystifying prompts in language models via perplexity estimation. In: Bouamor, H., Pino, J., Bali, K. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 10136–10148. Association for Computational Linguistics, Singapore (2023). https://doi.org/10.18653/v1/2023.findings-emnlp.679, https://aclanthology.org/2023.findings-emnlp.679/

4. Imazato, K., Shimada, K.: Automatic few-shot selection on in-context learning for aspect term extraction. In: 2024 16th IIAI International Congress on Advanced Applied Informatics (IIAI-AAI), pp. 15–20 (2024)
5. Kaji, N., Kitsuregawa, M.: Automatic construction of polarity-tagged corpus from HTML documents. In: Proceedings of the COLING/ACL 2006 Main Conference Poster Sessions, pp. 452–459. Association for Computational Linguistics (2006). https://aclanthology.org/P06-2059
6. Kurihara, S., Mizumoto, T., Inui, K.: Construction of a target-evaluation expression dataset for sentiment analysis on twitter (in Japanese). In: Proceedings of the Twenty-fourth Annual Meeting of the Association for Natural Language Processing, pp. 344–347 (2018). https://www.cl.ecei.tohoku.ac.jp/resources/twitter_target_review/
7. Liu, H., Wang, Y.: Towards informative few-shot prompt with maximum information gain for in-context learning. In: Bouamor, H., Pino, J., Bali, K. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 15825–15838. Association for Computational Linguistics, Singapore (2023). https://doi.org/10.18653/v1/2023.findings-emnlp.1060, https://aclanthology.org/2023.findings-emnlp.1060
8. Lu, Y., Bartolo, M., Moore, A., Riedel, S., Stenetorp, P.: Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 8086–8098. Association for Computational Linguistics (2022). https://doi.org/10.18653/v1/2022.acl-long.556, https://aclanthology.org/2022.acl-long.556
9. Monarch, R.M.: Human-in-the-Loop Machine Learning: Active Learning and Annotation for Human-Centered AI. Manning (2021)
10. Peng, K., et al.: Revisiting demonstration selection strategies in in-context learning. In: Ku, L.W., Martins, A., Srikumar, V. (eds.) Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 9090–9101. Association for Computational Linguistics, Bangkok (2024). https://doi.org/10.18653/v1/2024.acl-long.492, https://aclanthology.org/2024.acl-long.492/
11. Schouten, K., Frasincar, F.: Survey on aspect-level sentiment analysis. IEEE Trans. Knowl. Data Eng. **28**(3), 813–830 (2015)
12. Zhang, W., Li, X., Deng, Y., Bing, L., Lam, W.: A survey on aspect-based sentiment analysis: tasks, methods, and challenges. IEEE Trans. Knowl. Data Eng. **35**(11), 11019–11038 (2022)
13. Zhu, C., Shimada, K., Taniguchi, T., Ohkuma, T.: STAYKATE: hybrid in-context example selection combining representativeness sampling and retrieval-based approach – a case study on science domains. CoRR abs/2412.20043, pp. 1–11 (2024). https://arxiv.org/abs/2412.20043

IRIS: Rapid Curation Framework for Iterative Improvement of Noisy Named Entity Annotations

Ken Yano[1(✉)], Makoto Miwa[1,2], and Sophia Ananiadou[1,3]

[1] National Institute of Advanced Industrial Science and Technology, Tokyo 135-0064, Japan
yano.ken@aist.go.jp
[2] Toyota Technological Institute, Nagoya 468-8511, Japan
makoto-miwa@toyota-ti.ac.jp
[3] University of Manchester, Oxford Rd, Manchester M13 9PL, UK
sophia.ananiadou@manchester.ac.uk

Abstract. We propose IRIS (IteRative Improvement dS-ner), a rapid curation framework for the iterative improvement of noisy named entity annotations. The framework aims to provide an efficient and rapid method for curating noisy entity annotations initially made by distantly supervised named entity recognition (DS-NER). Unlike many existing entity annotation tools, which focus primarily on annotation from scratch, IRIS is designed to provide more efficient methods for improving entity annotations initially generated by DS-NER. This is enabled by robust annotation search capabilities and the automatic annotation capabilities suggested by the NER model, which is initialized with the DS-NER model and later incrementally updated during annotation. We also adopt Active Learning (AL), which allows curators to work with documents suggested by the system, rather than manually selecting them. This work also analyzes the trade-offs among different strategies for iteratively updating the NER model, such as selecting training samples and the strategies for fine-tuning the new NER model based on the previous model or training it from scratch.

Keywords: distantly supervised named entity recognition · entity annotation · curation web tool

1 Introduction

Annotating named entities is time-consuming and requires much human effort, especially in the biomedical domain, where the types of named entities are diverse. Hence, few existing natural language processing (NLP) tools support the target entity types, forcing manual annotation from scratch, which requires special skills and domain knowledge. To cope with these dilemmas, distantly supervised named entity recognition (DS-NER) based on domain dictionaries is efficient in reducing such annotation efforts [12,14,17,19].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 58–69, 2026.
https://doi.org/10.1007/978-3-031-97144-0_6

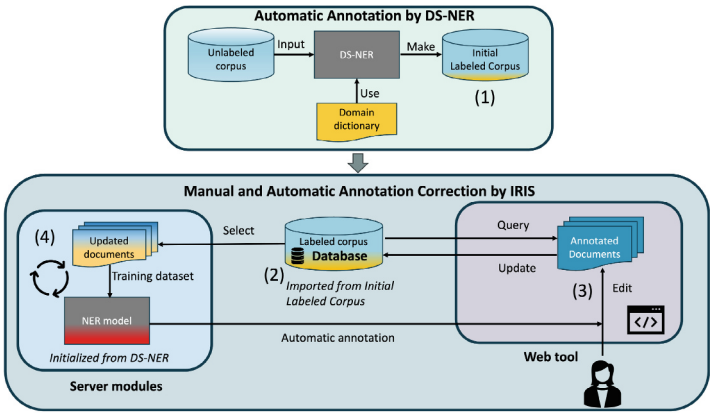


Fig. 1. The workflow of IRIS. (1) First, DS-NER annotates entity mentions automatically using a domain dictionary on unlabeled documents. (2) Initial annotated documents are imported into a database. (3) Curators use the web tool to search and correct annotations manually or automatically fix them according to the suggestions by the NER model. (4) During curation, the curators request that the NER model initialized with the DS-NER model be updated iteratively.

However, the quality of DS-NER annotations is severely dependent on distant supervision, such as the domain dictionary. Hence, human verification is necessary to verify and correct the results of automatic annotation, obtaining high-quality annotations suitable for secondary use, such as training datasets for machine learning and benchmark generation [10]. On the other hand, Active Learning (AL) has also been proven to be an efficient method of reducing manual annotation efforts [5,6,15].

However, to adopt AL, a small subset of reliable manual annotations is often necessary to train an NER model that can suggest the most informative unlabeled samples for annotation. Hence, the initial settings for entity annotation that adopt DS-NER and AL are crucial. Our proposed method aims to iteratively improve DS-NER's noisy entity annotation to address this discrepancy. Figure 1 overviews the workflow of the proposed IRIS curation framework. Based on an unlabeled corpus and target entity types, we generate domain dictionaries for each target entity type using an off-the-shelf ontology or thesaurus database such as UMLS [2]. Then, using the generated domain dictionary, the DS-NER model annotates named entities to create a possibly noisy annotated corpus. Then, the initially labeled corpus is imported into an annotation database (Elasticsearch). The web annotation tool offers user-friendly annotation search capabilities. It can retrieve target entity annotations by exact or partial match using keywords or extending the match condition based on entity concepts. The tool also enables full-text search by free phrases. Moreover, besides manual search, it adopts AL to prompt users to annotate the most informative documents. The NER model on a server module initialized with the DS-NER is iteratively updated by the curator's request using the training dataset selected from the updated labeled

documents. The web tool provides automatic annotation by comparing the current annotations with the new annotations suggested by the latest NER model.

To verify the efficacy of the automatic annotation, we conduct simulation-based experiments to evaluate the trade-offs among different strategies for iteratively updating the NER model regarding the selection of training samples and fine-tuning method to update the NER model. The results of the simulation suggest that (1) iterative update of the DS-NER model is possibly effective only when the size of the manually curated annotations are relatively small, and training a supervised NER model from scratch becomes more effective when the size becomes large, (2) using accumulated curated annotations to fine-tune the NER model is more favorable than using only newly updated annotations since the last update of the NER model. In our case study, the threshold for the size of accumulated annotations in terms of sentences was roughly 100, about 1.1% of 9080 in total, as described in finding 1.

To summarize, this paper aims (1) to present a novel entity curation framework called IRIS to describe its architecture and user interface, and (2) to evaluate the iterative improvement of noisy annotations made by DS-NER using simulation.

2 Related Work

There exist numerous entity annotation tools. brat rapid annotation tool [13] is an intuitive web-based tool with great visualization capabilities. It enables entity annotation and structured annotation, such as relations and events. WebAnno [18] is a web-based annotation tool that enables various annotation tasks using the concept of the annotation layer. INCEpTION [4], the updated version of WebAnno, includes human-in-the-loop annotation support by employing active learning and powerful corpus search functions. APLenty [6] adopts active and proactive learning, in which a learning algorithm can choose representative and informative data for annotation, thus helping reduce human annotation efforts. PubAnnotation [3] is a module-based annotation tool that enables the reuse of various annotation tools and resources. These existing entity annotation tools assume one starts annotations from scratch.

Instead of manual annotation, various methods of automatic entity annotation by DS-NER using domain-specific dictionaries have been proposed [12,14,17,19]. However, because DS-NER methods often have low recall issues due to the low coverage of the domain dictionary, improved techniques have been proposed, such as positive and unlabeled (PU) learning algorithms. For instance, Conf-MPU [7] proposed a risk minimization loss to improve performance, and DISTANT [17] proposed a spy-based PU learning method.

Some annotation tools support active annotations such as PubAnnotation [3] and INCEpTION [4]. For instance, INCEpTION [4] employs active annotation by suggesting one possible annotation, such as entity spans. Another line of annotation frameworks is based on the UIMA[1] specification, allowing interoperability and integration of different text processing components [9].

[1] https://uima.apache.org/.

3 System Description of IRIS

When IRIS launches, you can select active (Active Learning) or passive (Search & Edit) annotation mode on the top page to start annotating. One can return to the top page to change the annotation mode anytime. The main UI of the IRIS web tool is shown in Fig. 2.

3.1 Passive and Active Mode

In passive mode, the user searches for documents to curate using search keywords, as shown in Fig. 2(B). The relevance score of BM25 [11] sorts the search results. The supported keywords include document ID, entity mentions, and free-text phrases. One can input multiple query keywords by joining them with Boolean conditions: AND, OR, and NOT. For entity mention keywords, the match condition can be further controlled by three match modes: 1) exact, 2) partial, and 3) concept. The concept match is based on the concept of annotated entities. Under the hood, if available, the DS-NER associates an extracted entity mention with its concept name, such as a UMLS concept ID. Keyword-based searches in passive mode allow for a cursory view of the entire annotated document. This helps to identify the tendency and frequency of similar annotation errors, which in turn helps one plan their curation work.

In active mode, the framework selects the most informative documents for curation as shown in Fig. 2(A). In this mode, the framework ranks each document based on its confidence score, which is calculated by averaging the maximum probability of the NER model's outputs for each token. Then the framework sorts the records in ascending order by confidence score. When annotating from scratch, as the NER model has not been trained yet, it sorts documents by length in descending order [6]. The web tool's system settings allow users to set the number of suggested documents for active annotation and the confidence score threshold.

3.2 Annotation

Users can choose manual or automatic annotation correction in active and passive mode annotation by selecting the corresponding tab in the right pane, as shown in Fig. 2(C). The following explains the details of each annotation method.

In manual annotation correction, the currently selected document is shown in the upper part of the right pane, as shown in Fig. 3(A). Annotated entities are shown in different colors according to their entity types. The matched part of the entity, including text phrases, is shown in boldface when it matches the query keywords. In this text area, any entity annotation or text phrase can be directly added to the query keywords by selecting it and then opening the context menu to add it to the current query keywords. This method allows users to narrow the search results more efficiently. One can also toggle the display of annotation highlights to avoid distracting one's attention.

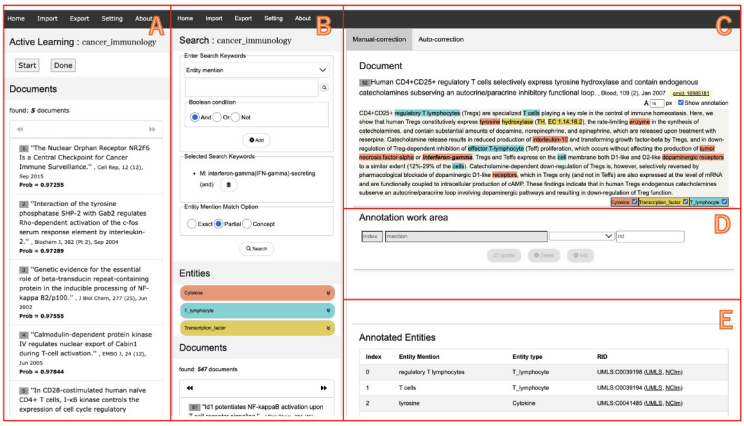


Fig. 2. UI of IRIS web tool (A) Side panel in active mode (B) Side panel in passive mode (C) Content of target document (D) Annotation work area (E) List of annotations. The right-hand panes (C), (D), and (E) are common to the passive and active modes.

One can add, update, or delete entity annotations in the Annotation work area, shown in the middle of Fig. 3(A). To update or delete an existing entity annotation, select the target annotation by clicking on it to load it in the annotation work area. The currently selected entity blinks to indicate that the following actions will be applied to this annotation. To update the span of the annotation, select a new span with the mouse. To add new annotations, one first selects the span of the new mention. To add or update an entity, select an entity type from the list, optionally enter the entity's resource ID (RID), and click the add or update button, respectively. (Note that we define RID as a more general term for the concept ID.) To delete, one presses the delete button. The table summarizing the currently registered entity annotations is at the bottom of Fig. 3(A).

In automatic annotation correction, the current annotations can be replaced by the new annotations suggested by the latest NER model. The UI of automatic annotation correction is shown in Fig. 3(B). One can request the server module to annotate the currently selected document by pressing the 'annotate' button. Then, the suggested new annotation is shown below the current annotation. On average, it takes less than 15 s, including the time it takes to load the NER model, to annotate one document with the NER model. Newly added or updated (either by changing their span or type) entity annotations are shown in red bold font, and removed or updated old entity annotations are marked with red wavy underlines. If you agree with the new annotation, you can replace it with the old one by pressing the 'OK' button.

3.3 Iterative Update of NER Model

The NER model under the framework is initialized with the trained DS-NER, which adopts Conf-MPU [20]. The update request of the NER model during

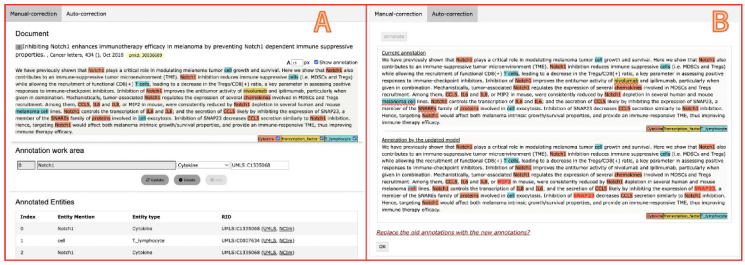


Fig. 3. (A) Manual annotation correction. The selected annotation is displayed in the annotation work area, where one can update, delete, or add a new annotation. The bottom is the list of annotations. (B) Automatic annotation correction. One can compare the current annotation with the new annotation suggested by the updated NER model. Sample text was cited from [8]

curation is triggered by a request from the setting menu and sent to the server modules via a CGI interface, as shown in Fig. 1. On the server side, updated documents are retrieved from the annotation database and split into training and validation datasets in a 9:1 ratio. Then, the NER model is trained using the training split and evaluated using the validation split. The training continues until the patience count is reached, which is set to five. Once the training is finished, the updated NER model is saved and registered in a table within the annotation database. The trade-offs concerning different strategies for the training dataset and the iterative update of the NER model are analyzed in detail in Sect. 4.

4 Analysis

The efficient methodology for updating the NER model based on the training dataset and the loss function used in training is not apparent. To investigate this, we conduct a simulation-based analysis.

4.1 Dataset

The BC5CDR dataset [16] contains annotations for chemical and disease entities. The dataset is split into training, validation, and testing subsets, each containing 500 documents. We concatenated the training and validation subsets to create the corpus for initial noisy annotation and used the testing subsets for evaluation.

We denote the training documents as $\mathbb{D} = \{D^i | i = 1, \ldots, 1,000\}$ and testing documents as $\mathbb{T} = \{T^i | i = 1, \ldots, 500\}$. Let sentences included in the i-th document $D^i = [\mathbf{S}^i_1, .., \mathbf{S}^i_{i^K}]$, tokens in the j-th sentence of the i-th document are defined as $\mathbf{S}^i_j = [x^i_{j,1}, .., x^i_{j,j^L_i}]$. Here we assume that the human annotation for each token is either non-entity (0), chemical (1), or disease (2), and we define the annotations corresponding to $\mathbf{S}^i_j$ as $\mathbf{A}^i_j = [y^i_{j,1}, .., y^i_{j^L_i}]$, where $y^i_{j,k} \in \{0, 1, 2\}$.

Table 1. Micro-average of Precision (P), Recall (R), and F1 score (F1) of BC5CDR by the distantly supervised methods using the UMLS-based dictionary and the BigDict used by Zhou et al. [20]. BERT-CRF is a fully supervised method.

	UMLS Dict			BigDict					
	P	R	F1	P	R	F1	P	R	F1
BERT-CRF	n/a	n/a	n/a	n/a	n/a	n/a	83.24	89.14	86.09
Dict Matching	44.68	48.41	46.47	70.45	85.16	77.11	n/a	n/a	n/a
DS-NER	67.81	50.26	57.73	74.49	81.25	77.72	n/a	n/a	n/a
PU-DS-NER	69.69	63.78	66.61	75.87	86.20	79.22	n/a	n/a	n/a

The domain dictionaries for chemical and disease entity types are constructed as follows.

1. Retrieve the root UMLS concept IDs for chemical and disease entity types, which are `C0220806` (Chemicals) and `C0012634` (Disease).
2. Retrieve the narrower concept IDs that fall under each root concept ID using UMLS Metathesaurus.
3. Collect names for each concept ID found in the previous step and use them as dictionary entries.
4. Clean the dictionary by deleting duplicate dictionary entries.

As a result, the generated domain dictionaries contained 275,318 and 116,160 entries for chemical and disease, respectively.

4.2 Initial DS-NER Model Training

We adopted the Positive and Unlabeled (PU) learning method proposed by Zhou et al. [20] and refer to the DS-NER model as PU-DS-NER in the following discussion. PU-DS-NER is trained in two steps as follows. In the first step, we search for chemical or disease mentions in $\mathbb{D}$ using string matching and the domain dictionary to assign binary labels. Then, we train the token-based binary classifier to detect entity spans using the Confidence-based PU learning [20]. We apply the trained entity span classifier to $\mathbb{D}$ to assign the confidence value $conf \in (0, 1)$ for each token of $\mathbb{D}$, and denote the confidence values as $\mathbb{C} = \{C^i | i = 1, \ldots, 1,000\}$. In the second step, we train the multi-class classifier to classify each token into non-entity (0), chemical (1), or disease (2) using $\mathbb{D}$ and $\mathbb{C}$ as the training dataset. For the loss function, we use the risk minimization loss proposed by Zhou et al. [20], which utilizes the confidence values provided in $\mathbb{C}$. Table 1 shows the performance of the PU-DS-NER, DS-NER (which are trained without using PU learning by replacing the risk minimization with the cross-entropy loss), and naive dictionary (Dict) matching (using the UMLS-based dictionary described above). We also show the performances of these models when using the tailored BigDict from Zhou et al. [20] for reference. It also includes the performance of a fully supervised method, which uses SciBERT [1] with a CRF layer on top of its

output. All the performances were measured in precision, recall, and F1-score using $\mathbb{T}$ as the test dataset.

We denote the initial NER model initialized with the PU-DS-NER model as $\mathcal{M}_{init}$ and define the noisy initial annotation made by applying $\mathcal{M}_{init}$ to $\mathbb{D}$ as $\mathbb{Z}$ in the following discussion.

4.3 Simulation of Iterative Update of NER Model

We simulate the iterative update of the model as follows. First, we randomly split human annotation $\mathbb{A} = \left\{\mathbf{A}_j^i \mid i \in \{1, \ldots, 1,000\}, j \in \{1, \ldots, j_i^L\}\right\}$ into 500 sets denoted as $\{A_2^s | s = 1, \ldots, 500\}$, each containing gold human annotations corresponding to two documents in $\mathbb{Z}$ with noisy annotations. On average, each A_2^s contains 20 annotated sentences.

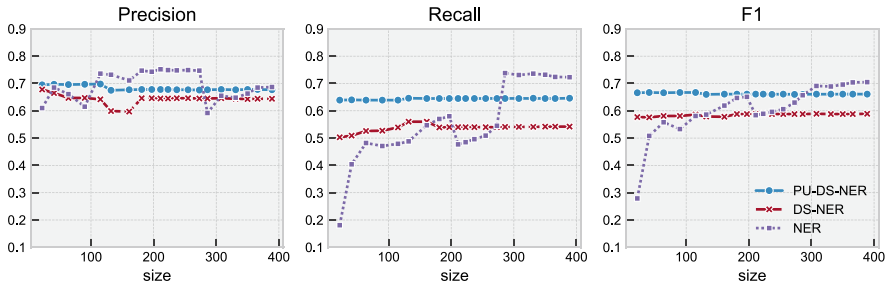
To simulate the human annotation, for the k-th simulation round, we select two documents in $\mathbb{Z}$ corresponding to A_2^k and replace the original noisy annotations with those of A_2^k. We run the simulation 20 times, resulting in 40 initial documents in $\mathbb{Z}$ being replaced with the gold human annotations at the end of the simulation. We construct the training dataset for each simulation round using two strategies to obtain the updated NER model denoted by $\mathcal{M}_k$.

- No-accum: Use only A_2^k as the training dataset for the k-th simulation to fine-tune $\mathcal{M}_k$ based on $\mathcal{M}_{k-1}$
- Accum: Use accumulated $\{A_2^n | n = 1, \ldots, k\}$ as the training dataset for the k-th simulation to fine-tune $\mathcal{M}_k$ based on $\mathcal{M}_{k-1}$

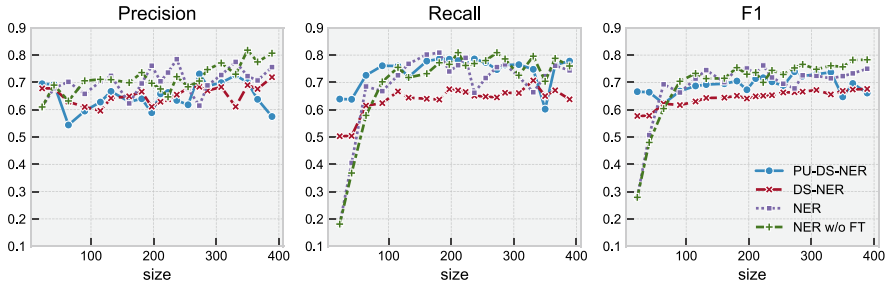
Using the above two strategies, we update PU-DS-NER and DS-NER and evaluate the performance for each simulation round. For PU-DS-NER, we update the binary classifier for entity span detection and the multi-class classifier for entity detection. We also evaluate the baseline NER model, identical to BERT-CRF in Table 1, but is trained from scratch.

4.4 Results

Figures 4a and 4b show the results of No-accum and Accum, respectively. The X-axis shows the cumulative number of human-annotated sentences replaced with the corresponding original noisy annotations over the 20 rounds of the simulation. In Fig. 4a, we can see that the f1 performance of both PU-DS-NER and DS-NER does not improve so much. On the other hand, the baseline NER gradually improves its performance. On the contrary, Fig. 4b shows quite different results. All the models show a gradual increase in f1 performance throughout the simulation. However, we notice that the performance increases of PU-DS-NER and DS-NER are much slower than those of the baseline NER. After the fourth simulation round, which corresponds to approximately 80 on the x-axis, the models based on PU-DS-NER and DS-NER lag significantly behind the baseline NER. NER without FT is the same as the baseline NER, except it re-trains the model from scratch for every simulation round. Except for NER w/o FT, all



(a) No-accum: The models are updated using only a subset of human annotation denoted by A_2^k as the training dataset for the k-th simulation.



(b) Accum: The models are updated by using all human annotations denoted by $\{A_2^n | n = 1, \ldots, k\}$ as the training dataset for the k-th simulation.

Fig. 4. Performance change over 20 simulations by PU-DS-NER, DS-NER, and the baseline NER. The X-axis is the accumulated number of human-annotated sentences replaced with the corresponding original noisy annotations.

other models are fine-tuned from the model generated in the previous simulation round.

These results suggest that iterative update of PU-DS-NER is only efficient when the size of human-annotated sentences is relatively small ($\ll 100$). Suppose the accumulated size of human-annotated sentences becomes large (> 100). Replacing DS-NER with a supervised NER model trained from scratch using only the human-annotated sentences is probably better. This threshold size corresponds to about 1.1% of 9080 total sentences, 185 (0.964%) entity mentions out of 19235, and 88 (1.959%) entity mention types out of 4525 within the total gold annotations contained in the training documents $\mathbb{D}$.

Regarding the research question of how to detect the convergence of the performance of updated NER and, consequently, the corrected annotations, we suggest the following solution. We can measure the performance of the NER model over the last couple of updates against the validation dataset; if the performance score does not increase, we can judge that the model $\mathcal{M}_{last}$ has converged. The above analysis will illuminate the NER model's iterative update behavior. This is

crucial for developing a better iterative updating strategy for DS-NER and showing the trade-offs among different methods for updating NER models under the dynamic change of available human-annotated documents while curating noisy annotations.

5 Conclusion

We propose IRIS (IteRative Improvement dS-ner), a rapid curation framework for the iterative improvement of noisy named entity annotations. The framework aims to provide an efficient and rapid method for curating noisy entity annotations initially made by distantly supervised named entity recognition (DS-NER). Unlike many existing entity annotation tools, which focus primarily on annotation from scratch, IRIS is designed to provide more efficient methods for correcting noisy entity annotations by combining passive and active annotations and employing Active Learning, which uses an iteratively updated NER model during curation work. This is enabled by robust search capabilities based on the database and the automatic annotation capabilities suggested by the NER model, which is initialized with the DS-NER model and incrementally updated during the annotation process.

References

1. Beltagy, I., Lo, K., Cohan, A.: SciBERT: a pretrained language model for scientific text. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 3615–3620. Association for Computational Linguistics, Hong Kong (2019). https://aclanthology.org/D19-1371
2. Bodenreider, O.: The unified medical language system (UMLS): integrating biomedical terminology. Nucleic Acids Res. **32**(Database-Issue), D267–D270 (2004). https://doi.org/10.1093/nar/gkh061
3. Kim, J.D., Wang, Y., Fujiwara, T., Okuda, S., Callahan, T.J., Cohen, K.B.: Open Agile text mining for bioinformatics: the PubAnnotation ecosystem. Bioinformatics **35**(21), 4372–4380 (2019). https://doi.org/10.1093/bioinformatics/btz227
4. Klie, J.C., Bugert, M., Boullosa, B., Eckart de Castilho, R., Gurevych, I.: The INCEpTION platform: machine-assisted and knowledge-oriented interactive annotation. In: Proceedings of the 27th International Conference on Computational Linguistics: System Demonstrations, pp. 5–9. Association for Computational Linguistics, Santa Fe (2018). https://aclanthology.org/C18-2002
5. Li, M., Nguyen, N., Ananiadou, S.: Proactive learning for named entity recognition. In: BioNLP 2017, pp. 117–125. Association for Computational Linguistics, Vancouver (2017). https://aclanthology.org/W17-2314
6. Nghiem, M.Q., Ananiadou, S.: APLenty: annotation tool for creating high-quality datasets using active and proactive learning. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 108–113. Association for Computational Linguistics, Brussels (2018). https://aclanthology.org/D18-2019

7. Peng, M., Xing, X., Zhang, Q., Fu, J., Huang, X.: Distantly supervised named entity recognition using positive-unlabeled learning. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp. 2409–2419. Association for Computational Linguistics, Florence (2019). https://aclanthology.org/P19-1231
8. Qiu, H., Zmina, P.M., Huang, A.Y., Askew, D., Bedogni, B.: Inhibiting Notch1 enhances immunotherapy efficacy in melanoma by preventing Notch1 dependent immune suppressive properties. Cancer Lett. **434**, 144–151 (2018). https://doi.org/10.1016/j.canlet.2018.07.024
9. Rak, R., Rowley, A., Ananiadou, S.: Collaborative development and evaluation of text-processing workflows in a UIMA-supported web-based workbench. In: Calzolari, N., Choukri, K., Declerck, T., Doğan, M.U., Maegaard, B., Mariani, J., Moreno, A., Odijk, J., Piperidis, S. (eds.) Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC 2012). pp. 2971–2976. European Language Resources Association (ELRA), Istanbul (2012). https://aclanthology.org/L12-1572/
10. Rehbein, I., Ruppenhofer, J.: Detecting annotation noise in automatically labelled data. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1160–1170. Association for Computational Linguistics, Vancouver (2017). https://aclanthology.org/P17-1107
11. Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. Found. Trends Inf. Retr. **3**(4), 333–389 (2009). https://doi.org/10.1561/1500000019
12. Shang, J., Liu, L., Gu, X., Ren, X., Ren, T., Han, J.: Learning named entity tagger using domain-specific dictionary. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 2054–2064. Association for Computational Linguistics, Brussels (2018). https://aclanthology.org/D18-1230
13. Stenetorp, P., Pyysalo, S., Topić, G., Ohta, T., Ananiadou, S., Tsujii, J.: brat: a web-based tool for NLP-assisted text annotation. In: Proceedings of the Demonstrations at the 13th Conference of the European Chapter of the Association for Computational Linguistics, pp. 102–107. Association for Computational Linguistics, Avignon (2012). https://aclanthology.org/E12-2021
14. Trieu, H.L., Miwa, M., Ananiadou, S.: Named entity recognition for cancer immunology research using distant supervision. In: Proceedings of the 21st Workshop on Biomedical Language Processing, pp. 171–177. Association for Computational Linguistics, Dublin (2022). https://aclanthology.org/2022.bionlp-1.17
15. Vacareanu, R., Noriega-Atala, E., Hahn-Powell, G., Valenzuela-Escarcega, M.A., Surdeanu, M.: Active learning design choices for NER with transformers. In: Calzolari, N., Kan, M.Y., Hoste, V., Lenci, A., Sakti, S., Xue, N. (eds.) Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024), pp. 321–334. ELRA and ICCL, Torino (2024). https://aclanthology.org/2024.lrec-main.30
16. Wei, C.H., et al.: Assessing the state of the art in biomedical relation extraction: overview of the BioCreative V chemical-disease relation (CDR) task. Database **2016** (2016)
17. Yano, K., Miwa, M., Ananiadou, S.: DISTANT: distantly supervised entity span detection and classification. In: The 22nd Workshop on Biomedical Natural Language Processing and BioNLP Shared Tasks, pp. 171–177. Association for Computational Linguistics, Toronto (2023). https://aclanthology.org/2023.bionlp-1.14

18. Yimam, S.M., Gurevych, I., Eckart de Castilho, R., Biemann, C.: WebAnno: a flexible, web-based and visually supported system for distributed annotations. In: Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics: System Demonstrations, pp. 1–6. Association for Computational Linguistics, Sofia (2013). https://aclanthology.org/P13-4001
19. Yongming, N., Yanping, C., Yongbin, Q., Ruizhang, H., Ruixue, T., Ying, H.: A joint model for entity boundary detection and entity span recognition. J. King Saud Univ. - Comput. Inf. Sci. **34**(10, Part A), 8362–8369 (2022). https://www.sciencedirect.com/science/article/pii/S1319157822002907
20. Zhou, K., Li, Y., Li, Q.: Distantly supervised named entity recognition via confidence-based multi-class positive and unlabeled learning. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 7198–7211. Association for Computational Linguistics, Dublin (2022). https://aclanthology.org/2022.acl-long.498

Fusion-in-LLM: A Parallel Approach to Improve Retrieval-Augmented Generation for Quiz-Based Question Answering

Yuuki Tachioka(✉)

Denso IT Laboratory, Shimbashi, Minato City, Tokyo 105-0004, Japan
tachioka.yuki@core.d-itlab.co.jp
https://www.d-itlab.co.jp/en/

Abstract. Retrieval-Augmented Generation (RAG) enhances large language models (LLMs) by integrating external knowledge for question-answering (QA) tasks. However, effectively utilizing multiple retrieved contexts remains a challenge. Traditional sequential RAG methods process contexts incrementally, leading to incorrect answer revisions, as our experiments show. On the other hand, fusion-in-decoder approaches process all contexts simultaneously. Inspired by this method, we propose fusion-in-LLM, a parallel approach in which LLMs generate independent answers for each retrieved context, selecting the final answer via majority voting or LLM-based selection. This method allows simultaneous processing, mitigating context interference, while improving efficiency. We evaluated fusion-in-LLM on quiz-based QA tasks using the AI-O dataset. Experimental results show that fusion-in-LLM outperforms sequential RAG, particularly when utilizing the top 5 to 7 retrieved contexts. Additionally, fusion-in-LLM reduces computational overhead compared to sequential methods by using parallel processing. These findings demonstrate that fusion-in-LLM effectively improves retrieval-augmented QA in terms of accuracy and efficiency, making it a promising approach to handle multiple contexts in LLM-driven QA tasks.

Keywords: Retrieval-augmented generation · Large language models · Quiz-based question answering

1 Introduction

The capability of large language models (LLMs) to generate responses is inherently limited by the knowledge encoded within their training data. To address this limitation, Retrieval-Augmented Generation (RAG) [2,3,12] has emerged as a promising approach that enhances LLM by incorporating external knowledge retrieval. In particular, passage retrieval plays a critical role in open-domain question answering (QA) [4,7,11]. However, when multiple diverse contexts are retrieved, the challenge lies in effectively utilizing them without introducing misleading information, which could result in incorrect responses [17].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 70–81, 2026.
https://doi.org/10.1007/978-3-031-97144-0_7

Quiz-style QA tasks [1] present unique challenges due to the careful design of the quiz questions to obscure direct answers. The passages retrieved may contain irrelevant or misleading information, and the keywords within the question itself often do not directly relate to the correct answer. Passage retrieval for quiz-style QA tasks requires a more nuanced approach than simply selecting a highly ranked retrieved passage.

Given that the effectiveness of RAG depends heavily on the quality of the retrieved passages [5,6], it is crucial to select the appropriate context. However, retrieved passages may be influenced by explicitly mentioned answer choices, leading to misleading contextual information and incorrect predictions. Many RAG implementations, such as LlamaIndex, employ a sequential approach in which the top-ranked contexts are processed one at a time, allowing the LLM to determine whether to revise its initial response. However, as shown in our experiments, this sequential RAG method suffers from a significant drop in accuracy when incorporating multiple contexts (e.g., top-2 or top-3 passages) compared to using only the most relevant context. The reason for this decline is that including more contexts introduces more irrelevant information, causing the LLM to revise correct answers incorrectly. Additionally, the sequential nature of this method results in increased computational overhead, as the LLM must generate an answer for each new context separately.

An alternative approach to deal with multiple contexts is the fusion-in-decoder (FiD) method [9,10,15], which concatenates all retrieved contexts and inputs them into the decoder simultaneously. Unlike sequential RAG, FiD enables parallel processing of all contexts, leveraging a majority voting effect that mitigates errors introduced by misleading contexts.

Inspired by this, we propose fusion-in-LLM, an approach that utilizes LLMs in parallel for QA tasks. Rather than sequentially refining an answer, fusion-in-LLM independently generates responses for each retrieved context in parallel. The final answer is then determined by majority voting or by allowing the LLM to select the most appropriate response, similar to the re-ranking strategies [16]. This parallelization not only enhances response accuracy but also significantly reduces computation time compared to sequential methods.

Through experiments on quiz-based QA tasks using AI-O [1], we demonstrate that fusion-in-LLM achieves superior performance over the traditional sequential RAG approach. By optimizing answer selection through parallel processing and context evaluation, fusion-in-LLM provides a more robust and efficient methodology for retrieval-augmented QA.

2 Retrieval-Augmented Generation for Question-Answering

2.1 Dense Passage Retrieval

To solve QA tasks using LLM, the relevant passages are retrieved by Dense Passage Retrieval (DPR) [10]. A question q and a set of passages p such as Wikipedia are encoded into fixed-dimension embeddings $E_q(q)$ and $E_p(p)$ using a question

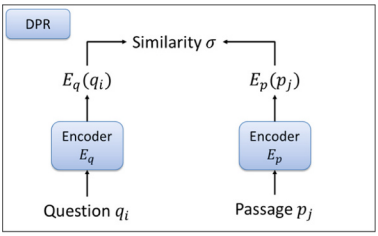


Fig. 1. Deep passage retriever.

encoder E_q and a passage encoder E_p, respectively, as shown in Fig. 1. The relevant passages are extracted and the top n passages are ranked by similarity scores σ, where $\sigma(q, p)$ is a similarity function defined as the inner product of the embeddings, $E_q(q)^\top E_p(p)$, with $\top$ representing the transpose of a vector. These retrieved passages are provided as a context and input into the LLM along with the question. Because the process of retrieving relevant passages is mature, the main challenge lies in how to effectively input multiple diverse contexts into the LLM.

To train both encoders, the loss function (1) is optimized, representing the negative log likelihood of the positive passage:

$$\mathcal{L}(q_i, p_i^+, \mathcal{P}_i^-) = -\log \frac{\exp(\sigma(q_i, p_i^+))}{\exp(\sigma(q_i, p_i^+)) + \sum_{p^- \in \mathcal{P}_i^-} \exp(\sigma(q_i, p^-))}, \tag{1}$$

where p_i^+ is a positive example for the i-th question q_i, while $\mathcal{P}_i^-$ is a set of n negative examples $\{p_{i,1}^-, \ldots, p_{i,n}^-\}$.

2.2 Retrieval-Augmented Generation (RAG)

For RAG, the passage retrieved by DPR is inputted into the LLM along with the question, as shown in Fig. 2. The retrieved contexts serve as context, and the query is treated as a question, which is provided in the prompt shown in Fig. 3. The LLM is instructed to respond concisely with only the answer[1]. Since this method only utilizes the most relevant passage (top-1 passage), if the retrieved passage is incorrect, it becomes difficult to generate the correct answer, as demonstrated in later experiments.

2.3 Fine-Tuning of LLM with Low-Rank Approximation (LoRA) for RAG

To effectively utilize contexts and derive correct answers, the LLM is fine-tuned. Using training data, context-answer pairs are collected for the cases where the

[1] This instruction is not strictly followed; therefore, only the text before the first newline character is treated as the answer.

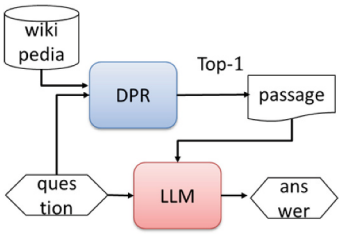


Fig. 2. Retrieval-augmented generation (RAG).

```
You are a quiz show contestant.
Using the provided context, answer the query concisely in under 20
characters.
Context: ...
Query: ...
### Answer:
```

Fig. 3. Prompt for conducting RAG.

context was relevant but the generated answer was incorrect. The correct answers are appended after the `### Answer` prompt in Fig. 3, and the LLM is fine-tuned using LoRA to reproduce these outputs.

In QA tasks, concise answers (often in single words) are typically required. When there is a specific pattern in how answers should be formulated, fine-tuning the response style using LoRA is an effective approach.

2.4 Sequential RAG (Refine Prompt)

Figure 4 illustrates the most widely used sequential RAG method. To handle multiple contexts retrieved by DPR, the contexts are input sequentially as top $\{1, 2, \ldots, n\}$, and the LLM is prompted to decide whether to rewrite the previous

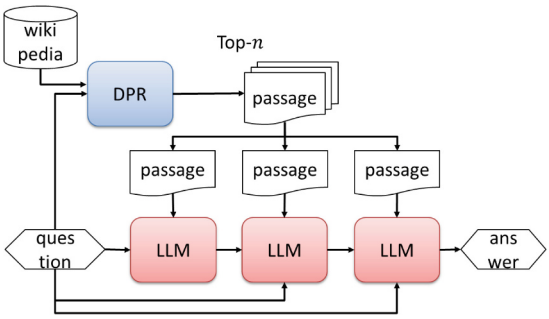


Fig. 4. Sequential RAG using LLMs with refine prompt.

answer based on the new context. This method is standard in general-purpose frameworks like LlamaIndex[2].

```
You are a quiz show contestant.
Using the provided context, answer the query concisely in under 20
characters. You may rewrite the original answer using the new
context. If the new context is unhelpful or unclear, repeat the
original answer.
New context: ...
Query: ...
Original answer: ...
### Answer:
```

Fig. 5. Refine prompt for sequential RAG.

The prompt to implement this is shown in Fig. 5. The LLM decides whether to rewrite the original answer based on the new context. If uncertain, the LLM is instructed to repeat the original answer. This method operates the LLM sequentially, requiring that the previous response generation be completed before proceeding, which results in the disadvantage of taking n times the generation time.

3 Fusion-in-LLM

3.1 Generation of Answer Candidates by Multiple LLM Outputs

In the FiD method [10], the question and related passages are concatenated, vectorized by the encoder, and combined as n inputs to the decoder. This enables the decoder to select the correct answer by processing all contexts in parallel. Similarly, for LLMs, we propose the fusion-in-LLM that generates the answer for each context in parallel and selects the final answer from the generated answers in the previous step. As shown in Fig. 6, contexts are inputted into multiple LLMs separately, and the majority vote of the outputs determines the final answer. Our approach operates LLMs in parallel, allowing the process to be completed within the time required for a single generation if multiple computational units are used, providing an advantage.

3.2 Exclusion of Inappropriate Answers Based on Comparison with the Query

One of the advantages of our methods is the ability to exclude queries that already contain the answer, thanks to the parallel answer generation. It is evident that answers containing phrases directly from the query are inappropriate. For instance, in response to the query "In the movie 'West Side Story', what is the name of the other gang besides the Sharks?", the answer "Sharks" is clearly unsuitable. Such responses are excluded before the answer selection process as inappropriate answers.

[2] https://www.llamaindex.ai/.

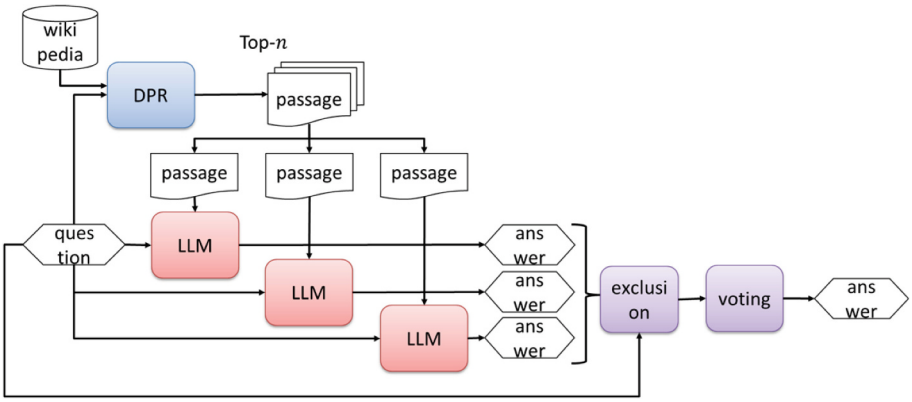


Fig. 6. Fusion-in-LLM.

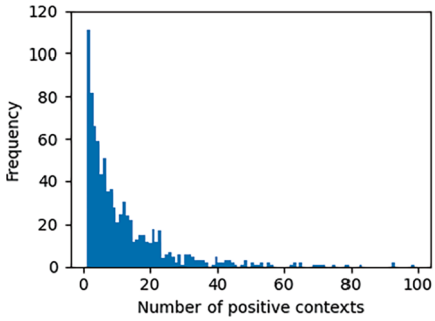


Fig. 7. Histogram of number of positive contexts.

3.3 Answer Selection

Majority Voting. The LLM generates a maximum of n answers corresponding to the top-n contexts. Since there may be cases where no answer is generated or inappropriate answers are excluded, as described in the previous section, the actual number of answers is often less than n. The final answer is determined using majority voting. In the event of a tie, the answer with a higher-ranked context is selected. For example, if the top-5 answers corresponding to the contexts of rank 1 to 5 are `[a, a, b, b, c]`, the final selected answer would be `a`.

Figure 7 shows the histogram of the number of positive contexts per query in the dataset used in subsequent experiments. While most queries have only one positive context, some queries have multiple positive contexts due to the division of passages. Since answers derived from multiple positive contexts are likely to be the same, majority voting is expected to be an effective approach.

LLM Selection. The LLM is provided with at most n answer candidates obtained using the above method and is tasked with selecting the most appro-

```
You are a quiz show contestant. Please select the most appropriate answer
for the query from the provided choices.
Query:...
Choices: ...
### Answer:
```

Fig. 8. Prompt for selecting an answer from choices.

priate one. The LLM is used to answer multiple choice questions. The choices in the prompt shown in Fig. 8 represent these answer candidates. For LLM, this process effectively becomes answering a multiple-choice question without RAG. Since duplicate candidates appear multiple times in the choices, the LLM can take into account the frequency of occurrence when selecting an answer. Because the multiple-choice format is expected to be simpler than the original problem, the LLM should be able to use its embedded knowledge to derive the correct answer.

4 Experiments Using AI-O

4.1 Experimental Setup

The dataset from the second AI-O task was used for the experiment. The AI-O task[3] was designed to evaluate AI question-answer capabilities in a quiz format. A key characteristic of AI-O is that it focuses on questions whose answers remain unchanged over time and are presented in Japanese [1].

AI-O consists of three datasets: training (22,335 instances), validation (1,000 instances), and evaluation (1,000 instances). Since the correct answers for the evaluation set are not publicly available, the model was trained on the training set, and the performance was evaluated using the validation set. The evaluation was carried out on the basis of the exact matching of the strings.

The relevant passages were retrieved from Wikipedia using DPR. The model setup followed the FiD baseline system provided by AI-O[4]. The top-30 related passages were extracted using DPR and Faiss. These passages were then input as contexts for RAG processing by the LLM. The authors provided a baseline code for this component in the transfer-2 (RAG task) system [8,14][5]. The LLM used was a 7 billion parameter Japanese LLM based on Llama2-7B, fine-tuned with instruction tuning for various Japanese tasks [13][6].

[3] https://sites.google.com/view/project-aio/home.

[4] https://github.com/ntcirtransfer/transfer2/tree/main/RAG/AIO3_FiD_baseline. The fourth AI-O of the most recent focuses on buzzer-specific tasks; therefore, the system developed for the third AI-O was adopted as a baseline.

[5] https://github.com/ntcirtransfer/transfer2/tree/main/RAG/transfer2_llm_baseline.

[6] https://huggingface.co/elyza/ELYZA-japanese-Llama-2-7b-fast-instruct.

4.2 Results and Discussion

The results are shown in Table 1. Using pre-trained Japanese LLM, the accuracy was 19.1% for queries without contexts, indicating that the knowledge inherent in the LLM alone is insufficient and RAG is essential. When RAG was applied with the top-1 context, the accuracy improved to 34.0%.

Table 1. Accuracy (exact match) [%] on the Question-Answering Task.

w/o RAG	RAG	Sequential RAG		LoRA+RAG
	top-1	top-2	top-3	top-1
19.1	34.0	27.6	23.0	40.5

Table 2. Number of samples corresponding to the correctness of top-1 retrieved passage by DPR and answer generated by LLM.

		LLM(+RAG)		total
		correct	wrong	
DPR	relevant	1842	774	2616
	irrelevant	488	4896	5384
total		2330	5670	8000

Sequential RAG was applied to utilize multiple contexts, but the accuracy decreased as the number of contexts increased (top-2 and top-3), dropping from 34.0% to 23.0%. This decline was due to incorrect revisions of the correct answers. At least for this task, sequential RAG was found to be ineffective.

To teach effective LLM use of contexts, 8,000 randomly selected pairs of "query+context" → "answer" were generated from the training set, focusing only on the most relevant context. As shown in Table 2, the data were classified into four patterns in terms of whether DPR provided a relevant context or whether LLM generated a correct answer. A total of 1,846 cases were correctly answered by both. The LLM accuracy was 70% for the 2,616 cases where DPR provided a relevant passage.

For the 5,384 cases where the context provided by the DPR was incorrect, the accuracy was only 9%. Considering that the accuracy without RAG was 19.1%, it is evident that incorrect contexts significantly impact the final performance.

To address the remaining 30% (774 cases in which the context was relevant but the LLM answer was incorrect), a fine-tuning was performed using the LoRA method described in Sect. 2.3. As a result, the accuracy improved to 40.5%. Teaching effective use of contexts proved to be beneficial for performance enhancement.

Figure 9 shows the Hit rate of DPR. It presents the relevance of a single context and cumulative accuracy. This indicates that the context of top-1 is

relevant in approximately 40% of the cases. When considering up to the top 30 passages, the cumulative accuracy reaches around 80%. This suggests that useful contexts exist even in lower-ranked passages.

The accuracy achieved by majority voting in the proposed fusion-in-LLM method is shown in Table 3. The majority vote for answers up to the top-3 contexts improved the accuracy to 44.5%. This demonstrated that the proposed fusion-in-LLM is more effective than sequential RAG that utilizes multiple contexts in QA tasks.

Table 3. Accuracy [%] with fusion-in-LLM.

top-n	3	5	7	10	20	30
Majority voting	44.5	46.5	46.3	45.3	42.4	41.9
LLM selection	44.3	46.1	46.2	45.7	43.4	43.1
Oracle	57.3	63.6	66.9	69.5	74.3	76.9

Although the accuracy degradation when using multiple contexts was relatively smaller, no significant differences were observed between LLM selection and majority voting. Since retrieved passages tend to overlap, answers generated from the same article are likely to be duplicated, leading to improved accuracy through majority voting. Similarly, LLM selection probably depends on the frequency of occurrence when determining the best answer.

Figure 10 compares the accuracy of majority voting and the oracle method (the proportion of cases where at least one correct answer was included in the top n answers). The shape of the oracle accuracy curve closely resembles the cumulative accuracy shown in Fig. 9, confirming that obtaining the correct context significantly increases the likelihood of the LLM producing the correct answer. The highest accuracy, 46.3%–46.7%, was achieved in the top 5 to the top 7 contexts. However, there remains a considerable gap between the oracle performance and the best results from majority voting. In cases where a correct answer

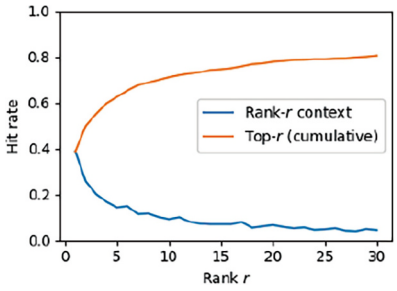


Fig. 9. Hit rate of retrieved passages by DPR.

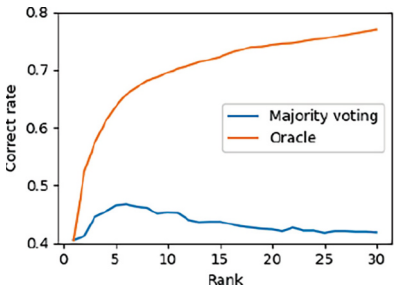


Fig. 10. Accuracy by majority voting and oracle.

is only found in lower-ranked contexts (e.g., beyond rank 20), majority voting becomes ineffective. Thus, additional techniques are necessary to provide useful information to select the appropriate answer from the candidates of the answers.

Figure 11 shows the number of answers included in the top-30 predictions. In the case of majority voting, it is the maximum when the number of supported predictions is three. In the oracle case, it is maximum when the number of those is one, which cannot be solved by a majority voting scheme.

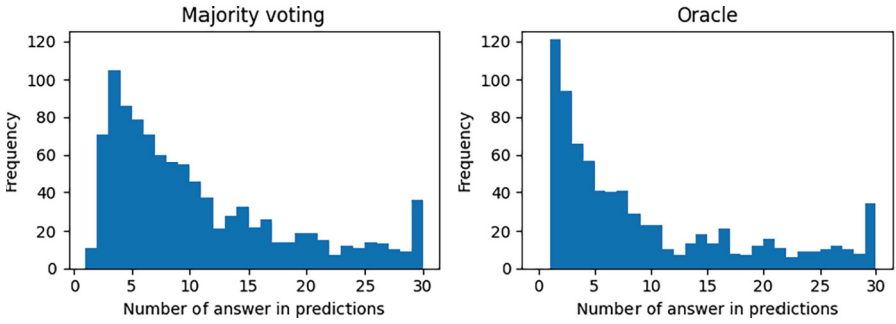


Fig. 11. Number of supported predictions for final answer (majority voting) and gold answer (oracle).

5 Conclusion and Future Work

This study explored improving RAG for QA tasks by effectively utilizing multiple retrieved contexts. We found that sequential RAG decreases accuracy as more contexts are introduced. To address this, we proposed fusion-in-LLM, a parallel processing method in which LLMs generate answers independently, followed by selection via majority voting or LLM-based ranking. This approach allows simultaneous context processing, unlike sequential RAG, which must process contexts incrementally.

Experiments on AI-O quiz-based QA tasks showed that fusion-in-LLM outperforms sequential RAG, particularly when using the top 5–7 contexts, while also improving computational efficiency. However, lower-ranked contexts (beyond the top-7) still contain valuable information, suggesting that refining answer selection strategies could further enhance accuracy. Future work will focus on improving selection mechanisms and expanding to other QA domains to evaluate the broader applicability of our approach.

References

1. Ariyama, T., Suzuki, J., Suzuki, M., Tanaka, R., Akama, R., Nishida, K.: Achievements and challenges in Japanese question answering: insights from quiz competition results. Nat. Lang. Process. **31**(1), 47–78 (2024). https://doi.org/10.5715/jnlp.31.47
2. Fan, W., et al.: A survey on RAG meeting LLMs: towards retrieval-augmented large language models. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 6491–6501 (2024). https://doi.org/10.1145/3637528.3671470
3. Gao, Y., et al.: Retrieval-augmented generation for large language models: a survey. arXiv preprint (2023). https://arxiv.org/abs/2312.10997
4. Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M.W.: REALM: retrieval-augmented language model pre-training. In: Proceedings of the 37th International Conference on Machine Learning. ICML 2020. JMLR.org (2020)
5. He, H., Zhang, H., Roth, D.: Rethinking with retrieval: faithful large language model inference (2022). https://arxiv.org/abs/2301.00303
6. Huang, W., Lapata, M., Vougiouklis, P., Papasarantopoulos, N., Pan, J.: Retrieval augmented generation with rich answer encoding. In: Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1012–1025 (2023). https://doi.org/10.18653/v1/2023.ijcnlp-main.65
7. Izacard, G., Grave, E.: Leveraging passage retrieval with generative models for open domain question answering. In: Merlo, P., Tiedemann, J., Tsarfaty, R. (eds.) Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume, pp. 874–880 (2021). https://doi.org/10.18653/v1/2021.eacl-main.74
8. Joho, H., Keyaki, A., Tachioka, Y., Yamamoto, S.: Building test collections for Japanese dense information retrieval technologies and beyond. In: Proceedings of The First Workshop on Evaluation Methodologies, Testbeds and Community for Information Access Research (EMTCIR 2024). NII (2024)
9. de Jong, M., et al.: FiDO: fusion-in-decoder optimized for stronger performance and faster inference. In: Findings of the Association for Computational Linguistics: ACL 2023, pp. 11534–11547 (2023). https://doi.org/10.18653/v1/2023.findings-acl.732
10. Karpukhin, V., et al.: Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 6769–67 (2020). https://doi.org/10.18653/v1/2020.emnlp-main.550
11. Khattab, O., Zaharia, M.: ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (2020). https://arxiv.org/abs/2004.12832
12. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS 2020 (2020)
13. Sasaki, A., Hirakawa, M., Horie, S., Nakamura, T.: ELYZA-Japanese-LLaMA-2-7b (2023). https://huggingface.co/elyza/ELYZA-japanese-Llama-2-7b
14. Tachioka, Y., Terao, Y.: ditlab at the NTCIR-18 transfer-2. In: the 18th NTCIR Conference on Evaluation of Information Access Technologies. NII (2025)

15. Wang, C., Yu, H., Zhang, Y.: RFiD: towards rational fusion-in-decoder for open-domain question answering. In: Findings of the Association for Computational Linguistics: ACL 2023, pp. 2473–2481 (2023). https://doi.org/10.18653/v1/2023.findings-acl.155
16. Wang, S., et al.: Evidence aggregation for answer re-ranking in open-domain question answering. In: International Conference on Learning Representations (2018). https://openreview.net/forum?id=rJl3yM-Ab
17. Wei, Z., et al.: GARAG: a general adaptive question-answering system based on RAG. In: Proceedings of the 2024 International Conference on Cloud Computing and Big Data, pp. 442–447 (2024). https://doi.org/10.1145/3695080.3695156

Unveiling Emotional Signature in Conspiracy Topics on Social Media Through Vector Analysis

Luigi Lomasto[1(✉)], Nicola Lettieri[2], Delfina Malandrino[1], Valerio Mosca[1], and Rocco Zaccagnino[1]

[1] Department of Computer Science, University of Salerno, Salerno, Italy
{llomasto,dmalandrino,rzaccagnino}@unisa.it, v.mosca18@studenti.unisa.it
[2] INAPP, Rome, Italy
n.lettieri@inapp.org

Abstract. This study explores the emotional signature of online discussions within conspiracy, non-conspiracy, and real echo chamber communities on Reddit. Using a RoBERTA-based emotion recognition model, we create numerical representations of emotional intensities across 27 categories. Cosine similarity analysis reveals that real echo chambers exhibit the highest emotional homogeneity, with strong internal alignment and reinforced emotional tones. Conspiracy communities show moderate-to-high emotional similarity, suggesting partial echo chamber behavior, whereas non-conspiracy discussions display the lowest internal consistency, indicating greater emotional diversity.

Furthermore, Principal Component Analysis (PCA) highlights a continuum of emotional reinforcement, where non-conspiracy subreddits demonstrate wider emotional dispersion, while conspiracy forums cluster closer to real echo chambers. These findings suggest that emotional vectorization serves as an effective method for identifying echo chamber dynamics and distinguishing conspiracy communities from more diverse online discussions. Our approach provides a foundation for further research into emotion-driven behavioral patterns in digital environments, with potential implications for misinformation detection and online discourse analysis.

1 Introduction

Information Disorder (ID) has become a major issue in the past decade, particularly due to the spread of unchecked content on social media (SM). Researchers are working to address this challenge using methods afferent to Artificial Intelligence *(AI)*, Natural Language Processing *(NLP)*, Simulation Dynamics *(SD)*, and Fact-Checking Activities *(FA)*. When studying ID, it is important to consider several factors, such as the topics covered by fake news *(FN)*, how users behave on SM, and the emotions and feelings expressed in both the news and

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 82–93, 2026.
https://doi.org/10.1007/978-3-031-97144-0_8

users' reactions. The factors just listed can be used in several ways to counter the phenomenon under consideration. The first approach, known as fake news detection *(FND)*, involves making strategies and implementing *AI* and *NLP* models capable of determining whether a news is true or false. Unfortunately, this type of solutions cannot completely solve the problem, as the models offer poor results when they have to classify breaking news. A second and more recent approach is to study how news are shared on *SM* so as to detect patterns and anomalies related to the sharing of fake or unverified content. Generally, simulative, agent-based methods are employed with which we try to replicate the phenomenon to understand its evolution and how to intervene [23]. To best simulate the behavior of users and their actions on SMs, the sphere of emotions should be considered. Emotions and sentiments are among the biggest challenges in addressing the ID phenomenon, as they play a key role in shaping user behavior on social media, affecting how people comment, react, and share content [2]. Thankfully, Emotion Recognition (ER) within NLP has made great progress and is now commonly used in Social Media Analysis (SMA) to understand emotions in posts and comments. Identifying emotions like happiness, anger, fear, and disbelief provides valuable insights for various purposes, including business, marketing, social events, and understanding personal beliefs. A direct consequence of emotions on social networks, is the formation of groups of users who share the same opinions and ideas or beliefs with respect to a specific topic. These subnetworks are commonly called *Echo Chambers (EC)* [18]. In the *EC*, users with similar opinions about a topic interact together without engaging with opposing viewpoints with users that have a different idea [8]. This dynamic often leads to the uncontrolled spread of unverified or harmful content, which worsens the problem of ID. This is particularly common with conspiracy topics like *Flat Earth theory, anti-vaccine beliefs, COVID-19 misinformation, and 5G concerns* [20]. Identifying these particular subnets and understanding their formation and evolution could make it possible to limit the uncontrolled sharing of unverified or false information.

This paper presents a preliminary study by which we aim to investigate the possible emotional similarity between subnetworks some of that considered *conspiracy* and potentially related to *EC*. This assumption is based on [6,7,20] where the authors demonstrate that the opinion polarization in EC is generally related to conspiracy topics. Our research question (***RQ***) is: *Can the emotion expressed by social media users be used to detect conspiracy topics potentially related to* ***EC*** *without analyze the textual contents?* To do so, we have constructed emotional vectors (*Emotional Signature*) for specific Reddit[1] topics by using an ER model based on RoBERTA [15][2] and provided by HuggingFace[3]. *NLP* through vector analysis and cosine similarity provides a method to assessantic similarity between words, as shown in [19]. We applied this concept to compare emotional vectors in order to identify similarities and differences between several subnetworks. The

[1] https://www.reddit.com.
[2] https://huggingface.co/SamLowe/roberta-base-go_emotions.
[3] https://huggingface.co.

preliminary results show how by using vectors it is possible to find significant correlation between conspiracy subnetworks that can result in potentially EC. Results allow to, on one hand offer the possibility to study the emotional evolution of groups on SM so as to intervene to block formation of potential ECs, and on the other hand classify existing potential ECs through their *Emotional Signatures (ES)*. To support our results we have compared the selected topics with real Echo Chambers (*REC*) proposed by [20]. The paper is organized as follows: Sect. 2 provides background and related work about ED, EC and ID phenomenon. Section 3 briefly describes the defined dataset and the costructed vectors. In Sect. 4 we show and describe the results and finally in Sect. 5 we provide some consideration and potential future works.

2 Background and Related Work

In this section we provide an overview about the related work and we'll describe the main concept concern to the background. More in details we'll deepen relevant concept such as: *Emotion Detection*, *Information Disorder* and *Echo Chamber*.

Emotion Detection (ED) is an interdisciplinary field that aims to identify and analyze emotions expressed in various modalities, including text, speech, images, and videos. It combines principles from psychology, linguistics, computer science, and machine learning to capture the underlying affective states of individuals. In NLP, emotion detection focuses on analyzing textual data such as social media posts, reviews, and comments. This involves classifying text into emotion categories such as happiness, sadness, anger, fear, or surprise. Traditional approaches rely on lexicons or rule-based methods that match words to predefined emotional states. Recent advancements leverage deep learning models, including Recurrent Neural Networks (RNNs), Convolutional Neural Networks (CNNs), and transformers like BERT [10], to capture contextual and semantic nuances in text. For speech-based emotion detection, acoustic features such as pitch, tone, and rhythm are analyzed to infer emotions. In image and video modalities, facial expression recognition plays a significant role, utilizing computer vision techniques and models like Convolutional Neural Networks to interpret facial cues and gestures. Emotion detection has several applications, including sentiment analysis for business and marketing, mental health monitoring, human-computer interaction, and social media analysis. Despite its potential, challenges remain, such as handling ambiguous or mixed emotions, cultural variations in emotional expression, and the ethical implications of privacy and misuse of data. Ongoing research focuses on improving multimodal emotion detection, which combines information from multiple sources (e.g., text, speech, and facial expressions) to enhance accuracy and reliability. This holistic approach aims to bridge gaps in single-modal analyses and achieve a deeper understanding of human emotions. The survey [3] provides a comprehensive overview of recent developments in the use of transformer-based architectures and their integration techniques to analyze and interpret emotions in multiple modalities. It also explores the

various challenges encountered in this field and the ways transformers address these issues. By considering the emotions in the ID phenomenon, the survey [16] describe how the emotions has an important impact in the misinformation with a particular focus on advanced fusion methods.

Information Disorder (ID) is studied mainly through two approaches: *fake news Detection* using machine learning approaches [4], and *Simulation Techniques* in order to understand how disinformation spreads on social media. *fake news Detection* has been widely researched, with studies categorized into *style-based models*, *source-based models*, *behavioral-based models*, and *knowledge-based models*.

- *Style-based models* analyze the writing style of news articles to identify features typical of fake news. The paper [5] focuses on linguistic characteristics such as syntax, sentiment, and readability, while more advanced approaches based on BERT ([14]) consider contextual semantics.
- *Source-based models* assess the trustworthiness of news sources through *credibility scores*, as demonstrated in [21,22], and [11].
- *Behavioral-based models* investigate social and behavioral aspects of users. [13] and [1] apply approximate rough set theory to study Information Disorder and how online communities shift their views over time.
- *Knowledge-based models* rely on fact-checking by comparing news content with verified facts. For example in [17] we have defined the *Negativity Score*, a sentiment-based metric used as feature for fake news detection.

Considering *Simulation* approaches an important research activity is represented by [23] in which the authors analyze the virality of the disinformation inside an *EC*. In [24] is proposed a framework based on LLM-based simulation to evaluate and counter polarization phenomena. The paper [12] proposes a simulation model, which integrates artificial intelligence tools on behavioral data, in order to establish recommendation technologies, to explore the evolution of social network structures. Generally the main concept used to study the *ID* phenomenon throught simulations is the *EC*.

Echo Chambers (ECs) are social network structures where users are mainly exposed to like-minded opinions, reinforcing beliefs and discouraging diverse perspectives. They contribute to polarization and misinformation, studied through network analysis, sentiment analysis, and agent-based modeling. Key drivers include algorithmic filtering, confirmation bias, and homophily. ECs amplify conspiracy theories and misinformation, especially during elections and crises, eroding trust in institutions. Mitigation strategies involve platform design changes, media literacy, and critical thinking promotion. Ongoing research focuses on detecting ECs and countering their influence.

3 Dataset Construction and Emotional Vectors Definition

Our study is based on data collected by using the Reddit platform. Specifically, the sub-networks, or subreddits, were selected according to the very definition

of echo chambers. Once the subreddits to analyze have been identified, datasets were created by extracting posts and comments from the "hot" section of each subreddit, covering the period between November and December 2024. Subsequently, the datasets were filtered: posts and comments with only one or two words, as well as those labeled [deleted] or [removed], were discarded.

The subreddits identified as potential conspiracy (A) and potentially related to EC are: *communism, conspiracy, AskThe_Donald and flatearth*. To compare these potentially conspiracy topics we have selected sub-reddits groups considered non conspiracy *(B)* in order to understand the emotional differences between them. The non conspiracy groups are: *shakespeare, AskReddit, books and changemyview*. The Table 1 show the main info about the constructed datasets for the selected subreddits groups.

Table 1. Reddit Dataset Overview by Category

File Name	Total Posts	Total Comments	Oldest Date	Newest Date	Avg Words per Body Category
r/communism	748	7538	2023-11-26	2024-03-29	143 Conspiracy (A)
r/conspiracy	895	49828	2024-10-03	2024-11-23	37 Conspiracy (A)
r/AskThe_Donald	980	12842	2024-10-09	2024-12-09	30 Conspiracy (A)
r/flatearth	856	31323	2020-09-13	2024-12-04	33 Conspiracy (A)
r/shakespeare	967	18628	2022-01-22	2024-11-22	47 Not Conspiracy (B)
r/AskReddit	825	31520	2024-11-22	2024-11-24	25 Not Conspiracy (B)
r/books	114	13027	2024-11-18	2024-12-04	47 Not Conspiracy (B)
r/changemyview	402	63809	2024-10-16	2024-11-24	77 Not Conspiracy (B)
r/thedonald	440	3218	2019-04-16	2022-01-19	19 Real Echo Chamber (REC)
r/Fuckthealtright	993	16767	2024-08-31	2025-01-29	30 Real Echo Chamber (REC)
r/EnoughTrumpSpam	701	6486	2021-01-07	2025-01-29	32 Real Echo Chamber (REC)

Once the collection and structuring of the datasets were completed, the study proceeded to the emotion recognition phase. For this analysis, we have used *roberta-base-go_emotions*[4], an emotion detection model provided by *Hugging Face*. The model is a fine-tuned version of the RoBERTa base model, specifically trained for multi-label emotion classification using the GoEmotions dataset [9]. This dataset comprises 28 distinct emotion labels, enabling the model to identify multiple emotions within a single text input. The goal of this phase was to identify and quantify the emotions expressed in the collected texts, providing a numerical representation of the intensity of each emotion. The model generates, for each text, a table containing *Emotion Labels* (the identified emotions e.g., joy, sadness, anger) and *Confidence Scores* (the associated confidence levels for each emotion). We have used this information to build *Emotional Vectors* for each item of our datasets. The results of this analysis were organized into a new CSV file. Each row of the file represents the emotional profile of a text, including scores for all emotions (27 emotions plus a "neutral" label). In short, each dataset contains vectors of 28 dimensions. We have decided to remove

[4] https://huggingface.co/SamLowe/roberta-base-go_emotions.

the *neutral* emotion, because it did not contribute any information and showed behavior common to all topics. From the processed files, an average emotion vector was calculated for each dataset (and therefore for each subreddit) using the arithmetic mean. These vectors represent the aggregated emotional profile of each dataset and serve as the basis for subsequent comparative analyses. To support our results we have compared A and B Topics with real Echo Chambers (*REC*) presented in Table 1 and provided by [20].

4 Experiment and Results

This section will show the results obtained during the experimental phase. The results can be divided conceptually in 2 parts. In the first one 4.1, we compare the Emotional Signature in order to detect similarities and differences between A and B topics. In the second experimentations phase 4.2, we compare our A and B topics with real Echo Chambers (*REC*) proposed by [20].

4.1 Emotional Signature Similarities

To understand and detect the similarity between A and B topics, we have evaluated the cosine similarity by using emotional vectors previously evaluated as described in Sect. 3. We have evaluated Emotional Signatures that represent the average emotional vector for each topic. The bar plots in Fig. 1 are based on the global average vectors computed from the emotional averages of topics A (Conspiracy) and B (Not Conspiracy). The orange bars show **B** content, while the blue bars represent **A**. As seen in the figure, emotions like *approval* and *realization* are much stronger in non-conspiracy content. This suggests that users in B are more likely to agree with or accept the information they read. On the other hand, conspiracy-related discussions (A) show more *confusion* and *disappointment*, pointing to feelings of uncertainty or frustration. Positive emotions like *optimism*, *excitement*, and *love* are more common in B, while negative emotions such as *anger*, *sadness*, and *annoyance* appear with similar intensity in both groups. Emotions like *disgust*, *relief*, and *gratitude* are the least expressed across both categories. Overall, B subreddits tend to express more positive emotions, while A content reflects more doubt and dissatisfaction.

To find out whether there are more discriminating emotions than others, we have decided to run the experiments using sets of N different emotions selected by considering the emotions with higher variance value. This activity can be considered as features selection step. The number of emotions N took the following values: 3, 10 and 27 emotions, respectively. Comparison of emotional signatures for cosine similarity was done by comparing the three different combinations *A-A, A-B, B-B*. The sets of emotions used are: **3 emotions** (*"approval", "realization", "caring"*), **10 emotions** (*"approval", "realization", "caring", "optimism", "curiosity", "excitement", "love", "disappointment", "confusion", "surprise"*), and **All emotions**. The analysis of cosine similarity matrices between emotional vectors of conspiracy and non-conspiracy subreddits provides valuable insights

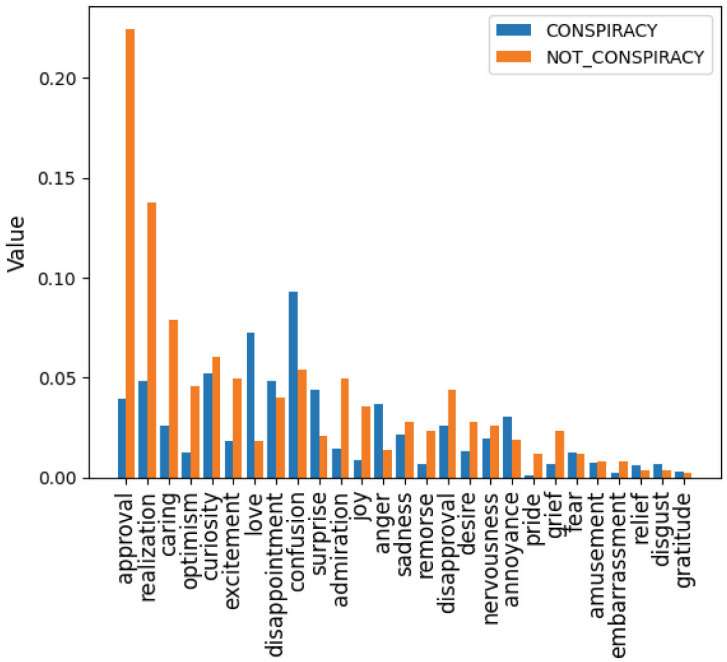
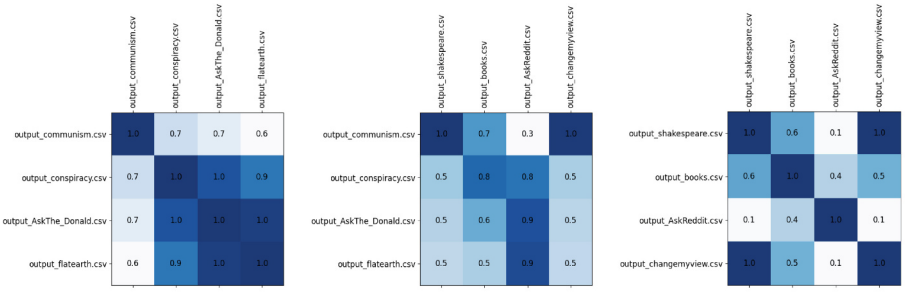


Fig. 1. Global Sentiment for A and B topics.



(a) 3 emotions A-A (b) 3 emotions A-B (c) 3 emotions B-B

Fig. 2. Comparison of cosine similarity with 3 Emotions.

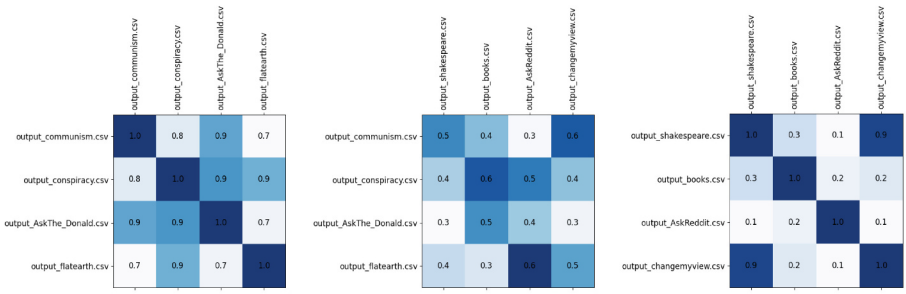
into the emotional structure of these online communities. By varying the number of emotions used in the vector representation, we observe key differences in emotional coherence and divergence across subreddit types. In the Table 2 are reported the main consideration about the obtained results (Fig. 2).

4.2 Real Echo Chambers Emotions

In order to compare the emotional aspects and differences between selected *A* and *B* topics with *Real Echo_ Chamber* (REC), we have used subReddit listed in Table 1 and provided by [20]. The comparison was made using all 27 emotions. The cosine similarity matrices and Principal Component Analysis (PCA) scatter plots showed in Figs. 5a, 5b, 5c and 6 provide complementary insights into the emotional alignment of *A, B, and REC* subreddits, discussed in Table 3 (Figs. 3 and 4).

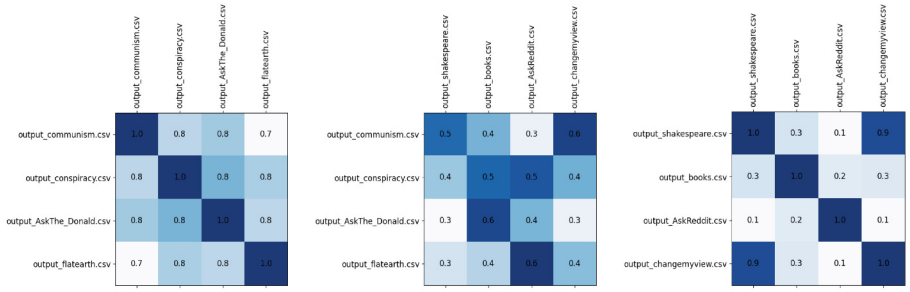
Table 2. Summary of Cosine Similarity Analysis Across Subreddit Categories

Comparison	Observation	Explanation
A vs B	High similarity with 3 emotions	Broad emotion groups are not enough to tell communities apart
	Lower similarity with 10 and 27 emotions	More detailed emotions show clearer differences
Within A	High similarity (above 0.7)	Conspiracy subreddits share similar emotional tone
	Slight drop with more emotions	Shows more detailed emotional differences between them
	Strong clustering of emotions	Suggests an echo-chamber effect in these communities
Within B	More variation in similarity scores	These subreddits talk about many different topics
	Similarity depends on topic	Literature and debate subreddits have more similar emotions
	Lower emotional unity than A	Non-conspiracy groups express a wider range of emotions



(a) 10 emotions A-A (b) 10 emotions A-B (c) 10 emotions B-B

Fig. 3. Comparison of cosine similarity with 10 Emotions.

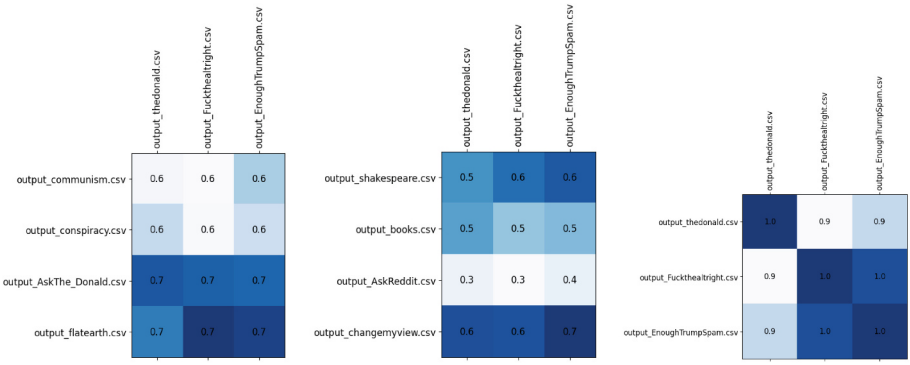


(a) 27 emotions A-A (b) 27 emotions A-B (c) 27 emotions B-B

Fig. 4. Comparison of cosine similarity with 27 Emotions.

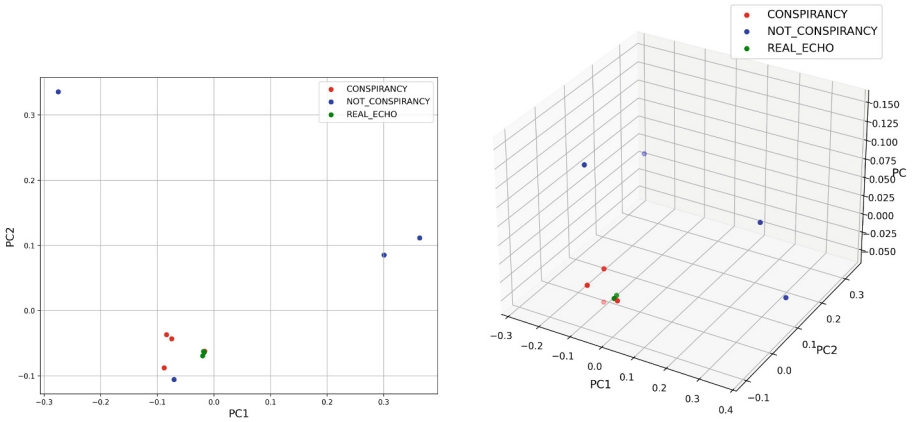
Table 3. Emotional Similarity and PCA Clustering Summary (27 Emotions)

Subreddit Group	Cosine Similarity Insight	PCA Clustering Insight
REC-REC	Very high internal similarity; emotional tone is nearly identical across all REC subreddits	Green points are tightly grouped in 2D/3D space, confirming strong emotional alignment
A-REC	Moderate-to-high similarity, close to REC levels	Red points form a compact cluster, with slight variation–indicating partial emotional homogeneity
B-REC	Lowest internal similarity; emotional tone varies more between subreddits	Blue points are widely dispersed, especially in 3D, showing emotional diversity across communities



(a) A-REC (b) B-REC (c) REC-REC

Fig. 5. Comparison of cosine similarity with All Emotions with REC.



(a) 2D PCA Scatter Plot (b) 3D PCA Scatter Plot

Fig. 6. Comparison of A, B, and REC subreddits using PCA with all emotions.

5 Conclusion and Future Works

The results of this study show clear emotional differences between echo chambers and more open communities, focusing on conspiracy and non-conspiracy topics. Conspiracy-related subreddits have strong emotional similarity, showing shared feelings and group alignment. In contrast, non-conspiracy communities show more emotional variety, which reflects openness to different views. Considering the ***RQ***, the results support the idea that emotional signatures can help identify conspiracy discussions and their possible link to echo chambers, without needing full content analysis. Cosine similarity shows that conspiracy subreddits are emotionally aligned, but not as much as real echo chambers, which are highly uniform and reinforce polarized views. Non-conspiracy subreddits show lower emotional similarity, suggesting more diverse emotional content. Principal Component Analysis (PCA) confirms a clear separation between conspiracy, non-conspiracy, and real echo chamber groups. The emotional patterns of conspiracy and real echo chamber subreddits are also very close, supporting our conclusions and those of [20]. Future work could study how these emotions change over time and under external events. It should also test this method on other platforms and explore ways to reduce the impact of echo chambers while encouraging more balanced conversations.

Acknowledgement. This work was partially supported by project SERICS (PE00000014) under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU.

References

1. Abbruzzese, R., Gaeta, A., Loia, V., Lomasto, L., Orciuoli, F.: Detecting influential news in online communities: an approach based on hexagons of opposition generated by three-way decisions and probabilistic rough sets. Inf. Sci. **578**, 364–377 (2021)
2. Bakir, V., McStay, A.: Fake news and the economy of emotions: problems, causes, solutions. Digit. J. **6**(2), 154–175 (2018)
3. Belaref, A., Séguier, R.: Multimodal transformer fusion for emotion recognition: a survey. In: 2024 6th International Conference on Natural Language Processing (ICNLP), pp. 107–113. IEEE (2024)
4. Capuano, N., Fenza, G., Loia, V., Nota, F.D.: Content-based fake news detection with machine and deep learning: a systematic review. Neurocomputing **530**, 91–103 (2023). https://doi.org/10.1016/j.neucom.2023.02.005
5. Choudhary, A., Arora, A.: Linguistic feature based learning model for fake news detection and classification. Expert Syst. Appl. **169**, 114171 (2021). https://doi.org/10.1016/j.eswa.2020.114171
6. Cinelli, M., et al.: Conspiracy theories and social media platforms. Curr. Opin. Psychol. **47**, 101407 (2022)
7. Cinelli, M., Morales, G.D.F., Galeazzi, A., Quattrociocchi, W., Starnini, M.: Echo chambers on social media: a comparative analysis. arXiv preprint arXiv:2004.09603 (2020)
8. Del Vicario, M., et al.: Echo chambers: emotional contagion and group polarization on Facebook. Sci. Rep. **6**(1), 37825 (2016)
9. Demszky, D., Movshovitz-Attias, D., Ko, J., Cowen, A., Nemade, G., Ravi, S.: Goemotions: a dataset of fine-grained emotions. arXiv preprint arXiv:2005.00547 (2020)
10. Devlin, J.: BERT: pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805 (2018)
11. Dias, N., Pennycook, G., Rand, D.G.: Emphasizing publishers does not effectively reduce susceptibility to misinformation on social media (2020)
12. Donkers, T., Ziegler, J.: De-sounding echo chambers: simulation-based analysis of polarization dynamics in social networks. Online Soc. Netw. Media **37**, 100275 (2023)
13. Gaeta, A., Loia, V., Lomasto, L., Orciuoli, F.: A novel approach based on rough set theory for analyzing information disorder. Appl. Intell. **53**(12), 15993–16014 (2023)
14. Kaliyar, R.K., Goswami, A., Narang, P.: FakeBERT: Fake news detection in social media with a BERT-based deep learning approach. Multimed. Tools Appl. **80**(8), 11765–11788 (2021)
15. Liu, Y.: RoBERTa: a robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692, vol. 364 (2019)
16. Liu, Z., Zhang, T., Yang, K., Thompson, P., Yu, Z., Ananiadou, S.: Emotion detection for misinformation: a review. Inf. Fusion 102300 (2024)
17. Lomasto, Lettieri, M.Z.: Sentiment impact on fake news detection: a preliminary study. In: Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU) (2024)
18. Mahmoudi, A., Jemielniak, D., Ciechanowski, L.: Echo chambers in online social networks: a systematic literature review. IEEE Access (2024)

19. Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J.: Distributed representations of words and phrases and their compositionality. In: Advances in Neural Information Processing Systems, vol. 26 (2013)
20. Morini, V., Pollacci, L., Rossetti, G.: Toward a standard approach for echo chamber detection: reddit case study. Appl. Sci. **11**(12), 5390 (2021)
21. Przybyła, P., Borkowski, P., Kaczyński, K.: Countering disinformation by finding reliable sources: a citation-based approach. In: 2022 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. IEEE (2022)
22. Qureshi, K.A., Malick, R., Sabih, M., Cherifi, H.: Deception detection on social media: A source-based perspective. Knowl.-Based Syst. **256**, 109649 (2022)
23. Törnberg, P.: Echo chambers and viral misinformation: modeling fake news as complex contagion. PLoS One **13**(9), e0203958 (2018)
24. Wang, C., Liu, Z., Yang, D., Chen, X.: Decoding echo chambers: LLM-powered simulations revealing polarization in social networks. arXiv preprint arXiv:2409.19338 (2024)

LLM-Supervised Multilingual Skill Extraction and Classification from Job Ads

Jakob Mørup Wang[1(✉)] and Zhiru Sun[2]

[1] Odense, Denmark
jakob@skipconnections.com
[2] University of Southern Denmark, Odense, Denmark
zhiru@sdu.dk

Abstract. This paper presents a pipeline for extracting, classifying, and representing skills requested in job advertisements to enable demand-side labor market analysis. Our contributions include: (1) addressing the annotation bottleneck by leveraging scalable, taxonomy-aligned LLM supervision for training a lightweight sentence encoder, (2) expanding skill extraction to include implicit skill requirements as well as the explicit mentions typically targeted in prior work, and (3) representing skills as distributions to robustly support downstream tasks despite the fluid, overlapping nature of skill definitions. Concretely, we compile 3M+ postings from 10k+ sources and sample 500k+ sentences to fine-tune paraphrase-multilingual-mpnet-base-v2 for identifying skill requests and mapping them to the 13,896-skill ESCO taxonomy, supervised by GPT-4o mini. The outcome is normalized per-ad skill distributions, aggregated from sentence-level distributions weighted by request probability.

Keywords: Skill extraction · Skill classification · LLM supervision · Taxonomy alignment · Labor market analysis

1 Introduction

Shifting market conditions, social transformations, and digital innovations are rapidly reshaping the nature of work [6]. Tracking which skills are in demand and how that demand changes over time can inform curriculum design, guide workforce development initiatives, and help individuals align their career paths with emerging opportunities. Online job advertisements (job ads) provide a rich public signal, yet their free text is unstructured, ambiguous, and often skills are implied rather than explicitly requested [1].

While structured taxonomies such as European Skills, Competences, Qualifications and Occupations (ESCO) [7] offer a useful reference, automatic extraction and classification remains complex. Many of ESCO's 13,896 skills overlap, and even expert annotators disagree [12,15], suggesting it is not a problem that can be 'solved', but one that must be approached pragmatically.

J. M. Wang—Independent.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 94–104, 2026.
https://doi.org/10.1007/978-3-031-97144-0_9

Skill extraction techniques have evolved alongside natural language processing, and with the introduction of transformer models [5], fine-grained span detection has become viable [18–20]. This is valuable for locating explicit terms, but often overlook implicit skill requests. Others operate at the sentence-level, using distant supervision with lexical matches of ESCO labels [3], annotating samples with varied context, but still relying on explicit terms. Document-level multi-label classification [1] attempts to capture references across an entire ad but risks losing granularity and context in longer texts. Recent efforts also explore embedding sentences or full ads and clustering them [8,10], useful for identifying broad occupational profiles, but lacking precision on specific skill requirements.

More recently, generative large language models (LLMs) have been employed directly for skill extraction [12,13] or summarization [11]. Others use LLMs to create synthetic training data for fine-tuning a sentence encoder [4] or as part of a multi-step pipeline involving candidate ranking and verification [2,16]. While these applications show promise, relying on LLM-generated training data may be too limiting to be effective outside narrow, curated benchmark datasets [3], and employing LLMs at inference time can be computationally expensive at scale.

Consequently, it remains challenging to simultaneously extract both explicit and implicit skills from noisy, multilingual job ads, classifying them effectively against a comprehensive taxonomy like ESCO without sacrificing computational efficiency, and representing them adequately for robust downstream analysis.

Building on this prior work while acknowledging that any perfect mapping to a 13,896-entry taxonomy overstates the uniqueness of each skill, we propose a pragmatic pipeline that embraces overlap and implicit requests. First, sampled sentences are annotated by an LLM using ESCO candidates retrieved through a lightweight encoder. Using this annotated data, the same encoder is then fine-tuned to detect and classify skill requests, yielding normalized per-ad skill distributions that preserve valuable information for downstream aggregation. We work within a bilingual (Danish/English) context, but the multilingual properties of the encoder and taxonomy suggests potential for broader application.

2 Method

We detail our pipeline as summarized in Fig. 1. We begin with data collection and preprocessing before describing taxonomy alignment and encoder selection, retrieval-augmented LLM supervision, multi-objective fine-tuning, and normalized-distribution skill representation. Finally, we explain and justify our evaluation and metrics.

2.1 Data Collection and Preprocessing

We take off from a structured starting point, crawling the search engine result pages and archives of Jobindex, the largest Danish job portal that aggregates most national job ads, and we collect 3M+ ads from 10k+ domains spanning the past decade. Our approach handles plain HTML pages as well as nested iframes,

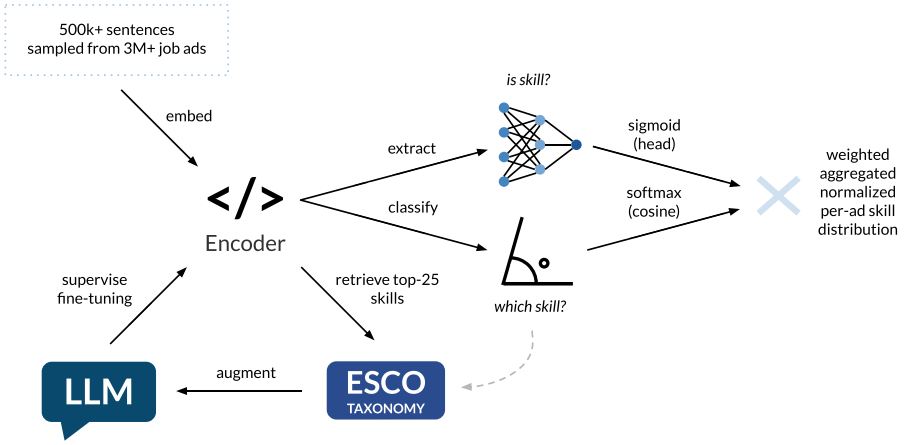


Fig. 1. Overview of our pipeline utilizing the sentence encoder twice.

dynamically generated content, and PDF files to ensure broad coverage. Once we have the raw pages we apply simple heuristics to isolate the relevant text. Using CSS selectors in BeautifulSoup and tailored regular expressions we strip away navigation, ads and boilerplate. To capture context around bulleted requirement lists we look for sentences that precede a list and prepend them to each list item before splitting into individual sentences. The result is a sentence-segmented corpus with sufficiently low residual noise for further application.

2.2 Taxonomy Alignment and Encoder Selection

We choose ESCO as taxonomy for its comprehensive multilingual coverage, providing both two-three word labels and longer explanatory descriptions for 13,896 skills across 28 European languages, including Danish [7]. To select the most promising skill anchor text and pretrained sentence encoder, we evaluate the performance of Danish and English labels and descriptions combined with different models in matching their remaining counterparts, e.g. how well Danish labels match both English labels and Danish and English descriptions measured by cosine similarity of the vector embeddings. We find the combination of English labels and paraphrase-multilingual-mpnet-base-v2 [14] shows the highest overall performance and is therefore chosen for downstream retrieval and fine-tuning.

2.3 Retrieval-Augmented LLM Supervision

We randomly sample 500k+ sentences from our corpus and generate embeddings with paraphrase-multilingual-mpnet-base-v2. For each sentence we retrieve the 25 most similar ESCO skills by cosine similarity and bundle the candidate skill labels and descriptions alongside the sentence into a structured English prompt [17]. We opt for exactly 25 candidates primarily to keep the prompt length

manageable while still allowing the model to consider a relatively broad set of skills. Even if the majority of sentences are in Danish, we find that English prompting gives better instruction following.

Using this prompt, GPT-4o mini classifies whether the sentence requests a skill of the applicant, and if so, which of the 25 candidate skills are relevant, if any. The prompt explicitly instructs the model to consider implicit requests and a broad interpretation of skills. We conduct manual spot checks on approximately 0.5% of examples, iterating prompt refinements to reduce false positives. The process yields 536,767 annotated sentences, of which 295,548 are labeled as requesting at least one skill, covering 10,658 (76.7%) of the ESCO skill categories.

2.4 Multi-objective Fine-Tuning

We fine-tune the pretrained sentence encoder in two concurrent objectives. A small feed-forward head classifies whether each sentence requests a skill. It consists of a two-step linear transformation with a regularizing 0.25-rate dropout, and a GELU activation for consistency with the encoder architecture. To reduce noise, it is trained with a false-positive penalty where z_s is the classifier's logit for sentence s, y_s is the true label (1 for requesting skills, 0 otherwise), and λ is the false-positive penalty ($\lambda = 0.01$). The loss for this objective is

$$\text{Loss}(z_s, y_s) = \text{BCE}(\sigma(z_s), y_s) \times (1 + \lambda \cdot \mathbb{I}[z_s > 0 \wedge y_s = 0]) \tag{1}$$

where BCE is the Binary Cross-Entropy and $\mathbb{I}$ is the indicator function.

Simultaneously, we optimize the embedding space for ESCO alignment by treating cosine similarities to all 13,896 skill embeddings (detached from gradient updates) as logits in a cross-entropy ranking loss.

The annotated data is split based on stratified sampling by ESCO skill category to ensure balanced coverage. The cardinalities of the training, validation and test datasets are 517,765, 9,349, and 9,653, respectively.

To stabilize learning, we freeze the bottom nine transformer layers [9], apply layer-wise learning-rate decay on the unfrozen layers (top layer LR = 1e-6), and train the classification head with a separate AdamW optimizer (LR = 1e-3, no weight decay). Batches are balanced with 96 positive (skill requests) and 96 negative samples, and each epoch samples an equal number of examples per ESCO skill to mitigate class imbalance. Because of this approach, epochs are concluded after processing 5 random samples for each of the 13,896 ESCO skills, and convergence is reached after 21 epochs, corresponding to approximately 2.8 full passes over the training data given our per-skill sampling.

Early stopping uses a combined criterion on the skill classification metrics CAP, RP@5, and MRR (see Sect. 2.6). Full implementation details and overview of tested hyperparameters are available in a public repository [17].

2.5 Normalized-Distribution Representation

We compute a sentence-level sigmoid skill request probability $\hat{y}(s)$ on the binary head output and a softmax distribution (with temperature $T = 0.01$) over all

ESCO skills $p(s)$ on cosine-similarity logits from the sentence encoder output, and then define the raw score for skill i in sentence s as

$$\text{score}_i(s) = \hat{y}(s) \times p(s)_i. \tag{2}$$

To reduce noise we discard any skill whose score does not exceed $\alpha = 0.25$. For a job ad j with sentences $s_1, \ldots, s_{n_j}$ the aggregated score for skill i is

$$S_i(j) = \sum_{k=1}^{n_j} \text{score}_i(s_k)\, \mathbb{I}[\text{score}_i(s_k) > \alpha]. \tag{3}$$

We then normalize to obtain a per-ad distribution that sums to one,

$$\widehat{S}_i(j) = \frac{S_i(j)}{\sum_{k=1}^{|\text{ESCO}|} S_k(j)}, \tag{4}$$

ensuring each ad contributes equally regardless of verbosity. Finally, for any corpus subset $\mathcal{J}$ we compute the average skill profile as

$$\widetilde{S}_i(\mathcal{J}) = \frac{1}{|\mathcal{J}|} \sum_{j \in \mathcal{J}} \widehat{S}_i(j). \tag{5}$$

With this representation we preserve information from overlapping skill signals instead of discarding them by collapsing probabilities into hard labels, and we support downstream comparison and segmentation across variables of interest without bias from verbose or terse postings.

2.6 Evaluation and Metrics

To our knowledge, prior work does not evaluate under comparable circumstances for sentence-level, implicit skill request extraction against ESCO. Therefore, we do not claim or aim for SOTA but instead provide performance metrics as context. As a baseline we use a public synthetic sentence dataset with 10 GPT-3.5-generated English examples per ESCO skill [4], augmented with our real non-skill sentences, training this variant under the same configuration as our final model. We evaluate both our final LLM-supervised model and the synthetic-data model, and include the untuned encoder for classification context.

First, we measure sentence-level extraction as a binary classification problem and report accuracy, precision, and recall on our Danish/English test split, the Danish BIO-annotated dataset KOMPETENCER [19] with spans converted to binary labels, and the English datasets TECH, HOUSE, and TECHWOLF [3,4], which all three consists exclusively of skill-related sentences.

Second, we measure taxonomy-aligned classification as both a probability-distribution task and a ranking task, reporting Correct Assigned Probability (CAP), R-Precision@5 (RP@5), and Mean Reciprocal Rank (MRR), defined as

$$\text{CAP} = \frac{1}{N}\sum_{n=1}^{N}\sum_{g\in\mathcal{G}_n} p_n[g], \tag{6}$$

$$\text{RP@5} = \frac{1}{N}\sum_{n=1}^{N}\frac{1}{\min\{5,|\mathcal{G}_n|\}}\sum_{k=1}^{5}\text{Rel}(n,k), \tag{7}$$

$$\text{MRR} = \frac{1}{N}\sum_{n=1}^{N}\frac{1}{\text{rank}(n)}, \tag{8}$$

where p_n is the temperature-scaled softmax ($T = 0.01$) over ESCO logits, $\mathcal{G}_n$ the set of ground-truth skills, $\text{Rel}(n,k)$ an indicator of whether the kth predicted skill is correct, and $\text{rank}(n)$ the position of the first correct skill. CAP aligns with our normalized-distribution objective, while RP@5 and MRR offer retrieval-style comparators used in prior work [2,4].

Finally, we examine practical utility with a brief case study of research-skill demand at the University of Southern Denmark (SDU). We identify the 20 ESCO skill embeddings closest to "research", compute their mean normalized weight per quarter, and plot them alongside the quarterly count of SDU job ads.

3 Results

As described in Sect. 2.6, we report sentence-level extraction and taxonomy-aligned classification results, and we examine a small case study of our normalized-distribution representation. Finally, we conclude with an error-type analysis from spot checks.

3.1 Sentence-Level Extraction

For identifying sentences requesting skills, Table 1 shows that our LLM-annotated model outperforms the synthetic-data baseline (using LLM-annotated negatives) on the Danish/English test split, achieving 0.83 accuracy versus 0.39. Interestingly, on the KOMPETENCER dataset our model performs slightly worse, illustrating the difference between a span-level explicit approach and a sentence-level implicit one. To illustrate, we show examples of disagreement for KOMPETENCER where our final model identifies implicit skills that are not included in the span annotations:

- "Mange af opgaverne løses [...] skolens undervisere og elever" (Many of the tasks are carried out in close contact with the training center and the cafeteria as well as the school's educators and students)
  Synth: no skill, Final: *liaise with educational staff* (0.68)
- "Som leder i enheden vil man [...] fagligt og kulturelt" (As leader of the unit, one/you will have the opportunity to contribute to building a whole new organization, professionally and culturally)
  Synth: no skill, Final: *leadership principles* (0.90)

Table 1. Skill extraction performance (identifying sentences requesting skills) across evaluation datasets, comparing our final model with the synthetic baseline.

Dataset	n	%	Accuracy		Precision		Recall	
			Final	Synth	Final	Synth	Final	Synth
Test Split	9,653	0.75	**0.83**	0.39	0.95	**0.99**	**0.82**	0.32
KOMPETENCER	1,334	0.20	0.76	**0.83**	0.49	**0.73**	**0.95**	0.39
TECH	405	1.00	**0.98**	0.79	-	-	**0.98**	0.76
HOUSE	323	1.00	**0.99**	0.79	-	-	**0.99**	0.77
TECHWOLF	326	1.00	**0.99**	0.84	-	-	**0.98**	0.80

Note: n is number of unique sentences. % is proportion labelled as skill requests.
See Sect. 2.6 for more details.

3.2 Taxonomy-Aligned Classification

For associating ESCO skills with sentences, Table 2 shows that our final model performs comparably to the synthetic baseline across the test split and public benchmarks. Analysis of the top-5 predicted skills reveals that 80% of matches are shared between our final model and the synthetic-data variant, compared to just 64% shared with the untuned model, suggesting that fine-tuning on real LLM-annotated data offers a benefit similar to synthetic data for skill classification, while also enabling implicit-skill extraction as shown in Sect. 3.1.

Table 2. Skill classification performance (associating ESCO skills with sentences) across evaluation datasets, comparing our final model with the synthetic baseline and the untuned model.

Dataset	CAP			RP@5			MRR		
	Final	Synth	Untuned	Final	Synth	Untuned	Final	Synth	Untuned
Test Split	**0.37**	0.36	0.32	0.40	0.40	0.36	**0.55**	0.53	0.49
TECH	**0.34**	0.33	0.29	**0.48**	0.47	0.42	**0.46**	0.45	0.42
HOUSE	**0.26**	0.25	0.20	**0.40**	0.39	0.34	**0.39**	0.38	0.33
TECHWOLF	0.29	0.29	0.23	0.43	0.43	0.38	**0.42**	0.41	0.34

Note: CAP is Correct Assigned Probability. RP@5 is R-Precision@5.
MRR is Mean Reciprocal Rank. See Sect. 2.6 for more details.

3.3 Normalized-Distribution Representation Case Study

For examining the practical utility of our normalized-distribution representation, we apply our pipeline to job ads from University of Southern Denmark (SDU) between 2014 and 2023 to examine trends for research-related skills. Figure 2 shows the total mean normalized weight rising from approximately 0.013 in Q1 2014 to 0.033 in Q4 2023, revealing a shift in employer demand, regardless

of fluctuations in the number of job ads. As the figure illustrates, aggregating the distributional weights of the top 20 ESCO skills semantically related to "research" produces an outcome that may not have emerged if skill signals had been collapsed to discrete labels.

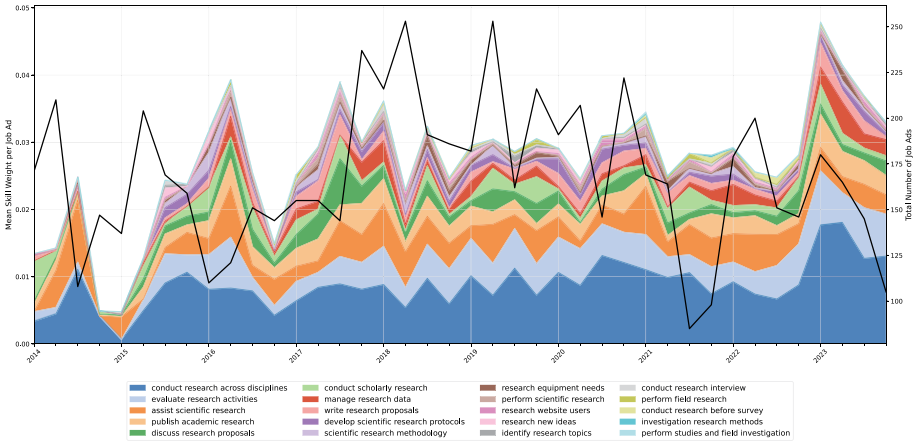


Fig. 2. Quarterly mean research-related skill weights (stacked area) and SDU job ad volume (line) from 2014–2023.

3.4 Error Analysis

Finally, spot checking model output also reveals errors. Examples include the model attributing the ESCO skill *dispatch ambulance* to a mention of "CPR", interpreting it as cardiopulmonary resuscitation, when it actually references the Danish organization "Det Centrale Personregister". In the milder area, the model attributes the ESCO skill *apply statistical analysis techniques* to the sentence "• R coding and multivariate statistical analysis", but misses the ESCO skill *R*.

However, most errors result from residual parsing noise, where sentences stem from a "Similar positions" section that was unsuccessfully filtered. Finally, other errors stem from an overly literal interpretation of implicit requirements, where for instance the model associates the skill *application process* with the invitation to "Apply". These, and other observations, are explored further in the discussion.

4 Discussion

We reflect on the boundaries of our approach, outline paths for improvement, and summarize the key takeaways.

4.1 Limitations

ESCO's 13,896 skills offer a rich taxonomy, highly useful for capturing specific nuances and delimiting skills. However, it imposes excessive granularity. While our normalized distributions embrace overlap, they can be too fine-grained for many downstream tasks. With that said, preserving this level of detail remains essential to mitigate the risk of skewing analysis results (see Sect. 3.3), even if suitable measures need to be in place to accommodate it.

Oppositely, working at the sentence level sacrifices broader context. Fragments drawn from headings, list items or link text can slip through as noise (e.g. our "Similar positions" parsing errors), yet full document modeling introduces its own complexity and may dilute sentence-level precision.

This leads to the reliance on GPT-4o mini introducing annotation biases and occasional misinterpretations, for instance associating *dispatch ambulance* with "CPR" (see Sect. 3.4). Prompt refinements brought on by manual spot checks mitigate many errors, but a systematic audit of LLM biases is necessary.

Furthermore, our bilingual (Danish/English) focus demonstrates the pipeline's multilingual capacity, yet performance may degrade in languages or domains poorly covered by ESCO or the chosen encoder. Expansion will likely require further encoder pretraining or taxonomy alignment efforts.

Embracing implicit requests boosts recall but risks diffuse outputs. Our case study on "research" skills (Sect. 3.3) shows how distributional aggregation can reveal trends that discrete labels would miss, but the error analysis (Sect. 3.4) shows how it can also lead to overly interpretive results. Striking the right balance between inclusiveness and specificity remains an open challenge.

Overall, classification of skills is inherently interpretive and shaped by evolving cultural and institutional norms. Any system that assigns labels at scale risks reinforcing existing biases, overlooking certain groups' capabilities or overemphasizing others. Regular audits, diverse and updated training samples, and iterative sanity checks are therefore essential safeguards.

4.2 Future Work

To address excessive granularity we plan experiments with data-driven meso-skill groupings, clustering ESCO entries by semantic similarity, co-occurrence, and normalized weights after classification to leverage the rich nuance in registering skills, but before downstream analysis to facilitate clear results.

Improving text preprocessing is the next step. Leveraging HTML structure or training a lightweight model to strip boilerplate and preserve meaningful context can reduce residual noise. Incorporating this context into the pipeline by including surrounding sentences, adopting sliding windows, or even considering multilingual sentence-segmentation models should help capture implicit requests more reliably in noisy settings. Extending our pipeline to additional languages and specialized labor domains will test its generality and robustness.

Following this, we will explore human-in-the-loop validation and LLM-based adjudication to audit and ensure responsible annotation biases, addressing both

the documented errors and the outlined ethical concerns regarding systematic bias against certain groups.

Finally, an iterative loop where the fine-tuned encoder retrieves refined ESCO candidates for a subsequent LLM annotation pass merits investigation, as it could potentially offer a path to progressively enhance both precision and recall.

4.3 Conclusion

We have introduced a pipeline combining LLM supervision with a lightweight multilingual sentence encoder to address three core challenges in skill extraction. First, we tackle the annotation bottleneck by using retrieved ESCO candidates to generate high-quality labels at scale. Second, we extend beyond explicit mentions to capture implicit skill requests, achieving 0.95 precision and 0.82 recall on a noisy Danish/English test split. Third, we represent skills as normalized per-ad distributions, enabling fine-grained labor-market analysis that surfaces trends such as the near threefold rise in "research"-related skills at SDU.

The results demonstrate a viable path for leveraging noisy, multilingual job ads when equipped with scalable, taxonomy-aligned supervision. Labor-market analysts, educators and policymakers can benefit when monitoring evolving skill demands, informing curriculum design and guiding workforce development.

Future work includes clustering micro-skills into meso-groups, improving preprocessing to remove noise, and adding human-in-the-loop validation to boost reliability. With these refinements, the pipeline can provide increasingly nuanced and reliable representations of skill dynamics in the labor market.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Bhola, A., Halder, K., Prasad, A., Kan, M.Y.: Retrieving skills from job descriptions: a language model based extreme multi-label classification framework. In: Scott, D., Bel, N., Zong, C. (eds.) Proceedings of 28th COLING, Barcelona, Spain (Online), pp. 5832–5842. International Committee on Computational Linguistics (2020). https://doi.org/10.18653/v1/2020.coling-main.513
2. Clavié, B., Soulié, G.: Large language models as batteries-included zero-shot ESCO skills matchers (2023). https://doi.org/10.48550/arXiv.2307.03539
3. Decorte, J., Hautte, J.V., Deleu, J., Develder, C., Demeester, T.: Design of negative sampling strategies for distantly supervised skill extraction. CoRR abs/2209.05987 (2022). https://doi.org/10.48550/ARXIV.2209.05987
4. Decorte, J.J., Verlinden, S., Hautte, J.V., Deleu, J., Develder, C., Demeester, T.: Extreme multi-label skill extraction training using large language models (2023). https://doi.org/10.48550/arXiv.2307.10778
5. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. In: Burstein, J., Doran, C., Solorio, T. (eds.) Proceedings of NAACL-HLT 2019, Minneapolis, Minnesota, pp. 4171–4186. ACL (2019). https://doi.org/10.18653/v1/N19-1423

6. Di Battista, A., et al.: Future of jobs report 2023. In: World Economic Forum, Geneva, Switzerland (2023). https://www.weforum.org/reports/the-future-of-jobs-report-2023
7. European Commission: ESCO (European skills, competences, and occupations) database (2024). https://esco.ec.europa.eu/. Accessed 31 Dec 2024
8. Kavas, H., Serra-Vidal, M., Wanner, L.: Enhancing job posting classification with multilingual embeddings and large language models. In: Dell'Orletta, F., Lenci, A., Montemagni, S., Sprugnoli, R. (eds.) Proceedings of 10th CLiC-it, pp. 440–450. CEUR Workshop Proceedings (2024). https://aclanthology.org/2024.clicit-1.53/
9. Lee, J., Tang, R., Lin, J.: What would elsa do? Freezing layers during transformer fine-tuning (2019). https://doi.org/10.48550/arXiv.1911.03090
10. Leon, F., Gavrilescu, M., Floria, S.A., Minea, A.A.: Hierarchical classification of transversal skills in job advertisements based on sentence embeddings. Information **15**(3) (2024). https://doi.org/10.3390/info15030151
11. Li, N., Kang, B., Bie, T.D.: Skillgpt: a restful API service for skill extraction and standardization using a large language model. CoRR abs/2304.11060 (2023). https://doi.org/10.48550/ARXIV.2304.11060
12. Li, N., Kang, B., De Bie, T.: Llm4jobs: unsupervised occupation extraction and standardization leveraging large language models. Knowl.-Based Syst. **316**, 113302 (2025). https://doi.org/10.1016/j.knosys.2025.113302
13. Nguyen, K., Zhang, M., Montariol, S., Bosselut, A.: Rethinking skill extraction in the job market domain using large language models. In: Hruschka, E., Lake, T., Otani, N., Mitchell, T. (eds.) Proceedings of NLP4HR 2024, St. Julian's, Malta, pp. 27–42. ACL (2024). https://aclanthology.org/2024.nlp4hr-1.3/
14. Reimers, N., Gurevych, I.: Sentence-BERT: sentence embeddings using Siamese BERT-networks. In: Inui, K., Jiang, J., Ng, V., Wan, X. (eds.) Proceedings of EMNLP-IJCNLP 2019, Hong Kong, China, pp. 3982–3992. ACL (2019). https://doi.org/10.18653/v1/D19-1410
15. Sibarani, E.M., Scerri, S., Morales, C., Auer, S., Collarana, D.: Ontology-guided job market demand analysis: a cross-sectional study for the data science field. In: Proceedings of SEMANTiCS 2017, pp. 25–32. Association for Computing Machinery, New York (2017). https://doi.org/10.1145/3132218.3132228
16. Vrolijk, J., Poslavsky, V., Bijl, T., Popov, M., Mahdavi, R., Shokri, M.: Ontology learning for ESCO: leveraging LLMs to navigate labor dynamics. In: Razniewski, S., Kalo, J., Singhania, S., Pan, J.Z., Nguyen, T., Zhang, B. (eds.) Proceedings of 2nd KBC-LM and 3rd LM-KBC Workshops at ISWC 2024, Baltimore, USA. CEUR Workshop Proceedings, vol. 3853. CEUR-WS.org (2024). https://ceur-ws.org/Vol-3853/paper2.pdf
17. Wang, J.M., Sun, Z.: skillextraction (2024). https://github.com/jakobmwang/skillextraction. Accessed 09 May 2025
18. Zhang, M., van der Goot, R., Plank, B.: ESCOXLM-R: multilingual taxonomy-driven pre-training for the job market domain. In: Rogers, A., Boyd-Graber, J., Okazaki, N. (eds.) Proceedings of ACL 2023, pp. 11871–11890. ACL (2023). https://doi.org/10.18653/v1/2023.acl-long.662
19. Zhang, M., Jensen, K., Plank, B.: Kompetencer: fine-grained skill classification in danish job postings via distant supervision and transfer learning. In: Proceedings of 13th LREC, pp. 436–447. European Language Resources Association (2022). https://doi.org/10.48550/arXiv.2205.01381
20. Zhang, M., Jensen, K., Sonniks, S., Plank, B.: Skillspan: hard and soft skill extraction from english job postings. In: Proceedings of NAACL 2022. ACL (2022). https://doi.org/10.48550/arXiv.2204.12811

Cross-Domain Named Entity Recognition: A Resource-Efficient Transfer Learning Approach

Gianmaria Balducci[1,2(✉)], Elisabetta Fersini[1], and Enza Messina[1]

[1] University of Milano-Bicocca, Milan, Italy
g.balducci1@campus.unimib.it
[2] PMI Reboot S.r.l., Milan, Italy

Abstract. Named Entity Recognition (NER) is a fundamental task in natural language processing that involves identifying and classifying entities such as names, locations, and organizations within text. Although public NER resources such as annotated datasets and annotation services exist across various domains, no single resource typically supports all entity types required for specific downstream applications. Additionally, the availability of training data to effectively develop NER systems for different domain classification schemas is often limited due to constraints on time, quality, and annotation costs. In this paper, we propose to address these issues by a transfer learning approach, validating the hypothesis that with limited resources, the target domain labels and their distributions can be learned by exploiting features obtained from the source domain. The proposed approach achieves comparable performance with respect to fine-tuning state-of-the-art transformer-based models, using a limited amount of resources in terms of GPU, CPU and RAM.

Keywords: Transfer Learning · NER · Resource Consumption

1 Introduction

Named Entity Recognition (NER) [1] serves as a fundamental task in data extraction, aiming to identify and categorize named elements in content according to predefined classes. Traditional NER systems operate under the assumption that training and test data share the same feature space and underlying distribution. However, when adapting a NER system to a new target classification schema, this assumption may not hold. Since new data may exhibit different representation characteristics and follow distinct distributions compared to the source domain, the absence of appropriate training sets poses significant challenges regarding human effort, resource allocation, and time investment [2]. To address these limitations, we propose an efficient supervised approach that aims to transfer knowledge from a source domain to a target domain

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 105–115, 2026.
https://doi.org/10.1007/978-3-031-97144-0_10

for named entity recognition purposes. The proposed approach enables learning target domain labels and distributions by exploiting token-level features derived from transformer-based models pre-trained on source domains, including attention patterns, hidden states, and probability vectors. In particular, by leveraging transformer model characteristics across different tagsets, satisfactory results can be achieved using only 10% of the available training set. We evaluate the proposed approach on three benchmark datasets, resulting in six distinct transfer scenarios. Results demonstrate that our approach achieves comparable performance to fine-tuned transformer-based models but with reduced computational resources.

2 Related Work

NER systems often require adaptation when shifting from a source to a target domain. This issue has received significant attention in recent years, with several novel approaches emerging to address the inherent difficulties of aligning divergent conceptual frameworks. Early work in domain adaptation for NER primarily focused on rule-based mappings and simple statistical approaches. However, the emergence of neural approaches has dramatically transformed the research landscape [3]. Such methods [4] employ noise-contrastive estimation to align semantically similar concepts in the embedding space while using regularization techniques to preserve original semantic knowledge. This approach demonstrated significant improvements over previous methods on benchmark datasets. Grounding on word embedding techniques, Fersini et al. [2] explored domain adaptation specifically for NER systems. Their research showed that carefully constructed word embeddings can effectively bridge the gap between source and target domains by capturing underlying semantic relationships. Recent advances have leveraged self-alignment and contrastive learning techniques to improve cross-domain adaptation. [5] introduced a self-alignment pretraining approach specifically designed for biomedical entities. Semi-supervised learning has emerged as another powerful paradigm for domain adaptation [6]. For domains with limited resources, knowledge distillation has proven to be an effective strategy. Chen and Smith [7] explored knowledge distillation as a means of transferring NER capabilities across domains. The advent of large pre-trained language models has significantly impacted domain adaptation research. Models like BioBERT [8] and SciBERT [9] have demonstrated impressive capabilities in specialized domains. Li et al. [10] proposed OntologyBERT, which incorporates explicit ontological knowledge during the pre-training phase. Starting from PLMs, parameter-efficient fine-tuning has gained significant attention as a computationally efficient approach. Jia et al. [11] proposed a Parameter-Efficient Domain Adaptation framework that fine-tunes only a small subset of model parameters while maintaining 97–99% of full fine-tuning performance. As domain adaptation often faces data scarcity challenges in target domains, few-shot and zero-shot learning approaches have gained significant traction. Zhang et al. [12] demonstrated that with carefully designed prompting strategies, large language

models can perform reasonably well at NER tasks even across different ontological frameworks with minimal examples. Although the above-mentioned investigations represent a fundamental step towards the adaptation of NER systems, one main issue remains open: most of the available approaches are often computationally expensive, requiring resources that lead to high energy consumption, making them costly and less sustainable for real world applications. In this paper, we validate the hypothesis that we can transfer the knowledge acquired from a source to a target domain to accurately recognize entities, exploiting as much as possible pre-trained language models without the necessity of fine-tuning and guaranteeing a substantial reduction in energy consumption and execution time, thereby resulting in a more cost-effective solution.

3 The Proposed Approach

3.1 NER Transfer Scenario

Transfer learning in NER involves adapting a model trained on a source domain S with entity types Y_S to a target domain T with potentially different entity types Y_T. Traditional approaches typically require substantial labeled data in the target domain and computationally expensive fine-tuning. In this paper, we address the following scenario:

1. Labeled data in the target domain is limited;
2. Computational resources for fine-tuning are minimal;
3. Entity types and distributions differ between source and target domains.

In particular, we verify the hypothesis that trivial token-level representations from transformer models pre-trained on a source domain contain sufficient information to accurately recognize entities in a target domain, requiring only a small portion of labeled target data for training classifiers with limited computational resources. To this purpose, we aim at creating a feature space, without the necessity of the expensive costs of fine-tuning, that is able to characaterize entities in the source domain and to learn the relationship with the target domain.

3.2 Input Space Representation

In order to reuse as much as possible the knowledge about the language of existing pre-trained models, therefore reducing the energy consumption costs required to fine-tune them, we extracted the following token representations. In particular, a token i can be represented as follows:

– **Contextualized Hidden States**: this is denoted by the hidden state representation from the final layer of a pre-trained language model, which captures rich contextual information about each token. For a token i, we derive a vector $h_i \in \mathbb{R}^d$, where d is the hidden dimension.

- **Attention Scores**: this representation derives from the multi-head attention weights from the final layer of a pre-trained language model. In particular, each token is represented through an attention tensor $A_i \in \mathbb{R}^{m \times n \times n}$, where m is the number of attention heads and n is the sequence length. Each tensor A_i is reshaped into a one-dimensional vector a_i to create an attention-based feature representation for each token.
- **Source Probability Distribution**: this representation corresponds to the probability distribution of each entity mention in the source domain. Given a token i that belongs to an entity mention that has been recognized by a pre-trained NER system in the source domain, its probability distribution across the source tagset is taken as a suitable representation. For a token i, we therefore derive a vector $p_i \in \mathbb{R}^{|Y_T|}$, where Y_T is the target tagset.

3.3 Learning Objective

The vectors introduced above allow us to create an input space for representing each entity mention that can be subsequently used to learn how to map a predicted source type $y_s \in Y_S$ to a target type $y_t \in Y_T$. In particular, let $\Phi_S = \{\phi_{s_1}, \phi_{s_2}, \ldots, \phi_{s_n}\}$ be a set of entity mentions annotated according to a source tagset Y_S and M_S be a pre-trained language model underlying a NER system trained on the source domain. Let $\Psi_T = \{\psi_{t_1}, \psi_{t_1}, \ldots, \psi_{t_r}\}$ be a (small) set of entity mentions that can be used to train a model in the target domain under the tagset Y_T. Rather than fine-tuning a pre-trained model using Ψ_T, our main objective is to train shallow classifiers on the input space previously defined to learn how to map the prediction of a model trained in the source domain into a target domain. Given a token i, we represent it using its Contextualized Hidden States vector h_i, its Attention Score vector a_i and its Source Probability Distribution vector p_i. In its general form, the input space X for a token i is denoted as $X_i = h_i \oplus a_i \oplus p_i$, where $\oplus$ denotes the concatenation operator.

Our main goal is to learn a function $f : X_i \rightarrow y_t \in Y_T$:

$$g(X_i) = \arg \max_{y_t \in Y_T} f(X_i, y_t) \tag{1}$$

To accomplish this task, any shallow machine learning algorithm can be adopted to learn f for any source-target transfer. In our case, we adopted the set C of classifiers, i.e. *Support Vector Machine, Stochastic Gradient Descent, Passive Aggressive, Perceptron, Multi-Layer Perceptron* . Each classifier is trained independently on each feature space or subspace that can be composed of hidden states, attention patterns, and probability vectors, resulting in a diverse set of base classifiers with varying strengths and error patterns.

3.4 Heterogeneous Bayesian Model Averaging

To leverage the complementarity of different classifiers and feature types, we grounded on a Bayesian Model Averaging (BMA) ensemble approach initially proposed in [13]. In particular, we trained the above mentioned machine learning

models using $X_i = h_i \oplus a_i \oplus p_i$ or any of its subspaces (e.g. $X_i = h_i \oplus a_i$, $X_i = a_i$, etc.). The resulting set of trained models are combined according to a BMA ensemble paradigm, which considers both the marginal probability and the reliability of each model to generate the final prediction. The final ensemble is composed of the optimal combination of base classifiers from all possible sets of models trained on different feature spaces. Therefore, since we train the models on heterogeneous spaces, we will denote the proposed approach as Heterogeneous Bayesian Model Averaging (HBMA). A graphical representation of the proposed approach is reported in Fig. 1.

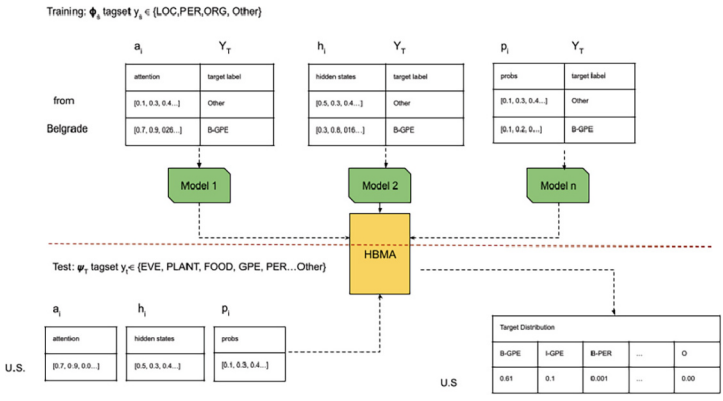


Fig. 1. Workflow of the proposed approach.

This ensemble approach enables effective knowledge transfer by exploiting complementary information captured in different feature subspaces and relying on the capability of each model with respect to each class label.

4 Datasets and Baselines

We report here the details of the benchmark datasets used for the experimental evaluation, i.e. OntoNotes 5.0 [14], CoNLL-2003 [15], and MultiNERD [16]. Each dataset presents distinct characteristics in terms of size, annotation schema, and linguistic properties, making them particularly suitable for evaluating cross-domain adaptations. The datasets entity tag sets are listed below:

- **CoNLL03**: *Person, Organization, Location, Miscellaneous*
- **Ontonotes 5.0**: *Person, Organization, Location, GPE, Facility, Time, Date, Money, Cardinal, Percent, Product, Event, Work Of Art, Ordinal, Language, Law, Norp, Quantity*
- **MultiNERD**: *Person, Location, Organization, Animal, Bio, Celestial Body, Disease, Event, Instrument, Media, Mythology, Plant, Time, Vehicle, Food*

Table 1 reports a few statistics related to the considered datasets. The granularity of the tagsets presents both challenges and opportunities for adapting

Table 1. Dataset statistics

Feature	OntoNotes 5.0	CoNLL-2003	MultiNERD
Domains	News, broadcast, telephone, web, religious	News	News, wiki, social media, web
Entity Types	18	4	15
Total Entities	153,092	35,089	54,020
Languages	English, Chinese, Arabic	English, German	11 languages
Publication Year	2013	2003	2021
Annotation Scheme	Hierarchical	Flat	Hierarchical
Token Count	∼1.6M	∼301K	∼365K (English)

NER systems from a source to a target domain. For instance, the LOC category in CoNLL-2003 can include both GPE and LOC in OntoNotes. An additional challenge can be grasped by considering MultiNERD, which introduces specialized categories like *Animal* and *Plant* that do not have any direct correspondence in the other datasets. The proposed approach has been compared against fine-tuning one of the most promising transformer-based architectures suitable for NER purposes, i.e. BERT-NER [17]. More precisely, the BERT-NER models implemented with transformers library from Huggingface are pre-trained on Ontonotes5.0[1], pre-trained on MultiNERD[2] and pre-trained on ConLL03[3] For each tansfer-learning scenario reported in Table 2, source BERT-NER models are fine-tuned using a batch size of 32 for 3 epochs.

5 Performance Measures and Experimental Setup

5.1 Entity Recognition Capabilities

The primary metrics for NER evaluation are entity-level precision, recall, and F1 score [15]. It is important to note that since each pre-trained NER exploits the BIO tagging schema, an entity is considered correctly predicted only if both boundaries and type are exactly matched. This strict evaluation is the standard approach in NER benchmarking and reflects real-world requirements where both the span and the semantic type of an entity must be correct to be useful.

5.2 Resource Consumption

To assess the efficiency of our approach, we measured several indicators:

- **CPU usage (%)**: Average CPU usage during training and inference.
- **GPU usage (GB)**: The amounts of GPU memory (GB) used during training and inference, sampled at 0.2-second intervals.

[1] https://huggingface.co/djagatiya/ner-bert-base-cased-ontonotesv5-englishv4.
[2] https://huggingface.co/medxiaorudan/bert-base-cased-finetuned-MultiNERD-SystemA.
[3] https://huggingface.co/datasets/eriktks/conll2003.

- **RAM usage (GB)**: memory footprint in terms of GB.
- **Execution Time (s)**: seconds to process target domain test set.

These measurements enable direct comparison of the proposed approach with the fine-tuning of pre-trained NER mechanisms, which is particularly relevant in resource-constrained environments. To ensure a fair evaluation and account for potential hardware-specific performance variations, we conducted experiments considering the following hardware configurations: **CPU** AMD Ryzen 7 5700G 8-Core Processor (3.8 GHz base, 4.6 GHz boost), **GPU** NVIDIA GeForce RTX 3090 with 24 GB GDDR6X memory, **RAM** 64 GB DDR4-3200 MHz. The experiments were executed on Ubuntu 20.04 LTS with NVIDIA (version 525.105.17) and CUDA toolkit 12.0. PyTorch 2.0.1 and the Hugging Face Transformers library (version 4.30.2) have been used to implement the transformer components. For the machine learning classifiers, we used *scikit-learn 1.3.0*.

5.3 Transfer Learning Scenarios

We evaluated our approach on all possible source-target domains adopting the three benchmark OntoNotes 5.0, CoNLL-2003, and MultiNERD. This resulted in the following transfer scenarios:

Table 2. Transfer scenarios

Source	Target	Transfer Complexity
OntoNotes 5.0	CoNLL-2003	High (18 4 types)
CoNLL-2003	OntoNotes 5.0	High (4 18 types)
OntoNotes 5.0	MultiNERD	Medium (18 15 types)
MultiNERD	OntoNotes 5.0	Medium (15 18 types)
CoNLL-2003	MultiNERD	High (4 15 types)
MultiNERD	CoNLL-2003	High (15 4 types)

Each configuration denotes unique challenges in terms of entity type mapping. For example, transferring from CoNLL-2003 to OntoNotes 5.0 requires a shift from a coarse-grained 4-type tagset to a fine-grained 18-type schema, while in the opposite direction (from OntoNotes 5.0 to CoNLL-2003), it is necessary to shift from a high number of possible labels into broader classes. For each transfer learning scenario, we experimented with three different sizes of the training set, i.e. 5000, 2500 and 1200 instances. These settings represent scenarios with a limited amount of data to assess both model capabilities and resource consumption.

6 Results

We report in this section a detailed analysis of the entity recognition capabilities and the resource consumption of the proposed approach, comparing the results with the state of the art represented by fine-tuning BERT-NER models pre-trained on source domains. In particular, in Table 3, a comparison between

Table 3. Model Comparison: Resource consumption vs Prediction Capabilities

Experiment	Model	Training			Inference				F1
		CPU (%)	RAM (GB)	GPU (GB)	Execution Time (s)	CPU (%)	RAM (GB)	GPU (GB)	
MultiNERD ↓ CoNLL		**5K Training Instances**			**CoNLL Test Set**				
	HBMA	30.89	**11.64**	**1.71**	**23.25**	**4.32**	**1.57**	**1.06**	**0.864**
	Bert	**6.19**	13.21	15.34	24.28	10.5	2.94	1.40	0.843
		2.5K Training Instances			**CoNLL Test Set**				
	HBMA	28.36	**7.20**	**1.70**	23.38	**4.22**	**1.56**	**1.06**	**0.858**
	Bert	**4.48**	13.18	15.48	**19.29**	11.2	2.80	1.40	0.857
		1.2K Training Instances			**CoNLL Test Set**				
	HBMA	28.15	**4.92**	**1.66**	**23.40**	**4.28**	**1.57**	**1.06**	**0.856**
	Bert	**7.42**	13.16	20.03	23.67	9.9	2.99	1.40	0.846
CoNLL ↓ MultiNERD		**5K Training Instances**			**MultiNERD Test Set**				
	HBMA	47.63	17.86	**1.56**	**124.43**	23.41	**2.86**	**1.00**	0.910
	Bert	**6.50**	**15.68**	16.87	215.75	**17.7**	11.70	1.40	**0.912**
		2.5K Training Instances			**MultiNERD Test Set**				
	HBMA	40.02	**9.15**	**1.48**	**125.46**	23.26	**2.82**	**1.00**	0.901
	Bert	**6.90**	15.41	16.81	216.74	**18.9**	11.74	1.40	**0.903**
		1.2K Training Instances			**MultiNERD Test Set**				
	HBMA	42.45	**6.18**	**1.34**	**125.27**	23.07	**2.78**	**1.00**	**0.891**
	Bert	**11.88**	15.40	17.00	215.57	**19.0**	11.64	1.40	0.867
Ontonotes ↓ MultiNERD		**5K Training Instances**			**MultiNERD Test Set**				
	HBMA	39.82	17.21	**1.53**	**106.94**	12.91	**2.91**	**1.00**	0.907
	Bert	**6.76**	**13.25**	17.24	241.59	**9.9**	12.10	1.41	**0.917**
		2.5K Training Instances			**MultiNERD Test Set**				
	HBMA	48.41	**9.04**	**1.42**	**107.3**	**12.91**	**2.90**	**1.00**	0.895
	Bert	**10.5**	12.94	14.21	238.29	16.8	12.28	1.41	**0.899**
		1.2K Training Instances			**MultiNERD Test Set**				
	HBMA	55.49	**6.01**	**1.20**	**106.97**	**12.94**	**2.91**	**1.00**	0.886
	Bert	**4.35**	13.08	16.73	239.47	16.8	12.28	1.41	**0.889**
Ontonotes ↓ CoNLL		**5K Training Instances**			**CoNLL Test Set**				
	HBMA	28.40	**10.00**	**1.66**	23.32	**4.30**	**1.59**	**1.06**	**0.890**
	Bert	**6.67**	13.24	15.18	**19.16**	11.2	2.7	1.40	0.851
		2.5K Training Instances			**CoNLL Test Set**				
	HBMA	29.71	**6.51**	**1.83**	**23.13**	**4.34**	**1.58**	**1.06**	**0.879**
	Bert	**7.52**	13.23	15.37	27.35	9.2	2.65	1.40	0.858
		1.2K Training Instances			**CoNLL Test Set**				
	HBMA	36.03	**4.45**	**1.77**	23.00	**4.41**	**1.58**	**1.08**	**0.878**
	Bert	**4.10**	13.21	14.33	**19.79**	9.9	2.92	1.40	0.850
MultiNERD ↓ Ontonotes		**5K Training Instances**			**Ontonotes Test Set**				
	HBMA	46.60	18.27	**1.71**	**51.59**	**6.42**	**1.53**	**1.08**	0.769
	Bert	**6.22**	**14.51**	20.40	55.38	17.9	4.41	1.40	**0.842**
		2.5K Training Instances			**Ontonotes Test Set**				
	HBMA	51.51	**10.49**	**1.62**	**51.51**	**6.30**	**1.52**	**1.08**	0.759
	Bert	**6.53**	15.30	17.65	55.67	17.8	4.43	1.40	**0.827**
		1.2K Training Instances			**Ontonotes Test Set**				
	HBMA	43.15	**5.81**	**1.63**	**50.84**	**6.39**	**1.52**	**1.08**	0.734
	Bert	**6.48**	15.28	17.29	55.8	17.7	4.47	1.40	**0.807**
CoNLL ↓ Ontonotes		**5K Training Instances**			**Ontonotes Test Set**				
	HBMA	64.09	18.61	**1.71**	**55.03**	**4.41**	**1.61**	**1.09**	**0.831**
	Bert	**4.39**	**13.22**	19.86	61.64	9.8	4.80	1.41	0.827
		2.5K Training Instances			**Ontonotes Test Set**				
	HBMA	72.66	**10.85**	**1.61**	**54.05**	**4.39**	**1.61**	**1.09**	**0.799**
	Bert	**8.07**	13.23	18.01	79.52	13.6	4.87	1.41	0.789
		1.2K Training Instances			**Ontonotes Test Set**				
	HBMA	88.97	**6.55**	**1.54**	**55.33**	**4.19**	**1.41**	**1.09**	**0.775**
	Bert	**4.03**	12.55	14.12	87.30	10.6	4.88	1.41	0.723

HBMA and Bert fine-tuned (3 epochs) is shown, reporting both resource consumption and entity recognition capabilities. The transfer from MultiNERD to CoNLL reveals a good trade-off between efficiency and prediction capabilities. We can easily note that HBMA not only outperforms the fine-tuned Bert in terms of F1 measure, but its resource consumption is approximately 8–12 times less in terms of GPU during training. The transfer from CoNLL to MultiNERD shows a substantial difference in inference times between approaches. In fact, HBMA takes ∼125 s while BERT ∼215 s, implying that the proposed approach is able to achieve comparable F1 with respect to the model baseline but at half of the inference time. When considering the transfer from Ontonotes to MultiNERD, we can similarly highlight that HBMA achieves comparable performances in terms of F1 score, but with a substantially lower resource consumption at inference time. Regarding the transfer from Ontonotes to CoNLL, HBMA is able to significantly outperform the state-of-the-art baseline also using a decreasing number of training instances. The transfer scenario from MultiNERD to Ontonotes represents the only case where BERT consistently outperforms HBMA across the different training set sizes. Despite performance disparities, HBMA maintains its resource efficiency advantageous, particularly in GPU utilization. Finally, also in the case of transfer from CoNLL to Ontonotes we can highlight that HBMA outperform the state-of-the-art Bert-based model, still maintaining a reduced resource consumption at inference time.

As a general remark, we can affirm that HBMA generally achieves competitive or superior performance compared to BERT fine-tuning, particularly when a limited amount of data is available in the target domain. The proposed approach uses higher CPU resources (28–89%) compared to BERT fine-tuning (4–13%), but requires consistently less RAM during both training and inference. For what concerns GPU Utilization, in the training phase, HBMA demonstrates substantially lower GPU requirements (1.3–1.8 GB) compared to BERT fine-tuning (14–20 GB) and maintains lower GPU usage during the inference phase compared to the fine-tuning approach. Regarding Inference Time, HBMA shows significantly faster inference for some dataset combinations. Additionally, HBMA exploits a consistently smaller quantity of RAM compared to BERT fine-tuning. BERT fine-tuning can achieve marginally superior performance in some dataset combinations (e.g. from MultiNERD to Ontonotes) but at significantly higher resource consumption costs. This suggests that HBMA may be particularly suitable for low-resource languages, specialized domains, or applications where labeled data acquisition is too expensive or challenging. The substantial differences in resource utilization translate into significant implications for deployment costs, energy consumption, and environmental impact. HBMA's lower GPU and RAM requirements during both training and inference represent substantial advantages for resource-constrained environments or large-scale deployments where efficiency is of paramount importance. In particular, if we assume a system that should be maintained operational continuously, we can easily estimate the reduced energy consumption of the proposed approach with respect to the state of the art. For instance, considering the hardware infrastructure used in these experiments,

when processing MultiNERD (32k sentences), considering GPU, the energy consumption per second is approximately 0.097 kWh, while CPU contributes to an additional 0.018 kWh per second. For MultiNERD, the reduced inference time (from 215 s to 125 s) saves 90 s × 0.097 kWh for what concerns the GPU and 90 s × 0.018 kWh for what regards CPU, which roughly corresponds to ∼10.35 kWh on a full inference cycle. Over extended periods, scaling to millions of sentences per day, this reduction compounds into significant cost and carbon footprint reductions. In a real-world deployment scenario, where inference is performed continuously (e.g. on news articles or social media content), HBMA not only reduces the computational overhead but also significantly lowers energy consumption, potentially saving 40–50% of the power with respect to the state-of-the-art BERT-NER. The source code and the full ablation study are available at https://github.com/MIND-Lab/CrossDomainNER.

7 Conclusions

In this paper, we presented a transfer learning approach for Named Entity Recognition that enables the accurate learning of target domain labels from source domain features. The proposed method demonstrates competitive performance compared to fine-tuned transformer-based models while requiring significantly lower computational resources. HBMA guarantees 55–60% faster inference times compared to Bert fine-tuning, making it particularly suitable for real-time applications. Additionally, HBMA reduces GPU memory requirements by approximately 91% during both training and inference compared to Bert fine-tuning, enabling deployment in resource-constrained environments. These findings show that reusing the knowledge of pre-trained language models not only allows to achieve comparable performance with respect to fine-tuned transformer architectures but at lower computational costs.

References

1. Augenstein, I., Derczynski, L., Bontcheva, K.: Generalisation in named entity recognition: a quantitative analysis. Comput. Speech Lang. **44**, 61–83 (2017)
2. Nozza, D., Manchanda, P., Fersini, E., Palmonari, M., Messina, E.: LearningToAdapt with word embeddings: domain adaptation of named entity recognition systems. Inf. Process. Manag. **58**(3), 102537 (2021)
3. Ullah, N., et al.: An effective approach to detect and identify brain tumors using transfer learning. Appl. Sci. **12**(11), 5645 (2022)
4. Kolyvakis, P., Kalousis, A., Kiritsis, D.: Deepalignment: unsupervised ontology matching with refined word vectors. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, vol. 1, pp. 787–798 (2018)
5. Sung, M., Jeon, H., Lee, J., Kang, J.: Self-alignment pretraining for biomedical entity representations. In: Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 312–323 (2022)

6. Wang, L., Zhang, Y., Sun, X.: Cross-ontology neural entity recognition with semi-supervised domain adaptation. IEEE/ACM Trans. Comput. Biol. Bioinf. **20**(2), 1126–1137 (2023)
7. Chen, Y., Smith, P.: Knowledge distillation for ontology transfer in low-resource domains. In: Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, pp. 704–715 (2023)
8. Lee, J., et al.: Biobert: a pre-trained biomedical language representation model for biomedical text mining. Bioinformatics **36**(4), 1234–1240 (2020)
9. Beltagy, I., Lo, K., Cohan, A.: Scibert: a pretrained language model for scientific text. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 3615–3620 (2019)
10. Li, T., Jiang, Z., Qin, L., Zhang, X., Wang, S., Wu, L.: Ontologybert: integrating ontological knowledge in pre-trained language models for concept extraction. In: Proceedings of the 29th International Conference on Computational Linguistics, pp. 2578–2588 (2022)
11. Jia, C., Wu, Y., Zhang, Y.: PEDA: parameter-efficient domain adaptation for named entity recognition. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, pp. 1578–1590 (2023)
12. Zhang, K., Zhao, R., Feng, Y.: Few-shot cross-ontology named entity recognition with prompt-based contrastive learning. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, pp. 9472–9486 (2023)
13. Fersini, E., Messina, E., Pozzi, F.A.: Sentiment analysis: Bayesian ensemble learning. Decis. Support Syst. **68**, 26–38 (2014)
14. Weischedel, R., et al.: OntoNotes Release 5.0 (2013)
15. Sang, E.F.T.K., De Meulder, F.: Introduction to the CoNLL-2003 shared task: language-independent named entity recognition. In: Proceedings of the 7th Conference on Natural Language Learning at HLT-NAACL 2003, pp. 142–147 (2003)
16. Tedeschi, S., Navigli, R.: MultiNERD: a multilingual, multi-genre and fine-grained dataset for named entity recognition (and disambiguation). In: Carpuat, M., de Marneffe, M.-C., Ruiz, I.V.M. (eds.) Findings of the Association for Computational Linguistics: NAACL 2022, pp. 801–812 (2022)
17. Liu, Z., Jiang, F., Yuxiang, H., Shi, C., Fung, P.: NER-BERT: a pre-trained model for low-resource entity tagging (2021)

Explainable AI for NLP: Enhancing Transparency in Sentiment Analysis and Named Entity Recognition

Feras Elkharrat(✉), Mohamed Ghoniem, Ahmed Sherif, and Caroline Sabty

German International University, New Administrative Capital, Egypt
{feras.elkharrat,mohamed.ghoniem}@student.giu-uni.de,
{ahmed.sherif,caroline.sabty}@giu-uni.de

Abstract. As Natural Language Processing (NLP) models grow more complex, explaining their predictions becomes increasingly challenging—particularly for tasks involving structured outputs or multilingual input. This paper introduces adaptations of three explainability methods; SHAP, LIME and Integrated Gradients; to enhance interpretability for Sentiment Analysis and Named Entity Recognition on Arabic–English code-switched data. To the best of our knowledge, this is the first work to systematically adapt these techniques for structured prediction in this language setting. We propose class-specific filtering for SHAP, a language-aware perturbation strategy for LIME, and a refined token-level IG approach (X-DIGG) for more faithful attributions. To improve accessibility, we present Explainify, an interactive tool that combines token-level visualizations with natural language summaries. A human-grounded user study and quantitative analysis demonstrate that our adaptations significantly improve interpretability across tasks.

Keywords: Natural Language Processing · Explainable AI · Sentiment Analysis · Named Entity Recognition · SHAP · LIME · Integrated Gradients · Code-switching

1 Introduction

Deep neural networks have achieved state-of-the-art performance in many Natural Language Processing (NLP) tasks, but their opaque decision-making processes pose explainability challenges. This has driven research in Explainable Artificial Intelligence (XAI) for NLP, leading to the development of techniques to interpret model predictions. Local Interpretable Model-Agnostic Explanations (LIME) and SHapley Additive exPlanations (SHAP) typically estimate feature importance by perturbing inputs and observing changes in output [11]. Gradient-based methods such as Integrated Gradients (IG) attribute importance by accumulating a model's gradients along an input interpolation [22]. These methods have proven effective for text classification, where predictions often depend on individual keywords. In contrast, tasks like Named Entity Recognition (NER) and Sentiment Analysis (SA) require the model to consider interactions between multiple tokens and contextual cues. Conventional XAI techniques often fall short in

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 116–127, 2026.
https://doi.org/10.1007/978-3-031-97144-0_11

such cases; for example, presenting a ranked list of important words fails to capture how word order or context shaped the model's decision [11]. Existing explanation tools are not well equipped to handle inputs or outputs with rich internal structure.

Code-switched text, where two or more languages are intermixed, poses challenges for NLP models due to abrupt language shifts and mixed vocabularies that degrade performance [7]. From an explainability standpoint, such inputs require capturing language transitions and cross-lingual context; factors most explanation methods overlook [24]. In this work, we focus on Arabic–English code-switched data as a case study, a language pair with distinct explainability challenges. To the best of our knowledge, this is the first work to systematically adapt SHAP, LIME and IG for structured tasks like NER and SA on this language pair. LIME's random perturbations may replace an Arabic word with an unrelated English token (or vice versa), producing nonsensical variants and unreliable importance estimates. SHAP and IG similarly treat features independently, missing the multilingual structure and often misattributing importance. These methods assume a monolingual, homogeneous feature space; when applied to bilingual inputs like ours, such assumptions break down and lead to incomplete or misleading explanations. This underscores the need for XAI methods tailored to structured, multilingual data.

In response, we adapt and extend SHAP, LIME and IG for structured and code-switched NLP tasks, focusing on NER and SA. First, we propose an adaptation of SHAP for sequence labelling, which computes token-level attributions while accounting for context within the sequence. This sequence-aware SHAP method ensures that explanations reflect how neighbouring words (even in another language) influence the prediction. Next, we modify LIME to be language-aware for code-switched text: instead of perturbing words in isolation, our approach generates multilingual perturbations that preserve grammatical consistency and maintain the correct language at each position. We also extend IG to structured outputs in a multilingual setting by developing Extended Discretized Integrated Greedy Gradients (X-DIGG), a multi-output variant tailored for SA and token-based models that helps users interpret opaque models more intuitively. It employs principal component analysis (PCA) to project an interactive 3D scatter plot that is accessible, easy to use and colour blind friendly.

To support these adaptations, we developed Explainify, a lightweight explanation tool that wraps our modified SHAP and LIME methods to deliver intuitive visual and natural language explanations for NLP tasks like NER and SA on Arabic–English code-switched text, using Llama 3.2 to enhance clarity and accessibility for users.

For evaluation, we conduct a human-centered study using surveys to assess the clarity and usefulness of the explanations generated by LIME and SHAP within our Explainify tool. Feedback was collected from both technical and non-technical users to ensure broad interpretability. In contrast, the X-DIGG technique is evaluated using quantitative metrics, including comprehensiveness, sufficiency and log-odds, to objectively measure explanation quality. The paper is structured as follows: Sect. 2 reviews related work; Sect. 3 presents our adapted XAI techniques, including X-DIGG; Sect. 4 describes the Explainify tool; Sect. 5 outlines the experimental setup; Sect. 6 discusses the results; and Sect. 7 concludes with future directions.

2 Related Works

While explainability has been explored in SA and NER, most prior work focuses on monolingual or unstructured settings. Limited research has addressed structured outputs or multilingual inputs; particularly code-switched data; across both tasks. In SA, early explainability tools like LIME [15] approximated local behaviour but lacked scalability. SHAP [13] improved attribution fairness using Shapley values, while IG [22] introduced gradient-based methods. However, both face limitations in computational cost and baseline dependence, respectively. For Arabic SA, AraBERT [2] improved contextual understanding but lacked interpretability. Later efforts combined Naïve Bayes with CNN-LSTM [21], hybrid models [8] and rule-based methods [4]. SHAP-based visualizations were recently used to enhance transparency in Arabic sentiment prediction [1].

BERT-WMAL [14] combined lexicon-based and contextual embeddings to enhance interpretability in multilingual sentiment settings. Multimodal frameworks incorporating text, audio and visual inputs have also been proposed for more holistic sentiment understanding [23]. NER introduces additional challenges due to its span-based structure. Token-level attribution in LIME and SHAP often breaks span coherence, and both face scalability issues. Alternatives include amnesic probing [5], trigger-based justifications [12], influence functions [3] and attention-guided gradient methods [9], though span-aware explanations remain underexplored.

Multilingual and code-switched NER also lack interpretability. Models like mBERT and XLM-RoBERTa struggle with mixed-language boundary detection. While SpanBERT [10] and LUKE offer improved span representations, they remain opaque without dedicated XAI adaptations. Despite these efforts, no existing XAI framework supports both SA and NER in code-switched settings. IG, in particular, remains underused.

3 Explainable AI Techniques

XAI techniques help interpret the inner workings of machine learning models, especially deep learning in NLP. This section presents the three methods adapted in this paper: SHAP, LIME and X-DIGG, a variant of IG. We describe their application to SA and NER, with X-DIGG used specifically for SA. SHAP offers consistent, theory-based attributions for both local and global insights. LIME's model-agnostic design enables interpretable local surrogate models, useful for analyzing individual predictions. X-DIGG enhances gradient-based attribution to reveal deeper insights into complex model behaviour.

3.1 SHapley Additive exPlanations

SHAP is a game-theoretic interpretability method that assigns importance scores to features based on their marginal contribution to a model's prediction [13,18]. It computes Shapley values by perturbing input features and measuring their impact on the output, capturing both local and contextual effects. SHAP is model-agnostic and has

been widely adopted for many-to-one tasks such as SA, where the output is a single label.

In our SA setting, SHAP was used without modification. A custom wrapper linked the model to SHAP's kernel explainer, enabling local visualizations through heatmaps, bar charts and force plots. For global insights, we perturbed a representative dataset and aggregated token contributions to identify high-impact words per sentiment class.

However, standard SHAP does not support many-to-many tasks like NER, where each token has an associated output label. To address this, we introduced structural modifications to adapt SHAP for sequence labeling. Specifically, we implemented a class-index filtering mechanism that enabled class-specific explanations (e.g., I-PERS, I-LOC) by returning per-class outputs through a custom prediction function. SHAP was then applied iteratively across tokens and classes to generate span-aware explanations.

Local explanations used heatmaps to show each token's influence on predicted labels, revealing inter-token dependencies. SHAP's bar and text plots were adapted for token-level attribution. For global analysis, we preserved stop words to maintain contextual integrity, visualizing per-class and cumulative feature importances. These extensions enable token-level interpretability for structured NLP tasks.

3.2 Local Interpretable Model-Agnostic Explanations

LIME is a model-agnostic interpretability method that explains individual predictions by fitting a simple surrogate model around the input instance [15, 18]. It perturbs inputs by masking or removing tokens and assigns importance scores based on how these changes affect the model's output. LIME is well suited to many-to-one tasks such as SA, where it identifies influential words for single-label predictions. In our SA setup, we used LIME without modification. A custom prediction function returned class probabilities, and input sentences were perturbed by masking tokens. LIME's visualization tools then highlighted the words most responsible for each sentiment classification.

To adapt LIME for many-to-many tasks like NER, we implemented a custom prediction function that accepts a target token index and returns its predicted label. During each iteration, LIME perturbed the input while evaluating the output for the selected token, enabling token-level attributions within their full sentence context.

This process was applied across all tokens, producing visual explanations that captured how surrounding words influenced each prediction. In contrast to SHAP's class-wise filtering, LIME's adaptation focused on token-wise analysis while retaining its surrogate modeling approach, demonstrating its flexibility for sequence-level interpretability.

3.3 Extended Discretized Integrated Greedy Gradients

Integrated Gradients computes feature importance by accumulating gradients along a path from a baseline to the actual input [22]. As computing the exact integral is inefficient, it is typically approximated, as shown in Eq. 1.

$$IG_i(x) \approx (x_i - x_i^0) \cdot \sum_{k=1}^{m} \frac{\partial f\left(x^0 + \frac{k}{m}(x - x^0)\right)}{\partial x_i} \cdot \frac{1}{m} \tag{1}$$

where m is the number of interpolation steps, x^0 is the baseline, x is the actual input and $\frac{k}{m}$ represents discrete values of the scaling factor α at each step. However, selecting a suitable baseline remains a challenge, as it affects explanation quality. While initially developed for vision tasks, IG has been adapted for NLP. Discretized Integrated Gradients (DIG) [19] addresses the discrete nature of text by guiding integration through valid word embeddings, avoiding unnatural transitions and improving interpretability in SA.

Standard IG often produces invalid token interpolations that lack semantic coherence. To overcome this, X-DIGG introduces a path-integrated approach through contextualized, sentence-specific embeddings. It builds an embedding dictionary and filters variants using cosine similarity (threshold $< 85\%$), ensuring smoother attribution paths. For misclassifications, X-DIGG highlights how specific phrases influence predictions, helping detect attribution outliers or ambiguous embeddings. It employs 3D PCA scatter plots and K-means++ clustering to visualize semantic clusters and identify attribution hotspots. X-DIGG is model-agnostic, supports token-based models in multilingual and code-switched settings, and integrates LLM-driven token replacement. Using Llama 3.2 [6], it generates alternative tokens and recomputes predictions to refine interpretation.

4 Explainify

Explainify is a proof-of-concept interactive tool designed to make model explanations accessible to both technical and non-technical users. Given an input sentence, the tool routes it through either a NER or SA model, generates predictions and returns explanations using adapted SHAP and LIME techniques. Each prediction is accompanied by interactive visualizations; such as token-level heatmaps, bar charts and natural language summaries generated via Llama 3.2 to enhance clarity and interpretability. Explainify integrates multiple XAI methods into a unified, simple interface. In SA, explanations highlight the words most influential to the predicted sentiment. In NER, each token is explained in relation to its entity label, emphasizing the influence of the surrounding context. All components, predictions, explanations and summaries are delivered in real time through an intuitive interface that allows users to input a sentence, observe predictions and immediately understand the reasoning behind them.

5 Experimental Setup

This section outlines the setup for evaluating the adapted explainability techniques, including tasks, datasets, model architectures and evaluation metrics. The BiLSTM (with attention for X-DIGG) model from [20] was used for its compatibility with token-level attribution methods and ease of integration with explainability tools.

5.1 Test Cases and Datasets

The explainability techniques were evaluated on SA and NER tasks on Arabic-English code-switched datasets. SA and NER test sets were sourced from [20] and [16,17],

respectively. For Explainify's SA task, two test cases were used: one with a correct prediction and another with a misclassification. Similarly, NER was evaluated using two cases; one showing correct entity classification and one illustrating misclassification. Each test case included the input text, predicted label and the explanation generated by the interpretability method. Natural explainability was also assessed using correct and incorrect cases to evaluate the alignment between explanation and prediction.

X-DIGG evaluation focused on misclassified samples to address model limitations. Samples were chosen based on expressive language, the presence of emojis, a minimum of three tokens and subjectivity of annotations. Hyperparameters were tuned by testing different values for interpolation steps (m), neighbourhood size (K) and masking threshold ($top_k\%$). The tested values for interpolation steps m were {1, 5, 12, 30, 50}. For neighbourhood size, K values included {25, 50, 100, 250, 350, 500}. The masking threshold was explored from 2% up to 100% of the vocabulary.

The results indicated that interpolation steps had the most significant impact. Setting m to 30 provided a good balance between attribution quality and computational efficiency. Although setting m to 50 produced slightly more stable results, it caused a 60% increase in execution time, which was impractical for real-world use. As reported in [19], changes in neighbourhood size did not notably affect performance. The best masking threshold was found at 20% of the total vocabulary. This choice maintained informative feature selection while avoiding excessive noise.

Based on these insights, we used the configuration with m set to 30, K set to 500 and the masking threshold set to 20% in subsequent experiments. This setup allowed for a detailed analysis of explanations on misclassified samples, enabling comparison between the model's reasoning for predicted classes and potential alternatives.

5.2 Evaluation Metrics

The evaluation of the XAI techniques combined human-grounded and functionally grounded approaches. SHAP, LIME and Explainify's natural language explanations were assessed through structured surveys with both technical and non-technical participants. This evaluation focused on three aspects: clarity (ease of understanding), usefulness (support in interpreting decisions) and fidelity (alignment with the model's reasoning).

In contrast, X-DIGG was evaluated using grounded metrics. Comprehensiveness measured the confidence drop when key tokens were masked, indicating their importance. Sufficiency assessed whether a minimal token subset could preserve the prediction and log-odds quantified confidence shifts when influential tokens were removed. Stronger explanations are indicated by higher comprehensiveness and lower sufficiency and log-odds scores. Global SHAP outputs were not quantitatively assessed due to the lack of standardized NLP metrics, leaving global interpretability as an open challenge.

6 Results and Analysis

This section presents results and analysis of SHAP, LIME and X-DIGG for SA, as well as SHAP and LIME for NER. We include example explanations from X-DIGG and Explainify, along with quantitative metrics and human evaluation scores.

6.1 X-DIGG

We tested X-DIGG on multiple samples and 3 were handpicked from the mispredicted test set to show off some of the possible use cases of X-DIGG. These samples were chosen according to some criteria; such as presence of emojis, abundance of expression, having 3 or more tokens and subjectivity of the original annotation.

Sample 1 (Poor Generalization): Figure 1 shows a misprediction of "Neutral" instead of "Negative" for the sample "dislikes عشان علي غزلان" ("dislikes for Ali Ghezlan"). The 3D graph below visualizes token attributions. While the most important token was correctly identified, the model misunderstood the context. Replacing "dislikes" with "hates" via the LLM led to the correct prediction, indicating that the model failed to generalize to unseen data due to limited training exposure.

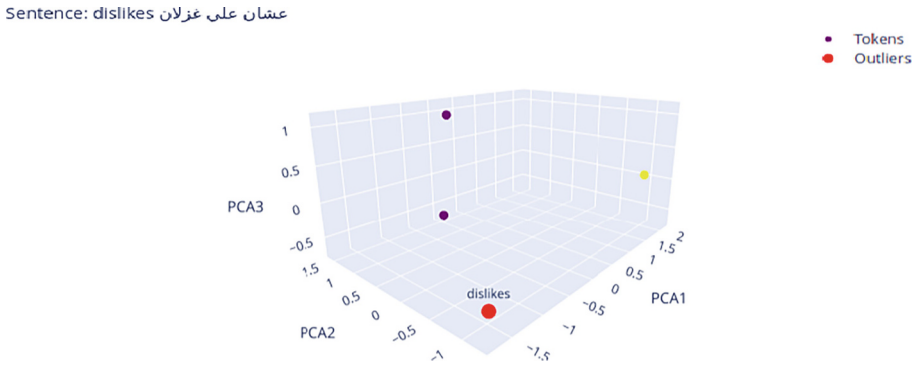


Fig. 1. Sample 1: "Neutral" instead of "Negative" (Non-red dots indicate a cluster)

Sample 2 (Emojis and Annotation Subjectivity): In sample 2, a degree of human error can be observed in the annotation process. The sample "free guy & old بليز مراجعة" ("Please review Free Guy & Old" followed by a heart-eyes emoji and a flower emoji) was annotated as "Neutral" instead of "Positive". The model was able to identify both emojis that, to some subjective extent, indicate a positive sentiment. While this is technically a request and not a review, it is a friendly request and the emojis are a clear indicator of a positive sentiment. Moreover, it showed that the model can identify requests from strict orders, which can possess a different sentiment.

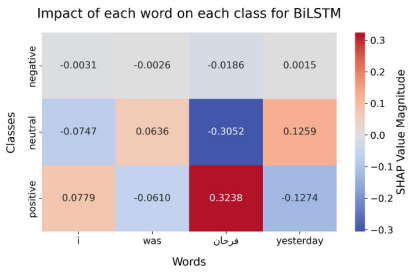
Sample 3 (Sarcasm and Figurative Language): The final sample is a showcase of the model's inability to understand figurative language. The sample "هو الراجل ده vampire ولا ايه ده بيصغر في السن" ("Is this man a vampire or what? He's getting younger in age") was predicted as "Negative" instead of "Positive" and showcases the model's shortcomings in terms of sarcastic and figurative language. In this run of X-DIGG, the model was confused and unable to identify what was wrong with the sentence, resorting

to a negative sentiment when age was mentioned. This shows that the model was unable to identify the sarcasm in the sentence and the figurative language used.

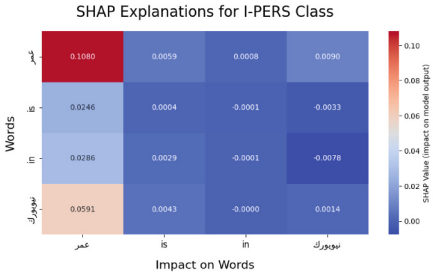
For evaluation metrics, Comprehensiveness $= f(x) - f(x_{\text{masked}})$ shows removed features' contribution (positive if prediction drops). On the other hand, Sufficiency $= f(x) - f(x_{\text{top-}k})$ indicates whether top-k features suffice to maintain predictions. Lastly, Log-Odds $= \log\left(\frac{f(x_{\text{masked}})}{1-f(x_{\text{masked}})+\epsilon}\right) - \log\left(\frac{f(x)}{1-f(x)+\epsilon}\right)$ quantifies each feature's directional influence (ϵ stabilizes division). Higher comprehensiveness scores are better while lower sufficiency and log-odds scores are better. Our model scored a comprehensiveness score of 0.813, a sufficiency score of -0.187 and a log-odds score of -16.749 (average of 80 mispredicted samples).

6.2 Explainify

To further analyze the explainability techniques evaluated in this work, we present token-level visualizations generated by LIME and SHAP for both NER and SA. These visualizations highlight how individual tokens contribute to the model's final decision, offering deeper insights into the interpretability of structured text tasks.



(a) SHAP Heatmap visual explanations for SA sentence "I was فرحان yesterday" ("I was happy yesterday").



(b) SHAP Heatmap visual explanations for NER sentence "عمر is in نيويورك" ("Omar is in New York").

Fig. 2. SHAP Heatmap explanations for different NLP task types.

Figure 2 demonstrates SHAP-based token-level explanations for two structurally different NLP tasks. In Fig. 2a, the model performs a many-to-one task by assigning a single sentiment label to the entire sentence. The token "فرحان" ("happy") contributes most significantly to the prediction, while other tokens such as "i", "was" and "yesterday" have comparatively lower impact, highlighting the emotional weight of the word "فرحان" ("happy") in driving sentiment classification.

In Fig. 2b, SHAP values are shown for a many-to-many task, focusing on the I-PERS class in a Named Entity Recognition setting. The token "عمر" ("Omar") plays the biggest role in predicting a person entity. Also, nearby words like "is" and "نيويورك" ("New York") also contribute positively, though with smaller values. This means that the model considers the surrounding context when making predictions, not just the target

word. These visualizations highlight how SHAP helps explain how the model uses both key tokens and context, making its decision process easier to understand.

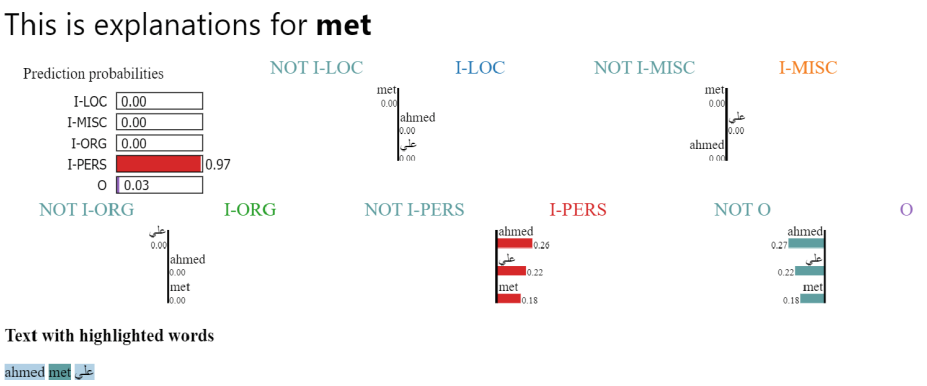


Fig. 3. LIME visual explanation for NER showing a model misclassification for the token "met" in the sentence "Ahmed met على"("Ahmed met Ali").

Figure 3 presents a LIME-based explanation for a NER task, focusing on the token "met". Although "met" is a verb and not an entity, the model incorrectly predicts it as a person (I-PERS) with high confidence. The LIME explanation shows that this error is strongly influenced by its surrounding tokens, "Ahmed" and "على" ("Ali"), both correctly identified as person entities. This example demonstrates how contextual bias can lead to misclassification, especially in patterns such as "PERSON verb PERSON." Such insights emphasize the value of interpretability methods like LIME in uncovering hidden model behaviour and support the case for improving training data diversity to handle such constructs more reliably.

These visualizations showcase the differences and complementarities between LIME and SHAP in their approach to token attribution. For NER, LIME provides fine-grained, token-specific attributions, highlighting how individual tokens influence their predicted entities. SHAP, on the other hand, offers class-specific heatmaps that illustrate each token's contribution to all possible entity classes, capturing the multi-label nature of NER. For SA, LIME delivers localized, instance-level feature importance, while SHAP reveals deeper token interactions and their collective effect on sentiment prediction. Together, these methods offer a comprehensive view of model reasoning,

6.3 User Study Results

A user study was conducted with 30 participants: 10 with technical backgrounds and 20 with non-technical backgrounds. Technical participants evaluated SHAP, LIME and the natural language explanations generated by Explainify on both correct and incorrect predictions. Non-technical participants assessed only Explainify's outputs. Although

SHAP and LIME were not shown to non-technical users, the results demonstrate Explainify's effectiveness in communicating model behaviour to broader audiences.

As shown in Table 1, Explainify achieved the highest clarity ratings, particularly for incorrect predictions, and was rated comparably to SHAP in usefulness and fidelity. Here, the standard deviation (SD) indicates how consistent participants' ratings were. Lower SD values represent more agreement among users. While SHAP was preferred for analyzing individual predictions (Table 2), Explainify was the top choice for overall interpretability. Non-technical users also rated Explainify highly in clarity and usefulness (Table 3), underscoring its accessibility across user groups.

Table 1. Evaluation Summary – Technical Participants

Method	Clarity (±SD)		Usefulness		Fidelity	
	Corr.	Inc.	Corr.	Inc.	Corr.	Inc.
Explainify	4.4 ± 0.84	3.9 ± 0.57	90%	75%	85%	70%
SHAP	4.2 ± 1.32	3.7 ± 1.06	90%	75%	85%	70%
LIME	4.0 ± 0.67	3.2 ± 1.03	–	–	–	–

Table 2. Preferred Method – Technical Participants

Preference	Percentage
SHAP (Correct Predictions)	70%
SHAP (Incorrect Predictions)	80%
Explainify (Overall Interpret.)	40%
SHAP (Overall Interpret.)	30%
LIME (Overall Interpret.)	10%
Context-dependent Preference	20%

Table 3. Evaluation – Non-Technical Participants (Explainify Only)

Metric	Correct	Incorrect
Clarity (±SD)	4.40 ± 0.68	3.90 ± 0.57
Usefulness	90%	75%

7 Conclusion and Future Work

We proposed adapted versions of SHAP, LIME and X-DIGG to improve explainability in SA and NER for Arabic–English code-switched data. SHAP produced structured attributions but was computationally intensive, while LIME offered faster results with less stability in sequence-based contexts. X-DIGG performed well for SA but requires further refinement for many-to-many tasks and real-world deployment. To enhance accessibility, we developed Explainify, a tool that combines visual and natural language explanations. However, Explainify currently supports only SA and NER, while global interpretability remains an open challenge. Future work will focus on optimizing SHAP and IG for real-time applications, extending IG to NER using attention-guided mechanisms, improving LIME's robustness for sequence tasks, expanding Explainify's coverage to additional NLP tasks and integrating transformer-based models to enhance performance.

References

1. Abdelwahab, Y., Kholief, M., Sedky, A.: Justifying Arabic text sentiment analysis using explainable AI (XAI): Lasik surgeries case study. Information **13**(11), 536 (2022)
2. Antoun, W., Baly, F., Hajj, H.: Arabert: transformer-based model for Arabic language understanding. arXiv preprint arXiv:2003.00104 (2020)
3. Athira, A., Kumar, S.M., Chacko, A.M.: A systematic survey on explainable AI applied to fake news detection. Eng. Appl. Artif. Intell. **122**, 106087 (2023)
4. Diwali, A., Dashtipour, K., Saeedi, K., Gogate, M., Cambria, E., Hussain, A.: Arabic sentiment analysis using dependency-based rules and deep neural networks. Appl. Soft Comput. **127**, 109377 (2022)
5. Elazar, Y., Ravfogel, S., Jacovi, A., Goldberg, Y.: Amnesic probing: behavioral explanation with amnesic counterfactuals. Trans. Assoc. Comput. Linguist. **9**, 160–175 (2021)
6. Grattafiori, A., et al.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024)
7. Hamed, I., Sabty, C., Abdennadher, S., Vu, N.T., Solorio, T., Habash, N.: A survey of code-switched Arabic NLP: progress, challenges, and future directions. In: Proceedings of the 31st International Conference on Computational Linguistics, pp. 4561–4585 (2025)
8. Hicham, N., Karim, S., Habbat, N.: Customer sentiment analysis for Arabic social media using a novel ensemble machine learning approach. Int. J. Electr. Comput. Eng. **13**(4), 4504 (2023)
9. Huang, Z., Xu, W., Yu, K.: Bidirectional LSTM-CRF models for sequence tagging. arXiv preprint arXiv:1508.01991 (2015)
10. Joshi, M., Chen, D., Liu, Y., Weld, D.S., Zettlemoyer, L., Levy, O.: Spanbert: improving pre-training by representing and predicting spans. Trans. Assoc. Comput. Linguist. **8**, 64–77 (2020)
11. Kokalj, E., Škrlj, B., Lavrač, N., Pollak, S., Robnik-Šikonja, M.: Bert meets shapley: extending shap explanations to transformer-based classifiers. In: Proceedings of the EACL Hackashop on News Media Content Analysis and Automated Report Generation, pp. 16–21 (2021)
12. Lin, B.Y., et al.: Triggerner: learning with entity triggers as explanations for named entity recognition. arXiv preprint arXiv:2004.07493 (2020)
13. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 4765–4774 (2017)
14. Polignano, M., Basile, V., Basile, P., Gabrieli, G., Vassallo, M., Bosco, C.: A hybrid lexicon-based and neural approach for explainable polarity detection. Inf. Process. Manag. **59**(5), 103058 (2022)
15. Ribeiro, M.T., Singh, S., Guestrin, C.: Why should i trust you? Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1135–1144 (2016)
16. Sabty, C., Elmahdy, M., Abdennadher, S.: Named entity recognition on Arabic-English code-mixed data. In: 2019 IEEE 13th International Conference on Semantic Computing (ICSC), pp. 93–97. IEEE (2019)
17. Sabty, C., Sherif, A., Elmahdy, M., Abdennadher, S.: Techniques for named entity recognition on Arabic-English code-mixed data. Int. J. Transdiscip. AI **1**(1), 44–63 (2019)
18. Salih, A.M., et al.: A perspective on explainable artificial intelligence methods: shap and lime. Adv. Intell. Syst. 2400304 (2024)
19. Sanyal, S., Ren, X.: Discretized integrated gradients for explaining language models. arXiv preprint arXiv:2108.13654 (2021)

20. Sherif, A., Sabty, C.: Sentiment analysis for Egyptian Arabic-English code-switched data using traditional neural models and advanced language models. In: International Conference on Speech and Computer, pp. 54–69. Springer (2025)
21. Suleiman, D., Odeh, A., Al-Sayyed, R.: Arabic sentiment analysis using Naïve Bayes and CNN-LSTM. Informatica **46**(6) (2022)
22. Sundararajan, M., Taly, A., Yan, Q.: Axiomatic attribution for deep networks. In: International Conference on Machine Learning, pp. 3319–3328. PMLR (2017)
23. Wu, S., Wang, X., Wang, L., He, D., Dang, J.: Enriching multimodal sentiment analysis through textual emotional descriptions of visual-audio content. arXiv preprint arXiv:2412.10460 (2024)
24. Xie, K., Babbar, N., Chen, V., Turura, Y.: Enhancing multilingual language models for code-switched input data. arXiv preprint arXiv:2503.07990 (2025)

Utilization of SLMs as Generator in RAG

Kouya Abe(✉) and Hiroyuki Shinnou

Department of Computer and Information Sciences, Ibaraki University, Hitachi, Japan
{24nm701t,hiroyuki.shinnou.0828}@vc.ibaraki.ac.jp

Abstract. Retrieval-Augmented Generation (RAG), a method for integrating external knowledge into large language models (LLMs), is typically composed of a retriever, which searches for relevant information, and a generator, which produces responses based on the retrieved content. This study aims to examine whether RAG can function effectively when using a small language model (SLM) as the generator, assuming the retriever has sufficiently high performance. In our experiments, we conducted a question-answering (QA) task using the JQaRA dataset under an idealized condition where the retriever was assumed to always retrieve the correct document. The results indicate that even when the retriever demonstrates high performance, using a more capable model as the generator leads to an overall improvement in response accuracy. However, a certain number of test cases were observed where a lower-performing SLM produced the correct answer, while a higher-performing SLM failed. To address this, we investigated the potential of ensemble RAG, which combines multiple SLMs. The results suggest that incorporating multiple SLMs as an ensemble within the generator mechanism could significantly improve overall accuracy.

Keywords: RAG · Generator · SLM · QA task · Ensemble RAG

1 Introduction

Large language models (LLMs) have achieved impressive performance in natural language processing (NLP), but they often struggle with out-of-distribution knowledge, including domain-specific or recent information. This limitation frequently results in hallucinations - plausible but factually inaccurate outputs [5]. To address this issue, various approaches have been proposed to incorporate external knowledge into LLMs [1,3,4].

One prominent framework is *Retrieval-Augmented Generation (RAG)* [7], which augments input text with retrieved documents to improve factual accuracy. RAG typically consists of a retriever, responsible for retrieving relevant documents via vector similarity, and a generator, which produces responses conditioned on both the query and retrieved content. High retrieval accuracy is crucial to ensure that relevant information is effectively obtained. As the retriever is a core component of RAG, extensive research has been conducted to enhance

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 128–139, 2026.
https://doi.org/10.1007/978-3-031-97144-0_12

its performance [2]. The generator is responsible for producing text based on the documents retrieved by the retriever; in other words, it generates the final response.

In this study, we investigate the impact of model selection on performance and generated outputs by incorporating a small language model (SLM) as the generator and conducting a question-answering (QA) task. For our experiments, we assume an idealized scenario in which the retriever operates flawlessly, always retrieving a single relevant document that corresponds to the input text. The primary objective of this study is to evaluate the feasibility of using an SLM as the generator in RAG when the retriever achieves sufficiently high performance.

2 Related Work

In recent years, the use of SLMs has gained increasing attention due to constraints on computational resources and response latency [14]. The work by Pierre Lepagnol et al. [6] investigates the effectiveness of SLMs in zero-shot classification tasks, demonstrating that smaller models can achieve comparable or even superior performance to larger models on specific tasks. Similarly, Guillermo Marco et al. [9] report that a fine-tuned version of BART-large, configured as an SLM, outperformed the average human writer in short creative writing tasks. However, these studies focus on standalone performance evaluations of SLMs without incorporating retrieval modules, and do not consider the performance of SLMs within a RAG framework.

It is also well known that ensembling LLMs can lead to performance improvements [8], and recent studies have extended this idea to SLMs as well. For instance, Ensemble Bayesian Inference by Haru-Tada Sato et al. [12] demonstrates that by aggregating the outputs of multiple SLMs using Bayesian inference, it is possible to match the performance of LLMs, even when the ensemble includes weaker models. Their approach showed effectiveness in bilingual aptitude evaluation tasks (Japanese-English). Similarly, Ensemble SuperICL by Lingyi Yang et al. [11] integrates predictions from multiple SLMs into the in-context learning process of an LLM. Their method achieved state-of-the-art results across four tasks, including medical domain applications, with performance gains ranging from 3% to 20% over previous approaches.

In this study, we investigate the ensemble use of SLMs as generators in a RAG setup, aiming to leverage their efficiency and complementary strengths for improved generation quality.

3 Experimental Dataset

For this experiment, we utilized the JQaRA (Japanese Question Answering with Retrieval Augmentation) dataset[1], which was publicly released by Yuichi Tateno from hotchpotch.

[1] https://huggingface.co/datasets/hotchpotch/JQaRA.

3.1 JQaRA Dataset

The JQaRA dataset consists of 100 candidate documents per question, serving as the retrieval target. A simplified overview of the dataset structure is illustrated in Fig. 1, and an example of its contents is provided in Appendix 1. The questions and correct answers in JQaRA originate from JAQKET: A Japanese QA Dataset Based on Quiz Questions[2]. JAQKET is a high-quality, diverse Japanese QA dataset, notably distinguished by the fact that its answers correspond to Wikipedia article titles.

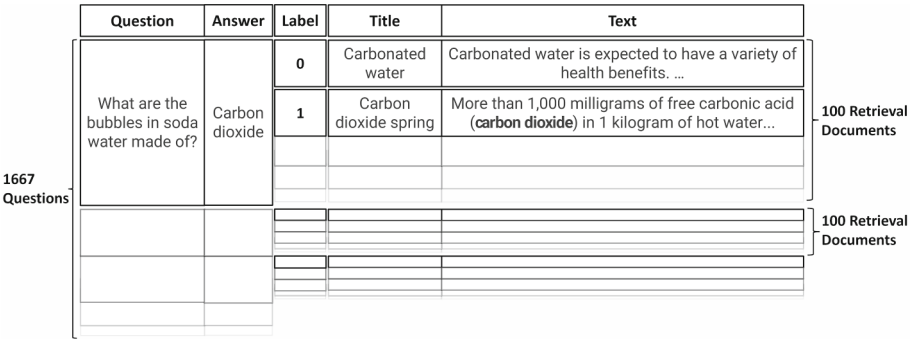


Fig. 1. This figure provides an overview of the test cases in the JQaRA dataset. While the content is translated into English here, the actual data is entirely in Japanese. In the second entry from the top, the label is assigned as "1" because the "Text" field contains the same noun as the "Answer" field.

Retrieval Document Acquisition Method. The retrieval documents in JQaRA dataset were obtained from singletongue/wikipedia-utils - passages-c400-jawiki-20230403[3], which segmented Japanese Wikipedia articles into chunks of up to 400 characters. To retrieve relevant documents for each question, sentence vector similarity was computed using embeddings models. To mitigate biases associated with using a single embeddings model and enhance retrieval diversity, the dataset creator employed five different embeddings models. For each query, the top 500 most similar documents were retrieved separately using each of the five embeddings models. The retrieved results were then re-ranked using Reciprocal Rank Fusion (RRF), and the top 100 documents with the highest scores were selected as candidate documents. Each retrieved document was linked to the title of the Wikipedia article from which it was extracted, enabling each query to be associated with both the corresponding article titles and relevant retrieved passages.

[2] https://www.nlp.ecei.tohoku.ac.jp/projects/jaqket/.
[3] https://huggingface.co/datasets/singletongue/wikipedia-utils.

Ground-Truth Label Assignment Method. Among the 100 documents associated with each query, a ground-truth label was assigned if the body text or title of a document contained a partial match of the correct answer string. In such cases, the document was considered relevant to the query.

4 Experimental Method

In this experiment, we utilized a subset of the JQaRA dataset, which consists of 1,667 questions, each associated with 100 retrieved documents, resulting in a total of 166,700 test samples. Among these, we conducted experiments using 16,204 samples that were assigned ground-truth labels.

4.1 LLMs Used in the Experiment

As the SLMs incorporated into the generator, we employed eight models, as listed in Table 1.

Table 1. The table provides the model names and the number of parameters. Throughout the remainder of this paper, we refer to each model using the abbreviations defined in the table.

Model Name	Abbreviation	Params
line-corporation/japanese-large-lm-1.7b-instruction-sft[a]	Line-1.7B	1.7B
google/gemma-2-2b-jpn-it[b]	Gemma-2B	2B
line-corporation/japanese-large-lm-3.6b-instruction-sft[c]	Line-3.6B	3.6B
rinna/japanese-gpt-neox-3.6b-instruction-ppo[d]	Rinna-3.6B	3.6B
llm-jp/llm-jp-3-3.7b-instruct[e]	LLM-jp-3.7B	3.7B
microsoft/Phi-3-mini-128k-instruct[f]	Phi-3-mini	3.8B
tokyotech-llm/Swallow-7b-instruct-hf[g]	Swllow-7B	7B
elyza/Llama-3-ELYZA-JP-8B[h]	Elyza-8B	8B

[a] https://huggingface.co/line-corporation/japanese-large-lm-1.7b-instruction-sft
[b] https://huggingface.co/google/gemma-2-2b-jpn-it
[c] https://huggingface.co/line-corporation/japanese-large-lm-3.6b-instruction-sft
[d] https://huggingface.co/rinna/japanese-gpt-neox-3.6b-instruction-ppo
[e] https://huggingface.co/llm-jp/llm-jp-3-3.7b-instruct
[f] https://huggingface.co/microsoft/Phi-3-mini-128k-instruct
[g] https://huggingface.co/tokyotech-llm/Swallow-7b-instruct-hf
[h] https://huggingface.co/elyza/Llama-3-ELYZA-JP-8B

Additionally, for comparison in terms of accuracy, we included OpenAI's gpt-3.5-turbo-instruct as an additional baseline model.

4.2 Procedure

From the JQaRA dataset described in Sect. 3, we selected 16,204 samples with a ground-truth label of 1 and incorporated them into prompts structured as shown in Table 2. These prompts were then provided as input to the LLM.

Table 2. The left side shows the actual Japanese prompt used, while the right side presents its English translation.

以下のコンテキストを使用して質問に回答してください。	Please answer the question using the following context.
———-	———-
コンテキスト: {title}: {context}	Context: {title}: {context}
———-	———-
質問: {question}	Question: {question}
回答:	Answer:

Specifically, the text, title, and question fields from the JQaRA dataset were mapped to the context, title, and question fields in the prompt, respectively. By structuring the prompt in this manner, each query and its corresponding retrieved context were treated as a single input instance for the LLM. This setup assumes an ideal retriever, meaning that it always retrieves exactly one relevant document for each query. For implementation, we utilized LangChain as the framework. Additionally, the temperature parameter for all language models was set to 0 to ensure deterministic outputs. The evaluation metric was based on whether the correct answer for the test query was at least partially included in the model-generated response. The experiment was conducted iteratively, replacing the generator with each of the nine language models: the eight SLMs listed in Sect. 4.1 and an additional LLM for comparison. The accuracy of each model on the test dataset was then analyzed.

5 Experimental Results

We conducted the experiment using the methodology described in Sect. 4, and the results are summarized in Table 3.

As shown in Table 3, incorporating the correct context information into the prompt using the RAG framework improves QA task accuracy.

The accuracy is strongly influenced by the number of model parameters, exhibiting a general trend where larger models achieve higher accuracy. However, the extent of accuracy improvement does not appear to be proportional to the number of parameters. We hypothesize that models with lower baseline (i.e., without RAG) accuracy tend to benefit more from RAG integration. This suggests a trade-off between a language model's intrinsic knowledge and its adaptability to externally provided context.

Table 3. Indicates the accuracy of each model on the test dataset.

Model Name	w/o RAG	w/RAG	Differences
Line-1.7B	0.205	0.578	+0.373
Gemma-2B	0.062	0.625	+0.563
Line-3.6B	0.376	0.683	+0.307
Rinna-3.6B	0.408	0.697	+0.289
LLM-jp-3.7B	**0.558**	0.809	+0.251
Phi-3-mini	0.018	0.689	**+0.671**
Swallow-7B	0.580	**0.854**	+0.274
ELYZA-8B	0.425	0.850	+0.425
gpt-3.5-turbo-inst.	0.484	0.844	+0.360

Additionally, we included gpt-3.5-turbo-instruct as a baseline for comparison with the SLMs. Notably, models in the 7B parameter class outperformed gpt-3.5-turbo-instruct.

6 Considerations on Ensemble Models

6.1 Comparison of Each SLM

We analyzed which models were capable of correctly answering each test sample. For this analysis, we selected pairs of SLMs (excluding gpt-3.5-turbo-instruct) and categorized the test samples into three groups:

- Both models provided the correct answer
- Only one model provided the correct answer
- Both models failed to provide the correct answer

This process was conducted for all 28 possible model pairings, selecting two models from the eight SLMs used in the experiment. The results for Line-1.7B is presented in Tables 4. For additional results, refer to Appendix B.

These results indicate that, despite Line-1.7B being able to produce correct answers, more than 5% of the total 16,204 test samples could not be correctly answered by models with a larger number of parameters. Furthermore, the analysis revealed that for all model pairings, a non-negligible number of test samples existed where only one of the two models generated the correct answer. This suggests that each model possesses specialized strengths that cannot be determined solely based on the number of parameters.

6.2 Pseudo-Construction of Ensemble RAG

To investigate whether different models can complement each other's strengths, we propose a pseudo-construction of an Ensemble RAG. In this study, we consider two types of Ensemble RAG configurations:

Table 4. The results of the comparison between Line-1.7B (baseline) and each SLMs (comparison target) in terms of their ability to produce correct answers for each test sample are presented below. BothCor: both models answered correctly; BaseOnly: only the baseline model was correct; CompOnly: only the comparison model was correct; BothInc: both models were incorrect.

Model A	Model B	BothCor	BaseOnly	CompOnly	BothInc
Line-1.7B	Gemma-2B	0.405	0.173	0.219	0.203
Line-1.7B	Line-3.6B	0.461	0.118	0.222	0.200
Line-1.7B	Rinna-3.6B	0.452	0.126	0.245	0.177
Line-1.7B	LLM-jp-3.7B	0.507	0.071	0.303	0.119
Line-1.7B	Phi-3-mini	0.425	0.153	0.265	0.157
Line-1.7B	Swallow-7B	0.523	0.055	0.330	0.091
Line-1.7B	ELYZA-8B	0.518	0.061	0.332	0.089

– An Ensemble RAG using three SLMs
– An Ensemble RAG using four SLMs

For determining correctness, we adopt two different criteria based on the number of models that successfully generate the correct answer. Thus, the four examined Ensemble RAG configurations are as follows:

– Case 1 - Three-SLM Ensemble (at least one correct): If at least one of the three SLMs correctly answers a given test sample, the sample is considered correctly answered.
– Case 2 - Three-SLM Ensemble (at least two correct): If at least two of the three SLMs correctly answer a given test sample, the sample is considered correctly answered.
– Case 3 - Four-SLM Ensemble (at least one correct): If at least one of the four SLMs correctly answers a given test sample, the sample is considered correctly answered.
– Case 4 - Four-SLM Ensemble (at least two correct): If at least two of the four SLMs correctly answer a given test sample, the sample is considered correctly answered.

For each case, we evaluated all possible combinations of the eight SLMs used in the experiment, excluding gpt-3.5-turbo-instruct, and analyzed their accuracy. Specifically, there were 56 possible combinations for Case 1 and Case 2, and 70 possible combinations for Case 3 and Case 4. A box plot summarizing the results is presented in Fig. 2, while Table 5 provides the highest and lowest accuracy values for each case.

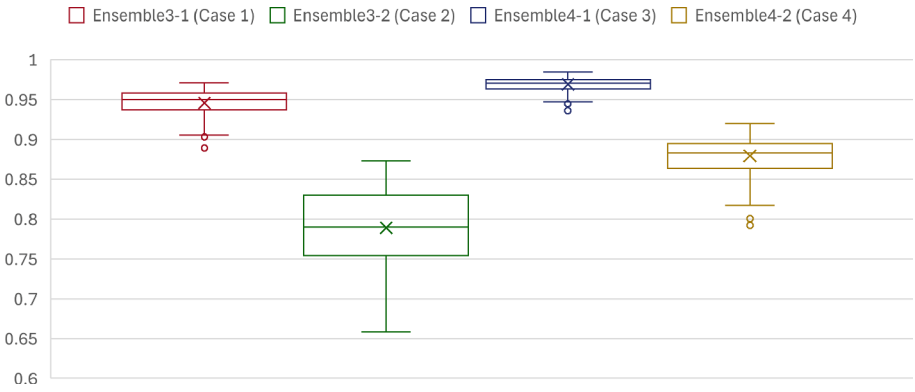


Fig. 2. This figure illustrates the accuracy of the pseudo-ensemble RAG. The vertical axis represents the accuracy, while the four cases, from left to right, correspond to Case 1, Case 2, Case 3, and Case 4.

Table 5. This table presents the highest accuracy (top row) and lowest accuracy (bottom row) for each case, along with the corresponding model combinations.

Case	Accuracy	Combinations
Case 1	0.971	Phi-3-mini, wallow-7B, ELYZA-8B
	0.889	Line-1.7B, Gemma-2B, Rinna-3.6B
Case 2	0.873	LLM-jp-3.7B, Swallow-7B, ELYZA-8B
	0.658	Line-1.7B, Gemma-2B, Rinna-3.6B
Case 3	**0.984**	line-3.6, Phi-3-mini, Swallow-7B, ELYZA-8B
	0.936	Line-1.7B, Gemma-2B, Line-3.6B, Rinna-3.6B
Case 4	0.920	Phi-3-mini, LLM-jp-3, Swallow-7B, ELYZA-8B
	0.792	Line-1.7B, Gemma-2B, Line-3.6B, Rinna-3.6B

These results indicate that even when using SLMs, an ensemble approach combining multiple models can achieve an accuracy rate of approximately 97% to 98%. However, it is important to recognize that the results presented in this study are based on idealized assumptions. Specifically, we assume that the retriever consistently retrieves exactly one relevant document per query, and we do not consider how the final answer should be selected from multiple outputs generated by different SLMs. In practical scenarios, retrieval errors are inevitable and can significantly impact overall system performance. Moreover, the system may fail to select the correct answer from among the candidates, resulting in an incorrect final output. Evaluating the system under realistic retrieval conditions remains a key direction for future research.

Ensemble Methods. One possible approach is to select answers that appear consistently across multiple models. However, imposing stricter conditions—such as requiring at least two models to produce the correct answer, as in Case 2 or Case 4—can lead to a reduction in overall ensemble accuracy. Another potential strategy involves model-based answer evaluation, where each SLM assesses the responses generated by itself and by other models. The responses are then re-ranked using methods such as Reciprocal Rank Fusion (RRF), and the highest-ranked answer is selected as the final output.

Thus, exploring effective methods for extracting the correct answer from multiple model outputs will be a central focus of future work.

7 Conclusion

This study aimed to investigate whether small language models (SLMs) could achieve sufficient performance when the retriever in Retrieval-Augmented Generation (RAG) has a high retrieval capability. In the experiment, a single relevant context, which was expected to lead to the correct answer, was provided in the prompt, and the model was tasked with generating a response. The results demonstrated that even when the retriever's performance is sufficiently high, using a more powerful LLM leads to higher accuracy in Question Answering (QA) tasks. Furthermore, an analysis of each SLM's ability to correctly answer test data suggested that different models have specialized areas of strength. To explore this, we investigated ensemble RAG approaches using multiple SLMs. The findings showed that in up to 98% of test cases, at least one SLM was capable of generating the correct answer. Future work will focus on developing effective ensemble RAG strategies utilizing SLMs to further enhance performance.

Acknowledgments. This research was supported by the collaborative research project "Classification of Reading Ambiguities for Text-to-Speech and Construction of a Dataset for Reading Estimation Tasks" at the National Institute for Japanese Language and Linguistics, as well as by JSPS KAKENHI Grant Number 23K11212.

A Examples of Experimental Test Data

Table 6 presents three test samples from the evaluation dataset used in the RAG experiment described in Sect. 4.1.

Table 6. This table presents an example from the JQaRA dataset. The original text is shown first, followed by its English translation.

Label	Text	Title	Question	Answer
1	絶対零度(ぜったいれいど、アブソリュートゼロ、英: Absolute zero)は、熱力学上の最低温度である摂氏−273.15度。	絶対零度(曖昧さ回避)	摂氏ではマイナス273.15度にあたる、全ての原子の振動が停止する最も低い温度を何というでしょう?	["絶対零度"]
1	Absolute zero is the lowest temperature in thermodynamics, equivalent to -273.15 degrees Celsius.	Absolute zero (disambiguation)	What is the lowest temperature at which all atomic motion ceases, corresponding to -273.15 degrees Celsius?	["Absolute zero"]
0	物理学においては、絶対温度において切りのよい数字である 300K (27° C)が室温とされる場合が多い。	室温	摂氏ではマイナス273.15度にあたる、全ての原子の振動が停止する最も低い温度を何というでしょう?	["絶対零度"]
0	In physics, a rounded value of 300K (27°C) is often considered room temperature in terms of absolute temperature.	Room temperature	What is the lowest temperature at which all atomic motion ceases, corresponding to -273.15 degrees Celsius?	["Absolute zero"]

B Comparison Results of Each SLM

Table 7 provide the remaining comparison results for all model pairings discussed in Sect. 6.1.

Table 7. The results of the comparison between Model A (baseline) and Model B (comparison target) in terms of their ability to produce correct answers for each test sample are presented below. BothCor: both models answered correctly; BaseOnly: only the baseline model was correct; CompOnly: only the comparison model was correct; BothInc: both models were incorrect.

Model A	Model B	BothCor	BaseOnly	CompOnly	BothInc
Gemma-2B	Line-3.6B	0.462	0.162	0.221	0.155
Gemma-2B	Rinna-3.6B	0.494	0.130	0.203	0.172
Gemma-2B	LLM-jp-3.7B	0.565	0.059	0.244	0.131
Gemma-2B	Phi-3-mini	0.476	0.148	0.213	0.162
Gemma-2B	Swallow-7B	0.576	0.049	0.278	0.097
Gemma-2B	ELYZA-8B	0.589	0.036	0.261	0.114
Line-3.6B	Rinna-3.6B	0.516	0.167	0.181	0.136
Line-3.6B	LLM-jp-3.7B	0.590	0.093	0.220	0.098
Line-3.6B	Phi - mini	0.490	0.192	0.199	0.118
Line-3.6B	Swallow-7B	0.609	0.074	0.245	0.073
Line-3.6B	ELYZA-8B	0.604	0.079	0.246	0.071
Rinna-3.6B	LLM-jp-3.7B	0.617	0.080	0.192	0.110
Rinna-3.6B	Phi-3-mini	0.510	0.187	0.179	0.123
Rinna-3.6B	Swallow-7B	0.635	0.062	0.218	0.084
Rinna-3.6B	ELYZA-8B	0.636	0.062	0.214	0.088
LLM-jp-3.7B	Phi-3-mini	0.582	0.227	0.107	0.083
LLM-jp-3.7B	Swallow-7B	0.734	0.075	0.119	0.071
LLM-jp-3.7B	ELYZA-8B	0.731	0.078	0.119	0.072
Phi-3-mini	Swallow-7B	0.606	0.084	0.248	0.063
Phi-3-mini	ELYZA-8B	0.609	0.080	0.241	0.070
Swallow-7B	ELYZA-8B	0.763	0.091	0.087	0.059

References

1. Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-rag: learning to retrieve, generate, and critique through self-reflection (2023)
2. Chen, J., Lin, H., Han, X., Sun, L.: Benchmarking large language models in retrieval-augmented generation (2023)
3. Dziri, N., Madotto, A., Zaiane, O., Bose, A.J.: Neural path hunter: reducing hallucination in dialogue systems via path grounding (2021)
4. Gao, L., Ma, X., Lin, J., Callan, J.: Precise zero-shot dense retrieval without relevance labels (2022)
5. Ji, Z., et al.: Survey of hallucination in natural language generation. ACM Comput. Surv. **55**(12), 1–38 (2023). https://doi.org/10.1145/3571730

6. Lepagnol, P., Gerald, T., Ghannay, S., Servan, C., Rosset, S.: Small language models are good too: an empirical study of zero-shot classification (2024). https://arxiv.org/abs/2404.11122
7. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks (2021)
8. Li, J., Zhang, Q., Yu, Y., Fu, Q., Ye, D.: More agents is all you need (2024). https://arxiv.org/abs/2402.05120
9. Marco, G., Rello, L., Gonzalo, J.: Small language models can outperform humans in short creative writing: a study comparing SLMs with humans and LLMs. In: Rambow, O., Wanner, L., Apidianaki, M., Al-Khalifa, H., Eugenio, B.D., Schockaert, S. (eds.) Proceedings of the 31st International Conference on Computational Linguistics, pp. 6552–6570. Association for Computational Linguistics, Abu Dhabi (2025). https://aclanthology.org/2025.coling-main.437/
10. Masatoshi, S., Jun, S., Kouji, M., Kyousuke, N., Naoya, I.: Jaqket: construction of a Japanese question answering dataset based on quiz questions (2020). (in Japanese)
11. Mojarradi, M.M., Yang, L., McCraith, R., Mahdi, A.: Improving in-context learning with small language model ensembles (2024). https://arxiv.org/abs/2410.21868
12. Sato, H.T., Matsuzaki, F., ichiro Takahashi, J.: Ensemble bayesian inference: leveraging small language models to achieve llm-level accuracy in profile matching tasks (2025). https://arxiv.org/abs/2504.17685
13. Tateno, Y.: Jqara: a benchmark dataset for Japanese question answering with retrieval-augmented generation (in Japanese). https://huggingface.co/datasets/hotchpotch/JQaRA
14. Wang, F., et al.: A comprehensive survey of small language models in the era of large language models: techniques, enhancements, applications, collaboration with llms, and trustworthiness (2024). https://arxiv.org/abs/2411.03350

Repurposing Annotation Guidelines to Instruct LLM Annotators: A Case Study

Kon Woo Kim[1,2(✉)], Rezarta Islamaj[3], Jin-Dong Kim[4], Florian Boudin[5], and Akiko Aizawa[1,2]

[1] The Graduate University for Advanced Studies, SOKENDAI, Hayama, Kanagawa, Japan
{ken1204,aizawa}@nii.ac.jp
[2] National Institute of Informatics, Chiyoda, Tokyo, Japan
[3] National Library of Medicine, Bethesda, MD, USA
Rezarta.Islamaj@nih.gov
[4] Database Center for Life Science, Kashiwa, Chiba, Japan
jdkim@dbcls.rois.ac.jp
[5] Japanese-French Laboratory of Informatics, CNRS, Nantes University, Chiyoda, Tokyo, Japan
florian.boudin@univ-nantes.fr

Abstract. This case study explores the potential of repurposing existing annotation guidelines to instruct a large language model (LLM) annotator in text annotation tasks. Traditional annotation projects invest significant resources–both time and cost–in developing comprehensive annotation guidelines. These are primarily designed for human annotators who will undergo training sessions to check and correct their understanding of the guidelines. While the results of the training are internalized in the human annotators, LLMs require the training content to be materialized. Thus, we introduce a method called moderation-oriented guideline repurposing, which adapts annotation guidelines to provide clear and explicit instructions through a process called LLM moderation. Using the NCBI Disease Corpus and its detailed guidelines, our experimental results demonstrate that, despite several remaining challenges, repurposing the guidelines can effectively guide LLM annotators. Our findings highlight both the promising potential and the limitations of leveraging the proposed workflow in automated settings, offering a new direction for a scalable and cost-effective refinement of annotation guidelines and the following annotation process.

Keywords: Large Language Models · Text Annotation · Annotation Guidelines

1 Introduction

Most annotation projects establish detailed annotation guidelines to effectively direct human annotators. In theory, well-crafted guidelines contain sufficient

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 140–151, 2026.
https://doi.org/10.1007/978-3-031-97144-0_13

information to guide annotators' decisions even in the most nuanced cases, and the quality of the resulting annotations is closely tied to the clarity and precision of these guidelines. Consequently, many projects invest significant time and cost in developing labor-intensive yet comprehensive annotation guidelines, which serve as a highly valuable community resource. Incidentally, manual processes can often introduce unintended biases and human errors, producing inconsistencies in the finalized dataset, which could result in unexpected performance fluctuations of the downstream model [16]. To counteract this, researchers build annotation tools [9] and organize annotation tasks [10,11] to minimize unintended biases and human errors.

Meanwhile, large language models (LLMs) are increasingly recognized as viable alternatives to human labor across various domains and show the potential to assist human annotators and even automate at a significantly lower cost and in a much shorter time than manual annotation. Approaches to harness their capabilities range from zero-shot and few-shot methods to full fine-tuning [12,15,19].

In this case study, we investigate the potential of repurposing annotation guidelines to instruct LLMs in text annotation. As LLM capabilities increasingly approach those of human annotators, leveraging these extensive community resources becomes more promising. If LLM annotators can be effectively directed using existing guidelines, it would represent a significant benefit to the community. The study aims to evaluate this potential, assess the existing gaps, and offer insights into the remaining challenges in this area.

During the repurposing of annotation guidelines for LLM annotators, we focus on the moderation process typically used to train new human annotators. This training aims to bridge the gap between the intended interpretation of the guidelines and the annotators' understanding. To pinpoint this gap, trainee annotators first annotate sample texts, and any deviations from the gold annotations reveal areas that need improvement. While human annotators internalize this feedback, LLM annotators require training content to be explicitly materialized. We argue that the necessity for training underscores deficiencies in the original guidelines and that this training knowledge must be captured through revisions of annotation guidelines. In this work, we call this process moderation-oriented guideline repurposing. Our experiments on disease name annotations using the NCBI Disease Corpus reveal both the promising potential of employing annotation guidelines and the challenges that remain. This paper presents these findings in detail.

Our contributions are:

- We provide explicit instructions to an LLM annotator by repurposing existing domain-specific annotation guidelines.
- We introduce a workflow that mimics the human moderation process by automatically updating and adapting the guidelines for LLM annotations.
- We illustrate our approach to the NCBI Disease Corpus by applying the LLM annotation workflow to its annotation guidelines, which we repurposed for this study.
- We investigate the performance gap between LLM annotations and human annotations to show how the LLM annotator comprehends natural language differently in the annotation task.

Prompt:

You are a professional biomedical text annotator. Your task is to annotate all disease mentions in the given text using <category=X>...</category> tags, where X is the type of entity:

SpecificDisease, DiseaseClass, CompositeMention, or Modifier.

{*Input Text*}

Prompt:

You are a professional biomedical text annotator. Your task is to annotate all disease mentions in the given text using <category=X>..</category> tags based on the guidelines below.

Biomedical Annotation Guidelines
{The NCBI Disease Corpus Annotation Guidelines}
- The document includes "What to Annotate", "What NOT to Annotate", and "Examples"

{*Input Text*}

Fig. 1. Simple Prompting (Left) and Guidelines in the Prompt (Right)

2 Related Work

The use of crowd-sourcing workers has been a common practice to obtain natural language annotations until recently. However, studies have shown that the quality of annotated data from crowd workers is not guaranteed due to their limited expertise and knowledge [8]. As a result, researchers have explored alternative methods to ensure annotation quality [3,18]. To collect annotations more effectively, recent studies have focused on utilizing LLM annotators as an alternative to crowd workers or fine-tuned models. Recent studies suggest that LLM annotators have the potential to speed up the annotation process and even surpass the quality of crowd-sourced human workers for text annotation tasks. For example, ChatGPT showed text annotation performance comparable to both trained annotators and Mturk workers in certain datasets, including tweets, news articles, and FewRel [4,6,21], at lower costs. In addition, LLMs helped to improve the annotation process in certain domain-specific tasks [14] and fields, such as medical information extraction [7] and medicine [13], when properly aggregated with human annotators. A recent study attempted to utilize annotation guidelines to improve performance [17]. However, it still required some fine-tuning and did not fully utilize annotation guidelines because it only contained schema including label names, descriptions, and few-shot examples. In addition, it transformed annotation guidelines into a coding style, and therefore, it was difficult to adopt fine-grained annotation guidelines that cover various edge cases.

3 Methodology

To integrate comprehensive and detailed annotation guidelines for higher accuracy and consistency, we designed a workflow that the LLM annotator mimics the human moderation process using annotation guidelines and moderation process. Our methodology consists of several stages, progressing from a simple baseline to a full human-like annotation process.

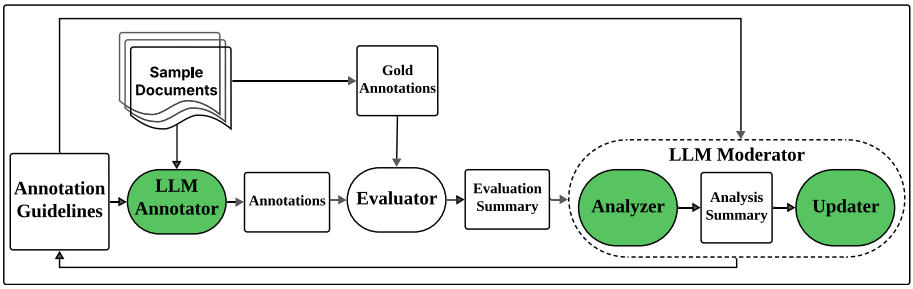


Fig. 2. The Workflow of Moderation-oriented Guideline Repurposing

3.1 Baseline Annotation and Guideline Integration

For the baseline, annotations were generated using a simple prompting method (as illustrated in Fig. 1). The prompt is divided into three parts: (i) a role definition as a text annotator to regulate the knowledge scope; (ii) a brief description of the annotation task; and (iii) guidelines on output format and category, following the original dataset. To validate the effectiveness of human annotation guidelines, we incorporated the annotation guidelines directly into the prompt. The remainder of the prompt remains the same as in the baseline to ensure a fair comparison, as shown in Fig. 1. The guidelines consist of three sections– "What to Annotate", "What NOT to Annotate", and "Examples"– equivalent to approximately 1,000 words, which is around 3 PDF pages.

3.2 Moderation-Oriented Guideline Repurposing

We created a workflow that mimics the human moderation process with the LLM annotator and LLM moderator to examine whether it can substitute the annotation task. In order to more closely mimic the human moderation process, we developed an iterative workflow with 3 phases (refer to Fig. 2). Following the general human annotation process, we began with the assumption that annotators annotate a small size of samples based on some initial gold annotations.

Phase 1: Annotation

- Provide a size of sample text documents from the dataset along with the original annotation guidelines formatted in a text file for improved readability.
- Instruct the LLM annotator to annotate the input text based on the provided guidelines.

Phase 2: Evaluation and Summary

- Compute the average F1 score in Strict Match against the gold annotations. If the average F1 score falls below threshold, the moderation process is initiated.

– Compare and extract discrepancies between LLM-generated annotations and gold annotations using the evaluator, including false positives (FPs), false negatives (FNs), and category mismatches.

Phase 3: Moderation

– Use the LLM moderator, which performs as both analyzer and updater, to analyze these discrepancies and generate a report that includes both the causes of the errors and potential solutions.
– Review the original guideline based on the generated report and update specific sections accordingly or provide detailed examples for edge cases.

3.3 Human-in-the-Loop Moderation

Additionally, a human expert, involved as an updater in Phase 3, conducted an error analysis and manually revised the guideline based on the LLM report to demonstrate the further potential of our proposed methodology.

4 Experiments

4.1 Dataset

To test our methodology, we selected the NCBI Disease Corpus [5]. It is comprised of 793 PubMed abstracts with 6,892 disease mentions and 790 unique disease concepts mapped to Medical Subject Headings (MeSH) or Online Mendelian Inheritance in Man (OMIM). Annotations are divided into four categories: Specific Disease (Mentions that can be linked to one specific definition that does not include further categorization), Modifier(A textual string may refer to a disease name, but it may modify a noun phrase), Disease Class(Mentions that could be described as a family of multiple specific diseases), and Composite Mention(A textual string may refer to two or more separate disease mentions). We selected this corpus for our experiment for two main reasons: 1. The NCBI Disease Corpus provides relatively shorter yet clear and structured annotation guidelines that can be used as a stepping stone to test our hypothesis. 2. It has been widely used in various model evaluations, such as BioBERT and SciBERT [2,12], which allows us to evaluate the significance and practical applicability of our proposed method.

4.2 Parameter Settings

For experiments, we selected GPT-4o as the LLM annotator and moderator, which is one of the most well-performing LLM models today. The size of sample text documents in Phase 1 is set to 5 to (i) loosen the burden of LLM moderator's revision complexity and (ii) account for real-world cases where only a small amount of gold annotation is available. The threshold used to break the loop in Phase 2 is set to 0.8, motivated by the lowest inter-annotator agreement observed in the NCBI Disease Corpus.

4.3 Evaluation Criteria

We tested four evaluation criteria: Strict Match, Strict Match without Category Matching, Soft Match, and Soft Match without Category Matching. We also included Match without Category to measure the performance of disease name annotations without considering category confusion, reflecting requirements of downstream applications (e.g., PubTator [20]). We included Soft Matches as evaluation criteria not to show score improvements but to demonstrate the insufficiency of the span offset regulation when the LLM annotator is given annotation guidelines. A prior research also demonstrated that boundary free approach can improve inter-annotator agreement and reduce required annotation time [1]. Our criteria also provide insights into how well category handling is performed. Another reason is that strict boundary matching for annotation tasks in real-world cases is often challenging unless human judgments are moderated well, even if fine-grained annotation guidelines are provided.

5 Results

The results in Table 1 show that incorporating annotation guidelines significantly improves the accuracy of LLM annotations. Overall, F1 score in the Strict Match evaluation improved from 0.36 to 0.58. Given the annotation guidelines, as shown in Table 2, the LLM annotator achieved 0.70 on Specific Diseases. Table 2 shows that the performance of the LLM annotator improved in Specific Diseases significantly as this category had clearer annotation guidelines requirements compared to the other categories. Incorporating the annotation guidelines outperforms the Baseline method in all criteria. This suggests that disease annotations can be better directed toward the gold annotations by instructing the LLM annotator based on the annotation guidelines used for generation of the gold annotations. The big gap between the Strict and Soft Match results indicates that precisely regulating the selection of the text spans to be annotated as disease names is still challenging. For all methods, matches without category tend to yield better F1 scores. This gap between with and without category matching indicates that category confusion, such as Disease Class vs. Specific Disease vs. Modifier, can be a major issue for the LLM annotator in a situation where category distinction is explicitly required.

The human-in-the-loop approach shows noticeable improvements, indicating that there is still room for more refinement for the LLM moderators. It can reduce category errors with human judgment in correctly defining detailed categories for the LLM annotator. To be more specific, Composite Mention typically requires more complex spans with multi-words or nested concepts. For Disease Class, LLMs may do comparatively better if the prompts or training examples highlight unseen higher-level terms. The LLM annotator frequently gets confused with modifiers due to the inability to distinguish when certain terms can fit into multiple categories or the failure to label them separately. The performance can be even stronger if it recognizes these disease mentions well, but boundary mismatching can penalize Strict Match heavily.

Table 1. Comparison of Annotation Performance under Different Matching Criteria. P: Precision, R: Recall, F1: F1 score. w/o Cat.: Without Category Matching

Method	Strict Match			Strict Match (w/o Category)			Soft Match			Soft Match (w/o Category)		
	P	*R*	F1	*P*	*R*	F1	*P*	*R*	F1	*P*	*R*	F1
Baseline	0.38	0.35	0.36	0.59	0.54	0.57	0.50	0.46	0.48	0.77	0.70	0.74
GuidelinePrompt	0.57	0.39	0.46	0.82	0.57	0.67	0.66	0.46	0.54	0.94	0.66	0.78
Our Method	0.56	0.45	0.50	0.82	0.65	0.73	0.62	0.49	0.55	0.90	0.72	0.80
Human-in-the-loop	0.67	0.51	0.58	0.82	0.62	0.71	0.72	0.55	0.62	0.90	0.69	0.78

Table 2. Comparison of Annotation Performance by Category. P: Precision, R: Recall, F1: F1 score. w/o Cat.: Without Category Matching

Category	Method	Strict Match			Strict Match (w/o Cat.)			Soft Match			Soft Match (w/o Cat.)		
		P	R	F1	P	R	F1	P	R	F1	P	R	F1
Composite Mention	Baseline	0.08	0.35	0.14	0.40	0.35	0.37	0.14	0.60	0.23	0.66	0.70	0.68
	Guideline	0.36	0.50	0.42	0.57	0.65	0.61	0.50	0.70	0.58	0.86	0.90	0.88
	Our Method	0.24	0.25	0.24	0.62	0.40	0.49	0.43	0.45	0.44	0.90	0.65	0.76
	Human-in-the-Loop	0.06	0.10	0.07	0.47	0.10	0.17	0.22	0.40	0.29	0.78	0.50	0.61
Disease Class	Baseline	0.29	0.18	0.22	0.57	0.35	0.43	0.47	0.30	0.37	0.78	0.55	0.65
	Guideline	0.44	0.13	0.20	0.81	0.29	0.43	0.58	0.17	0.27	0.94	0.36	0.53
	Our Method	0.35	0.25	0.29	0.70	0.38	0.49	0.44	0.31	0.37	0.88	0.46	0.61
	Human-in-the-Loop	0.32	0.14	0.20	0.58	0.19	0.29	0.42	0.18	0.25	0.83	0.26	0.40
Modifier	Baseline	0.07	0.04	0.05	0.14	0.53	0.22	0.10	0.06	0.07	0.26	0.64	0.36
	Guideline	0.39	0.15	0.22	0.75	0.56	0.64	0.43	0.17	0.24	0.89	0.60	0.72
	Our Method	0.33	0.08	0.12	0.65	0.62	0.64	0.38	0.09	0.14	0.77	0.67	0.72
	Human-in-the-Loop	0.61	0.44	0.51	0.77	0.65	0.70	0.64	0.47	0.54	0.82	0.67	0.74
Specific Disease	Baseline	0.52	0.53	0.52	0.74	0.60	0.66	0.66	0.68	0.67	0.92	0.77	0.84
	Guideline	0.62	0.56	0.59	0.85	0.63	0.72	0.72	0.65	0.68	0.96	0.74	0.84
	Our Method	0.62	0.67	0.65	0.86	0.74	0.79	0.67	0.72	0.70	0.92	0.80	0.86
	Human-in-the-Loop	0.78	0.64	0.70	0.89	0.72	0.80	0.83	0.68	0.75	0.95	0.79	0.86

6 Discussion

We conducted an error analysis of the LLM annotator results in collaboration with humans and the LLM. We compared gold annotations with the LLM-generated annotations, identified FPs, FNs, and mis-categorized annotations, and the annotations categorized into two categories - that is in our case: disease and uncertain. Through the error analysis, we found two main reasonable causes resulting in unexpectedly lower F1 scores when incorporating annotation guidelines with the LLM annotator - scope mismatch and category mismatch.

Table 3. Category of Influencing Factors in the Dataset

Category	Description	Examples
Ambiguous Abbreviations and Acronyms (50.9%)	Short forms with multiple interpretations. Variability in capitalization or punctuation can contribute to ambiguity	*DM* (diabetes mellitus or myotonic dystrophy), *CT*, *CD*, *IVF*, *A-T*, *DMS*, *PWS*, *TSD*
Generic or Vague Descriptors (17.7%)	Terms describing general concepts	*tumor*, *tumours*, *tumors*
Chromosomal and Genomic Anomaly Terms (4.8%)	Genetic phenomena that are related to diseases	*uniparental disomy*, *Maternal disomy 15*, *maternal UPD 15*, *UPD*
Descriptive or Phenotypic Features (10.0%)	Clinical descriptors relating to disease phenotypes	*early death*, *premature death*, *facial asymmetry*, *broad great toes*, *limb anomalies*
Incomplete or Non-Specific Genetic/Pathological Descriptions (2.4%)	Phrases indicating abnormalities	*genetic defect*, *gene defects*, *deficient activity of coproporphyrinogen III oxidase*
Miscellaneous or Low-Frequency Terms (14.2%)	Rare terms that may denote symptoms, components of a syndrome, or misparsed fragments	*hypomania*, *mania*, *MJD*, *contractures*, *congenital joint contractures*

6.1 Scope Mismatch

When NCBI Disease Corpus annotators were asked to annotate disease names, first they had to define the scope of the disease. For example, they included specific disease names, "liver disease", "Wilson's disease", "colorectal cancer", etc., general disease references, "inherited disorder", "autosomal dominant disease", "neuromuscular disorder", etc., as well as phenotypic terms, such as "hyperthermia", "iron overload", "hepatomegaly", "clinodactyly", "broad great toes", etc. Annotation projects often adopt specific terminologies or ontologies to base their definition of the scope. In the case of NCBI Disease Corpus [5], it was MeSH and OMIM that defined the scope of disease names to be annotated.

According to the NCBI Corpus, some of the most common disease mentions are as follows: Tumor (43), DM (39), G6PD deficiency (33), APC (29). However, we noticed that the most common disease mentions were not likely to be annotated by the LLM annotator. According to the error analysis, influencing factors are categorized into 6 types as shown in Table 3. The LLM annotator skips the annotation of disease mentions specifying gene names. For example, in the NCBI Disease Corpus, "adenomatous polyposis coli" is annotated as a disease mention because this term is listed as the MeSH heading for the "Familial Adenomatous Polyposis Coli" disease. However, the LLM annotator would struggle with this term, which is mostly known to refer to the gene name causing the disease. In the articles annotated in the NCBI Disease Corpus, the term "adenomatous poly-

posis coli" or its abbreviation "APC" are annotated as "Disease Modifier" since they are often found to refer to the gene, i.e., APC gene. However, if human expert annotators were to re-annotate the NCBI Disease Corpus today, terms such as *adenomatous polyposis coli*, which can refer to both a gene and a disease, would likely be prioritized under the gene tag, since the current NLP systems are able to perform multiple entity recognition tasks concurrently. Another example that is often mis-annotated by the LLM annotator is the term "tumor(s)" or its alternate spelling "tumour(s)". "Tumor" refers to an anatomical mass and could be considered as a biological symptom, compared to "cancer", which refers to the diseased state. However, in the MeSH terminology, "tumor" and "cancer" are both listed as synonyms for the term "Neoplasms" categorized as a disease, and annotated as such in the NCBI Disease Corpus.

Myotonic Dystrophy (DM), however, is a case of ambiguity. The LLM annotator in this case expects the abbreviation of Myotonic Dystrophy to be MD. However, the developers of MeSH terminology aimed to reduce ambiguity by not assigning the same text string to denote two different concepts. As such, since there already existed a concept named Muscular Dystrophy abbreviated "MD", Myotonic Dystrophy was referred to by its Greek name - dystrophia myotonica and abbreviated DM.

6.2 Category Mismatch

Category mismatches occur when the LLM annotators have difficulties in understanding the nuances of the annotation guidelines. For the case of NCBI Disease Corpus, the "Composite Mention" category defines a mention that refers to more than one disease, for example: "Saethre-Chotzen, Crouzon, and Pfeiffer syndromes". In this case, the LLM annotator only annotated "Pfeiffer syndromes" as a Specific Disease, creating a category mismatch. If these mentions were written in full rather than abbreviated with ellipsis, then all three would have been annotated as Specific Disease. Furthermore, the "Modifier" category defines any disease mention that is used as an adjective or modifies a noun, for example "APC" gene, "DM" mutation, "breast cancer" families, "aniridia" patients. The LLM annotator was more likely to skip mentions in the modifier context, especially if they referred to genes. Lastly, "Disease Class" denotes a mention that refers to a group of diseases, rather than a specific one, for example: "adenomas", "autosomal dominant disease", "cancer", "leukemia", etc. The LLM annotators had difficulties distinguishing between Specific Diseases and General Diseases.

7 Conclusion and Future Work

The performance of the LLM annotator gradually increased from 0.36 to 0.58 in the Strict Match F1 score after repurposing the annotation guidelines. Our moderation-oriented guideline repurposing involves two steps - 1. incorporating the guidelines into the prompt, which allows easier modifications, and 2.

moderation-oriented guideline revision, which better instructs the LLM annotator towards the gold annotations. This improvement required less time and cost than the fine-tuning methods, which focus solely on instructing the LLM annotator for an annotation task. In addition, LLM annotators have the potential to explain and adapt the latest scope of category definitions capturing the evolving landscape of language use. In future work, we plan to (i) enable the LLM to consult the original ontology/terminology as a reference for annotation while strictly adhering to intended concept scopes and guidelines; (ii) refine the discrepancy analysis and moderation process, including explicit cost reporting, to reduce human involvement as we gradually move towards fully automating the workflow for higher-quality annotations; and (iii) extend our workflow to additional datasets and domains to assess its generality and adopt more fine-grained evaluation metrics that capture guideline compliance for a more rigorous comparison of the original and refined annotation guidelines.

Acknowledgments. This work was supported by Cross-ministerial Strategic Innovation Promotion Program (SIP) on "Integrated Health Care System" Grant Number JPJ012425 and JSPS KAKENHI Grant Number 24K03231.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Andrade, G., Yada, S., Aramaki, E., et al.: Is boundary annotation necessary? Evaluating boundary-free approaches to improve clinical named entity annotation efficiency: case study. JMIR Med. Inf. **12**(1), e59680 (2024). https://doi.org/10.2196/59680
2. Beltagy, I., Lo, K., Cohan, A.: SciBERT: a pretrained language model for scientific text. In: Inui, K., Jiang, J., Ng, V., Wan, X. (eds.) Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 3615–3620. Association for Computational Linguistics, Hong Kong (2019). https://doi.org/10.18653/v1/D19-1371
3. Callison-Burch, C.: Fast, cheap, and creative: evaluating translation quality using Amazon's Mechanical Turk. In: Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing, pp. 286–295 (2009). https://doi.org/10.3115/1699510.1699548
4. Ding, B., et al.: Is GPT-3 a good data annotator? In: Rogers, A., Boyd-Graber, J., Okazaki, N. (eds.) Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, vol. 1: Long Papers, pp. 11173–11195. Association for Computational Linguistics, Toronto (2023). https://doi.org/10.18653/v1/2023.acl-long.626

5. Doğan, R.I., Leaman, R., Lu, Z.: NCBI disease corpus: a resource for disease name recognition and concept normalization. J. Biomed. Inf. **47**, 1–10 (2014). https://doi.org/10.1016/j.jbi.2013.12.006
6. Gilardi, F., Alizadeh, M., Kubli, M.: ChatGPT outperforms crowd workers for text-annotation tasks. Proc. Natl. Acad. Sci. **120**(30), e2305016120 (2023). https://doi.org/10.1073/pnas.2305016120
7. Goel, A., et al.: LLMs accelerate annotation for medical information extraction. In: Machine Learning for Health (ML4H), pp. 82–100. PMLR (2023)
8. Hirschman, L., Fort, K., Boué, S., Kyrpides, N., Islamaj Doğan, R., Cohen, K.B.: Crowdsourcing and curation: perspectives from biology and natural language processing. Database **2016**, baw115 (2016). https://doi.org/10.1093/database/baw115
9. Islamaj, R., Kwon, D., Kim, S., Lu, Z.: TeamTat: a collaborative text annotation tool. Nucleic Acids Res. **48**(W1), W5–W11 (2020). https://doi.org/10.1093/nar/gkaa333
10. Islamaj, R., et al.: NLM-Chem, a new resource for chemical entity recognition in pubmed full text literature. Sci. Data **8**(1), 91 (2021). https://doi.org/10.1038/s41597-021-00875-1
11. Islamaj, R., et al.: NLM-Gene, a richly annotated gold standard dataset for gene entities that addresses ambiguity and multi-species gene recognition. J. Biomed. Inf. **118**, 103779 (2021). https://doi.org/10.1016/j.jbi.2021.103779
12. Lee, J., et al.: BioBERT: a pre-trained biomedical language representation model for biomedical text mining. Bioinformatics **36**(4), 1234–1240 (2020). https://doi.org/10.1093/bioinformatics/btz682
13. Li, J.: A comparative study on annotation quality of crowdsourcing and LLM via label aggregation. In: ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 6525–6529 (2024). https://doi.org/10.1109/ICASSP48485.2024.10447803
14. Li, M., et al.: CoAnnotating: uncertainty-guided work allocation between human and large language models for data annotation. In: Bouamor, H., Pino, J., Bali, K. (eds.) Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 1487–1505. Association for Computational Linguistics, Singapore (2023). https://doi.org/10.18653/v1/2023.emnlp-main.92
15. Liyanage, C.R., Gokani, R., Mago, V.: GPT-4 as an x data annotator: unraveling its performance on a stance classification task. PLoS ONE **19**(8), e0307741 (2024). https://doi.org/10.1371/journal.pone.0307741
16. Luo, L., Lai, P.T., Wei, C.H., Arighi, C.N., Lu, Z.: BioRED: a rich biomedical relation extraction dataset. Brief. Bioinf. **23**(5), bbac282 (2022). https://doi.org/10.1093/bib/bbac282
17. Sainz, O., García-Ferrero, I., Agerri, R., de Lacalle, O.L., Rigau, G., Agirre, E.: GoLLIE: annotation guidelines improve zero-shot information-extraction. CoRR arxiv:2310.03668 (2023). https://doi.org/10.48550/ARXIV.2310.03668
18. Snow, R., O'connor, B., Jurafsky, D., Ng, A.Y.: Cheap and fast–but is it good? Evaluating non-expert annotations for natural language tasks. In: Proceedings of the 2008 Conference on Empirical Methods in Natural Language Processing, pp. 254–263 (2008). https://doi.org/10.3115/1613715.1613751
19. Wang, S., Liu, Y., Xu, Y., Zhu, C., Zeng, M.: Want to reduce labeling cost? GPT-3 can help. In: Moens, M.F., Huang, X., Specia, L., Yih, S.W.T. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2021, pp. 4195–4205. Association for Computational Linguistics, Punta Cana (2021). DOI: https://doi.org/10.18653/v1/2021.findings-emnlp.354

20. Wei, C.H., Kao, H.Y., Lu, Z.: PubTator: a web-based text mining tool for assisting biocuration. Nucleic Acids Res. **41**(W1), W518–W522 (2013). https://doi.org/10.1093/nar/gkt441
21. Zhu, Y., Zhang, P., ul Haq, E., Hui, P., Tyson, G.: Can ChatGPT reproduce human-generated labels? a study of social computing tasks. CoRR arxiv:2304.10145 (2023). https://doi.org/10.48550/ARXIV.2304.10145

Pretraining Data Exposure in Large Language Models: A Survey of Membership Inference, Data Contamination, and Security Implications

Ziyi Tong, Feifei Sun, and Le Minh Nguyen(✉)

Japan Advanced Institute of Science and Technology, Nomi, Japan
{s2320416,nguyenml}@jaist.ac.jp

Abstract. Large Language Models (LLMs) have become the predominant paradigm in NLP, advancing both research and industry. As model sizes and pretraining data grow, concerns about Pretraining Data Exposure (PDE) increase due to the scale and opacity of training datasets.

PDE refers to determining whether specific data appeared in an LLM's pretraining corpus. It is critical for ensuring evaluation integrity and protecting privacy, intersecting two key areas: data contamination and membership inference. Though conceptually related, these areas have often been studied in isolation. This paper offers the first unified survey of both under the PDE framework. We formalize PDE across exposure levels, review attack and defense methods, synthesize empirical findings, and highlight open challenges and future research directions.

Keywords: Data contamination · Membership inference attack · Pretraining Data Exposure(PDE)

1 Introduction

The rapid development of Large Language Models (LLMs) [1,51] along with the increasing size of pretraining datasets has introduced new challenges: how can we determine whether specific data was included in an LLM's pretraining corpus?

LLMs are trained on largely opaque datasets [48], often collected via automated web crawlers, making it impossible to determine which data points were included in the training. This raises serious security and privacy concerns [2,7,19,20,37,46]. Public evaluation datasets are particularly vulnerable to contamination [41,61], test-train overlap compromises validity, as models may memorize rather than generalize.

These concerns make it imperative to study Pretraining Data Exposure (PDE)–the problem of verifying whether specific data has been included in an LLM's pretraining corpus. In this paper, we approach PDE from a unified perspective by bridging two existing research domains: (1) data contamination [7,17], which investigates overlaps between training and evaluation datasets, and (2) membership inference [18,50], which seeks to determine whether specific

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 152–162, 2026.
https://doi.org/10.1007/978-3-031-97144-0_14

instances were part of the pretraining data. While MIA is often mentioned in data contamination studies, previous research [7,10,17–19] has typically treated it as a minor subset of the broader contamination problem. We position both fields as equally important, systematically integrating their latest attack and defense strategies into a comprehensive literature review.

At their core, both data contamination and membership inference share a common objective: *Determining whether a specific data point exists within the pretraining corpus of an LLM.*

Specifically, our key contributions are as follows.

1. **A unified review of PDE across two domains.** We propose a comprehensive framework for PDE, encompassing both instance-level and dataset-level exposure. To our knowledge, this is the first work to treat membership inference as equally important as data contamination, rather than as a subset. While the two areas overlap, we argue that MIA offers additional insight by enabling instance-level PDE detection.
2. **A novel taxonomy of PDE attack and defense strategies.** We present a new taxonomy for PDE attacks and defenses, organized by real-world scenarios and user types, to better align existing research with practical deployment contexts.
3. **Latest updates and future directions.** We provide an up-to-date review of PDE research, highlight key challenges and open problems, and propose directions for future work.

2 Background

In this section, we introduce key terms and concepts to provide a clear foundation for understanding the Pretraining Data Exposure (PDE) problem.

Membership Inference. Membership inference is an attack technique in which an adversary seeks to determine whether a specific data sample was part of a machine learning model's training dataset [13,18,38]. In the context of LLMs, membership inference attacks (MIAs) leverage the model's outputs [13], probability distributions [16,48], gradients [34,55], or internal representations to differentiate between member instances (samples seen during training) and non-member instances (unseen data). These attacks pose significant privacy risks, especially when models inadvertently memorize and disclose sensitive or private information [32].

Data Contamination. Data contamination is universally defined as the inclusion of evaluation data (input and/or labels) in training datasets of models [7,10,42,46]. Contamination causes models to memorize instead of generalize, inflating benchmark performance and undermining evaluation validity and generalization. Larger models often exhibit stronger inflation due to memorization

[30]. Traditional approaches for contamination detection include N-gram overlap, exact-match comparisons [42], and perplexity analysis [27], but these methods falter with approximate, noisy, or adversarial contamination [8,47]. Conventionally, strategies for preventing and mitigating data contamination focus on dataset design and curation [68], as well as innovations in evaluation protocols [46].

3 Definitions

This section formalizes PDE with mathematical definitions at both instance and dataset levels. Instance-level PDE aligns with membership inference attacks (MIA), while dataset-level PDE corresponds to data contamination. Therefore, following previous work [17], we provide definitions of instance-level PDE and dataset-level PDE.

3.1 Instance-Level PDE

Let D_M denote the pretraining data of an LLM M. The binary function $f(M, x)$ determines whether an individual instance x is seen by the model M:

$$f(M,x) = \begin{cases} 1 & \text{if } \exists x' \in D_M,\ b(x,x') = 1 \\ 0 & \text{if } \forall x' \in D_M,\ b(x,x') = 0 \end{cases} \tag{1}$$

If $f(M, x) = 1$, the instance x is considered **exposed** (seen by the model). If $f(M, x) = 0$, the instance x is considered **unexposed** (unseen by the model).

3.2 Dataset-Level PDE

A dataset D is **exposed** (partially seen by M) if at least one instance x in D is seen:

$$\exists x \in D, f(M,x) = 1 \tag{2}$$

A dataset is **fully exposed** if all instances within D are seen:

$$\forall x \in D, f(M,x) = 1 \tag{3}$$

3.3 Exposure Score for PDE

In addition to the binary definition, we define an exposure score to quantify how much of D has been seen by M:

$$PDE(D,M) = \frac{\sum_{x \in D} f(M,x)}{|D|} \tag{4}$$

where $PDE(D, M)$ represents the proportion of exposed instances in dataset D.

If $PDE(D, M) = 1$, the dataset is fully exposed. If $PDE(D, M) = 0$, the dataset is fully unexposed. If $0 < PDE(D, M) < 1$, the dataset is partially exposed.

4 Threat Models and Detection Strategies for PDE

4.1 Threat Model

We define the threat model for text-based LLMs, considering adversaries with query access but no access to pretraining data. Our analysis focuses on English-language models, including both open-source models and commercial APIs.

4.2 Taxonomies of Detection Strategies

Unlike previous studies [7,10,17–19], we categorize PDE detection methods based on real-world application scenarios and user types. Our taxonomy aligns existing research more closely with practical deployment settings. Empirically, we identify four key LLM application scenarios where pretraining data exposure presents significant challenges. Table 1 presents a taxonomy of scenarios, detailing descriptions, user types, and related security risks, offering a structured view of stakeholder interactions and vulnerabilities.

Besides the papers mentioned in Table 1, **benchmark contamination** scenario was also investigated in studies: [8,12,25,27,67]. Approaches such as perplexity analysis [27], exchangeability tests [41], and statistical tools like ConStat [9] offer mechanisms for identifying memorization and dataset overlap. However, detecting paraphrased or partially contaminated instances remains challenging, especially in black-box settings.

In **personal data exposure** scenario, beyond the works listed in Table 1, these studies also explored the personal data exposure problem: [4,53,57]. Overall, techniques like differential privacy reduce utility but fail to reliably protect rare or sensitive PII [32]. Innovations like MemHunter allow dataset-wide PII leakage verification with reduced computational costs [60]. However, detecting and addressing PII leakage remains a critical goal.

In the **copyrighted content** scenario, Expectation-Maximization MIA (EM-MIA): A state-of-the-art inference method refines membership probabilities, excelling in detecting copyrighted data under experimental setups [24]. Sampling-based MIAs (SaMIA): Improves inference without internal training dataset access [23]. Overall, despite progress in addressing copyright contamination and MIA risks, no single solution fully resolves the technical, legal, and ethical challenges, highlighting the need for integrated detection strategies.

In the **code & software security risks** scenario, papers mentioned in Table 1, confirm significant risks of PDE targeting programming-related text in LLMs, providing tailored methods (e.g., CodeMI [54], TraWiC [33]), metrics (e.g., token-level decoding [39], confidence score calibration [64]). However, programming-related text poses elevated risks due to deterministic structures, leading to the memorization of sensitive proprietary content (e.g., credentials, function templates). As a result, a high false positive rate arises in membership inference metrics. Moreover, deduplication strategies struggle with semantic equivalence in programming corpora [11], limiting their reliability in detecting nuanced contamination scenarios.

Table 1. Taxonomies of PDE detection methods based on LLM application scenarios and user types, associated with security risks.

Application Scenario	User Types	Scenario Description	Security Risks	Relevant Researches
NLP Benchmark Contamination	LLM developer	LLMs are trained on publicly available NLP benchmarks, leading to inflated evaluation results because the model has already seen the test set.	Model performance evaluation becomes unreliable, making it unclear whether the model is truly generalizing or just memorizing test examples.	e.g. [2,6,11,41,45,62,63]
Personal Data Exposure from Web Crawling	API User	LLMs scrapes massive web corpora, accidentally ingesting personal information, social media posts, and leaked databases.	If models memorize personal data, they might regurgitate sensitive details when prompted.	e.g. [32,56,66]
Copyrighted Content & Intellectual Property Risks	API Provider	LLMs trained on web data ingest copyrighted content, leading to legal concerns.	LLMs may directly output copyrighted content, raising ethical and legal issues.	e.g. [24,36,48,49,65]
Code & Software Security Risks	API Provider and API User	LLMs trained on public code repositories (e.g., GitHub, Stack Overflow) may leak proprietary or insecure code.	Models output license-violating code or insecure snippets.	e.g. [33,40,54,62,64]

5 Mitigation and Defense Mechanisms for PDE

This section outlines contamination mitigation and membership inference defenses.

Dynamic Benchmark. A dynamic benchmark is a regularly updating test dataset. Dynamic benchmarking offers a promising approach to mitigating data contamination in model evaluation [6,15,28,44,58,68]. For the coding area, there is [21]. Proposed evaluation methods, such as **contamination-free benchmarks** prevent test-train overlap and **dynamic dataset splits** update test sets post-training to ensure exclusion from pretraining data. These methods improve benchmark robustness by emphasizing generalization over memorization in the context of rapidly evolving LLMs. [3,7].

Private and Secure Benchmarking. A private, secure benchmark prevents data contamination by preserving dataset integrity and confidentiality during evaluation. [20] underscores the importance of protecting future datasets and provides practical strategies, including the encryption of test datasets. Cryptographic isolation and confidential computing frameworks were proposed to secure test datasets from contamination [5]. A secure benchmark prevents unintentional contamination and enables more reliable and trustworthy evaluation.

Automated Decontamination. Systems like AntiLeak-Bench [59] use automated frameworks to create contamination-free benchmarks. [26] presents a systematic strategy for preventing contamination, which includes periodic crawlers and a contamination detection mechanism. [68] tackles the contamination problem by identifying and modifying leaked samples, ensuring that their difficulty level remains unchanged.

Table 2. Comparison of SOTA PDE Methods Based on Code Availability, Dataset Availability, Benchmark Availability.

Method	Level	Types	Code Available?	Dataset Info?	Benchmark Info?
[2]	Dataset	Detection	×	×	✓
[24]	Instance	Detection	✓	✓	✓
[41]	Dataset	Detection	✓	✓	✓
[48]	Instance	Detection	✓	✓	✓
[11]	Dataset	Detection	×	✓	✓
[35]	Instance	Detection	✓	✓	×
[28]	Dataset	Mitigation	✓	✓	✓
[59]	Dataset	Mitigation	✓	✓	✓
[14]	Instance	Mitigation	×	✓	✓

Watermarking. [43] demonstrates that watermarking is effective for copyright protection and reduces the success rate of model inversion attacks. [29] brings TextMarker, which employs a backdoor-based membership inference method to protect sensitive data in pre-trained models, though it increases training complexity. Watermarking supports intellectual property enforcement, but its effects on training complexity and model robustness remain underexplored.

Machine Unlearning. Machine unlearning is, ideally, the ultimate solution for removing data from pre-trained models. Efforts have been made to leverage token-specific characteristics [52], benchmark the real-world knowledge unlearning [22], advocate for a minority-aware evaluation framework [56], and erase famous characters from the LLM [14,31]. However, the effectiveness of unlearning remains limited and requires further refinement.

Defending against Pretraining Data Exposure (PDE) involves trade-offs among privacy, robustness, scalability, and transparency. Preventative methods like dynamic and secure benchmarks reduce contamination but may limit reproducibility and openness. Automated decontamination scales well but struggles with paraphrased content. Watermarking aids intellectual property enforcement but can affect model generalization and remains legally uncertain. Machine unlearning enables post hoc data removal for privacy compliance, but is technically challenging and may impact performance. No single method suffices; effective PDE defense requires combining strategies tailored to deployment needs and data sensitivity.

5.1 Comparison of SOTA PDE Methods

As shown in Table 2, we provide a comparison of SOTA PDE methods based on code availability, dataset availability and benchmark availability.

6 Challenges and Future Directions

Despite progress in detecting PDE, major challenges remain. Identifying paraphrased, partially contaminated instances or low-occurrence instances is espe-

cially difficult. No single approach fully addresses the technical, legal, and ethical issues of PDE, underscoring the need for integrated solutions. Programming-related text poses added risk due to its deterministic nature, making it more prone to memorization and leakage of sensitive or proprietary content.

To address these challenges, future research should explore several promising directions. One is developing **unlearning** techniques that enable models to forget specific data points post-training without compromising overall performance. Advancements in **model explainability** can provide deeper insights into how models memorize and reproduce training data, ultimately leading to more effective mitigation strategies. **Semantic-level detection** enables identifying contamination beyond exact matches by capturing meaning-based similarities instead of relying solely on lexical overlap. Advancing this area can promote more secure, transparent, and accountable handling of PDE in LLMs.

7 Conclusion

This paper presents a comprehensive review of pretraining data exposure (PDE), unifying research on instance-level extraction from membership inference attacks and dataset-level extraction from data contamination. We summarize detection and defense methods, categorize them by user types and application scenarios, and analyze the strengths and limitations of the current work. Finally, we outline future directions to support deeper understanding and continued progress in this rapidly evolving field.

8 Limitation

Despite our efforts to be comprehensive, some relevant studies may have been omitted. This review focuses solely on text-based Large Language Models (LLMs), excluding other model types, multimodal systems, and research in multilingual or low-resource settings.

References

1. Anthropic: Introducing the next generation of claude (2024). https://www.anthropic.com/news/claude-3-family. Accessed 16 Mar 2025
2. Balloccu, S., Schmidtov'a, P., Lango, M., Dusek, O.: Leak, cheat, repeat: data contamination and evaluation malpractices in closed-source llms. In: Conference of the European Chapter of the Association for Computational Linguistics (2024). https://doi.org/10.48550/arxiv.2402.03927
3. Cao, J., Zhang, W., Cheung, S.: Concerned with data contamination? Assessing countermeasures in code language model (2024). https://doi.org/10.48550/arxiv.2403.16898
4. Carlini, N., Ippolito, D., Jagielski, M., Lee, K., Tramer, F., Zhang, C.: Quantifying Memorization Across Neural Language Models (2023). https://doi.org/10.48550/arXiv.2202.07646. http://arxiv.org/abs/2202.07646

5. Chandran, N., Sitaram, S., Gupta, D., Sharma, R., Mittal, K., Swaminathan, M.: Private benchmarking to prevent contamination and improve comparative evaluation of llms (2024). https://doi.org/10.48550/arXiv.2403.00393
6. Chen, S., et al.: Recent advances in large langauge model benchmarks against data contamination: from static to dynamic evaluation (2025)
7. Cheng, Y., Chang, Y., Wu, Y.: A survey on data contamination for large language models (2025). https://doi.org/10.48550/arXiv.2502.14425. http://arxiv.org/abs/2502.14425
8. Dekoninck, J., Muller, M.N., Baader, M., Fischer, M., Vechev, M.T.: Evading data contamination detection for language models is (too) easy (2024). https://doi.org/10.48550/arxiv.2402.02823
9. Dekoninck, J., Müller, M.N., Vechev, M.T.: Constat: performance-based contamination detection in large language models (2024). https://doi.org/10.48550/arXiv.2405.16281
10. Deng, C., et al.: Unveiling the spectrum of data contamination in language models: a survey from detection to remediation. arXiv preprint arXiv:2406.14644 (2024)
11. Deng, C., Zhao, Y., Tang, X., Gerstein, M., Cohan, A.: Investigating data contamination in modern benchmarks for large language models (2024). https://doi.org/10.48550/arXiv.2311.09783. http://arxiv.org/abs/2311.09783
12. Dong, Y., Jiang, X., Liu, H., Jin, Z., Li, G.: Generalization or memorization: data contamination and trustworthy evaluation for large language models (2024). https://doi.org/10.48550/arXiv.2402.15938
13. Duan, M., et al.: Do membership inference attacks work on large language models? (2024)
14. Eldan, R., Russinovich, M.: Who's harry potter? Approximate unlearning in llms. arXiv preprint arXiv:2310.02238 (2023)
15. Fan, Y.: Adeval: alignment-based dynamic evaluation to mitigate data contamination in large language models (2025). https://doi.org/10.48550/arxiv.2501.13983
16. Fu, W., Wang, H., Gao, C., Liu, G., Li, Y., Jiang, T.: Practical membership inference attacks against fine-tuned large language models via self-prompt calibration (2023). https://doi.org/10.48550/arXiv.2311.06062
17. Fu, Y., Uzuner, O., Yetisgen, M., Xia, F.: Does data contamination detection work (well) for llms? A survey and evaluation on detection assumptions. arXiv preprint arXiv:2410.18966 (2024)
18. Hu, H., Salcic, Z., Sun, L., Dobbie, G., Yu, P.S., Zhang, X.: Membership inference attacks on machine learning: a survey. ACM Comput. Surv. (CSUR) **54**(11s), 1–37 (2022)
19. Ishihara, S.: Training data extraction from pre-trained language models: a survey. arXiv preprint arXiv:2305.16157 (2023)
20. Jacovi, A., Caciularu, A., Goldman, O., Goldberg, Y.: Stop uploading test data in plain text: practical strategies for mitigating data contamination by evaluation benchmarks (2023). https://doi.org/10.48550/arXiv.2305.10160. http://arxiv.org/abs/2305.10160
21. Jain, N., et al.: Livecodebench: holistic and contamination free evaluation of large language models for code (2024). https://doi.org/10.48550/arXiv.2403.07974
22. Jin, Z., et al.: Rwku: benchmarking real-world knowledge unlearning for large language models. In: Neural Information Processing Systems (2024). https://doi.org/10.48550/arxiv.2406.10890
23. Kaneko, M., Ma, Y., Wata, Y., Okazaki, N.: Sampling-based pseudo-likelihood for membership inference attacks (2024). https://doi.org/10.48550/arXiv.2404.11262

24. Kim, G., Li, Y., Spiliopoulou, E., Ma, J., Ballesteros, M., Wang, W.Y.: Detecting training data of large language models via expectation maximization (2024). https://doi.org/10.48550/arXiv.2410.07582
25. Li, C., Flanigan, J.: Task contamination: language models may not be few-shot anymore (2023). https://doi.org/10.48550/arXiv.2312.16337. http://arxiv.org/abs/2312.16337
26. Li, Y., et al.: C^2leva: toward comprehensive and contamination-free language model evaluation (2024)
27. Li, Y.: Estimating contamination via perplexity: quantifying memorisation in language model evaluation (2023). https://arxiv.org/abs/2309.10677
28. Li, Y., Geurin, F., Lin, C.: Latesteval: addressing data contamination in language model evaluation through dynamic and time-sensitive test construction. In: AAAI Conference on Artificial Intelligence (2023). https://doi.org/10.48550/arxiv.2312.12343
29. Liu, Y., Hu, H., Zhang, X., Sun, L.: Watermarking text data on large language models for dataset copyright protection (2023). https://doi.org/10.48550/arXiv.2305.13257
30. Liu, Y.: An open source data contamination report for llama series models (2023). https://doi.org/10.48550/arxiv.2310.17589
31. Liu, Y., Zhang, Y., Jaakkola, T., Chang, S.: Revisiting who's harry potter: towards targeted unlearning from a causal intervention perspective. arXiv preprint arXiv:2407.16997 (2024)
32. Lukas, N., Salem, A., Sim, R., Tople, S., Wutschitz, L., Zanella-B'eguelin, S.: Analyzing leakage of personally identifiable information in language models (2023). https://doi.org/10.1109/SP46215.2023.10179300
33. Majdinasab, V., Nikanjam, A., Khomh, F.: Trained without my consent: detecting code inclusion in language models trained on code (2024). https://doi.org/10.1145/3702980
34. Mancera, G., DeAlcala, D., Fiérrez, J., Tolosana, R., Morales, A.: Is my text in your ai model? Gradient-based membership inference test applied to llms (2025)
35. Mattern, J., Mireshghallah, F., Jin, Z., Schölkopf, B., Sachan, M., Berg-Kirkpatrick, T.: Membership inference attacks against language models via neighbourhood comparison. In: Annual Meeting of the Association for Computational Linguistics (2023). https://doi.org/10.48550/arxiv.2305.18462
36. Meeus, M., Shilov, I., Faysse, M., de Montjoye, Y.A.: Copyright traps for large language models (2024). https://doi.org/10.48550/arXiv.2402.09363
37. Mireshghallah, F., Goyal, K., Uniyal, A., Berg-Kirkpatrick, T., Shokri, R.: Quantifying privacy risks of masked language models using membership inference attacks. In: Conference on Empirical Methods in Natural Language Processing (2022). https://doi.org/10.48550/arxiv.2203.03929
38. Mozaffari, H., Marathe, V.J.: Semantic membership inference attack against large language models (2024)
39. Nie, Y., Wang, C., Wang, K., Xu, G., Xu, G., Wang, H.: Decoding secret memorization in code llms through token-level characterization (2024). https://doi.org/10.48550/arXiv.2410.08858
40. Niu, L., Mirza, M.S., Maradni, Z., Pöpper, C.: Codexleaks: privacy leaks from code generation language models in github copilot (2023)
41. Oren, Y., Meister, N., Chatterji, N., Ladhak, F., Hashimoto, T.B.: Proving test set contamination in black box language models (2023). https://doi.org/10.48550/arXiv.2310.17623. http://arxiv.org/abs/2310.17623

42. Palavalli, M., Bertsch, A., Gormley, M.R.: A taxonomy for data contamination in large language models. CONDA (2024). https://doi.org/10.48550/arxiv.2407.08716
43. Panaitescu-Liess, M.A., et al.: Can watermarking large language models prevent copyrighted text generation and hide training data? (2024). https://doi.org/10.48550/arXiv.2407.17417
44. Qian, K., et al.: Varbench: robust language model benchmarking through dynamic variable perturbation. In: Conference on Empirical Methods in Natural Language Processing (2024). https://doi.org/10.48550/arxiv.2406.17681
45. Ravaut, M., et al.: How much are large language models contaminated? A comprehensive survey and the LLMSanitize library (2024). https://doi.org/10.48550/arXiv.2404.00699. http://arxiv.org/abs/2404.00699
46. Sainz, O., Campos, J.A., García-Ferrero, I., Etxaniz, J., Lacalle, O.L.D., Agirre, E.: NLP evaluation in trouble: on the need to measure LLM data contamination for each benchmark (2023).https://doi.org/10.48550/arXiv.2310.18018. http://arxiv.org/abs/2310.18018
47. Samuel, V., Zhou, Y., Zou, H.P.: Towards data contamination detection for modern large language models: limitations, inconsistencies, and oracle challenges (2024). https://doi.org/10.48550/arXiv.2409.09927. http://arxiv.org/abs/2409.09927
48. Shi, W., et al.: Detecting pretraining data from large language models (2024https://doi.org/10.48550/arXiv.2310.16789. http://arxiv.org/abs/2310.16789
49. Shilov, I., Meeus, M., de Montjoye, Y.A.: Mosaic memory: fuzzy duplication in copyright traps for large language models (2024). https://doi.org/10.48550/arXiv.2405.15523
50. Shokri, R., et al.: Membership inference attacks against machine learning models. arXiv Cryptography and Security (2016)
51. Team, G., et al.: Gemini: a family of highly capable multimodal models. arXiv preprint arXiv:2312.11805 (2023)
52. Tran, T., Liu, R., Xiong, L.: Tokens for learning, tokens for unlearning: mitigating membership inference attacks in large language models via dual-purpose training. Placeholder J. (2025)
53. Vakili, T., Dalianis, H.: Using membership inference attacks to evaluate privacy-preserving language modeling fails for pseudonymizing data. In: Nordic Conference of Computational Linguistics (2023)
54. Wan, Y., et al.: Does your neural code completion model use my code? A membership inference approach (2024). https://doi.org/10.48550/arXiv.2404.14296
55. Wang, J.G., Wang, J., Li, M., Neel, S.: Pandora's white-box: precise training data detection and extraction in large language models (2024)
56. Wei, R., et al.: Underestimated privacy risks for minority populations in large language model unlearning (2024). https://doi.org/10.48550/arxiv.2412.08559
57. Wen, Y., Marchyok, L., Hong, S., Geiping, J., Goldstein, T., Carlini, N.: Privacy backdoors: enhancing membership inference through poisoning pre-trained models. In: Neural Information Processing Systems (2024). https://doi.org/10.48550/arxiv.2404.01231
58. White, C., et al.: Livebench: a challenging, contamination-free llm benchmark (2024). https://doi.org/10.48550/arxiv.2406.19314
59. Wu, X., et al.: Antileak-bench: preventing data contamination by automatically constructing benchmarks with updated real-world knowledge (2024). https://doi.org/10.48550/arXiv.2412.13670

60. Wu, Z., Lou, J., Zheng, Z., Chen, C.: Memhunter: automated and verifiable memorization detection at dataset-scale in llms (2024). https://doi.org/10.48550/arXiv.2412.07261
61. Xu, C., Guan, S., Greene, D., Kechadi, M., et al.: Benchmark data contamination of large language models: a survey. arXiv preprint arXiv:2406.04244 (2024)
62. Yang, Z., et al.: Gotcha! this model uses my code! evaluating membership leakage risks in code models. IEEE Trans. Softw. Eng. (2023). https://doi.org/10.1109/tse.2024.3482719
63. Zhang, R., Bertran, M., Roth, A.: Order of magnitude speedups for llm membership inference. In: Conference on Empirical Methods in Natural Language Processing (2024). https://doi.org/10.48550/arxiv.2409.14513
64. Zhang, S., Li, H.: Code membership inference for detecting unauthorized data use in code pre-trained language models (2023). https://doi.org/10.48550/arXiv.2312.07200
65. Zhao, S., Zhu, L., Quan, R., Yang, Y.: Protecting copyrighted material with unique identifiers in large language model training (2024)
66. Zhao, Y.F., Zhang, J.: Does training with synthetic data truly protect privacy? (2025)
67. Zhou, K., et al.: Don't make your llm an evaluation benchmark cheater (2023). https://doi.org/10.48550/arXiv.2311.01964
68. Zhu, Q., et al.: Inference-time decontamination: reusing leaked benchmarks for large language model evaluation. In: Conference on Empirical Methods in Natural Language Processing (2024). https://doi.org/10.48550/arxiv.2406.13990

EcoRAG: A Multi-hop Economic QA Benchmark for Retrieval Augmented Generation Using Knowledge Graphs

Hanieh Khorashadizadeh[1(✉)], Sanju Tiwari[2], Farah Benamara[3], Nandana Mihindukulasooriya[4], Jinghua Groppe[1], Soror Sahri[5], Morteza Ezzabady[3], Frédéric Ieng[5], and Sven Groppe[1]

[1] Lübeck University, Lübeck, Germany
{hanieh.khorashadizadeh,jinghua.groppe,sven.groppe}@uni-luebeck.de
[2] Sharda University, Greater Noida, UP, India
tiwarisanju18@ieee.org
[3] Université de Toulouse, Toulouse, France
{farah.benamara,morteza.ezzabady}@irit.fr
[4] IBM Research, Yorktown Heights, NY, USA
nandana@ibm.com
[5] Université Paris Cité, Paris, France
{soror.sahri,frederic.ieng}@u-paris.fr

Abstract. Retrieval-Augmented Generation (RAG) enhances large language models (LLMs) by integrating external knowledge retrieval. However, existing multi-hop RAG benchmarks are limited by shallow reasoning depth, simple retrieval structures, and non-descriptive answers. We introduce **EcoRAG**, a novel **multi-hop economic QA benchmark** built on knowledge graphs (KGs). EcoRAG extends retrieval depth to seven-hop reasoning, incorporates complex subgraph structures, and leverages domain-specific economic knowledge. It enables a more realistic evaluation of retrieval and reasoning by bridging the gap between structured knowledge sources and generative models, while also providing a reusable benchmark to advance multi-hop RAG research.

Keywords: Knowledge Graph · Large Language Model · Retrieval Augmented Generation · Multi-Hop RAG Banchmark

1 Introduction

Retrieval-Augmented Generation (RAG) has emerged as a powerful approach to enhance large language models (LLMs) by integrating external knowledge retrieval. This enables more accurate, explainable, and up-to-date responses compared to models that rely solely on parametric memory. However, existing multi-hop RAG benchmarks often suffer from key limitations, including restricted reasoning depth, simplified retrieval structures, and non-descriptive answers. These limitations make it difficult to evaluate real-world multi-hop retrieval and reasoning effectively.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 163–173, 2026.
https://doi.org/10.1007/978-3-031-97144-0_15

To address these challenges, we introduce **EcoRAG**, a novel multi-hop economic QA benchmark designed for retrieval-augmented generation using knowledge graphs (KGs) [3]. EcoRAG is publicly available at[1]. Unlike prior benchmarks, EcoRAG enables extended multi-hop reasoning, introduces complex retrieval structures, and leverages domain-specific knowledge from economic datasets rather than general sources. By incorporating both converging (DAG) and divergent (polytree) subgraphs, our benchmark better reflects real-world retrieval complexity and reasoning dependencies. This benchmark is intended to support developers of future RAG approaches by providing a challenging setting that highlights the limitations of current systems, especially in multi-hop reasoning, and encourages the development of more robust and effective retrieval strategies.

1.1 Key Contributions

The key contributions of this paper are as follows:

- **Benchmarking Pipeline:** We propose a reusable pipeline that enables researchers and practitioners to create new benchmarks efficiently.
- **KG as Source Data:** Our dataset is derived from a structured KG, ensuring a more interconnected and comprehensive representation of knowledge.
- **Descriptive Answers:** Unlike prior benchmarks that focus on binary or entity-based answers, our benchmark supports rich, contextual responses, enhancing answer quality.
- **Domain-Specific Focus:** While most existing benchmarks use general datasets, our work is tailored for economic knowledge graphs, providing more relevant insights for financial and economic applications.

By integrating longer reasoning chains, structured retrieval challenges, and contextual answer generation, EcoRAG establishes a rigorous benchmark for evaluating retrieval-augmented generation in economic reasoning. This work sets a new standard for multi-hop RAG evaluation, paving the way for improved knowledge-intensive question answering.

2 Related Work

RAG has gained significant attention for its ability to enhance language models by incorporating external knowledge. However, existing multi-hop RAG benchmarks often suffer from limitations such as restricted hop counts, simple answer formats, and oversimplified retrieval structures. In this section, we review prior works that have attempted to evaluate multi-hop reasoning within RAG systems, highlighting their key contributions and constraints. We then position our benchmark as a more comprehensive alternative that addresses these shortcomings by incorporating longer reasoning chains, domain-specific data, and complex subgraph structures.

[1] https://github.com/haniehkh18/EcoRAG.

[6] introduced the MultiHop-RAG benchmark, which leverages an English news article dataset as the foundational RAG knowledge base. However, their approach has two key limitations: (1) The ground truth answers are confined to simple responses, including "yes," "no," entity names, or temporal indicators like "before" or "after." (2) The number of supporting evidence pieces, or hops, is capped at four.

CRAG [7] is a benchmark dataset covering five domains and eight question categories, including comparison, aggregation, and multi-hop questions, among others. However, the dataset is limited to two-hop questions, and its answers are non-descriptive, consisting only of single entity names rather than detailed explanations.

[2] created MINTQA by systematically extracting knowledge triplets from the English Wikidata and leveraging GPT-4o to generate multi-hop questions ranging from one to four hops. One limitation of their approach is that forming the retrieval corpus by sequentially linking entity-related facts oversimplifies the complexity of real-world documents, potentially leading to an overestimation of the effectiveness of retrieval-augmented methods. Table 1 presents a comparison of the characteristics of existing multi-hop RAG benchmarks and our proposed benchmark. As shown, none of the existing approaches provide descriptive ground truth, and the number of hops is limited to four, whereas our benchmark includes ground truth extending up to seven hops. Additionally, while all existing benchmarks focus on general knowledge, our approach is domain-specific, leveraging economic data. Another key distinction is that our benchmark incorporates complex subgraphs with both divergent and converging structures, which will be further explained in the following section.

3 Multi-hop QA Benchmark Creation

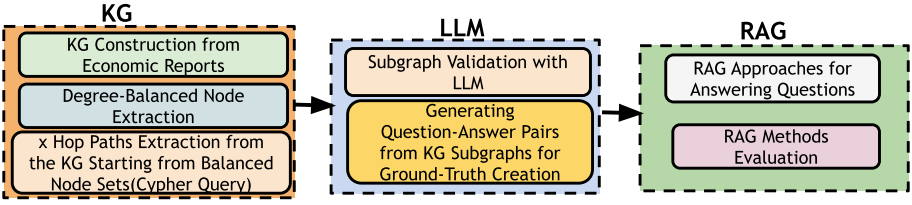


Fig. 1. Benchmark Creation Pipeline

3.1 Methodology

Figure 1 illustrates our pipeline, which consists of three main components: KG, LLM, and RAG. This pipeline is designed to be adaptable, allowing researchers

Table 1. Comparison with Existing Multihop RAG Benchmarks

Benchmark	Domain	KG Search	Descriptive Answers	Number of Hops
MultiHop-RAG [6]	General	✗	✗	4
CRAG [7]	General	✓	✗	2
MINTQA [2]	General	✗	✗	4
Our Approach	Economic	✓	✓	5

to construct RAG benchmarks in various domains. Unlike previous multi-hop benchmarks that primarily focus on binary (yes/no) answers or single-entity responses, our benchmark introduces descriptive answers, enhancing its comprehensiveness and applicability. Below, we provide a detailed description of each step within the components of our pipeline. Throughout the entire pipeline, we utilize GPT-4 as LLM.

KG Construction from Economic Reports. We first built the KG from 62 journals, reports, and economic bulletins; further details of the KG construction process can be found in [4]. The KG dataset is derived from multiple reputable sources, including the World Bank, the International Monetary Fund (IMF), and the European Central Bank.

Degree-Balanced Node Extraction. To ensure structural diversity in the benchmark, we selected starting nodes from the KG based on their degree, grouped into distinct ranges: nodes having degrees 1–9, 10–30, 31–100, and 101–200. From each group, representative nodes were sampled to initiate subgraph extraction. In addition, high-degree nodes such as Inflation, Euro Area, Governing Council, and Firms were deliberately included due to their centrality and relevance. This degree-balanced sampling strategy avoids over-reliance on highly connected nodes, enabling broader coverage of the KG topology and facilitating the generation of QA pairs that require reasoning across varied structural configurations.

X-Hop Paths Extraction from the KG Starting from Balanced Node Sets (Cypher Query). To enable multi-hop reasoning in a KG, it is essential to extract meaningful paths that connect relevant entities. This section focuses on retrieving structured subgraphs that serve as the foundation for question answering.

Types of Path-Based Reasoning in a Knowledge Graph
Path-based reasoning in multi-hop question answering involves following connections between entities in a KG to infer answers. These paths can have different structures depending on how entities are connected and how the information needs to be retrieved. We classify multi-hop paths based on graph structure. Subgraphs are extracted based on path reasoning structure stated in [1].

Linear Paths (Chain Reasoning). A linear path (chain reasoning) is the simplest type of multi-hop path. It follows a direct sequence of connected entities. The structure is A → B → C → D (Fig. 2).

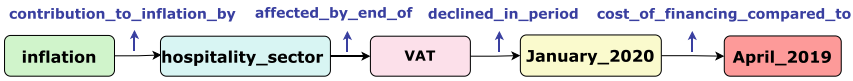


Fig. 2. Linear Subgraph

Converging Paths(Directed Acyclic Graph (DAG)). The structure is (A → B → C) (A → D → C) (Fig. 3).

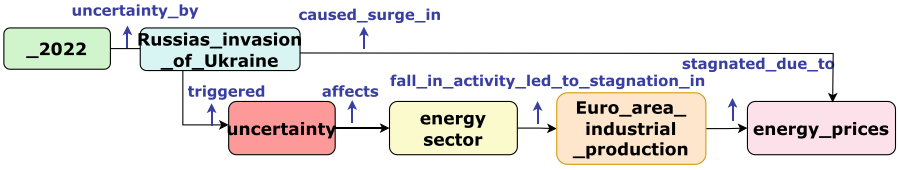


Fig. 3. Converging Subgraph

Divergent Paths (Polytree). The structure is (A → B),(A → C),(A → D) (Fig. 4 and Table 2).

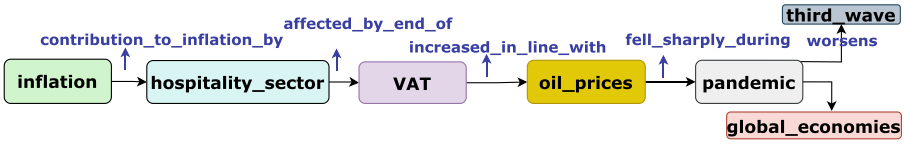


Fig. 4. Divergent Subgraph

Path Validation with LLM. To ensure that the retrieved reasoning path is logical and coherent, we employ LLMs for validation. Rather than evaluating each individual triple in isolation, we assess the entire path, ensuring that it maintains a central narrative flow. This approach allows us to verify that the path forms a meaningful sequence of knowledge transitions rather than just a collection of loosely connected facts. By leveraging LLM-based prompting, we systematically check whether the inferred multi-hop path aligns with human reasoning and preserves semantic consistency throughout. Figure 5 illustrates the prompt used for path validation. Among the converging paths, only 18.7% were found to be illogical, while 17.64% of the divergent paths exhibited similar issues. These relatively low rates of non-logical paths indicate the high structural and semantic quality of our constructed knowledge graph (Fig. 6).

Table 2. Some Examples from the Benchmark

Hops #	Subgraph(s)	Question	Answer
4	1-(energy price inflation)–[reflecting increase of inflation]→(HICP)–[type of inflation]→(euro area)–[price pressures remained subdued due to]→(significant economic slack) 2-(energy price inflation)-[reflecting increase of inflation]→(HICP)–[type of inflation]→(euro area)–[price pressures remained subdued due to]→(low wage pressures) Divergent	How did energy price inflation reflected in the HICP relate to the state of price pressures in the euro area?	Energy price inflation, as reflected in the Harmonised Index of Consumer Prices (HICP), indicated an increase in inflation. However, the price pressures in the euro area remained subdued due to significant economic slack and low wage pressures
6	1-(euro area)–[in line with]→(United Kingdom)–[uses approach]→(rental equivalence approach)–[used for]→(monetary policy purposes) 2-(euro area)–[implies]→(consumption impulse)–[not compared with]→(United States)–[uses approach for]→(monetary policy purposes) Converging	What is the approach used by both the Euro Area and the United Kingdom for monetary policy purposes that is not compared with the approach used by the United States?	Both the Euro Area and the United Kingdom use the rental equivalence approach for monetary policy purposes, which is not compared with the approach used by the United States

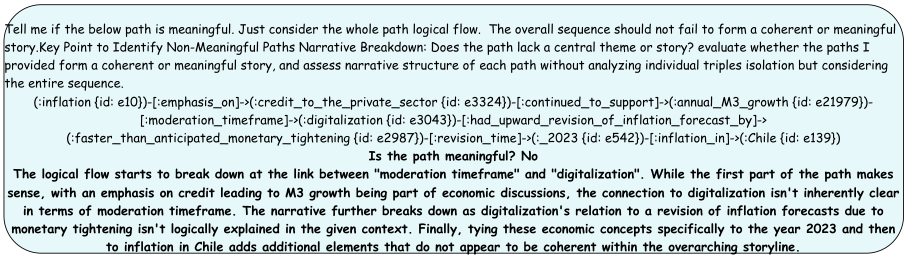


Fig. 5. Verify Subgraphs

Make a question from the below subgraph and answer only based on the subgraph provided.
Ensure you only use the subgraph info to produce the question and answer.
(:inflation {id: e10})-[:emphasis_on]->(:credit_to_the_private_sector {id: e3324})-
[:continued_to_support]->(:annual_M3_growth {id: e21979})-[:moderation_timeframe]->
(:digitalization {id: e3043})-[:had_upward_revision_of_inflation_forecast_by]->
(:faster_than_anticipated_monetary_tightening {id: e2987})-[:revision_time]->(:_2023 {id: e542})

Fig. 6. QA Generation from Subgraph

4 RAG Methods Evaluation

Overall, our dataset consists of 3,114 QA pairs derived from linear, converging, and divergent subgraphs, covering hop counts ranging from 4 to 7. We evaluate the benchmark across both the retrieval and generation phases while considering hop counts. For evaluation, we use subgraphs extracted from nodes across different degree ranges. We experiment with different subgraphs using Hybrid RAG, ReRank RAG and KG-RAG [5] to assess their effectiveness in multi-hop retrieval and answer generation.

4.1 Retrieval Evaluation

Retrieval performance is evaluated using three standard metrics: Mean Average Precision at K (MAP@K), Mean Reciprocal Rank at K (MRR@K), and Hit Rate at K (Hit@K). Table 3 reports retrieval performance across various RAG approaches on converging subgraphs, measured by Mean Reciprocal Rank (MRR), Mean Average Precision (MAP), and Hits@k across 4 to 7-hop settings. The Hybrid RAG method shows steadily improving retrieval performance with increasing hop count, suggesting that its combined dense and sparse strategy benefits from deeper subgraph exploration. In contrast, ReRank RAG achieves the highest MRR and Hits@k across nearly all hop levels, notably excelling at 5 and 7 hops (MRR = 0.75 and 0.5845; Hits@k = 0.75 and 0.82, respectively). This highlights the strength of re-ranking in prioritizing relevant paths. Notably, the baseline KG-RAG performs substantially worse across all metrics and hop depths, struggling particularly with deeper multi-hop paths (e.g., MAP = 0.0054 at hop 5). These results emphasize that converging reasoning structures, where multiple paths lead to a single target, challenge standard retrieval pipelines. The superior performance of ReRank RAG in this setting underlines the importance of post-retrieval relevance estimation in complex, multi-hop reasoning tasks.

Table 4 evaluates retrieval performance across divergent subgraphs, characterized by polytree structures where a central node branches into multiple directions. Among the methods compared, ReRank RAG performs best at hop count 4 with an MRR of 0.4742 and Hits@k of 0.5402, highlighting its ability to effectively prioritize relevant facts in shallow reasoning settings. However, unlike in converging scenarios, its performance declines for deeper paths, with both MRR and MAP dropping noticeably at hop counts 6 and 7. Hybrid RAG demonstrates more consistent behavior, with competitive performance across hops, peaking

at hop 6 with an MRR of 0.4247 and Hits@k of 0.4934. The KG-RAG baseline again lags behind, especially at lower hop counts (e.g., MAP@4 = 0.0110), though it shows marginal improvement with longer paths. Overall, these results reflect the greater difficulty divergent subgraphs pose for RAG systems due to their dispersed and less predictable information flow, emphasizing the need for more adaptive retrieval strategies in such settings. Figures 7 and 8 summarize how RAG retrieval performance varies with hop count across converging and divergent subgraph structures.

Table 3. Retrieval Performance – Converging Paths (Directed Acyclic Graph)

RAG Method	Hop	MRR@	MAP@	Hits@
Hybrid RAG	4	0.2166	0.0696	0.2866
	5	0.2976	0.0881	0.3571
	6	0.4391	0.1086	0.4828
	7	0.6860	0.1459	0.8152
ReRank RAG	4	0.3931	0.1529	0.6000
	5	0.7500	0.2178	0.7500
	6	0.5871	0.1403	0.7421
	7	0.5845	0.1081	0.8200
KG-RAG [5]	4	0.0438	0.0123	0.1111
	5	0.0271	0.0054	0.0455
	6	0.0470	0.0079	0.1496
	7	0.0946	0.0176	0.3333

Table 4. Retrieval Performance on Divergent Paths (Polytree)

RAG Method	Hop	MRR	MAP	Hit@
Hybrid RAG	4	0.3226	0.3226	0.3839
	5	0.2955	0.0794	0.3627
	6	0.4247	0.1194	0.4934
	7	0.3698	0.0841	0.5000
ReRank RAG	4	0.4742	0.1981	0.5402
	5	0.3180	0.1074	0.3678
	6	0.2862	0.0792	0.3448
	7	0.2862	0.0792	0.3448
KG-RAG [5]	4	0.0431	0.0110	0.1154
	5	0.0407	0.0088	0.1242
	6	0.1885	0.0337	0.3111
	7	0.1420	0.0203	0.1875

4.2 Generation Evaluation

Answer generation quality is evaluated using BLEU (n-gram lexical overlap), ROUGE (recall-based substring overlap), and Semantic Similarity (cosine similarity of embeddings to assess conceptual alignment). The results in Tables 5 and 6 offer insights into the generation performance of various RAG systems over both converging and divergent paths. For converging subgraphs, both Hybrid RAG and ReRank RAG exhibit high semantic consistency, with average semantic similarity scores exceeding 0.83 across all hop counts. However, BLEU scores and ROUGE-L F-measures decline as hop count increases, indicating that while the models capture the underlying meaning of the subgraph content, their lexical overlap with the reference answers diminishes. Notably, Hybrid RAG shows stronger semantic preservation at higher hops compared to KG-RAG, although KG-RAG leads in BLEU and ROUGE-L at hop 4. This suggests that KG-RAG may produce more surface-similar responses on simpler inputs, while Hybrid and ReRank models generalize better semantically in deeper reasoning chains.

In the case of divergent subgraphs, the trend is similar. KG-RAG achieves higher BLEU and ROUGE-L scores at lower hops, particularly at hop 5, but its semantic similarity is generally comparable to or slightly lower than the proposed methods. ReRank RAG and Hybrid RAG maintain strong semantic alignment but experience performance degradation in both lexical and semantic metrics as the number of hops increases. This performance drop reflects the increasing difficulty of generating coherent answers when reasoning over complex, branching structures.

Together, these results confirm that while KG-RAG is effective in producing fluent outputs for simpler paths, it underperforms in semantically grounded generation for deeper reasoning tasks. In contrast, the proposed methods demonstrate more robust handling of semantic consistency, supporting their effectiveness for multi-hop question answering in knowledge-intensive domains.

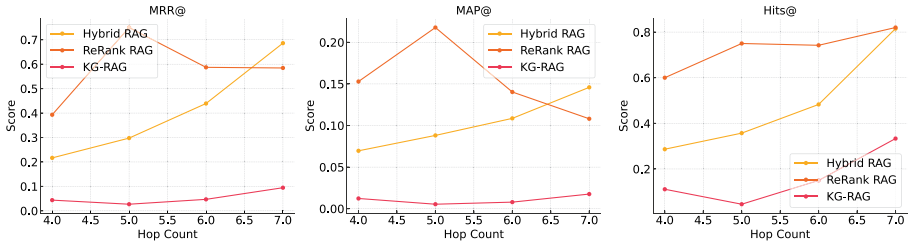


Fig. 7. Retrieval Performance- Converging Paths

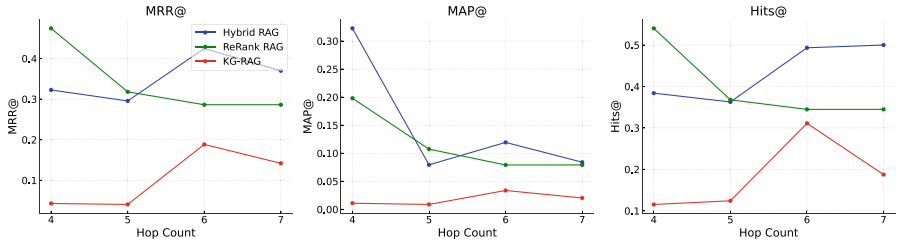


Fig. 8. Retrieval Performance- Divergent Paths

Table 5. Evaluation Metrics – Generation on Converging Paths (DAG)

RAG Method	Hop	BLEU	Sem. Sim.	ROUGE-L
Hybrid RAG	4	0.3266	0.8783	0.5124
	5	0.2001	0.8351	0.3993
	6	0.1740	0.8424	0.3566
	7	0.0776	0.8340	0.2620
ReRank RAG	4	0.3126	0.8891	0.5105
	5	0.1134	0.7641	0.2940
	6	0.1340	0.8330	0.3162
	7	0.0626	0.8287	0.2295
KG-RAG [5]	4	0.3825	0.8334	0.5971
	5	0.1981	0.8142	0.4341
	6	0.1730	0.8052	0.4050
	7	0.1313	0.8822	0.3669

Table 6. Generation Performance – Divergent Paths (Polytree)

RAG Method	Hop	BLEU	SemSim	ROUGE-L
Hybrid RAG	4	0.2114	0.8397	0.3943
	5	0.2462	0.8477	0.4517
	6	0.0966	0.8063	0.2667
ReRank RAG	4	0.2073	0.8353	0.3866
	5	0.2388	0.8432	0.4386
	6	0.0899	0.8070	0.2545
	7	0.0765	0.8370	0.2498
KG-RAG [5]	4	0.3170	0.8562	0.5435
	5	0.3591	0.8886	0.5893
	6	0.1878	0.8681	0.4238
	7	0.1532	0.8891	0.3905

5 Discussion and Conclusion

In this work, we introduced EcoRAG, a multi-hop economic QA benchmark designed for RAG using KGs. Unlike previous benchmarks, EcoRAG extends reasoning depth to seven-hop retrieval, incorporates complex subgraph structures, and emphasizes domain-specific economic knowledge, enabling a more comprehensive evaluation of multi-hop retrieval and generation.

By providing structured, interconnected knowledge, EcoRAG supports a more realistic assessment of how RAG models handle long-range dependencies and complex reasoning. The benchmark offers a reusable framework for evaluating retrieval performance, answer generation quality, and the integration of structured knowledge with generative models.

Despite recent progress, current RAG methods still struggle with effectively retrieving and synthesizing multi-hop information. As future work, we plan to develop improved RAG techniques specifically tailored to handle complex multi-hop question answering.

Acknowledgments. This work is funded by the German Research Foundation under the project number 490998901.

References

1. Bai, J., Lee, K.F., Hofmeister, M., Mosbach, S., Akroyd, J., Kraft, M.: A derived information framework for a dynamic knowledge graph and its application to smart cities. Future Gener. Comput. Syst. **152**, 112–126 (2024)
2. He, J., Hu, N., Long, W., Chen, J., Pan, J.Z.: Mintqa: A multi-hop question answering benchmark for evaluating llms on new and tail knowledge. arXiv preprint arXiv:2412.17032 (2024)
3. Khorashadizadeh, H., et al.: Research trends for the interplay between large language models and knowledge graphs. In: VLDB 2024 Workshop: The International Workshop on Data Management Opportunities in Unifying Large Language Models + Knowledge Graphs (LLM+KG), Guangzhou (2024). https://vldb.org/workshops/2024/proceedings/LLM+KG/LLM+KG-9.pdf
4. Khorashadizadeh, H., et al.: Construction and canonicalization of economic knowledge graphs with llms. In: Tiwari, S., Villazón-Terrazas, B., Ortiz-Rodríguez, F., Sahri, S. (eds.) Knowledge Graphs and Semantic Web, pp. 334–343. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-81221-7_23
5. Sanmartin, D.: Kg-rag: bridging the gap between knowledge and creativity. arXiv preprint arXiv:2405.12035 (2024)
6. Tang, Y., Yang, Y.: Multihop-rag: benchmarking retrieval-augmented generation for multi-hop queries. arXiv preprint arXiv:2401.15391 (2024)
7. Yang, X., et al.: Crag-comprehensive rag benchmark. Adv. Neural. Inf. Process. Syst. **37**, 10470–10490 (2024)

Short Papers (Industry Track)

Should Embedding-Based News Recommendation be Revisited? A Focus on the Differences Between News Publishers and Aggregators

Takumi Tamura[1(✉)], Yoichiro Ito[2], Masaki Aota[2], Kenta Yamada[2], and Shotaro Ishihara[2]

[1] University of Tsukuba, 1-2, Kasuga, Tsukuba-shi, Ibaraki 305-8550, Japan
tamura.takumi.ap@alumni.tsukuba.ac.jp
[2] Nikkei Inc., 1-3-7, Otemachi, Chiyoda-ku, Tokyo 100-8066, Japan
{yoichiro.ito,masaki.aota,kenta.yamada,shotaro.ishihara}@nex.nikkei.com

Abstract. Personalized news recommendation is crucial for engaging users on news websites. In this domain, there are numerous specific content-based methods that employ the profiles of news articles. Among these, embedding-based approaches (Embedding-Based News Recommendation; EBNR) are widely recognized for their efficacy. However, their performance for news publishers remains unclear, as prior EBNR research has primarily focused on news aggregators. News publishers and aggregators are generally distinct in the following two points: (1) news publishers' ability to leverage richer metadata, and (2) the narrower topic range of some news publishers that focus on specific affairs compared to news aggregators. This paper aims to provide practical insights into how these differences affect news recommendation. Through an online experiment with the Nikkei, Japan's leading financial news company, we reported that basic EBNR is not superior to simple content-based recommendation systems that directly utilize metadata as news profiles, and further analysis using open datasets shows that the narrowness of the topics covered by each news website affects the diversity of embeddings. Overall, our findings suggest that characteristics of news publishers impact EBNR performance, highlighting the need for future research on methods to mitigate these effects.

Keywords: News Websites · Distributed Representations · Newspapers

1 Introduction

With the increasing popularity of online news consumption, the significance of personalized news recommendation, tailored to individual user interests and

T. Tamura—Work done during an internship at Nikkei Inc.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 177–188, 2026.
https://doi.org/10.1007/978-3-031-97144-0_16

preferences, has also grown on news websites. Content-based recommendation systems are a prevalent method in the domain of news recommendation, as they are capable of recommending the latest news articles to users in real time [12]. These systems represent users and news articles as profiles and recommend news articles based on the similarity of both user and news profiles. The methods of calculating the profiles vary, including rule-based matching and deep learning techniques [18].

Among various content-based methods for news recommendation, embedding-based news recommendation (EBNR) [12] and its derivatives are widely recognized for their efficacy [18]. They are characterized by transforming users and candidate news articles into embeddings (vector representations) using word or sentence embedding technologies such as BERT (Bidirectional Encoder Representations from Transformers). News articles are recommended based on the similarity between the user and candidate news embeddings. EBNR's effectiveness has been evaluated by several news websites, and its derivatives have been proposed in various research papers [15–17].

We argue that previous studies about EBNR have primarily focused on websites of news aggregators, such as MSN (the MIND dataset [19]) and Yahoo! Japan [12]. News aggregators distribute news articles from various news publishers [5,10], such as The Washington Post, The New York Times, and The Guardian. On the other hand, most news publishers own their news websites or apps, which provide only their original news articles.

There are distinct differences between news publishers and aggregators [5], and the influence of the differences from the perspective of personalized news recommendation including the effectiveness of EBNR is still not clear. We assume the following key differences in terms of **Metadata Utilization** and **Variety of Topics**.

Metadata Utilization News publishers can leverage rich metadata while news aggregators often face challenges. News publishers own all news articles, including news metadata such as editor-supplied genres or keywords. In contrast, news aggregators generally have difficulty obtaining this news metadata. For example, Gulli [10] describes the anatomy of news aggregators and points out that automated news classifiers and clustering engines are necessary elements.

Variety of Topics News publishers that focus on specific affairs often have a narrower range of topics, whereas news aggregators cover a wide variety of topics. A previous study reported that users of news aggregators read more diverse news articles than of news publishers [6]. This difference in topic variety is likely to impact the diversity of the embeddings. If it is true, it should deteriorate the discriminability of each news article affecting the performance of EBNR [16].

Therefore, this paper addresses the following two research questions.

(RQ1) Are embedding-based approaches effective for news publishers compared to recommendation systems that utilize metadata directly as news profiles?

(RQ2) Do the differences in the variety of topics covered by each news website affect the diversity of news embeddings?

We conducted an online experiment (Sect. 3) and open data analysis (Sect. 4) to address the RQs. To address RQ1, an online experiment was conducted with the Japanese news publisher Nikkei[1], which is the largest financial newspaper in Japan. The experimental results show that the EBNR model did not outperform other simple content-based recommendation systems using news metadata directly as news profiles. In addition, we analyzed an open news dataset to address RQ2. Using the NOW (News on the Web) corpus [1], the five news websites including news aggregators and publishers were compared in terms of the relationship between the variety of topics and the diversity of embeddings. The results show that the diversity of embeddings is different depending on the characteristics of each news website, and suggesting that this difference affects the performance of EBNR in news publishers.

Overall, we point out that EBNR should be revisited for news publishers in terms of the differences between news publishers and aggregators. This paper highlights the importance of considering the characteristics of the target news publisher when designing EBNR on their websites or apps, and we acknowledge that methods to mitigate these effects of the differences, such as hybrid approaches, feature engineering, and model customization, need to be developed in future research.

Limitations. Although this paper aims to provide practical insights into the use of EBNR in news publishers, there are still some future works. Additional online experiments in other news publishers are needed to enhance the generalizability of our findings, and also different types of user behavior including the long-term effects should be observed. Due to proprietary information and data privacy concerns, a detailed description of the specific method, dataset, or code is not possible.

2 Related Works

News Publishers and Aggregators. We classify news websites into two categories: news publishers and aggregators. News publishers create and distribute original news articles to users. Conversely, news aggregators curate news articles collected from various news publishers, rather than making their own news article [5,10].

There are distinct differences between news publishers and aggregators, such as metadata utilization [10] and topic variety [6]. Several studies have investigated the interrelationships between news publishers and aggregators focusing on users' news consumption [3,4].

However, no prior research, to our knowledge, has examined the differences between news publishers and aggregators from the perspective of personalized

[1] https://www.nikkei.com/.

news recommendation. Given the potential impact of these differences on metadata utilization and topic variety, it is likely that they significantly influence news recommendation.

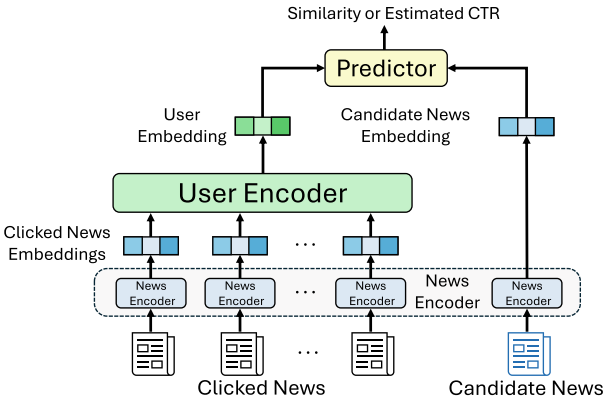


Fig. 1. General framework of embedding-based news recommendation (EBNR)

Embedding-Based News Recommendation (EBNR). Okura et al. [12] introduced EBNR to address item cold-start and performance challenges in recommending news to millions of users. They evaluated EBNR in a real-world deployment on Yahoo! Japan, a leading website as a news aggregator in Japan.

EBNR falls under the category of content-based recommendation systems and aims to predict the similarity score between users and candidate news articles. Figure 1 shows the general framework of EBNR. It consists of three main components: (1) **News encoder**, (2) **User encoder**, and (3) **Predictor**.

News encoders transform news content into news embeddings. These encoders typically leverage natural language processing (NLP) techniques, such as word embeddings [12]. Wu et al. [16] proposed using pre-trained language models like BERT as news encoders to achieve better recommendation performance.

User encoders generate user embeddings based on their clicked news (e.g., browsing history or session activities). They take user-clicked news embeddings from the news encoder as input and output a single embedding representing the target user. These encoders are often implemented using deep learning techniques such as GRU (Gated Recurrent Unit) [12], and other deep models [15,17].

Predictors calculate the similarity (or the click-through rate) between the user embedding from the user encoder and the candidate news embedding from the news encoder. For fast real-time recommendation, simple algorithms like the dot product are commonly used as predictors [12,15]. The similarities from the predictor are used for creating the ranking in the candidate news of the target user, and the top-k news articles are recommended [12,18].

There are several extensions of EBNR [18]. For instance, Wu et al. [17] extended the model's input to include not only user browsing history but also other user activities (e.g., likes, shares). Further research examined incorporating LLMs (Large Language Models) such as LLaMA or GPT3.5 into EBNR [11].

However, these studies do not focus on the differences between news publishers and aggregators. Many studies evaluated their EBNR models using benchmark datasets obtained from news aggregator access logs, such as the MIND dataset [19]. Although some studies used the dataset from news publishers, such as the Adressa dataset [8] based on the website of Norwegian newspapers Adresseavisen or the dataset from the Brazilian news website Globo [14], they also do not consider these difference. In this paper, we revisit EBNR on the point of the differences between news publishers and aggregators.

News Recommendation in News Publishers. Limited case studies on news recommendation for news publishers exist. Gulla et al. [9] examined the news recommendation processes of Scandinavian traditional media outlets. Additionally, Bloomberg [13] and The Washington Post [7] have published case studies detailing their news recommendation systems. These studies share similarities with our work; however, our research questions are distinct because they did not use EBNR, and the differences between news aggregators and publishers were not considered.

3 Online Experiment: Metadata Utilization

In this section, we describe our online experiment conducted on a mobile news app to address RQ1. The EBNR was compared to the simple content-based recommendation systems which they utilize news metadata directly as news profiles.

3.1 Environmental Setup

We conducted online experiments targeting the Nikkei. It is the largest financial newspaper in Japan, and it maintains a large-scale news website including mobile apps.

The online evaluation was conducted on the Nikkei Wave app, which is designed to explore the impact of innovative technologies by offering users access to experimental features. This app is available to paid subscribers of the Nikkei. Figure 2 illustrates an example of its interface. This app functions as a news timeline, displaying news articles chronologically based on the publication time of them. News articles are provided through various news recommendation systems, including automated recommendations alongside semi-automated options like "Editor's Pick." The experimental period spanned from November 25, 2023, to December 15, 2023, excluding weekends (because users' behavior in the financial newspaper Nikkei was distinct between workdays and weekends). A total of 2,632 users were randomly selected as the target users.

To assess the performance of each recommendation system, we calculated its CTR (click-through rate). CTR was defined as the ratio of the number of clicks to the total number of recommended news articles displayed. The CTR was first calculated for each individual user and then averaged across all users to obtain an overall CTR metric for each recommendation system.

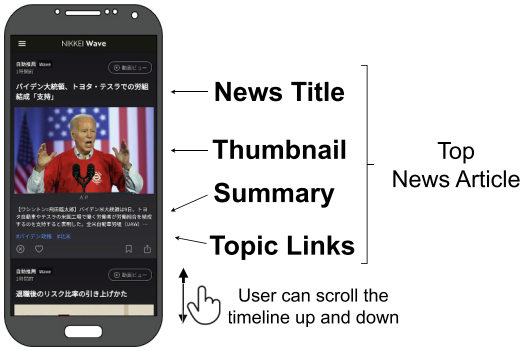


Fig. 2. Example of the timeline in the Nikkei Wave app. The topics are part of the news metadata (described in Sect. 3.2).

3.2 Recommendation Systems

In this experiment, we compared four content-based recommendation systems: (1) EBNR (2) Keywords and Genres Matching on Browsing History (KGM-BH) (3) Topics Matching based on Browsing History (TM-BH) (4) Topics Matching based on User Grouping (TM-UG). The latter three systems are rule-based recommendation systems that directly utilize news metadata as news profiles.

We utilized three types of news metadata in the recommendation systems: keywords, genres, and topics. Genres were the editor-provided broadest categories used to classify news articles (e.g., "politics," "markets"). Keywords were terms associated with the news articles (e.g., "The White House," "bank rate"), automatically extracted from the article text using a comprehensive keyword dictionary. This dictionary was regularly updated by human experts to ensure it reflected the latest terminology and trends. Finally, topics were editor-provided groupings broader than keywords but narrower than genres (e.g., "U.S. presidential election," "financial policy").

(1) EBNR model was designed for the Nikkei with reference to previous papers. It consists of three components: News Encoder, User Encoder, and Predictor (like Fig. 1).

The pre-trained BERT model[2] was used for news encoder similar to Wu et al. [16]. We made two types of news embeddings. Title and body embeddings

[2] https://huggingface.co/tohoku-nlp/bert-base-japanese-whole-word-masking.

were generated from the title and body up to 512 tokens[3] as vectors with 768 dimensions. We also generate metadata embeddings as vectors with 768 dimensions from the keywords and genres of the news articles. The keywords and genres were combined using `[SEP]` as a delimiter, for example: `Markets [SEP] Stock prices [SEP] ... [SEP] bank rate`, and that was embedded. Finally, the news encoder concatenated two embeddings as news embeddings. It should be noted that we did not finetune the BERT model to reduce the computational costs in training.

Similar to Okura et al. [12], we employed a simple GRU model for the user encoder. It generated user embeddings as vectors with the same dimensions as news embeddings from a maximum of 128 news embeddings, which were browsed by the user. The predictor was implemented by dot products, which is the simple well-known method [12,15].

The model was trained using access logs from the Nikkei's mobile apps, which included 20,342 users with paid subscriptions to the apps in August 2023. We verified its training success using the offline test dataset compared to the baseline called "Average" in Okura et al. [12]. This means that the average of clicked news embeddings is used instead of the user encoder.

We used the loss function based on the softmax function with reference to Wu et al. [15]. To mitigate popularity bias, we made some empirical changes to the function. Including the parameters of this change, all hyperparameters were tuned on the offline validation dataset. In the online experiments, the user embeddings were calculated daily based on their historical interactions.

(2) KGM-BH calculated the co-occurrence of keywords and genres (same as with EBNR) from the user's browsing history and recommended articles containing genre-keyword pairs frequently viewed by the user.

(3) TM-BH analyzed the frequency of topics in the user's browsing history and recommended articles associated with frequently viewed topics.

(4) TM-UG aimed to address user cold-start problems of the TM-BH and divided users into 12 groups based on occupation and age. It recommended articles based on the most frequently viewed topics within each user group.

3.3 Results and Discussion

Table 1 summarizes the results of the online evaluation. It is evident from the table that the EBNR model exhibits the lowest CTR among the evaluated recommendation systems. Note that our additional analyses confirmed that those results were not affected by biases such as exposure bias or position bias.

To further investigate this observation, we conducted a Dunnett's two-sided test, using the EBNR model as the control group and the remaining recommendation systems as the experimental groups. The results revealed that only the TM-BH method demonstrated a statistically significant difference in CTR

[3] Since most news articles in the Nikkei include a summary of the article at the beginning, there must be no deterioration by cutting off the longer texts.

Table 1. CTR of each recommendation system. Standard Error (SE), Variance, and p-values of Dunnett's test are also shown.

Recommendation systems	CTR±SE	Variance	p-value
EBNR	0.077 ± 0.0053	0.026	(control)
KGM-BH	0.080 ± 0.0059	0.030	0.999
TM-BH	0.153 ± 0.0156	0.087	$2.58 \cdot 10^{-8}$
TM-UG	0.100 ± 0.0099	0.059	0.141

compared to the EBNR model ($p < 0.05$). All other methods failed to achieve statistical significance ($p > 0.05$).

The results show that simple content-based recommendation systems, which utilize news metadata as news profile directly, perform well, suggesting that metadata utilization is an important factor for personalized news recommendations for news publishers. KGM-BH and TM-BH are similar in structure to EBNR, as they are content-based recommendation systems using the user's browsing history. The difference lies in the way news profiles are represented (metadata or embeddings). In particular, TM-BH, which was the best performing method, uses topic metadata. These topics are manually annotated by editors. This suggests that use of higher-quality metadata is important.

News metadata is commonly available at news publishers, but news aggregators may have difficulty accessing it. This is because they sometimes collect news articles through methods that do not involve receiving content directly from publishers, such as web scraping [10].

The effectiveness of EBNR in news publishers should be carefully reconsidered regarding metadata utilization. In news aggregators, the difficulty in obtaining rich metadata gives EBNR an advantage, conversely, the advantage should not be valid in news publishers.

4 Open Data Analysis: Variety of Topics

In this section, we investigated whether the differences in the variety of topics covered by each news website affect the diversity of news embeddings (RQ2). EBNR represents users and news articles as embeddings. It is generally assumed that these embeddings capture the topics of news articles or user interests [18].

The variety of topics covered by some news publishers is often narrower [6], and this may deteriorate the diversity of their embeddings. Lower diversity should affect the discriminability of each news embedding. Wu et al. [16] found the performance of EBNR is related to the discriminability of news embeddings through the t-SNE visualization of news embeddings, in other words, EBNR performs well when news embeddings form clear clusters depending on their topics. Therefore, the diversity of news embeddings, which may reflect the variety of topics covered by news websites, is likely to impact EBNR's performance. This

is expected to be particularly pronounced for news publishers specializing in specific news categories, such as financial newspapers.

We used an open dataset to embed English news articles from five different news websites, including aggregators, and compared the diversity of embeddings[4]. If the variety of topics on news websites is related to the diversity of embeddings, the difference should affect the behavior of EBNR.

4.1 Experimental Setup

An additional analysis was conducted using the publicly available the NOW corpus [1]. This dataset contains news titles and bodies from various English news websites. We selected one news aggregator (Yahoo!) and four news publishers in the NOW corpus, extracted 1,000 articles published in 2021 from them by random sampling, and encoded the news embeddings using the BERT model[5].

In this experiment, the following two diversity metrics were calculated. For both metrics, the lower the value, the more diverse the news website.

Average Similarity Average cosine similarity across all pairs, consisting of any two news embeddings in the news website.

GS Score originally proposed to classify music listeners as Generalists or Specialists as an metric utilizing cosine similarity [2], was adapted to assess the topic variety of news websites.

Table 2. Diversity metrics of each news website

News website		Average Similarity	GS Score
News Aggregator	Yahoo!	0.739	0.860
	(Yahoo! News)	0.748	0.865
	(Yahoo! Finance)	0.755	0.869
	(Yahoo! Sports)	0.748	0.865
News Publisher	New York Times	0.751	0.867
	Washington Post	0.742	0.862
	USA Today	0.723	0.850
	Reuters	0.751	0.867
All News Articles		0.727	0.853

[4] The Nikkei news articles in the online experiment (Sect. 3) are not adapted for a fair comparison of the diversity and those in the English news dataset, because it is written in Japanese and focused mainly on Japanese domestic news.

[5] https://huggingface.co/google-bert/bert-base-uncased.

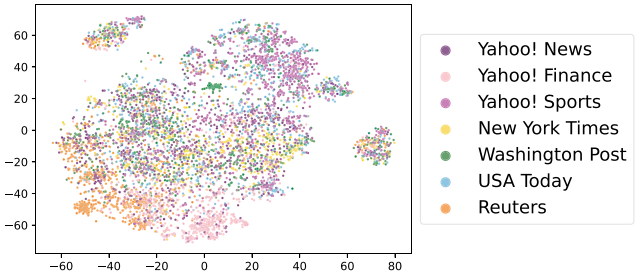


Fig. 3. t-SNE visualization of news embeddings from each news website, which illustrates that the clusters of each news website had been formed by data points.

4.2 Results and Discussion

Table 2 summarizes the diversity metrics calculated for each news website. Figure 3 also presents the visualization of the news embedding with t-SNE for supplementary information. From the figure, it can be discerned that the clusters of each news website had been formed by data points derived from embedded articles, but the data points belonging to some websites seem to spread widely and cover clusters of other websites.

Our results support the hypothesis that the topical variety of news publishers influences the diversity of news embeddings. The results in Table 2 indicate that the Reuters and the New York Times exhibit the highest average similarity and GS score, reflecting their narrower range of topics. Conversely, Yahoo! (news aggregator) displays lower scores.

However, some news publishers deviate from this trend. For example, the USA Today is the most diverse news website in Table 2. This is because the range of topics covered by news publishers can vary based on their specific characteristics. Some news publishers offer a broad range of topics, while others focus on a narrower domain. For instance, the USA Today categorizes its news on the front page under headings to "U.S.," "Elections," "Sports," "Entertainment," "Life," "Money," "Tech," "Travel," and "Opinion." In contrast, the Reuters categorizes its content under "World," "Business," "Markets," "Sustainability," "Legal," "Breakingviews," "Technology," and "Investigations."

These findings show that while exceptions exist, such as USA Today, the diversity of embeddings should deteriorate for news publishers with a limited range of topics. Wu et al. [16] point out that the performance of EBNR depends on the efficiency with which the model discriminates news embeddings based on the semantic information in the news articles. Considering this and our results, the lower diversity in news publishers suggests a disadvantage of EBNR in them.

5 Conclusion

This paper investigates the effectiveness of EBNR on news publishers' own websites or apps from a practical perspective. While EBNR is widely recognized its

efficacy in personalized news recommendation, no prior research has focused on the differences between news publishers and aggregators. Our results from the online experiment with Nikkei Inc. describe that the performance of EBNR in news publishers should be reconsidered in terms of richer metadata utilization, and our open data analysis suggests that narrower topic distributions within news publisher content influence the diversity of news embeddings. In conclusion, this paper points out that EBNR should be revisited when considering the characteristics of news publishers. We strongly recognize the need for future research to develop the EBNR extensions that mitigate the influences of the differences between news publishers and aggregators, such as metadata utilization and generating higher-quality embeddings.

Acknowledgments. We thank the developers of the Nikkei Wave app for their support.

References

1. NOW Corpus (News on the Web). https://www.english-corpora.org/now/
2. Anderson, A., et al.: Algorithmic effects on the diversity of consumption on Spotify. In: Proceedings of the Web Conference 2020, pp. 2155–2165 (2020)
3. Athey, S., Mobius, M., Pal, J.: The impact of aggregators on internet news consumption. Working Paper 28746, National Bureau of Economic Research (2021)
4. Calzada, J., Gil, R.: What do news aggregators do? Evidence from google news in Spain and Germany. Mark. Sci. **39**(1), 134–167 (2020)
5. Chowdhury, S., Landoni, M.: News aggregator services: user expectations and experience. Online Inf. Rev. **30**(2), 100–115 (2006)
6. Fletcher, R., Kalogeropoulos, A., Nielsen, R.K.: More diverse, more politically varied: How social media, search engines and aggregators shape news repertoires in the United Kingdom. New Media Soc. **25**(8), 2118–2139 (2023)
7. Graff, R.: How the Washington Post used data and natural language processing to get people to read more news (2015). https://knightlab.northwestern.edu/2015/06/03/how-the-washington-posts-clavis-tool-helps-to-make-news-personal/
8. Gulla, J., et al.: The Adressa dataset for news recommendation. In: Proceedings of the International Conference on Web Intelligence, pp. 1042–1048 (2017)
9. Gulla, J., et al.: Recommending news in traditional media companies. AI Mag. **42**(3), 55–69 (2021)
10. Gulli, A.: The anatomy of a news search engine. In: Special Interest Tracks and Posters of the 14th International Conference on World Wide Web, pp. 880–881 (2005)
11. Liu, Q., et al.: ONCE: Boosting content-based recommendation with both open- and closed-source large language models. In: Proceedings of the 17th ACM International Conference on Web Search and Data Mining, pp. 452–461 (2024)
12. Okura, S., et al.: Embedding-based news recommendation for millions of users. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 1933–1942 (2017)
13. Shah, D., et al.: News recommendations at scale at Bloomberg media: Challenges and approaches. In: Proceedings of the 10th ACM Conference on Recommender Systems, p. 369 (2016)

14. de Souza Pereira Moreira, G., Ferreira, F., da Cunha, A.M.: News session-based recommendations using deep neural networks. In: Proceedings of the 3rd Workshop on Deep Learning for Recommender Systems, pp. 15–23 (2018)
15. Wu, C., et al.: Neural news recommendation with multi-head self-attention. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pp. 6389–6394 (2019)
16. Wu, C., et al.: Empowering news recommendation with pre-trained language models. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 1652–1656 (2021)
17. Wu, C., et al.: FeedRec: news feed recommendation with various user feedbacks. In: Proceedings of the ACM Web Conference 2022, pp. 2088–2097 (2022)
18. Wu, C., et al.: Personalized news recommendation: methods and challenges. ACM Trans. Inf. Syst. **41**(1) (2023)
19. Wu, F., et al.: MIND: a large-scale dataset for news recommendation. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 3597–3606 (2020)

A Simple Ensemble Strategy for LLM Inference: Towards More Stable Text Classification

Junichiro Niimi[1,2(✉)]

[1] Meijo University, Nagoya, Aichi 4688502, Japan
jniimi@meijo-u.ac.jp
[2] RIKEN AIP, 1-4-1 Nihonbashi, Chuo-ku, Tokyo 1030027, Japan

Abstract. With the advance of large language models (LLMs), LLMs have been utilized for the various tasks. However, the issues of variability and reproducibility of results from each trial of LLMs have been largely overlooked in existing literature while actual human annotation uses majority voting to resolve disagreements among annotators. Therefore, this study introduces the straightforward ensemble strategy to a sentiment analysis using LLMs. As the results, we demonstrate that the ensemble of multiple inference using medium-sized LLMs produces more robust and accurate results than using a large model with a single attempt with reducing RMSE by 18.6%.

Keywords: large language models · natural language processing · sentiment analysis · marketing · electronic word-of-mouth

1 Introduction

In the field of marketing, accurate comprehension of customer's loyalty and preference is crucial as a customer relationship management (CRM) [29,33]. In particular, as consumers increasingly post opinions on social media and review platform, user-generated contents (UGCs) has become essential resource for market research [17]. Textual data have been utilized for company's various decision-making, such as product evaluation, feature extraction, and recommendation systems [3,21,25,34,45].

To extract, utilize, and understand consumer preferences from textual data, pre-processing through assigning labels is essential; however, these are labor-intensive tasks for humans. Manual labelling such as Amazon Mechanical Turk (MTurk) is costly, while traditional natural language processing (NLP) methods require specialized skills. In addition, data quality of crowdsourcing remains a serious concern [4,12]. Thus, handling big data becomes more challenging and often impractical despite the large amount of accumulated data.

Codes are available at GitHub: https://github.com/jniimi/ensemble_inference.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 189–199, 2026.
https://doi.org/10.1007/978-3-031-97144-0_17

With the advance of large language models (LLMs), several studies have proposed automated annotation models using LLMs [9,15,31]. Compared to human annotators, LLMs have highlighted advantages in processing speed, cost efficiency, and applicability across the wide range of tasks. However, the instability of LLM outputs has also been pointed out [31], which can lead to inconsistency of predictions. We therefore facilitate this variability through an ensemble approach by aggregating multiple inferences to acquire both stability and accuracy.

Therefore, this study proposes a sentiment analysis model that employs ensemble techniques, which not only stabilizes the classification but also reduces the effect of outlier generation in LLMs. The remainder of this paper is organized as follows. In Sect. 2, we review related works to contextualize this study. In Sect. 3, we introduce our proposed model, followed by an overview and results of the analysis in Sect. 4. Finally, we discuss the implications and challenges of this study in Sect. 5.

2 Related Work

First of all, sentiment analysis methods can be broadly classified into four groups: rule-based models (e.g., VADER [19]), machine learning [37,39], deep neural networks (DNNs) [10,43], and LLMs [22]. While many studies have shifted towards automated sentiment annotation using LLMs, as mentioned, the diversity in the responses [31] arises due to the nature of LLM inference, such as probabilistic token generation, prompt sensitivity, and inherent randomness. For business applications, reproducibility and consistency are crucial. In human annotation, multiple annotators are used to resolve discrepancies via majority voting, a concept that can be extended to LLM-based sentiment analysis.

Ensemble learning is typically achieved by combining multiple weak learners with diverse outputs to reduce the error rate and to generalize results [8]. One of the approaches of ensemble is voting, such as majority voting and weighted voting [16], which constructs robust predictors by combining multiple classifiers [8]. Ensemble learning has been applied to machine learning [32] and its utility also has been confirmed in the wide range of domains, including sentiment analysis [8,11,35,44]. Majority voting is utilized particularly for multinomial classification, in which each model votes for one class, and the class with the most votes is considered as the final predictor [44]. Given that LLMs generate probabilistic variations in their responses, ensemble learning can be realized by performing multiple inference iterations.

Despite the increasing use of LLMs, few studies have explored ensemble techniques to stabilize predictions. Most prior works have focused on single-run LLM annotations [27]. While a few studies [18,41] have applied ensemble-like techniques to LLM inference by assigning different characteristics to each LLM, they often rely on hierarchical models that vertically stacks LLMs, where multiple inferences are fed into another LLM for final decision-making. This approach, however, further increases the black-box nature of the model, which makes it difficult to interpret how individual predictions contribute to the final output.

Consequently, little research has systematically analyzed the effect of considering the repeated inferences in LLM-based sentiment analysis.

Therefore, this study addresses the gap by introducing a straightforward ensemble strategy to enhance stability and accuracy of the inference. We develop LLM sentiment analysis model which generates multiple workers inside the model and each worker votes the sentiment to generate more robust results.

3 Methodology

3.1 Pre-trained Models

Table 1 summarizes the models in this study: Meta AI's Llama family [13]. We adopt 70 billion (70B) and 8 billion (8B) models for Llama 3 and 7 billion (7B) model for Llama 2. Llama is trained with more than 15 trillion tokens from publicly available data sources. Furthermore, Meta's instruction-tuning [40] process of the model includes more than 10,000 manually annotated data in addition to instructional data, as well as reinforcement learning from human feedback (RLHF) [5]. Considering these learning processes, sufficient accuracy is expected for sentiment analysis of eWOM. Thus, no additional fine-tuning is performed in this study.

Table 1. Employed pre-trained models

model id	version	scale	precision	format
meta-llama/Llama-3-70B-Instruct	3	70B	4-bit	GGUF
meta-llama/Llama-3-8B-Instruct	3	8B	4-bit	GGUF
meta-llama/Llama-2-7B-Chat	2	7B	4-bit	GGUF

The model id represents the corresponding model identifier on Hugging Face (https://huggingface.co/models).

We limit our investigation to the Llama family. Even comparing within the Llama family, we can empirically investigate the effect of the model configuration, such as the model scale (i.e., number of parameters in the model; 7–70B) and version (version 2 and 3). To reduce the computational burden, this study quantizes each model in 4-bit with GGUF format [14] which is widely adopted in LLM community.

3.2 Base Models

In this study, an ensemble approach is constructed by unifying the inferences generated by different single LLMs (i.e., base models). As mentioned, since LLMs have been trained with the large-scale computing resources to ensure broad task generalizability and rapid applicability, we do not perform fine-tuning. Instead, we construct a one-shot model to ensure robust task execution. As shown in

```
### Instruction
You are a helpful assistant evaluating the review texts about the
restaurant. Please evaluate the review text and assign an integer score
ranging from 1 for the most negative comment to 5 for the most positive
comment.

### Review 1
User review: {example_review}
Output: {example_label}

### Review 2
User review: {user_review}
Output: _
```

Fig. 1. Basic Prompt

Fig. 1, we included a single annotated example in the prompt and enabled the model to generate an accurate response. The samples used for the instructions were randomly extracted from the dataset and excluded from the test set.

For the text-generation task, each base model predicts a single token that follows the prompt, corresponding to the underbar at the end of the prompt. To introduce probabilistic diversity in the responses, the model's reproducibility parameter is utilized. In most machine learning methods, specifying the initial value of the random number generator (i.e., seed value), ensures reproducibility in training and inference. In other words, by repeating inferences with different seed values, the model can generate the multiple outputs as if they were produced by multiple fictitious workers.

In this study, five virtual workers are established using different seed values. Let LLM^w denote the base model using seed value $w \in \{1, 2, \cdots, 5\}$, and let $prompt_i$ represent the input text corresponding to sample $i \in \{1, 2, \cdots, n\}$. The obtained sentiment value, denoted as s_i^w, is generated for sample i by worker w. Thus, the individual output is obtained as follows:

$$s_i^w = LLM^w(prompt_i) \tag{1}$$

3.3 Ensemble Strategy

To implement the ensemble learning by repeating the inferences, we develop the ensemble extension of 8B model and compare the performance with non-ensemble 70B model. Since the target variable is ordinal scale (1–5 stars), we adopt the median for the ensemble, which is robust to outlier predictions. Majority voting, on the other hand, is commonly employed in ensemble learning; however, it is vulnerable to the case when the number of votes is tied (e.g., two votes for 1, two for 2, and one for 3). Given our focus on the diversity of LLM outputs, the median is more appropriate for the aggregation.

For sample $i \in \{1, 2, \cdots, n\}$, the unified output, denoted as s_i^*, is obtained as follows:

$$s_i^* = median(s_i^w | s_i^w \in \{1, 2, \cdots, 5\}) \tag{2}$$

The aggregated sentiment score is computed using only valid predictions. In LLM-based sentiment analysis, models occasionally generate out-of-scope tokens, such as negative values, non-integer values, or non-digit characters. By taking the median, we mitigate the impact of such erroneous outputs and improve the robustness of the prediction.

4 Experiments

4.1 Dataset Overview

We use Yelp Open Dataset [42], an open dataset publicly available for academic research. Yelp is an online platform on which users post evaluations and reviews about various facilities, including restaurants, stores, and public institutions. Since our proposed method ensembles one-shot LLMs, performance is highly sensitive to pre-training data leakage. The Yelp dataset can only be obtained through an application process, so it is unlikely that an entire corpus was included for pre-training process of Llama family.

A key challenge in analyzing Yelp Dataset is the diversity of restaurants which vary in price range, cuisine, and service. While many studies [1,2,23,36] have analyzed user reviews using this dataset due to its reproducibility, few have employed LLMs for such analyses.

The dataset includes the ratings $rating_{jk} \in \{1, 2, \cdots, 5\}$ and review texts $review_{jk}$, which are posted by user j about restaurant k, where $j \in \{1, 2, \ldots, J\}$, $k \in \{1, 2, \ldots, K\}$. A single review was randomly selected for each user. If a user posted a review for the same restaurant multiple times, only the most recent review was considered for sampling. Thus, the sample size n equals the number of users J. Since all users and venues are unique, sample size $n = J \times K$.

In addition, each establishment is associated with category tags; allowing us to extract the target instances based on the tags. In this study, only restaurants holding a physical store in a fixed address were included in the analysis, and therefore we extracted only those tagged with Restaurant and excluded those of Fast Food, Food Trucks, Nightlife, and Bar. We selected 1,000 instances for the test set, and Table 2 lists the summary statistics.

4.2 Evaluation

We evaluate model performance using two metrics: concordance rate (Acc.) and root mean square error (RMSE). Since the target variable is ordinal, we assess not only prediction concordance but also the magnitude of errors using RMSE.

For the model comparison, we employ six reference models for the baseline. Among these, we implement three DNN-based models: BERT, Bi-LSTM, and

Table 2. Summary Statistics of the Dataset

$n = 1000$	Mean	Std	Min	Max
Characters	392.062	302.190	61	2425
Tokens	88.164	67.973	13	570
Stars	3.933	1.371	1	5

The numbers of tokens are counted using the Tiktoken tokenizer [30] which is also used in Llama 3.

CNN, based on previous studies [7,24,38]. The BERT model employs bert-large-uncased for both embedding and classification, while the Bi-LSTM and CNN models incorporate word embeddings (word2vec [26], FastText [20]) and respective architectures. For rule-based methods, we employ VADER, which computes polarity scores directly without training. For machine learning, we adopt a TF-IDF and Linear SVM combination based on previous studies, setting the TF-IDF dimensionality to 4,000. Additionally, we establish a chance-level benchmark by estimating sentiment label distributions from the training data and generating test predictions accordingly.

4.3 Results and Discussion

The results are shown in Table 3. First, non-ensemble LLMs (Model 2–4) outperformed all reference models. Notably, 70B model (Model 2) demonstrated the high performance compared to fewer-parameter models. Therefore, among the non-ensemble LLMs, the number of parameters significantly contributes on performance, which is consistent with the previous studies [28]. However, 70B model (Model 2) requires approximately 11 times the processing time of the 8B model (Model 3) for only a 3% increase in accuracy, which means that, even when analyzing only 1,000 samples, a total processing time differs by more than 16 h. Moreover, all Llama 3 models (Models 2–3) outperformed the Llama 2 model (Model 4). Notably, Llama 2 required more inference time than 80B model (Model 3), despite having fewer parameters. These results indicate that the model architecture and pre-training process have a significant impact on both the accuracy and processing speed.

In such a situation, the proposed ensemble model even outperformed the 70B model across RMSE and almost equivalent in accuracy. This indicates that, while the ensemble model achieved similar accuracy to the 70B model, it significantly reduced the margin of error in misclassification with less than half the processing time. Thus, a medium-sized model using ensemble approach with iterative inferences is significantly more efficient in terms of both training time and model performance than the larger model with a single inference.

Across the non-LLM baselines (Models 5–10), although the DNN-based models (Models 5–7) achieved high accuracy as expected, none of them surpassed LLM-based models. Additionally, VADER (Model 8) outperformed the machine-

Table 3. Comparison of the accuracy and the processing time

Model Name	Ensemble	RMSE	Acc.	Time (s)
LLMs				
1. Llama-3-8B-Instruct	adopted	**0.424**	**0.778**	27.215
2. Llama-3-70B-Instruct	–	**0.521**	**0.779**	64.879
3. Llama-3-8B-Instruct	–	**0.562**	**0.749**	5.443
4. Llama-2-7B-chat	–	**0.860**	**0.721**	5.986
Reference models				
5. BERT (Large) [7]	–	0.941	0.639	–
6. Bi-LSTM [24]	–	0.930	0.636	–
7. CNN [38]	–	1.098	0.596	–
8. VADER [19]	–	1.111	–	–
9. Linear SVM [6]	–	1.134	0.627	–
10. Random	–	1.850	0.358	–
Lift				
from the best single LLM (70B)		18.6%	–0.1%	58.1%
from the LLM baseline (8B)		32.7%	3.9%	–500.0%
from the best reference model		54.4%	17.9%	–

Note. Bold metric indicates that the model performs better than all references while cell shading represents the highest performance. Time represents the average duration to process one review.

Table 4. The Effects of Ensemble Inference

Sample	Workers $w \in \{1, 2, \cdots, 5\}$					s_i^*
	s_i^1	s_i^2	s_i^3	s_i^4	s_i^5	
1. consistent	4	4	4	4	4	4
2. similar	2	2	2	3	3	2
3. outlier	5	5	5	5	1	5

learning model (Model 9). Nevertheless, all models performed above the chance level (Model 10).

5 Conclusion

In this study, we demonstrated the effectiveness of a simple ensemble approach for sentiment analysis using LLMs. With the proposed model, which imitates human annotation, the results demonstrated a significant improvement over non-ensemble LLMs. By introducing the ensemble approach, prediction errors are compensated by each model, which significantly improves the accuracy (Table 4). Thus, the proposed approach enables a medium-sized model to achieve perfor-

mance comparable to a 70B model while requiring less than half the processing time. Moreover, even non-ensemble LLMs, compared with the well-known existing methods, can perform higher accuracy without any additional training. Thus, LLMs are highly useful for practical applications due to their large-scale pre-training.

This study has both academic and practical implications. First, from an academic perspective, our study suggests that simple ensemble methods can enhance the model performance using the randomness of LLM outputs. Unlike the other ensemble methods which rely on multiple independently trained models (e.g., bagging, boosting), our approach is based on the stochastic nature of LLM inferences. This indicates that this strategy is generally applicable to other pre-trained LLMs with different architecture or different number of parameters. Also, from a practical perspective, our findings provide two key implications. First, the ensemble of multiple inferences with a medium-sized model are much faster and more accurate than a single inference with a larger model. This efficiency can enhance the understanding of consumer preference and support quick decision-making for business. Second, since ensemble inference does not require additional training costs, practitioners can readily improve the tasks with this approach in their business environments.

Finally, this study remains several future challenges. First, all the base models employed the same example in the prompt; however, the performance of LLMs is highly sensitive to the prompt. Future research should explore the effects of different prompt configurations across the base models. Second, while this study focuses on ensemble of multiple models, it is necessary to examine the effect of combining different-scale models. This study treated all the outputs equally by using the median; however, to effectively integrate models with varying parameter counts, alternative ensemble techniques such as weighted averaging may be required. Third, although this study applied the ensemble of LLM inferences to marketing analysis on restaurant reviews, the proposed ensemble strategy itself is applicable to general text classification tasks. Future work should evaluate its effectiveness on broader benchmark across different domains to further validate the generalizability.

Acknowledgment. We are grateful to the two anonymous reviewers for their insightful comments and to Yelp for providing access to the valuable dataset. This work was supported by JSPS KAKENHI (Grant No. 24K16472).

Disclosure of Interests. Author JN has received consulting fees from private manufacturing company unrelated to the work presented in this paper. The authors have no other competing interests to declare.

References

1. Alamoudi, E.S., Alghamdi, N.S.: Sentiment classification and aspect-based sentiment analysis on yelp reviews using deep learning and word embeddings. J. Decis. Syst. **30**(2–3), 259–281 (2021). https://doi.org/10.1080/12460125.2020.1864106

2. Asghar, N.: Yelp dataset challenge: review rating prediction. arXiv preprint arXiv:1605.05362 (2016). https://doi.org/10.48550/arXiv.1605.05362
3. Chevalier, J.A., Mayzlin, D.: The effect of word of mouth on sales: online book reviews. J. Mark. Res. **43**(3), 345–354 (2006)
4. Chmielewski, M., Kucker, S.C.: An mturk crisis? Shifts in data quality and the impact on study results. Social Psychol. Pers. Sci. **11**(4), 464–473 (2020). https://doi.org/10.1177/1948550619875149
5. Christiano, P.F., Leike, J., Brown, T., Martic, M., Legg, S., Amodei, D.: Deep reinforcement learning from human preferences. Adv. Neural Inf. Process. Syst. **30** (2017)
6. Das, B., Chakraborty, S.: An improved text sentiment classification model using tf-idf and next word negation. arXiv preprint arXiv:1806.06407 (2018). https://doi.org/10.48550/arXiv.1806.06407
7. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, vol. 1 (Long and Short Papers), pp. 4171–4186. Association for Computational Linguistics (2019). https://doi.org/10.18653/v1/N19-1423
8. Dietterich, T.G.: Ensemble methods in machine learning. In: Kittler, J., Roli, F. (eds.) MCS 2000. LNCS, vol. 1857, pp. 1–15. Springer, Heidelberg (2000). https://doi.org/10.1007/3-540-45014-9_1
9. Ding, B., et al.: Is GPT-3 a good data annotator? In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, vol. 1: Long Papers, pp. 11173–11195. Association for Computational Linguistics (2023). https://doi.org/10.18653/v1/2023.acl-long.626
10. Do, H.H., Prasad, P., Maag, A., Alsadoon, A.: Deep learning for aspect-based sentiment analysis: a comparative review. Expert Syst. Appl. **118**, 272–299 (2019). https://doi.org/10.1016/j.eswa.2018.10.003
11. Dong, X., Yu, Z., Cao, W., Shi, Y., Ma, Q.: A survey on ensemble learning. Front. Comp. Sci. **14**(2), 241–258 (2019). https://doi.org/10.1007/s11704-019-8208-z
12. Douglas, B.D., Ewell, P.J., Brauer, M.: Data quality in online human-subjects research: comparisons between mturk, prolific, cloudresearch, qualtrics, and sona. PLoS ONE **18**(3), e0279720 (2023). https://doi.org/10.1371/journal.pone.0279720
13. Dubey, A., et al.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024). https://doi.org/10.48550/arXiv.2407.21783
14. Gerganov, G.: llama.cpp: LLM inference in C/C++ (2022). https://github.com/ggerganov/llama.cpp. Accessed 26 May 2024
15. Gilardi, F., Alizadeh, M., Kubli, M.: Chatgpt outperforms crowd workers for text-annotation tasks. Proc. Natl. Acad. Sci. **120**(30), e2305016120 (2023). https://doi.org/10.1073/pnas.2305016120
16. Hansen, L.K., Salamon, P.: Neural network ensembles. IEEE Trans. Pattern Anal. Mach. Intell. **12**(10), 993–1001 (1990)
17. Hartmann, J., Netzer, O.: Natural language processing in marketing. In: Artificial Intelligence in Marketing, pp. 191–215. Emerald Publishing Limited (2023)
18. Huang, Y., et al.: Ensemble learning for heterogeneous large language models with deep parallel collaboration. In: The Thirty-Eighth Annual Conference on Neural Information Processing Systems (2024)
19. Hutto, C., Gilbert, E.: Vader: a parsimonious rule-based model for sentiment analysis of social media text. In: Proceedings of the International AAAI Conference on

Web and Social Media, vol. 8, pp. 216–225 (2014). https://doi.org/10.1609/icwsm.v8i1.14550
20. Joulin, A., Grave, E., Bojanowski, P., Mikolov, T.: Bag of tricks for efficient text classification. In: Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics, vol. 2, Short Papers, pp. 427–431. Association for Computational Linguistics (2017). https://doi.org/10.18653/v1/e17-2068
21. Kauffmann, E., Peral, J., Gil, D., Ferrández, A., Sellers, R., Mora, H.: Managing marketing decision-making with sentiment analysis: an evaluation of the main product features using text data mining. Sustainability **11**(15), 4235 (2019). https://doi.org/10.3390/su11154235
22. Krugmann, J.O., Hartmann, J.: Sentiment analysis in the age of generative AI. Cust. Needs Solut. **11**(1), 3 (2024). https://doi.org/10.1007/s40547-024-00143-4
23. Liu, Z.: Yelp review rating prediction: machine learning and deep learning models. arXiv preprint arXiv:2012.06690 (2020). https://doi.org/10.48550/arXiv.2012.06690
24. Long, F., Zhou, K., Ou, W.: Sentiment analysis of text based on bidirectional lstm with multi-head attention. IEEE Access **7**, 141960–141969 (2019). https://doi.org/10.1109/ACCESS.2019.2942614
25. Luo, Z., Huang, S., Xu, F.F., Lin, B.Y., Shi, H., Zhu, K.: Extra: extracting prominent review aspects from customer feedback. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 3477–3486 (2018). https://doi.org/10.18653/v1/D18-1384
26. Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781 (2013). https://doi.org/10.48550/arXiv.1301.3781
27. Niimi, J.: Dynamic sentiment analysis with local large language models using majority voting: a study on factors affecting restaurant evaluation. arXiv preprint arXiv:2407.13069 (2024). https://doi.org/10.48550/arXiv.2407.13069
28. Nijkamp, E., Hayashi, H., Xiong, C., Savarese, S., Zhou, Y.: Codegen2: lessons for training llms on programming and natural languages. arXiv preprint arXiv:2305.02309 (2023). https://doi.org/10.48550/arXiv.2305.02309
29. Oliver, R.L.: Whence consumer loyalty? J. Mark. **63**(4_suppl1), 33–44 (1999)
30. OpenAI: tiktoken: a fast BPE tokeniser for use with OpenAI's models (2023). https://github.com/openai/tiktoken. Accessed 26 May 2024
31. Pangakis, N., Wolken, S., Fasching, N.: Automated annotation with generative ai requires validation. arXiv preprint arXiv:2306.00176 (2023). https://doi.org/10.48550/arXiv.2306.00176
32. Polikar, R.: Ensemble based systems in decision making. IEEE Circuits Syst. Mag. **6**(3), 21–45 (2006)
33. Reinartz, W., Krafft, M., Hoyer, W.D.: The customer relationship management process: its measurement and impact on performance. J. Mark. Res. **41**(3), 293–305 (2004)
34. Sardar, A., Manzoor, A., Shaikh, K.A., Ali, L.: An empirical examination of the impact of ewom information on young consumers' online purchase intention: mediating role of ewom information adoption. Sage Open **11**(4) (2021). https://doi.org/10.1177/21582440211052547
35. Silla, C.N., Freitas, A.A.: A survey of hierarchical classification across different application domains. Data Min. Knowl. Disc. **22**, 31–72 (2011)

36. Silva, F.d.S., dos Reis, J.C., Reis, M.S.: Seriema: a framework to enhance clustering stability, compactness, and separation by fusing multimodal data. In: International Conference on Applications of Natural Language to Information Systems, pp. 394–408. Springer, Heidelberg (2024). https://doi.org/10.1007/978-3-031-70239-6_27
37. Sun, Q., Niu, J., Yao, Z., Yan, H.: Exploring ewom in online customer reviews: sentiment analysis at a fine-grained level. Eng. Appl. Artif. Intell. **81**, 68–78 (2019). https://doi.org/10.1016/j.engappai.2019.02.004
38. Umer, M., et al.: Impact of convolutional neural network and fasttext embedding on text classification. Multimedia Tools Appl. **82**(4), 5569–5585 (2023). https://doi.org/10.1007/s11042-022-13459-x
39. Vermeer, S.A., Araujo, T., Bernritter, S.F., van Noort, G.: Seeing the wood for the trees: How machine learning can help firms in identifying relevant electronic word-of-mouth in social media. Int. J. Res. Mark. **36**(3), 492–508 (2019). https://doi.org/10.1016/j.ijresmar.2019.01.010
40. Wei, J., et al.: Finetuned language models are zero-shot learners. In: The Tenth International Conference on Learning Representations, ICLR 2022 (2022)
41. Xing, F.: Designing heterogeneous llm agents for financial sentiment analysis. ACM Trans. Manag. Inf. Syst. (2024). https://doi.org/10.1145/3688399
42. Yelp: Yelp Open Dataset, An all-purpose dataset for learning (2022). https://www.yelp.com/dataset. Accessed 20 Nov 2023
43. Zhang, L., Wang, S., Liu, B.: Deep learning for sentiment analysis: a survey. Wiley Interdisc. Rev. Data Min. Knowl. Disc. **8**(4), e1253 (2018). https://doi.org/10.1002/widm.1253
44. Zhou, Z.H.: Ensemble Methods: Foundations and Algorithms. CRC Press, Boca Raton (2012). https://doi.org/10.1201/b12207
45. Zhuang, Y., Kim, J.: A BERT-based multi-criteria recommender system for hotel promotion management. Sustainability **13**(14), 8039 (2021). https://doi.org/10.3390/su13148039

Testing Procedure Generation Based on Testing Requirements for Automotive Components with LLM Fine-Tuning

Man Yiu Chow(✉) and Mitsuhiro Kitani

Hitachi, Ltd., Research and Development Group, 292, Yoshida-cho, Kanagawa-ken, Japan
{manyiu.chow.dv,mitsuhiro.kitani.jd}@hitachi.com

Abstract. While a well-trained Transformer-based Large Language model has been proven to own an excellent ability to answer any question based on the input, the high accuracy of the answer output, especially for testing procedure generation based on testing requirements for automotive components focused in this paper, usually relies on either pre-training data with domain knowledge about the testing or input prompts with clear requirement instruction by well-dedicated prompt engineering. Since expensive pre-training data or instructing an overwhelming volume of untraceable tacit expertise in the input prompts is required in those general LLMs for procedure generation, it is not easy for most industries to adopt LLMs for their practical applications. This paper explores the model performance for the case when only limited domain-specific training data, around two hundred records, is available for our chosen models with fine-tuning and further provides insight into what level we can rely on fine-tuning for testing procedure generation based on Testing requirements for automotive components with BLEU evaluation. We chose Flan-T5-large and Mistral-7B as our target models with consideration for the model sizes with affordable fine-tuning methods and their few-shot prediction performance. Our results show that both fine-tuned target models can generate our expected procedures with more than 0.8 BLEU on average, while Mistral-7B can even derive new content for the procedure.

Keywords: Testing procedure · Testing requirement · LLM · QLoRA · T5 · automotive · Fine-tuning · GPT · Mistral

1 Introduction

While a well-trained Transformer-based Large Language model (LLM), especially for a larger size with good quality of pre-training, has been proven for their excellent ability [1, 2] to answer any question based on the input, the high accuracy of the answer output, which also includes testing procedure generation based on testing requirements for the system of automotive components (both hardware and software) focused in this paper, usually relies on large pre-training data consisting of domain knowledge for the automotive component testing. For those general pre-trained LLMs, clear prompt instructions input with few-shot examples and well-dedicated prompt engineering are required [3] when knowledge of a desirably specific discipline usually doesn't exist in the pre-training data for those models.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 200–209, 2026.
https://doi.org/10.1007/978-3-031-97144-0_18

In practice, reliance on the factors described above is usually infeasible since it is expensive and difficult to prepare large pre-training data. Furthermore, those requirement instructions for procedure generation in the input prompt involve an overwhelmingly large volume of untraceable domain-specific tacit expertise, which is hard to define. The more tacit information the users state in the input prompt, the more challenging LLM can generate the correct testing procedure due to the size of the model and the redundantly confusing information from the input prompt, especially for the lengthy information [4]. To deal with the problems, most researchers fine-tune the model with far less training data, which is more feasibly preparable and demands cheaper computational resources than pre-training data. However, only a little empirical information about the know-how and the effect of such fine-tuning, especially for generating sequential procedures with only a few hundred training data records, is available. Therefore, it is not easy for most industries to begin adopting LLMs for their practical applications due to the uncertainty about fine-tuning.

For this reason, our paper explores the model performance for the case when only limited domain-specific training data, around less than two hundred records, is available for our fine-tuning. Specifically considering the affordable cost for the users, we exploit normal fine-tuning for our target smaller-size model, and our larger-size model exploits computationally efficient QLoRA (Quantized Low-Rank Adapters) fine-tuning without a huge compromise of original full fine-tuning performance [5, 6]. Our result provides insight into how much we can rely on fine-tuning in testing procedure generation based on Testing requirements for automotive components or other comparable domains, especially for those with many specific variable terminologies, which only testing planners can understand.

2 Training Data and Approach of Testing Procedure Generation Based on Testing Requirements for Automotive Components

While there are different types of testing procedures, our paper focuses on Testing Procedure Generation based on Testing requirements for the system testing of automotive components, and we have testing requirements for input data and testing procedures for output data. In other words, the language models in our tool generate testing procedures based on testing requirements, and all of the testing is about the system diagnosis for automotive components. Those automotive components include ECU (Electronic control unit), motor, and miscellaneous safety monitoring sensors for electronic vehicles. In this section, we explain the details of our training data in Sect. 2.1 and our approach in Sect. 2.2 for the testing procedure generation.

2.1 Training Data for the Procedure Generation

For the content of our IO data, the input data is testing requirements, which contains a single long list about the environment and the requirements of the testing, whereas the output data consists of several parts of the testing procedure, including precondition setup, observation variable, and testing description. Both input and output data are written in English with a natural language style. As shown in Fig. 1, we separate our testing

procedure output into three parts for the three different language models, respectively, and those models all receive the same input of testing requirements. The reason for the separation is that the language models, especially the smaller-sized ones released earlier, are usually pre-trained on shorter sentences, and this pre-training method limits the token number along with the accuracy for handling long sentences [4, 7, 8]. In our case, the smaller size model, Flan-T5-large, is only capable of handling an input of 512 tokens at maximum.

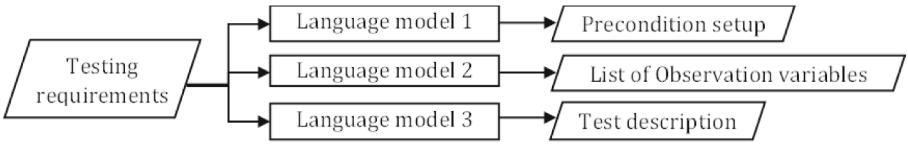


Fig. 1. The structure of our tool with three language models

For the fine-tuning data in this paper, we have 172 records of training data and 10 records of evaluation data for the model of testing description, 171 records of training data and 10 records of evaluation data for the model of Observation variable model, and 171 records of training data and 9 records of evaluation data for the model of precondition setup.

For the input of testing requirements, each content has an average of 168 words, and it specifically contains the testing environment, successful criteria, precondition criteria, and constraints for the testing. For the first output of the precondition setup, each setup has an average of 120 words and around 10 to 20 configuration steps for setting up the variable before the test starts. In the setup, each step contains an instruction sentence of around 10 words. For the second output of the observation variable list, each list has an average of 75 words and around 15 to 20 variable names from the system. In the list, each variable normally has around 2 to 3 words of unusual terminological names for component control, but occasionally around 10 words for the explanation of those components. For the third output of the testing description, each content has an average of 84 words. Each testing description has around 10 action descriptions, and each action contains around 8 words of testing instructions.

2.2 Approach of the Testing Procedure Generation

Instead of exploiting well-dedicated prompt engineering, each language model in our methods, shown in Fig. 1, learns the generation patterns for each part of the procedure from our training data by fine-tuning, when most of the pre-training corpus for language models do not contain the information of those tremendous numbers of generation rules. As shown in Fig. 2, the method of prompt engineering requires not only testing requirements, but also a long list of generation requirements which also need to be defined by complicated tracing or extraction methods [9] for the input prompt, and the inclusion makes the generation unscalable and hard to maintain with new updates from the untraceable rules. Even though we prefer the model to learn the rules during the pre-training stage, the reason behind the absence of generation rules in general pre-training data is

that the procedures of testing configurations usually depend on the unique calibration of the system along with the industrial testing methods, which are confidential from the public database and this requires additional fine-tuning to acquire the knowledge.

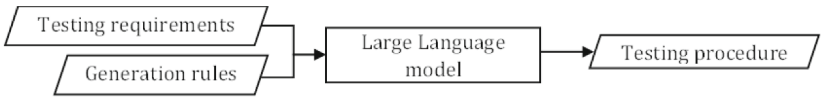


Fig. 2. The method of well-dedicated prompt engineering with generation rules

Moreover, domain-specific tacit expertise for the testing requirements depends on the divisive habits and preferences of engineers, and all these factors make it hard for all those engineers to explicitly figure out the precise generation rules in input prompts, let alone the high number of requirements. Therefore, it is more suitable to exploit fine-tuning in practice, and fine-tuning enables each of our language models to recognize the patterns of all the unspoken generation rules, including the calibration and common way of handling the testing procedures from the data acquired in past projects, for easier maintenance. As shown in Fig. 3, three models are responsible for learning the generation patterns by fine-tuning without any additional instruction of clear generation rules based on the given testing requirement input and three respective output parts of the testing procedure. In Sect. 3, we further detail how we chose our target language models for the three models stated in Fig. 3 and their fine-tuning methods for the generation.

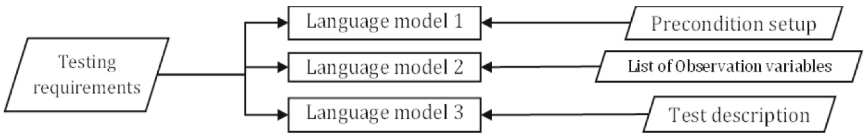


Fig. 3. The method of fine-tuning without any additional generation rules

3 Target LLM Models and Fine-Tuning Methods

This section provides the details of the two large language models chosen in this paper, which are well-known for their natural language task ability. For each target model, we provide the details of why we selected the model, along with the fine-tuning methods for each of those chosen models.

3.1 Flan-T5 Model and Its Fine-Tuning

Flan-T5 is the model based on the architecture of the T5 model but with fine-tuning enhancement by instruction datasets. [7, 10] The architecture of the original T5 model is a transformer-based encoder-decoder model, and the model is known for its ability to understand the relationship of all input tokens due to the bidirectional encoder structure

and flexible text generation due to its unidirectional decoder. The model has five sizes available, including "small" with around 77 million parameters, "base" with around 250 million parameters, "large" with around 770 million parameters, "3B" with around 3 billion parameters and "11B" with around 11 billion parameters. We chose the model as our target model because the whole T5 series has an affordably smaller size available, i.e. less than 1 billion parameters, than most of the Transformer-based decoders with comparable performance.

For affordable full fine-tuning costs and the ability to handle the complexity of procedure generation, we selected Flan-T5-large, which has fewer than 1 billion parameters but the most parameters among the T5 series in this paper. Specifically, the reason for choosing Flan-T5 is that its T5 model's structure is well-known for the features of a bidirectional encoder, which is critical for testing requirement understanding and unidirectional decoder procedure generation. With the extra fine-tuning enhancement, Flan-T5 has better performance on zero-shot tasks than the original T5 model, and this enhancement also implies that we can rely more on the model itself with less fine-tuning data required for our generation. Even though this enhancement is not significant, it is still critical for better pre-training performance when the reliance on the preparation of fine-tuning data has usually been difficult for most industries in the beginning.

For the fine-tuning method for Flan-T5-large, we adopted the most typical fine-tuning by updating all of the model parameters. In the hyperparameters of fine-tuning, we exploited its original Adam optimizer with a weight decay of 0.01 and selected 10 epochs.

3.2 Mistral-7B and Its QLoRA Fine-Tuning

Mistral-7B is a model based on a Transformer-based decoder that generates text given the input prompt. We chose Mistral over other Transformer-based decoder models because of its open-source and state-of-the-art performance. [11] The model has Sliding Window Attention that lets tokens at a higher layer access the information from the previous layer beyond the window size with less computation cost. [12] Alongside other training strategies, including grouped query attention, the model achieved better performance than other open-source models, such as Llama2, with cheaper computation costs, especially at the time when we decided on the model type for our tool.

For affordable fine-tuning costs regardless of its enormous size, we exploited QLoRA (Quantized Low-Ranking Adaptation) fine-tuning method to update a small set of additional trainable parameters instead of all the parameters in the base model [5]. We specifically inject new low-rank matrices into all linear layers except for the language modelling head on Mistral-7B in our method. Besides the small-scale update, QLoRA further quantizes the transformer model to 4-bit precision and uses paged optimizers with data type BFloart16 to handle memory spikes. With the implementation along with 2 gradient accumulation steps, [13] we can fit all the parameters of the model as well as the computation of the model update into a single GPU with 24 GB memory available during the whole fine-tuning. This implies that fine-tuning is more affordable for most industries when less computation cost is required.

For the hyperparameters of QLoRA fine-tuning, we chose 12 fine-tuning epochs with a warm-up ratio of 0.03 in the AdamW optimizer. For QLoRA setup, we selected

64 ranks for the size of the adaptation layer, with 16 alphas for the scaling of how the adaptation layer influences the base model.

4 Results

4.1 Evaluation Methods and Results

In the evaluation of the models for their procedure generation performance, we adopted the BLEU evaluation, which calculates the correlation between the generated procedure and our reference procedure word-by-word [14]. The evaluation method is appropriate for our generation since we expect our language model can generate the same setup for the procedural actions, variable names, variable values and descriptions for the testing procedure based on similar testing requirements.

As illustrated in Table 1, our evaluation results show that the Mistral-7B model generally performs better than the Flan-T5-large model, with a higher average BLEU score. However, the difference is small, especially when Mistral-7B has 10 times more parameters than Flan-T5-large. The evaluation also means that we should exploit small-size models for similar performance when features of other testing procedures are similar to our cases of less training data available. Depending on the generation requirements, our results further show that Mistral-7B tends to have more derivation ability by deriving new procedure content, such as new variable names, based on the unseen requirements, compared to Flan-T5-large. The derivation ability helps to flexibly define better generation rules for each requirement. The details of such observation are shown in Sect. 4.3.

Table 1. Model performance for each testing procedure type.

Model	Epoch	Testing procedure type	Average BLEU score
Flan-T5-large	10	Testing description	0.85
		List of Observation variables	0.88
		Precondition setup	0.93
Mistral-7B	12	Testing description	0.90
		List of Observation variables	0.89
		Precondition setup	0.97

4.2 Epoch and Overfitting

To find out the optimal epoch for our chosen model, we conducted more epochs than usual and observed the epoch with the minimum validation loss. In Fig. 4, we found that Mistral-7B has a more stable validation loss than Flan-T5-large with no drastic overfitting after a long fine-tuning epoch. Depending on the parts of the procedures, Flan-T5-large tends to have a bigger loss except for the Precondition configuration.

When good training also means the model can recognize the requirement pattern of our data without overfitting and respond well to unseen data with accurate procedures, our optimal epoch should not be too long, even though the validation loss does not increase after some time. Since choosing the longer one could also lead to overfitting to both training and validation data, we pinpoint the epoch where the improvement tends to stop. For this reason, we chose 12 epochs for Mistral-7B and 10 epochs for Flan-T5-large, in which the validation loss of each respective model started to stop improving for three parts of the testing procedure.

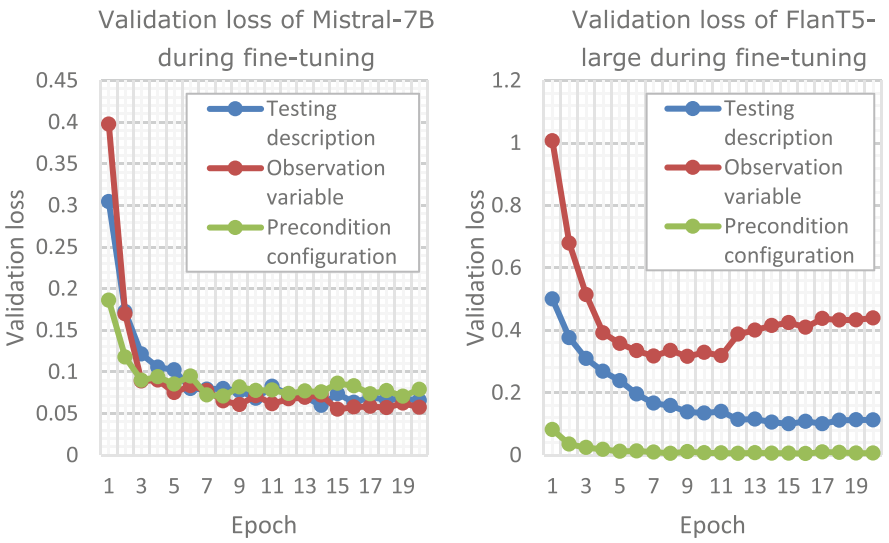


Fig. 4. The graph of validation loss of two target languages against epoch during fine-tuning

4.3 Samples of Model Generation

In this section, samples of model generation for each part of the testing procedure are listed to understand the generation ability of our models. For each part, the inputs of testing requirements are all the same for all the models, and we specially selected the sample, which has a different generation from the ground truth. Such selection can help us perceive the abilities and limitations of each model.

In sample generation for the precondition setup shown in Table 2, we changed the component angle from sine to cosine in the original testing requirements, and we expect that the variable "Sin" has a new name related to cosine in the new ground truth. Since the input of testing requirements does not exist in our training data, this further intelligently relies on the model's pre-training knowledge in addition to the knowledge from fine-tuning. Flan-T5-large can generate a procedure similar to the one in training data without any new variable name, but some steps/ variables are missing. Meanwhile, Mistral-7B can further create our expected variable name that never exists in our training data, even though it also changes a few steps in our expected procedure.

Table 2. Sample Generation for Testing Description.

Model	Generation output
Flan-T5-large	0:Set Precondition.,1:Ramp Sin from E[V] to X[V]., 2:Ramp Sin from X[V] to E[V]., 3:power2 off −> on., 4:Clear Err_code., 5: power2 off and, Set Sin to X[V]., 6:Set Sin to E[V]., Action7: power2 on.,
Mistral-7B	0:Set Precondition., 1:Ramp COS from E[V] to X[V]., 2:Ramp COS from X[V] to E[V]., 3:power2 off −> on., 4:Clear Err_code., 5:Set voltage to T[V]
Ground Truth	0:Set Precondition., 1:Ramp Cos from E[V] to X[V]., 2:Ramp Cos from X[V] to E[V]., 3:power2 off −> on., 4:Clear Err_code., 5:power2 off and, Set Cos to X[V]., 6:Set Cos to E[V]., 7:power2 on.,

In sample generation for the list of observation variables shown in Table 3, we fed our model with the input of the testing requirements, which is similar to some of the training data but has minor changes in wording. Flan-T5-large and Mistral-7B can generate similar variable names, which exist in our ground truth, except for the missing "HSL_XXX (RAM)" variable. Although Mistral-7B has chosen the "ASM_XXX(RAM)" variable, which does not exist in our ground truth, the variable was adopted in other similar testing from the past and exists in parts of our training data similar to the input. This also proves that our model can understand the requirement of how the variable is chosen from our training data through fine-tuning, even though they are not usual terminologies from general knowledge.

Table 3. Sample generation for the list of observation variables.

Model	Generation output
Flan-T5-large	Low voltage status(Low), Low voltage status(High), Low voltage power supply, System status, Error status 1, Error status 2, Error Flag 1, Error Flag 2, Error_data, Event ID, Controller status, Gate mode, COM signals, COM signals(Bus),
Mistral-7B	Low voltage status(Low), Low voltage status(High), Low voltage power supply, System status, Error status 1, Error status 2, ASM_XXX(RAM), Error Flag 1, Error Flag 2, Error_data, Event ID, Controller status, Gate mode, COM signals, COM signals(Bus),
Ground Truth	Low voltage status(Low), Low voltage status(High), Low voltage power supply, System status, Error status 1, Error status 2, HSL_XXX (RAM), Error Flag 1, Error Flag 2, Error_data, Event ID, Controller status, Gate mode, COM signals, COM signals(Bus),

In sample generation for the precondition setup shown in Table 4, we found that the Flan-T5-large model generates the setup that sticks to the steps from the testing requirements similar to the one in the ground truth with only one step missing. In contrast, the Mistral-7B model combined some other steps from the procedures corresponding to

other testing requirements similar to our input. There are two of the same steps in the output due to the combination. By this sample result, we observed that Flan-T5-large tends to inflexibly follow the past procedure, which makes the result closer to the ground truth. This is also a preferable result for the case when the requirements from testing requirements are strictly conventional and repetitive.

Table 4. Sample generation for Precondition setup

Model	Generation output
Flan-T5-large	1. Turn on power1., 2. Turn on power2., 3. Set temperature1 to Y degC., 4. Set temperature2 to Y degC., 5. Set temperature3 to Y degC., 6. Set temperature4 to Y degC., 7. Enable the High_voltage SW., 8. Enable the High_voltage manual SW., 9. Set High_voltage to M V., 10. Switch the control soruce to manual mode., 11. Set COM_Data_Mode to A., 12. Switch the control soruce to manual mode., 13. Set COM_Data_Toque to Y Nm., 14. Set Motor_Speed to E rpm,.,
Mistral-7B	1. Enable the Power supply., 2. Turn on power1., 3. Turn on power2., 4. Set temperature1 to Y degC., 5. Set temperature2 to Y degC., 6. Set temperature3 to Y degC., 7. Set temperature4 to Y degC., 8. Enable the High_voltage SW., 9. Enable the High_voltage manual SW., 10. Set High_voltage to M V., 11. Switch the control soruce to manual mode., 12. Switch the control soruce to manual mode., 13. Set HSL_StOpm to A., 14. Switch the control soruce to manual mode., 15. Set HSL_Tq to Y Nm., 16. Set Motor_Speed to E rpm, 17. Set ASG_Temp to A.,.,
Ground Truth	1. Enable the Power supply., 2. Turn on power1., 3. Turn on power2., 4. Set temperature1 to Y degC., 5. Set temperature2 to Y degC., 6. Set temperature3 to Y degC., 7. Set temperature4 to Y degC., 8. Enable the High_voltage SW., 9. Enable the High_voltage manual SW., 10. Set High_voltage to M V., 11. Switch the control soruce to manual mode., 12. Set COM_Data_Mode to A., 13. Switch the control soruce to manual mode., 14. Set COM_Data_Toque to Y Nm., 15. Set Motor_Speed to E rpm,

5 Summaries

In this paper, we explored our model fine-tuning approach for testing procedure generation based on testing requirements for automotive components, especially ECU, when less than 200 records of training data are available. While most industries hesitate to adopt LLM fine-tuning for the testing procedure generation based on their requirements due to the costly preparation of data and untraceable tacit expertise, our paper provides a valuable reference to the extent to which we can rely on fine-tuning for procedure generation according to the requirements.

Specifically, we selected two models, Flan-T5-large and Msitral-7B with affordable fine-tuning, and evaluated their performance for our procedure generation. While they both have relatively high BLEU scores with more than 0.8 in our result, Mistral-7B

shows more derivation ability, which is beyond our testing requirements for the model. Depending on the generation rules for the requirements, Flan-T5-large is better for sticking to the convention content for the similar requirements, whereas Mistral-7B tends to derive new content, especially for the unseen requirements based on the fine-tuning data. In the future, we will explore more reliable fine-tuning methods to improve accuracy with more complicated requirements and longer sequential procedure output.

References

1. Brown, T.B., et al.: Language models are few-shot learners. Neural Inf. Process. Syst. **33**, 1877–1901 (2020)
2. Chang, Y., et al.: A survey on evaluation of large language models. ACM Trans. Intell. Syst. Technol. **15**(3), Article 39, 1–45 (2024)
3. Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G.: Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing. ACM Comput. Surv. **55**(9), Article 195, 1–35 (2023)
4. Li, T., Zhang, G., Do, Q.D., Yue, X., Chen, W.: Long-context LLMs Struggle with Long In-context Learning. arXiv preprint arXiv:2404.02060 (2024)
5. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: efficient finetuning of quantized LLMs. In: NIPS '23: Proceedings of the 37th International Conference on Neural Information Processing Systems, article No. 441, p. 10088–10115 (2023)
6. Hu, E.J., et al.: LoRA: low-rank adaptation of large language models. In Proceedings of ICLR (2021)
7. Raffel, C., et al.: Exploring the limits of transfer learning with a unified text-to-text transformer. J. Mach. Learn. Res. **21**(140), 1–67 (2020)
8. Devlin, J., Chang, M., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. In: North American Chapter of the Association for Computational Linguistics (2019)
9. Zhao, L., et al.: Natural language processing for requirements engineering. ACM Comput. Surv. **54**(3), 1–41 (2021)
10. Chung, H.W., et al.: Scaling Instruction-Finetuned Language Models. arXiv preprint arXiv:2210.11416 (2022)
11. Jiang, A.Q., et al.: Mistral 7B. arXiv preprint arXiv:2310.06825 (2023)
12. Beltagy, I., Peters, M.E., Cohan, A.: Longformer: the long-document transformer. arXiv preprint arXiv:2004.05150 (2020)
13. Huang, Z., Jiang, B., Guo, T., Liu, Y.: Measuring the impact of gradient accumulation on cloud-based distributed training. In: IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing, pp. 344–354 (2023)
14. Papineni, K., Roukos, S., Ward, T., Zhu, W.J.: BLEU: a method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 311–318 (2002)

OpenThaiGPT 1.5: A Thai-Centric Open Source Large Language Model

Sumeth Yuenyong[1(✉)], Kobkrit Viriyayudhakorn[2,3], Apivadee Piyatumrong[4], and Jillaphat Jaroenkantasima[5]

[1] Department of Computer Engineering, Faculty of Engineering, Mahidol University, Salaya, Thailand
sumeth.yue@mahidol.edu
[2] iApp Technology Co., Ltd., Mohali, India
kobkrit@iapp.co.th
[3] Artificial Intelligence Entrepreneur Association of Thailand (AIEAT), Muang Pathum Thani, Thailand
[4] Big Data Institute (Public Organization), Bangkok, Thailand
apivadee.pi@bdi.or.th
[5] Department of Electrical Engineering, Faculty of Engineering, Kasetsart University, Bangkok, Thailand

Abstract. OpenThaiGPT 1.5 is an advanced Thai language chat model based on Qwen v2.5, finetuned on over 2,000,000 Thai instruction pairs. This report provides an engineering perspective on the model's development, capabilities, and performance. We discuss the model's architecture, training process, and key features, including multi-turn conversation support, Retrieval Augmented Generation (RAG) compatibility, and tool-calling functionality. Benchmark results demonstrate OpenThaiGPT 1.5's state-of-the-art performance on various Thai language tasks, outperforming other open-source Thai language models. We also address practical considerations such as GPU memory requirements and deployment strategies.

Keywords: Large Language Model · Thai Language Model · OpenThaiGPT

1 Model Architecture and Training

1.1 Base Model

OpenThaiGPT 1.5 is built upon the Qwen v2.5 architecture [5], leveraging its advanced capabilities as a foundation for Thai language modeling. The model is available in two sizes: 7 billion and 72 billion parameters, catering to different computational resource constraints and performance requirements. The 7B model was finetuned from Qwen/Qwen2.5-7B-Instruct on Huggingface, and the 72B model was finetuned from Qwen/Qwen2.5-72B-Instruct. Both base models have a vocabulary size of 152,064 and a maximum input length of 32,768.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 210–219, 2026.
https://doi.org/10.1007/978-3-031-97144-0_19

Inspection of the tokenizers and initial experimentation revealed that the Qwen 2.5 models already support the Thai language. For this reason, as well as due to the limitation of our computing, we opted not to perform any continued pre-training of the model with Thai data and start with instruction finetuning.

1.2 Finetuning Process

The model underwent extensive finetuning on a diverse dataset of over 2,000,000 Thai instruction pairs. This process was crucial in adapting the base model to the nuances of the Thai language and culture, enabling its effectiveness in handling Thai-specific domain questions. We used the NeMo[1] framework by Nvidia for finetuning. The 7B and 72B proposed models were trained using the LoRa technique [4]. The LoRa adaptor size r was 64, and α was 128. The learning rate was $1e^{-4}$. The global batch size was 32. We used 10% of the finetuning data as a validation set and restored the checkpoint with the lowest validation loss after training. Both models were trained on a server with 8x H100 GPUs, which was kindly provided to us by SiamAI[2].

Datasets. The finetuning dataset for OpenThaiGPT 1.5 comprises several high-quality, diverse Thai language datasets:

- **Thai Wiki Summary Dataset**: A collection of 3,000 cleaned rows generated from Wikipedia, focusing on summarization and information synthesis tasks.
- **Thai QA Multi-turn Answer Dataset**: This dataset contains 11,992 cleaned rows of multi-turn question-answering conversations, enhancing the model's ability to maintain context and generate coherent responses across multiple interactions.
- **Additional Proprietary Datasets**: Supplementary over 30 datasets covering various domains and task types, further enriching the model's knowledge and capabilities.
- **Synthetic Data**: We used Llama 3.1 405B [3] to generate synthetic data by prompting it to create variations of an existing instruction, generate counterfactual questions to existing facts, and to form questions about Wikipedia articles.
- **LLM as a Judge**: After generating synthetic data, we used another instance of Llama 3.1 405B [3] to review the quality of the generated data. We used in-context learning to inform the model of examples of low and high-quality instruction-answer pairs. We filtered the synthetic data to retain only instruction-answer pairs that the model judged high-quality.
- **English Data**: We would like the model to be English-Thai bilingual. Therefore, to alleviate catastrophic forgetting of English, we mixed in about 20% of English data from various datasets on Huggingface such as: yahma/alpaca-cleaned, OpenAssistant/oasst1, and (a subset of) Open-Orca/OpenOrca.

[1] https://github.com/NVIDIA/NeMo.
[2] https://siam.ai/.

These datasets were carefully curated and processed to ensure high-quality training data, contributing to the model's robust performance across a wide range of Thai language tasks and domains.

1.3 Alignment and Safety

We created a safety net for Reinforcement Learning from Human Feedback (RLHF) [6] to ensure the model does not generate or respond to rude or socially sensitive topics. Our approach began with compiling a list of impolite words and sensitive subjects to which the model should refuse to answer. We manually wrote examples that included these words and addressed sensitive issues. Additionally, we employed techniques such as Easy Data Augmentation [11] to enhance our dataset, and we also experimented with jailbreaking [12] some open-weight LLMs to generate more of this data locally and off-line. In total, we constructed around 5,000 records of alignment data and aligned the model using Direct Preference Optimization (DPO) [9], accessible through the model aligner module of NeMo.

2 Key Features

The finetuning process preserved all the key features of Qwen2.5.

2.1 Multi-Turn Conversation Support

OpenThaiGPT 1.5 preserved multi-turn conversation capability by having around 30% of the instruction finetuning dataset consist of multi-turn conversations. This allows for more natural and coherent dialogues. This feature is essential for applications requiring extended interactions, such as customer support or interactive learning systems.

2.2 Retrieval Augmented Generation (RAG)

The model supports RAG, enabling it to incorporate external knowledge sources during response generation. This feature significantly enhances the model's ability to provide accurate and up-to-date information, making it suitable for applications requiring access to large, dynamic knowledge bases.

2.3 Tool Calling Support

OpenThaiGPT 1.5 includes a tool-calling feature that allows it to execute predefined functions or make API calls based on user queries. This capability extends the model's functionality beyond text generation, enabling it to perform tasks such as retrieving real-time data or interacting with external systems.

2.4 Extended Context Handling

The model can process up to 131,072 input tokens and generate up to 8,192 tokens, allowing for detailed and complex interactions. This extended context window is particularly useful for tasks requiring the analysis of long documents or generating comprehensive responses.

3 Performance and Benchmarks

3.1 Evaluation Dataset

OpenThaiGPT Evaluation Dataset. We introduce the OpenThaiGPT Evaluation Dataset, a comprehensive collection of Thai-language benchmark exams that we developed to evaluate language models' performance across various educational and professional domains. This novel dataset comprises multiple standardized tests and specialized assessments:

- **Academic Standardized Tests**:
  - A-Level exams (2021–2022) covering Applied Mathematics, Biology, English, and Social Studies
  - ONET M3 (Grade 9) and M6 (Grade 12) exams from 2021–2025, including subjects like Thai, Mathematics, Social Studies, Science, and English
  - TGAT (Thai General Aptitude Test) focusing on Critical Thinking
  - TPAT1 (Thai Professional Aptitude Test) emphasizing Medical Ethics
- **Professional Certifications**:
  - Thai Investment Consultant Licensing Exams (IC Plain, IC Complex P2, P3) [10]
- **Natural Language Understanding Benchmarks**:
  - Facebook Belebele Thai [1]
  - XCOPA Thai for cross-lingual transfer learning [8]
  - XNLI 2.0 Thai for cross-lingual natural language inference [2]

The dataset features multiple-choice questions with verified answers, and includes additional annotations such as answerability flags and solution type indicators. All content has been verified by Thai speakers and is distributed under the Apache-2.0 License. The evaluation framework provides a rigorous testing environment for assessing Thai language models' capabilities across academic knowledge, professional expertise, and general language understanding. The dataset is open-source and publicly available at Huggingface[3], enabling researchers and developers to evaluate and compare different Thai language models using standardized benchmarks.

[3] https://huggingface.co/datasets/openthaigpt/openthaigpt_eval.

Thai Exam Benchmark. The Thai Exam Benchmark [7] is a comprehensive evaluation dataset designed to assess language models' performance on Thai educational and professional examinations. It consists of multiple-choice questions from various Thai national examinations including ONET (Ordinary National Educational Test), IC (Investment Consultant), TGAT (Thai General Aptitude Test), TPAT-1 (Thai Professional Aptitude Test), and A-Level exams. This benchmark provides a rigorous test of both language understanding and domain-specific knowledge across academic and professional contexts.

M3Exam. The M3Exam [13] is a multilingual, multimodal, and multilevel benchmark for evaluating the capabilities of large language models. It includes questions from various domains, including science, mathematics, and social studies, and is designed to test the model's understanding of context, reasoning, and knowledge application.

3.2 Evaluation Results

OpenThaiGPT 1.5 has been evaluated on these three datasets. The results demonstrate the model's superior performance compared to other open-source Thai language models (Tables 1, 2 and 3).

Table 1. Performance Comparison of Different Language Models in the 7B - 8B sizes on OpenThaiGPT Evaluation Dataset

Exam Name	Llama-3.1-8B		Typhoon-v1.5x-8b		Qwen2.5-7B		OpenThaiGPT1.5-7b	
	Correct	%	Correct	%	Correct	%	Correct	%
A Level	57/120	47.50	56/120	46.67	70/120	58.33	72/120	**60.00**
TGAT	18/50	**36.00**	16/50	32.00	16/50	32.00	18/50	**36.00**
TPAT1	22/40	55.00	21/40	52.50	23/40	**57.50**	23/40	**57.50**
Investment Consult	12/25	48.00	14/25	56.00	17/25	68.00	19/25	**76.00**
Facebook Beleble TH	146/200	73.00	156/200	78.00	158/200	79.00	162/200	**81.00**
XCOPA TH	138/200	69.00	159/200	79.50	161/200	80.50	162/200	**81.00**
XNLI 2.0 TH	110/200	55.00	113/200	**56.50**	106/200	53.00	109/200	54.50
O-NET M3 Thai	8/25	32.00	12/25	48.00	18/25	**72.00**	16/25	64.00
O-NET M3 Thai	10/20	50.00	15/20	75.00	18/20	**90.00**	16/20	80.00
O-NET M3 Math	3/16	18.75	4/16	25.00	5/16	**31.25**	5/16	**31.25**
O-NET M3 Science	11/26	42.31	12/26	**46.15**	12/26	**46.15**	12/26	**46.15**
O-NET M3 English	23/30	76.67	21/30	70.00	26/30	**86.67**	25/30	83.33
O-NET M6 Thai	19/65	29.23	31/65	47.69	30/65	46.15	35/65	**53.85**
O-NET M6 Math	3/17	17.65	5/17	**29.41**	5/17	**29.41**	5/17	**29.41**
O-NET M6 Social	24/55	43.64	28/55	50.91	31/55	56.36	32/55	**58.18**
O-NET M6 Science	9/28	32.14	12/28	42.86	16/28	**57.14**	16/28	**57.14**
O-NET M6 English	37/52	71.15	34/52	65.38	41/52	78.85	42/52	**80.77**
Total/Average	650/1169	55.60	709/1169	60.65	753/1169	64.41	769/1169	**65.78**

Table 2. Performance Comparison of Different Language Models in the 14B sizes on OpenThaiGPT Evaluation Dataset. Note that there are no 14B sized models for Llama-3.1 and Typhoon-v1.5x.

Exam Name	Qwen2.5-14B			OpenThaiGPT1.5-14b		
	Score	Total	%	Score	Total	%
A Level	74	120	61.67	78	120	**65.00**
TGAT	22	50	44.00	25	50	**50.00**
TPAT1	24	40	**60.00**	21	40	52.50
Investment Consult	19	25	**76.00**	18	25	72.00
Facebook Beleble TH	169	200	84.50	174	200	**87.00**
XCOPA TH	170	200	85.00	173	200	**86.50**
XNLI 2.0 TH	139	200	**69.50**	129	200	64.50
O-NET M3 Thai	19	25	76.00	21	25	**84.00**
O-NET M3 Social	18	20	**90.00**	18	20	**90.00**
O-NET M3 Math	7	16	**43.75**	2	16	12.50
O-NET M3 Science	13	26	50.00	14	26	**53.85**
O-NET M3 English	28	30	**93.33**	28	30	**93.33**
O-NET M6 Thai	34	65	52.31	37	65	**56.92**
O-NET M6 Math	4	17	23.53	7	17	**41.18**
O-NET M6 Social	33	55	60.00	34	55	**61.82**
O-NET M6 Science	14	28	50.00	16	28	**57.14**
O-NET M6 English	43	52	**82.69**	41	52	78.85
Total/Average	830	1169	71.09	836	1169	**71.51**

Table 3. Performance Comparison of Different Language Models in the 70B - 72B sizes on OpenThaiGPT Evaluation Dataset

Exam Name	Llama-3.1-70B		Llama-3-typhoon		Qwen2.5-72B		OpenThaiGPT1.5-72B	
	Correct	%	Correct	%	Correct	%	Correct	%
A Level	74/120	61.67	71/120	59.17	90/120	75.00	92/120	**76.67**
TGAT	20/50	40.00	23/50	46.00	24/50	**48.00**	23/50	46.00
TPAT1	20/40	50.00	21/40	52.50	22/40	**55.00**	22/40	**55.00**
Investment Consult	13/25	52.00	15/25	60.00	20/25	**80.00**	18/25	72.00
Facebook Beleble TH	176/200	88.00	175/200	87.50	180/200	**90.00**	180/200	**90.00**
XCOPA TH	171/200	85.50	169/200	84.50	180/200	90.00	181/200	**90.50**
XNLI 2.0 TH	126/200	63.00	125/200	62.50	131/200	65.50	141/200	**70.50**
O-NET M3 Thai	14/25	56.00	19/25	76.00	19/25	76.00	21/25	**84.00**
O-NET M3 Social	19/20	**95.00**	19/20	**95.00**	18/20	90.00	19/20	**95.00**
O-NET M3 Math	4/16	25.00	7/16	**43.75**	6/16	37.50	6/16	37.50
O-NET M3 Science	16/26	61.54	14/26	53.85	17/26	65.38	19/26	**73.08**
O-NET M3 English	28/30	93.33	28/30	93.33	29/30	**96.67**	29/30	**96.67**
O-NET M6 Thai	39/65	**60.00**	36/65	55.38	39/65	**60.00**	37/65	56.92
O-NET M6 Math	10/17	**58.82**	7/17	41.18	4/17	23.53	7/17	41.18
O-NET M6 Social	42/55	**76.36**	37/55	67.27	35/55	63.64	36/55	65.45
O-NET M6 Science	16/28	57.14	14/28	50.00	18/28	64.29	19/28	**67.86**
O-NET M6 English	43/52	82.69	38/52	73.08	45/52	86.54	47/52	**90.38**
Total/Average	831/1169	71.09	818/1169	69.97	877/1169	75.02	897/1169	**76.73**

For the Thai Exam Benchmark and M3Exam, OpenThaiGPT 1.5 (72B) achieved a score of 63.89% and 70.39% respectively, outperforming several other large language models, including some closed API-only models (Table 4):

Table 4. Benchmark Results on the Thai Exam Benchmark and the M3Exam.

Model	Average Score	Thai Exam Score	M3Exam Score
Claude 3.5 Sonnet (2024-06-20)	**68.41%**	**69.2%**	67.62%
OpenThaiGPT 1.5 72B	67.14%	63.89%	**70.39%**
GPT-4o (2023-05-13)	66.26%	63.89%	68.63%
Qwen2.5 72B Instruct	63.43%	60.53%	66.33%
OpenThaiGPT 1.5 14B	60.41%	58.41%	62.41%
Meta Llama 3.1 70B Instruct	59.38%	58.23%	60.52%
Llama 3 Typhoon v1.5x 70b Instruct	59.34%	58.76%	59.92%
OpenThaiGPT 1.5 7B	53.03%	52.04%	54.01%

These benchmark results highlight OpenThaiGPT 1.5's strong performance in Thai language understanding and generation tasks, positioning it as a leading option for Thai language AI applications.

4 Conclusion

We have developed and released OpenThaiGPT version 1.5 on Huggingface at openthaigpt/openthaigpt1.5-{size}b-instruct where {size} are 7, 14 or 72. There are based on Qwen2.5 family of models. Extensive experiments on Thai exams data showed that OpenThaiGPT1.5 is currently the most capable open model for the Thai language.

A OpenThaiGPT Evaluation Dataset Examples

Here are some examples from the OpenThaiGPT Evaluation Dataset.

Example 1: Investment Consultant Exam

Mr. A purchased units of Mutual Fund A at a price of 10.20 baht per unit. One year later, he sold those units at a price of 10.50 baht per unit. During the year, the mutual fund paid a dividend of 0.30 baht per unit. What is the rate of return that Mr. A received from his investment in Mutual Fund A? (Do not consider tax liabilities and other expenses incurred from investing in Mutual Fund A.)

- (1) 2.94%
- (2) 5.71%
- (3) 5.88%

– (4) 6.00%

(3) is the correct answer.

Example 2: O-NET M6 Thai Language

Which situation matches the meaning of the Thai idiom "pointing at a bird on a branch tip" (meaning to have unrealistic expectations or false hopes)?

– (1) Somsak was disappointed to receive only an honorable mention in the essay contest, saying he shouldn't have pointed at a bird on a branch tip
– (2) Somsri studied very hard but still ranked only third in class; this is what they call pointing at a bird on a branch tip
– (3) Somjit has been hinting to her mother several times that she wants a mobile phone, hoping her mother will buy it after exams; Somjit is pointing at a bird on a branch tip
– (4) Somjai received a sum of prize money and invited friends to go shopping for pretty clothes, saying now she can finally point at a bird on a branch tip
– (5) Somying keeps pestering her father for a car even though she's still in junior high school; this is called pointing at a bird on a branch tip

(5) is the correct answer.

Example 3: O-NET M6 Social Studies

When OPEC announced a reduction in oil production, causing global oil prices to rise, it affected the gasoline market, which is considered an essential commodity in Thailand. The domestic retail price of gasoline increased significantly, impacting people's lives in the country. Given this situation, if the government wants to help gasoline consumers, which measure should the government implement?

– (1) Set the gasoline price at equilibrium
– (2) Establish lower production quotas for gasoline
– (3) Set a price ceiling in the gasoline market
– (4) Set a price floor in the gasoline market
– (5) Set minimum wages in the oil production industry

(3) is the correct answer.

Example 4: O-NET M6 Science

A man with blood type AB and color blindness marries a woman with blood type O and normal vision (no family history of color blindness). They have three children together - one daughter and two sons. Which statement is correct?

– (1) Both sons have normal vision
– (2) The children will have either blood type AB or O

- (3) If they have another child, it will be female
- (4) All three children can donate blood to their mother
- (5) The daughter may or may not be a carrier of color blindness

(1) is the correct answer.

Example 5: O-NET M6 Mathematics

Given that the proposition "If Manee studies hard then Manee passes the exam" is false, and "Manee is class president or Manee passes the exam" is true, which of the following propositions must be true?

- (1) Manee doesn't study hard or passes the exam
- (2) Manee passes the exam and is not class president
- (3) Manee studies hard if and only if Manee passes the exam
- (4) If Manee is class president then Manee doesn't study hard
- (5) Manee is class president if and only if Manee fails the exam

(5) is the correct answer.

Example 6: Facebook Beleble Thai [1]

Let your hands be as relaxed as possible while still playing all the notes correctly. Try not to move your fingers unnecessarily. This method will minimize the fatigue you experience. Remember that you don't need to press the keys harder to make the sound louder like a piano. If you want to increase the volume of the accordion, use more pressure or speed with the bellows.

Which of the following is NOT a correct tip for successfully playing the accordion?

- (1) To make the sound louder, increase pressure when playing the notes
- (2) Try not to move fingers excessively to avoid using too much force
- (3) Be careful when playing notes while keeping hands as relaxed as possible
- (4) Increase the bellows speed to make the sound louder

(1) is the correct answer.

B Our Links

Official Website:
https://openthaigpt.aieat.or.th/

See our project on GitHub:
https://github.com/orgs/OpenThaiGPT/repositories

Hugging faces:
https://huggingface.co/openthaigpt

References

1. Bandarkar, L., et al.: The belebele benchmark: a parallel reading comprehension dataset in 122 language variants. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 749–775. Association for Computational Linguistics, Bangkok, Thailand and virtual meeting (August 2024). https://aclanthology.org/2024.acl-long.44
2. Conneau, A., et al.: Xnli: evaluating cross-lingual sentence representations. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics (2018)
3. Dubey, A., et al.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024)
4. Hu, E.J., et al.: Lora: low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685 (2021)
5. Hui, B., et al.: Qwen2. 5-coder technical report. arXiv preprint arXiv:2409.12186 (2024)
6. Ouyang, L., et al.: Training language models to follow instructions with human feedback. Adv. Neural Inf. Process. Syst. **35**, 27730–27744 (2022)
7. Pipatanakul, K., et al.: Typhoon: thai large language models. arXiv preprint arXiv:2312.13951 (2023)
8. Ponti, E.M., Glavaš, G., Majewska, O., Liu, Q., Vulić, I., Korhonen, A.: Xcopa: a multilingual dataset for causal commonsense reasoning. arXiv preprint arXiv:2005.00333 (2020)
9. Rafailov, R., Sharma, A., Mitchell, E., Manning, C.D., Ermon, S., Finn, C.: Direct preference optimization: your language model is secretly a reward model. Adv. Neural Inf. Process. Syst. **36** (2024)
10. Stock Exchange of Thailand: General investment advisor course. https://www.set.or.th/th/education-research/education/professional/ic/plain-products. Accessed 7 Nov 2024
11. Wei, J., Zou, K.: Eda: easy data augmentation techniques for boosting performance on text classification tasks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 6382–6388 (2019)
12. Xu, Z., Liu, Y., Deng, G., Li, Y., Picek, S.: A comprehensive study of jailbreak attack versus defense for large language models. In: Findings of the Association for Computational Linguistics ACL 2024, pp. 7432–7449 (2024)
13. Zhang, W., Aljunied, M., Gao, C., Chia, Y.K., Bing, L.: M3exam: a multilingual, multimodal, multilevel benchmark for examining large language models. Adv. Neural Inf. Process. Syst. **36**, 5484–5505 (2023)

Demo Papers

ESG-Consultant: Developing of an ESG Compliance Consulting Tool for Companies Using RAG

Angel Ontiveros[1,2], Irina Nikishina[1(✉)], Moritz Gomm[2], Christopher Schmitt[2], and Chris Biemann[1]

[1] University of Hamburg, Hamburg, Germany
{angel.ontiveros,irina.nikishina,chris.Biemann}@uni-hamburg.de
[2] Zühlke Engineering GmbH, Eschborn, Germany
{moritz.gomm,christopher.schmitt}@uni-hamburg.de

Abstract. Environmental, Social, and Governance (ESG) compliance has emerged as a crucial benchmark in contemporary corporate assessment. The present work develops and evaluates a software tool called *ESG-Consultant* to facilitate ESG compliance for companies. We also discuss two tasks related to the application of ESG: determining the *Applicability of ESG Law* to companies and *ESG Law Question Answering*. To the best of our knowledge, no previous works tackle these tasks specifically on ESG regulations. For both tasks, we create datasets and develop pipelines based on Retrieval Augmented Generation using LLMs. The results are quite promising, proving that our system can be effectively used to support, but under no circumstances replace practitioners. We hope this work contributes to more effective corporate practices for a sustainable future. The code of our tool and demonstration system are available online (https://github.com/angelonti/ai4esg_experiments, https://ai4esg-app.azurewebsites.net/ (pass1: e2c306nt for the annotations), https://www.youtube.com/watch?v=JKKGD3lFiUA).

Keywords: ESG Compliance · Retrieval Augmented Generation · Legal Data · Demonstration System

1 Introduction

This paper focuses on the Environmental, Social, and Governance (ESG) regulations which have become pivotal benchmarks in contemporary corporate assessment [8]. ESG is crucial for sustainable economic development, requiring businesses to positively impact their employees, families, communities, and society by aligning with sustainable development goals. The number of ESG regulations is growing significantly over the past decades, exceeding 2000 globally [3]. Many of these new regulations are mandatory, which pushes the companies to assess, understand, and implement ESG regulations using automatic methods.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 223–229, 2026.
https://doi.org/10.1007/978-3-031-97144-0_20

To the best of our knowledge, there is no published research implementing and evaluating a system using Large Language Models (LLMs)s to assist in ESG compliance. Therefore, in this paper, we develop and evaluate a system to fill the gap, gathering requirements from ESG practitioners. We provide support for two tasks in this demonstration system: the **ESG Law Question Answering task (ESG-QA)**, a vanilla Question Answering task with the texts from the ESG domain and the **ESG Law Applicability task**, a standard classification task, which aims at determining whether a ESG regulation applies to a company with specific parameters.

Fig. 1. Home screen of the ESG-Consultant tool.

The outcome of our research is a system called **ESG-Consultant** which aims to support companies and practitioners with Environmental Social, and Governance (ESG) compliance. The implemented prototype of **ESG-Consultant** is deployed, realizing our users' requirements, and manually tested by ESG experts. Additionally, the tool includes an annotations module, where users can annotate examples for the ESG Law Applicability task for further improvement of the system.

All in all, the main contributions of the paper are as follows:

- We present a demonstration system for assisting law experts with ESG regulations as well as present the methodology of developing such systems.
- We present an additional annotation module, for creating manual annotations for ESG regulations, which can be used for further improvement of the system.
- We introduce a novel domain-related benchmark: ESG Law Question Answering, and create a dataset for the evaluation.

Our system could be implemented not only for ESG regulations but also for any type of regulatory or legal texts. We use Apache License, Version 2.0, therefore, it can be also re-implemented for other legal tasks.

2 Related Work

Due to specifically complex language [7], specialized subdomains [10], and heterogeneous formats [4], the legal domain remains one of the most challenging NLP domains. However, more and more research on legal tasks now makes use of LLMs [4,7]. This has led to applications of NLP for the law being increasingly adopted in legal settings [6]. However, none of the previous work applied RAG to ESG-QA.

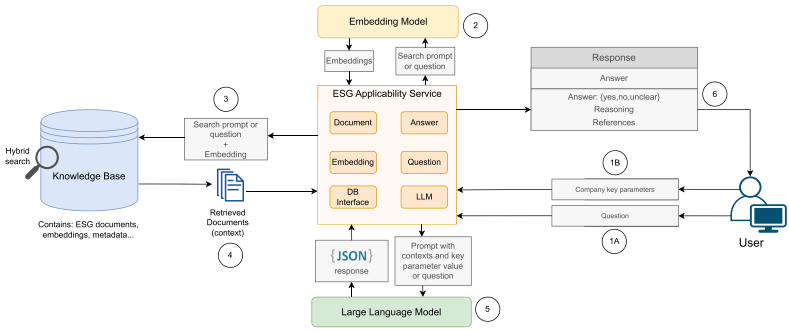


Fig. 2. Steps of the ESG-Consultant pipeline. The user inputs either key company parameters (country, number of employees, etc.) or a natural language question. As the output, the user receives a $\{yes, no, unclear\}$ answer to the applicability question with reasoning and references or a natural-language answer with the exact regulation.

3 System Design

ESG-Consultant is designed to be used in industry to assist experts in their day-to-day activities, and thus, increase efficiency in understanding and complying with ESG regulations. Figure 1 displays the home page of the system. The user is presented with a short introduction and the list of the system modules. Each of them can be accessed by clicking on their name on the left side panel.

System Architecture. Our system is developed in the Python programming language, and we use Poetry[1] as our dependency manager. To develop the user interface of our ESG-Consultant for ESG compliance we use streamlit[2]. It is an open-source Python library for the rapid development of user interfaces for data, and Artificial Intelligence (AI applications. We adhere to the SOLID [9] principles of software design whenever it aligns with our architecture goals and constraints. The backend can be split into two modules, a retrieval module, and a generation module. Figure 2 demonstrates the whole architecture pipeline.

[1] https://python-poetry.org/.
[2] https://streamlit.io/.

We use a single database (PostgreSQL) to store embeddings of eight ESG regulations. Seven of them are European Union law and were collected from EURLex[3] in text PDF format. The remaining one corresponds to Indian law[4] with a different text layout and consisting of a PDF with scanned images. We collect the texts using Unstructured[5] and divide them into chunks of preferred size in characters of 1024, max size 2048, and ended with an average character size over all chunks of 1201. The average number of chunks is ≈ 114.

In the retrieval stage (steps 1 through 4), we have the company's key parameters (1A) (which are added to the specific search prompt) or natural language questions (1B) as user input. Both are further transformed (step 2) into dense vector representations using an embedding model. In step 3, the search prompt or question text and its dense vector representation are used to perform a hybrid search (keyword and vector) with reciprocal rank fusion for relevant documents in our knowledge base, which contains previously stored ESG documents. We retrieve between 9–18 relevant documents (step 4) from our database. Then we use the cross-encoder model bge-reranker-v2-m3[6] to rerank and keep the top three most relevant documents. We use these techniques because several studies have confirmed task performance gains from hybrid search [5] and reranking [1] for retrieval. In contrast to our final pipeline naive RAG uses vector search only and no reranking.

In the generation stage (steps 5 through 6), we have either a prompt template for one of the key parameters for the company and its value or a natural language question and the top three relevant text chunks to be used as context.

4 Experiments

The ESG Law Question Answering task is a domain-specific task that takes a natural-language question about ESG regulations as an input, expecting a natural-language answer as the output. The ESG Law Applicability task aims at determining whether a specific regulation applies to a company based on its parameters or characteristics. To solve both tasks, we use the Retrieval Augmented Generation (RAG) pipeline which accepts a question as input, retrieves relevant excerpts, and using them in a prompt to the LLM, responds grounded on the retrieved excerpts.

Dataset. There is no existing dataset for ESG legislation, therefore, we generate it using GPT-4[7]. The development set comprises 273 question-answer pairs (327 is the average answer size in characters); the test set consists of 263 pairs (330 characters on average). An example of such data is as follows: *What is the*

[3] "EUR-Lex" accessed 05.07.2024, https://eur-lex.europa.eu/homepage.html.
[4] "Indian Standard Guidelines for Recycling of Plastics" accessed 05.07.2024, https://law.resource.org/pub/in/bis/S11/is.14534.1998.pdf.
[5] https://unstructured.io/.
[6] https://huggingface.co/BAAI/bge-reranker-v2-m3.
[7] https://platform.openai.com/docs/models/gpt-4-turbo-and-gpt-4.

definition of sustainability risk? 'Sustainability risk' means an environmental, social, or governance event or condition that, if it occurs, could cause an actual or a potential material negative impact on the value of the investment.

Methodology. For this task, we use question inputs directly from a user/dataset. Once a question is asked, we retrieve the top three most relevant text chunks from the database through similarity search techniques. In the generation stage, we have a single prompt template in which the retrieved context and the question are injected. The LLM-generated output is considered to be the final answer to the natural-language input question.

We compare two pipelines: naive (only the basic components of RAG with semantic search) and final (with the hybrid search with Reciprocal Rank Fusion (RRF) and a cross-encoder reranking module). We also experiment with two models: GPT-4 and SaulLM-7b (an LLM trained on legal texts) [2]. The number of retrieved documents k varies from $k = 1$ to $k = 3$.

Evaluation Metrics. used for the current study are standard: MAP, MRR, and Hit Rate for Retrieval Evaluation; SQuAD metrics (Exact Match (EM), Precision, Recall, F1) for the whole pipeline.

Table 1. SQuAD metrics of GPT-4 and SaulLM-7b models on the ESG Law Question Answering on dev and test datasets. The results are shown for the naive RAG approach *(n)* and the final pipeline.

Model	EM	F1	Precision	Recall
dev set				
GPT4-0613 (n)	**45.42**	73.94	71.55	85.03
GPT4-0613	44.69	**77.29**	**74.08**	**90.23**
SaulLM-7b (n)	10.26	63.34	64.61	73.92
SaulLM-7b	13.92	67.63	67.79	78.97
test set				
GPT4-0613	**44.86**	**78.87**	75.51	**92.02**
SaulLM-7b	14.83	72.90	**76.32**	79.13

Results and Discussion. Table 1 shows the comparative performance of naive RAG against our final RAG pipeline for GPT-4 and SaulLM-7b on dev set. The final pipeline yields a few percentage points improvements over the naive RAG pipeline. Similarly, we still observe that GPT-4 has the best performance for all metrics, therefore we stick to it for later experiments on the test dataset. Another interesting finding is that the SQuAD recall for GPT4 in our final pipeline achieves a value of 90%, this means the correct answer is within the model answer in 9 out of 10 questions. Finally, we evaluate our final pipeline on

Table 2. Retrieval metrics on the ESG Law Question Answering on the test dataset with varying the number of retrieved documents k from $k = 1$ to $k = 3$.

@k	MAP	MRR	Hit Rate
1	0.8479	0.8479	0.8479
2	0.8878	0.8878	0.9278
3	0.8945	0.8942	0.9468

the test split of our ESG Law Question Answering dataset. The results for the SQuAD metrics can be seen in Table 1, and the results for the retrieval metrics in Table 2. The results on the previously unseen test dataset are coherent with the development dataset.

5 Conclusion

In this paper, we developed the ESG-Consultant system for ESG compliance based on RAG pipeline and specified two tasks that these experts considered highly valuable for assisting in ESG compliance: the ESG Law Question answering task, and the ESG Law Applicability task. Moreover, we created a novel ESG QA dataset and evaluated on it several baselines. In future work, we plan to collect more data and work on multi-step reasoning.

References

1. Boytsov, L., Lin, T., Gao, F., Zhao, Y., Huang, J., Nyberg, E.: Understanding performance of long-document ranking models through comprehensive evaluation and leaderboarding. arXiv preprint arXiv:2207.01262 (2022)
2. Colombo, P., et al.: SaulLM-7B: a pioneering large language model for law. CoRR **abs/2403.03883** (2024)
3. ESG: ESG Policy Digest: December 2023 (2023). https://www.esgbook.com/esg-policy-digest-december-2023/
4. Ganguly, D., et al.: Legal IR and NLP: the history, challenges, and state-of-the-art. In: Kamps, J., et al. (eds.) Advances in Information Retrieval. ECIR 2023. LNCS, vol. 13982, pp. 331–340. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-28241-6_34
5. Gao, L., Dai, Z., Chen, T., Fan, Z., Van Durme, B., Callan, J.: Complement lexical retrieval model with semantic residual embeddings. In: Hiemstra, D., Moens, M.-F., Mothe, J., Perego, R., Potthast, M., Sebastiani, F. (eds.) ECIR 2021. LNCS, vol. 12656, pp. 146–160. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-72113-8_10
6. Guha, N., et al.: LegalBench: a collaboratively built benchmark for measuring legal reasoning in large language models. Adv. Neural Inf. Process. Syst. **36**, 44123–44279 (2024)
7. Katz, D.M., Hartung, D., Gerlach, L., Jana, A., Bommarito II, M.J.: Natural language processing in the legal domain. arXiv preprint arXiv:2302.12039 (2023)

8. Li, T.T., Wang, K., Sueyoshi, T., Wang, D.D.: ESG: research progress and future prospects. Sustainability **13**(21), 11663 (2021). https://doi.org/10.3390/su132111663
9. Martin, R.: Clean Architecture: A Craftsman's Guide to Software Structure and Design. United States of America: Pearson Education Inc., USA, 1st edn. (2017). https://dl.acm.org/doi/10.5555/3175742
10. Shui, R., Cao, Y., Wang, X., Chua, T.S.: A comprehensive evaluation of large language models on legal judgment prediction. In: Bouamor, H., Pino, J., Bali, K. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 7337–7348. Association for Computational Linguistics, Singapore (December 2023)

CaLuX: A Catalyst and Lubricant Properties Extraction System for Domain Experts

Catalina Riano(✉), Fumian Chen, and Hui Fang(✉)

University of Delaware, Newark, DE 19711, USA
{criano,fmchen,hfang}@udel.edu

Abstract. The literature is a valuable asset in many different areas. There are a great number of documents available within each knowledge discipline. This volume of information grows rapidly and leaves stakeholders with the challenge of reviewing all of these documents manually, creating the need for tools to extract relevant data in a more efficient and automatic manner. This data is particularly important for the domain experts in a wide range of fields, such as chemical engineering and material science, who rely on specialized records for their work, including the development of new models, experiments, and more. While existing tools can extract some common chemical properties from texts, currently, there is no comprehensive system available that focuses on extracting catalyst and lubricant properties from scientific literature. Catalysts and lubricants play pivotal roles in scientific research and industrial applications. Despite their significance, the lack of dedicated tools and structured databases presents a barrier to innovation in these fields. To address this gap, we introduce CaLuX, a Catalyst and Lubricant Properties Extraction System for domain experts specifically designed and tailored to extract catalyst and lubricant properties leveraging an existing chemistry-oriented Natural Language Processing (NLP) pipeline. CaLuX stands out for its ability to handle multiple document formats and produce structured data that can be used to build valuable databases. It offers a ready-to-use, straightforward interface for extracting properties from the scientific literature in the catalyst and lubricant fields, helping domain experts without the need for them to use their computational resources or to possess any coding skills.

Keywords: Information Extraction · Chemical Data · Catalyst and Lubricant Properties · Scientific Literature

1 Introduction

Fields like Chemical Engineering and Material Science generate a vast and rapidly expanding literature body in its many specialized areas. The sheer volume of documents, including research papers, patents, and technical reports,

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 230–236, 2026.
https://doi.org/10.1007/978-3-031-97144-0_21

makes it increasingly difficult for stakeholders to review and extract valuable information manually. Consequently, there is a critical need for automated tools that can efficiently analyze these documents and extract relevant data. This data is particularly important for the domain experts in a wide range of fields who relay on specific data for their work, including the development of new models, experiments, and more. Although there are tools available for extracting common chemical properties from text, no comprehensive system currently focuses on the specific and important domain of catalysts and lubricants.

Catalysts and lubricants play a pivotal role in both scientific research and industrial applications. Catalysts are essential for accelerating chemical reactions, optimizing energy efficiency, and reducing production costs in processes ranging from pharmaceuticals to petrochemicals. Lubricants, on the other hand, are critical for the growth of the oil and gas sector, including the automotive, aerospace, and manufacturing industries. Despite their significance, the lack of dedicated tools and structured databases for catalyst and lubricant properties presents a barrier to innovation and efficiency in these fields.

We present CaLuX[1], a novel system specifically designed to extract catalyst and lubricant properties from the scientific literature. CaLuX leverages an existing Chemistry-Oriented NLP pipeline [4] to automatically identify and extract relevant data from a variety of document formats, including PDFs, HTML, and plain text. The system generates structured data output that can be seamlessly integrated into databases, filling a critical need for organized and accessible data in the catalyst and lubricant domains. CaLuX offers a ready-to-use, straightforward end-to-end interface for extracting properties from the scientific literature in the catalyst and lubricant fields, helping domain experts with the need for such data, without the requirement for them to use their computational resources or possess any coding skills.

2 Related Work

In the chemistry field, there are specific tools to perform different NLP tasks, for example, ChemicalTagger [1], which does the parsing task of chemical experimental texts using rule-based techniques; OSCAR4 [7], which does NER tasks using maximum-entropy Markov model (MEMM); and ChemSpot [2], which does the Name Entity Recognition (NER) task using CRF with a dictionary method.

ChemDataExtractor (CDE) [3] is a state-of-the-art NLP tool that is chemistry aware, developed by the Molecular Engineering department of the University of Cambridge, which provides an NLP pipeline including sentence splitting, tokenization, part-of-speech (POS) tagging, phrase parsing, information extraction and inter-dependency resolution. In recent years, a second version of the tool was released ChemDataExtractor2.0 (CDE 2.0) [4] which uses a new approach, mainly taking advantage of nested models that are specialized to extract specific properties built in according to the need. This tool has been tested to extract mainly spectrographic properties [3], crystallography properties [4], yield

[1] CaLuX is hosted on a web server. Contact the authors to access the system.

strength properties [6], and Curie and Neel Temperatures [7]. To the best of our knowledge, this Chemistry-Oriented NLP Pipeline has not been used and do not provide the capabilities in extracting information about lubricants and catalysts. The full details of the CDE tool and its design are given by Swain et al. [3] and Mavracic et al. [4].

Additionally, most of these tools are available only as toolkits, which present a high barrier for domain experts without coding experience. Moreover, some lack user-friendly interfaces and require a lot of computational resources.

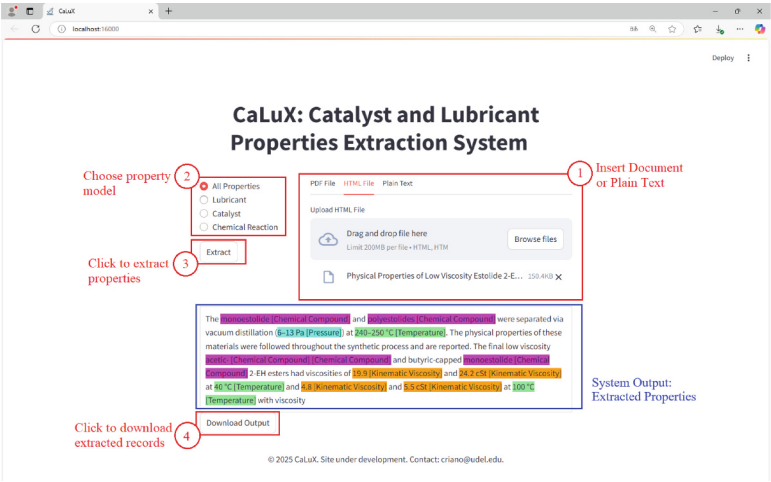


Fig. 1. CaLuX: Layout and Example of Output

3 CaLuX: System Overview

CaLuX: A Catalyst and Lubricant Properties Extraction System, is designed to extract catalyst and lubricant properties from scientific literature, easy and ready to use, it has a variety of properties available and also presents great potential for recurrent usage from domain experts without need of them to use their computational resources or possess coding skills. It is a straightforward process, yet a complex underlying structure. The tool takes an input document or plain text, processes it, and generates an output consisting of structured records related to the catalyst and lubricant properties.

3.1 System Interface

The system is designed to be simple and user-friendly, offering a straightforward interface that allows domain experts to extract the records easily. Figure 1 shows

the layout of the system. On the right side, there is a tab menu, where users can upload their document in one of three supported formats: PDF, HTML, or plain text.

On the left side of the interface, there is a radio button menu that allows users to choose the specific property model they wish to extract from the document. The available options include extracting all properties or just focusing on one of the following models: the lubricant model, the catalyst model, or the chemical reaction model. By providing the ability to select a specific model, rather than extracting all properties at once, the system gives the user more flexibility and helps optimize the extraction process for quicker results.

The properties encompass both quantitative and qualitative attributes and are organized into three primary models. These three models are collectively encapsulated within a general compound model, which identifies chemical compounds and can extract data about temperature, pressure, and phase. The three property models are:

- *Lubricant Model:* Properties include dynamic viscosity, kinematic viscosity, and density.
- *Catalyst Model:* Properties include catalyst type, catalyst weight, pore diameter, pore volume, Brunauer-Emmett-Teller (BET) surface area, and TOF (Turnover Frequency).
- *Chemical Reaction Model:* Properties include the type of reaction, yield, initial composition, initial concentration, and reaction time. As well, it incorporates a sub-model for the reactor, whose properties include the type of reactor, operational mode, space velocity, reaction rate, and residence time.

Once the user has selected their desired properties and clicked the Extract button, the extraction process will begin. When the extraction is complete, the extracted properties will be displayed on the screen, highlighted for easy identification. If the document is particularly lengthy, the system will only show a snippet of the extracted content. This snippet will provide a sample of the highlighted properties. Finally, the user can download the extracted records as a CSV file. This file will include not only the extracted properties but also the location of each property within the document, indicating the page where each property was found. This feature ensures that users can easily trace and verify the extracted information. Figure 1 shows an example of how the property extraction looks.

3.2 Technical Details

Figure 2 shows the general diagram of how CaLuX works. It is built leveraging an existing chemistry-oriented NLP system: CDE 2.0 [4].

Depending on the document format, different approaches are applied to ensure that all relevant content is extracted accurately. For PDF documents, specialized Python libraries are employed to identify and obtain the main document text, tables, and table captions while preserving content integrity. The

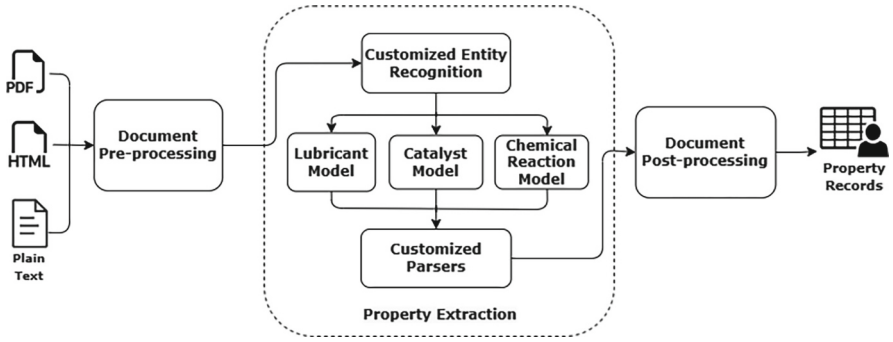


Fig. 2. CaLuX: General Diagram

PDF files are then converted into plain text format to facilitate further processing. On the other hand, if the document is in HTML format, HTML tags are leveraged to systematically extract the same elements–main text, tables, and captions–directly from the structured content.

On top of the chemistry-oriented NLP system, CDE 2.0 [4], a customized property extraction pipeline was created. First, the Customized Entity Recognition, in this step, the Named Entity Recognition (NER) task was improved by expanding the dictionary to include the most common catalysts found in the literature, suggested by domain experts. Additionally, a fine-tuned BERT tagger was incorporated to increase the model's capabilities for identifying chemical-named entities on the domains of interest.

The tool was further extended with new models designed to extract chemical properties that were not originally supported. These models were developed to recognize both the necessary property values and their corresponding units and dimensions. They are capable of extracting information from both text and tables. The extracted properties cover both quantitative and qualitative attributes and are structured into three main models: the Lubricant Model, Catalyst Model, and Chemical Reaction Model. Each of these models supports multiple properties, as described in Subsect. 3.1. Together, they are integrated within a broader General Compound Model, which was modified from the original tool to identify chemical compounds and extract additional data such as temperature, pressure, and phase.

To ensure accurate data extraction, customized parsers were developed for each of the new models, primarily using rule-based grammars. Additionally, the built-in Snowball machine learning algorithm was enabled as a parser and retrained to specialize in extracting information related to lubricants and catalysts.

As an output of the process, a Python dictionary is given, which consists of the normalized data from the tree constructed based on the parsing rules, in case a property model or several models are identified. This data is cleaned and

standardized before being organized into a tabular fashion to later be available to the user as a CSV file.

4 Conclusions and Future Work

CaLuX, the Catalyst and Lubricant Properties Extraction System, offers a comprehensive solution for extracting properties from the scientific literature in the catalyst and lubricant fields, contributing significantly to expert domains such as chemical engineers and material scientists, where this kind of data is highly needed. CaLuX stands out for its ability to handle multiple document formats and produce structured data that can be used to build valuable databases. The system was built leveraging an existing Chemistry-Oriented NLP Pipeline: CDE2.0 [4], which, to the best of our knowledge, has limited capabilities when it comes to the lubricant and catalyst domain, thus, CaLuX represents a solid addition to this existing tool.

CaLuX has a ready-to-use, straightforward end-to-end interface for extracting properties from the scientific literature in the catalyst and lubricant fields, allowing domain experts with a need for such data, without the requirement for them to use their computational resources or possess any coding skills. Also, CaLuX supports a variety of properties, presenting a great potential for recurrent usage from domain experts. Future work includes expanding CaLuX to other types of properties and providing more support for domain expert tasks. In addition, it includes optimizing the extraction time, improving the capabilities of supported document formats, and refining the extraction models.

Acknowledgments. This work was supported by the National Science Foundation under grant number CBET-22134471. The authors thank anonymous reviewers for their useful comments.

References

1. Hawizy, L., Jessop, D., Adams, N., Murray-Rust, P.: ChemicalTagger: a tool for semantic text-mining in chemistry. J. Cheminformatics **3**, 17 (2011). https://doi.org/10.1186/1758-2946-3-17
2. Rocktäschel, T., Weidlich, M., Leser, U.: ChemSpot: a hybrid system for chemical named entity recognition. Bioinformatics (Oxford, England) **28**, 1633–1640 (2012). https://doi.org/10.1093/bioinformatics/bts183
3. Swain, M.C., Cole, J.M.: ChemDataExtractor: a toolkit for automated extraction of chemical information from the scientific literature. J. Chem. Inf. Model. **56**(10), 1894–1904 (2016). https://doi.org/10.1021/acs.jcim.6b00207
4. Mavračić, J., et al.: ChemDataExtractor 2.0: autopopulated ontologies for materials science. J. Chem. Inf. Model. **61**(9), 4280–4289 (2021). https://doi.org/10.1021/acs.jcim.1c00446
5. Jessop, D.M., Adams, S.E., Willighagen, E.L., et al.: OSCAR4: a flexible architecture for chemical text-mining. J. Cheminformatics **3**(1), 41 (2011). https://doi.org/10.1186/1758-2946-3-41

6. Kumar, P., Kabra, S., Cole, J.: Auto-generating databases of yield strength and grain size using chemdataextractor. Scientific Data **9**, 292 (2022). https://doi.org/10.1038/s41597-022-01301-w
7. Court, C.J., Cole, J.M.: Auto-generated materials database of Curie and Néel temperatures via semi-supervised relationship extraction. Scientific Data **5**, 180111 (2018). https://doi.org/10.1038/sdata.2018.111

Author Index

A
Abdullah, Rana I-95
Abe, Kouya II-128
Abhin, B. I-216
Aguilar, Sergio Torres I-321
Ai, Xuguang I-49
Aizawa, Akiko II-140
Alekseev, Artem I-426
Alghamdi, Gheeda I-156
Al-Henaki, Lubna I-156
Aljasim, Hawra I-156
Al-Khalifa, Hend I-156
Alqubayshi, Hajar I-156
Al-Salman, Abdulmalik I-156
Al-Twailay, Hind I-156
Ananiadou, Sophia II-58
Aota, Masaki II-177
Aryan, Rishav I-457
Azman, Azreen I-173

B
Balducci, Gianmaria II-105
Banerjee, Debayan I-95
Baumgartner, David II-25
Benamara, Farah II-163
Bian, Jie I-350
Biemann, Chris II-223
Boonkwan, Prachya I-201
Bothera, Aabha I-457
Boudin, Florian II-140
Brokman, Aviv I-80
Butko, Miron I-426

C
Catapang, Jasper Kyle I-111
Chaichuk, Mikhail I-426
Chen, Fumian I-259, II-230
Chen, Jian I-442
Chen, Yin I-442
Chow, Man Yiu II-200
Chu, Hoang I-396
Chu, Huy I-396

D
Dey, Abhishek I-457
Do, Dinh-Truong I-186
Doan, Tu My II-25
Duan, Changxu I-34

E
Elkharrat, Feras II-116
Ernst, Damien II-3
Ezzabady, Morteza II-163

F
Fang, Hui I-259, II-230
Fersini, Elisabetta II-105
Frasincar, Flavius I-64, II-36
Fu, Xianghua I-442
Fukuoka, Keito I-472

G
Ghoniem, Mohamed II-116
Gomm, Moritz II-223
Groppe, Jinghua II-163
Groppe, Sven II-163
Gui, Zhou I-335
Gulla, Jon Atle I-365
Gupta, Shashank I-49

H
Han, Zhichao I-173
Hanwiboonwat, Supachoke I-201
Harth, Andreas I-335

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

R. Ichise (Ed.): NLDB 2025, LNCS 15837, pp. 237–239, 2026.
https://doi.org/10.1007/978-3-031-97144-0

Hasegawa, Rie I-380
Hato, Jumpei I-380
Havalgi, Akshay I-457
Hu, Miao I-126
Huang, Junbo I-95

I
Ichise, Ryutaro I-380
Ieng, Frédéric II-163
Imazato, Koki II-47
Inoshita, Keito I-3
Inoue, Naoya I-411
Ishihara, Shotaro II-177
Islamaj, Rezarta II-140
Ito, Yoichiro II-177

J
Jaroenkantasima, Jillaphat II-210
Jiang, Longquan I-95
Jiang, Yuhang I-49

K
Kamath S, Sowmya I-216, I-229
Kavuluru, Ramakanth I-49, I-80
Kazakova, Diana I-64
Khalid, Fatimah I-173
Khorashadizadeh, Hanieh II-163
Kille, Benjamin II-25
Kim, Jin-Dong II-140
Kim, Kon Woo II-140
Kitani, Mitsuhiro II-200
Klinkhamer, Jasmijn I-64
Kraft, Angelie I-95
Kumar M, Hemanth I-216
Kuty, Layla I-335

L
Lettieri, Nicola II-82
Lin, Zhiwei I-126
Liu, Yuan II-36
Lomasto, Luigi II-82
Long, Zi I-442
Louppe, Gilles II-3
Luu, Son T. I-291
Lv, Jinze I-442

M
Malandrino, Delfina II-82
Maraj, Amit I-142
Marshall, Adele H. I-126
Martin, Miguel Vargas I-142
McDermott, Christopher D. I-306
Messina, Enza II-105
Mihindukulasooriya, Nandana II-163
Miwa, Makoto II-58
Miyamori, Hisashi I-472, I-484
Mokeddem, Samy II-3
Möller, Cedric I-95
Mosca, Valerio II-82
Moskovskiy, Daniil I-19
Munasinghe, Lankeshwara I-306
Murali, Sidhaarth I-229
Mustaffa, Mas Rina I-173

N
Nakamura, Kosuke I-380
Nguyen, Dat Minh I-291
Nguyen, Kiet Van I-291
Nguyen, Le Minh I-411, II-152
Nguyen, Le-Minh I-186
Nguyen, Minh I-396
Nguyen, Ngoc Minh I-411
Nguyen, Phuc Huu I-275
Nguyen, Phuong I-396
Nguyen, Truc Mai-Thanh I-291
Nicho, Mathew I-306
Niimi, Junichiro II-189
Nikishina, Irina I-95, II-223

O
Ohkuma, Tomoko II-14
Ontiveros, Angel II-223
Otomura, Kotaro I-380
Özgöbek, Özlem I-365

P
Panchenko, Alexander I-19, I-95, I-426
Pirenne, Lize II-3
Piyatumrong, Apivadee I-201, II-210

Pletenev, Sergey I-19
Podishetty, Sanjay Kumar I-457

R
Racharak, Teeradaj I-243
Ragkhitwetsagul, Chaiyong I-243
Riano, Catalina II-230

S
Sabty, Caroline II-116
Sahri, Soror II-163
Sakhovskiy, Andrey I-95
Salnikov, Mikhail I-95
Sarikonda, Samhita I-457
Schmitt, Christopher II-223
Schraudner, Daniel I-335
Sherif, Ahmed II-116
Shimada, Kazutaka II-14, II-47
Shinnou, Hiroyuki II-128
Shirai, Kiyoaki I-186
Singh, Gaurav I-457
Somov, Oleg I-426
Sontesadisai, Chommakorn I-243
Srivastava, Saurabh I-457
Sugumaran, Vijayan I-216, I-229
Sun, Feifei II-152
Sun, Zhiru II-94
Sunetnanta, Thanwadee I-243

T
Tachioka, Yuuki II-70
Tamura, Takumi II-177
Taniguchi, Tomoki II-14
Thavornthaveekul, Chaichana I-201
Tiwari, Sanju II-163
Tong, Ziyi II-152
Tutubalina, Elena I-95, I-426

U
Ueno, Takato I-3
Usbeck, Ricardo I-95
Usmanova, Aida I-95

V
Vateekul, Peerapon I-201
Viriyayudhakorn, Kobkrit II-210

W
Wang, Jakob Mørup II-94
Welzl, Michael I-350

Y
Yadav, Vandana I-365
Yamada, Kenta II-177
Yamano, Mizuki I-484
Yan, Xi I-95
Yano, Ken II-58
Yuenyong, Sumeth II-210

Z
Zaccagnino, Rocco II-82
Zagoruyko, Sergey I-19
Zhang, Lemei I-365
Zhou, Xiaokang I-3
Zhu, Chencheng II-14
Zhu, Mengyao I-275